Chapter 1 Transaction Management and Lock Mechanism



HUAWEI

Foreword

- This chapter focuses on transaction processing and lock mechanisms in openGauss. Transaction concurrency control and lock mechanisms are crucial components for implementing ACID properties in openGauss databases. We will examine lock information for tables within the openGauss database and verify the functionality of SHARE, ACCESS SHARE, ROW EXCLUSIVE, and ACCESS EXCLUSIVE locks through reproduction.

HUAWEI

Objectives

- Upon completing this course, you will be able to:
  - Understand the principles and concurrency mechanisms of openGauss transaction processing.
  - Master the transaction state transitions and multi-version concurrency control (MVCC) mechanisms in openGauss.
  - Comprehend the working principles and distinctions among different types of locks.
  - Reproduce and experiment with different types of locks.

HUAWEI

Contents

1. **Transaction Management**
   - Transaction Concept
   - Concurrency Control Management
   - Transaction Status
   - Clog and CSNlog
   - MVCC Mechanism
   - Subtransactions
2. Lock Mechanism

HUAWEI

What Is a Transaction?

- A database transaction is a logical unit in the database execution process. It consists of a limited sequence of operations, including database reads and writes.
  - When a transaction is committed, the database system completes all operations in the transaction and saves the structure permanently.
  - If an operation in a transaction fails, all operations in the transaction are rolled back to the status before the transaction is executed.
  - Transactions are mutually independent and do not affect each other.
  - Not all sequences in database operations are transactions. A transaction has four basic properties — atomicity, consistency, isolation, and durability — known as the ACID properties.

HUAWEI

ACID Properties

- **Atomicity**
  - All operations in a transaction are either completed or not completed.
- **Consistency**
  - The database must remain consistent after any transaction. No transaction should have any adverse effect on the data in the database.
- **Isolation**
  - Each transaction is executed in isolation. There are four isolation levels.
- **Durability**
  - The transaction result is persistent.

Example: transferring CNY50 from account A to account B

begin

1. read(A)
2. A := A – 50
3. write(A)
4. read(B)
5. B := B + 50
6. write(B)

commit

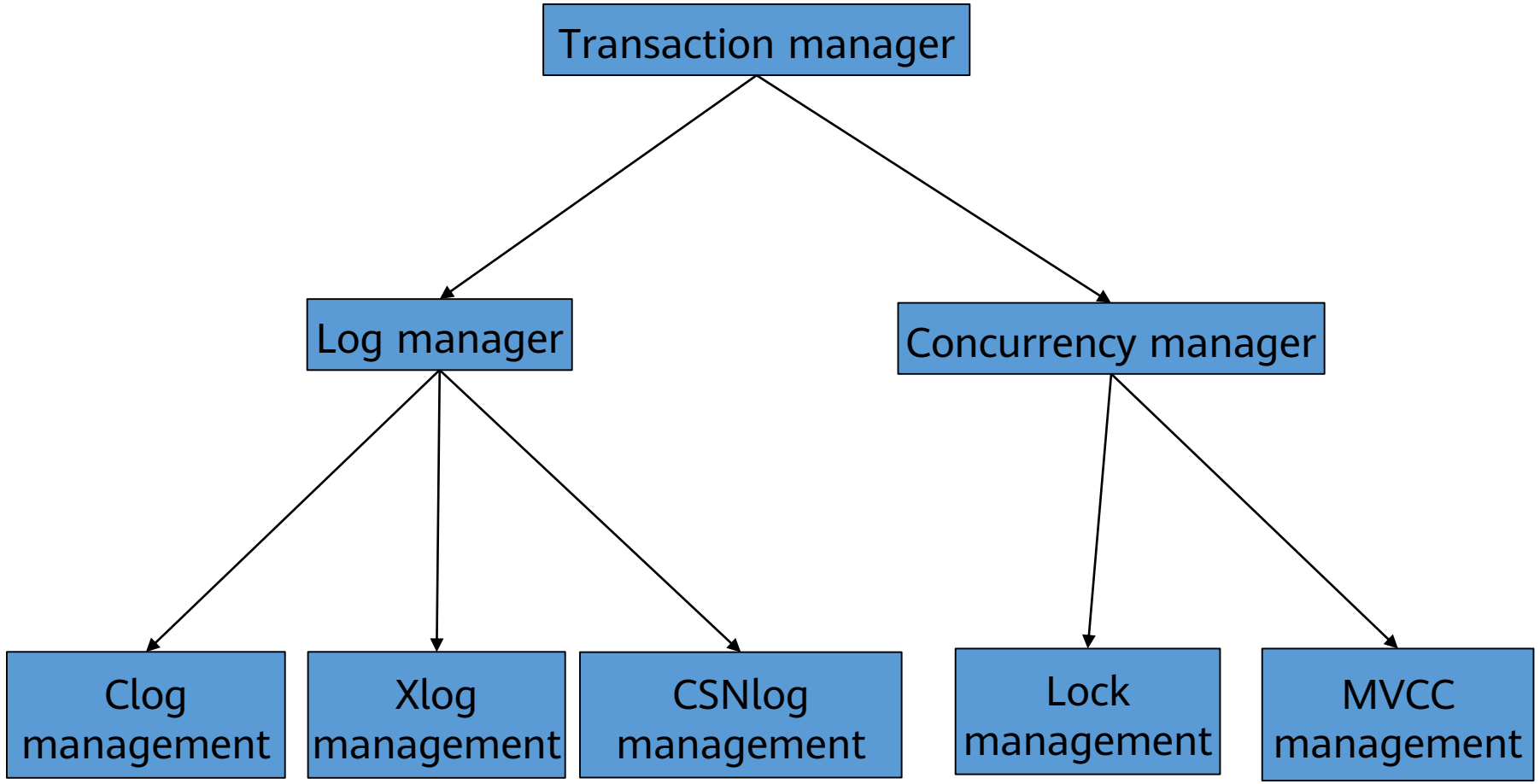
HUAWEI

Explicit Transaction

- Start a transaction block with the **BEGIN** command. All the following statements are executed in the transaction until an explicit COMMIT or ROLLBACK operation is given.
- When you make multiple related changes, executing multiple statements within a single transaction helps to ensure consistency. That is, other sessions cannot view the intermediate states until all related updates are completed.
- Automatically committed transaction:
  - By default (without the **BEGIN** command), the system executes transactions through the automatic commit. Each statement is executed in its own transaction and is implicitly committed upon successful execution. If the statement execution fails, a rollback is performed.

HUAWEI

Transaction Management Layer



HUAWEI

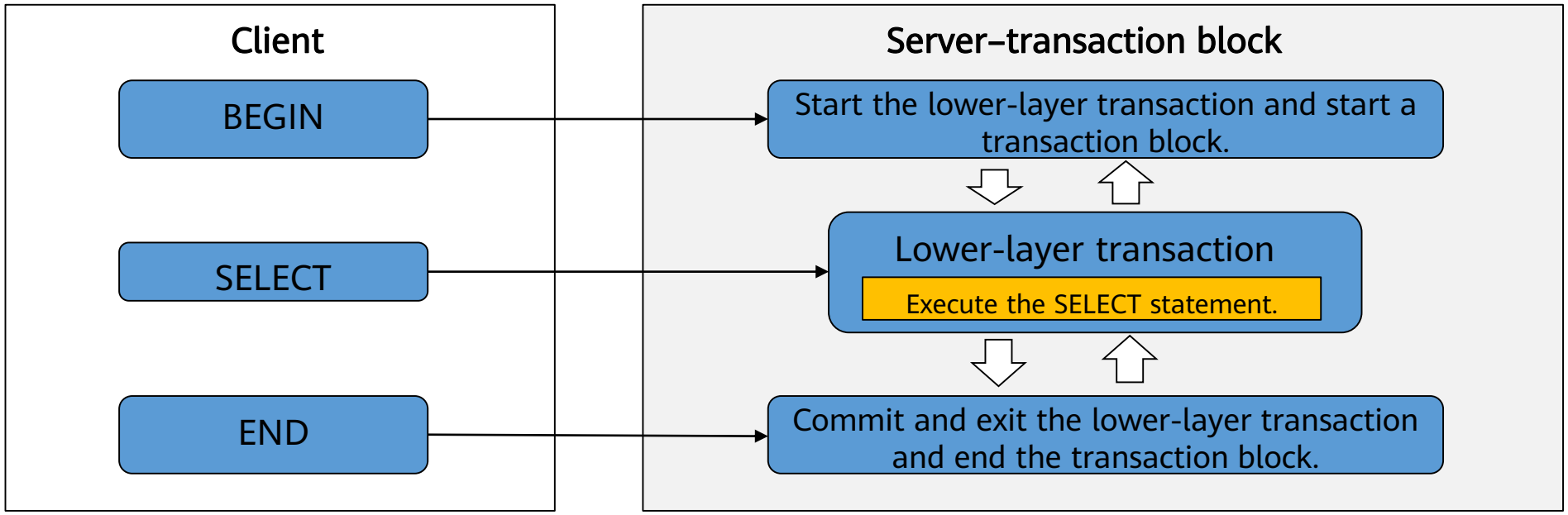
Understanding Transaction Management Layers

- The transaction manager is the hub of the transaction system. It is a finite state machine (FSM) in terms of implementation. It receives external commands or signals and determines the next process to execute based on the current transaction status.
- The concurrency manager is made up of various locks required for system concurrency control and multi-version concurrency control (MVCC). The read phase of the transaction uses MVCC. The read and write operations on tuples are not blocked. The write phase uses various locks to ensure the transaction isolation level.
- The log manager records the status of transaction execution and data changes, including transaction commitment logs (Clogs and CSNlogs) and transaction process logs (Xlogs).
  - Clogs and CSNlogs record the transaction execution result.
  - Clog specifies whether the transaction is committed.
  - CSNlog indicates the sequence number of the committed transaction.
  - Xlogs record data changes during the transaction and contains certain redundant data for restoration.

HUAWEI

Transaction Execution Layer

```
BEGIN;
SELECT * from tableA;
END;
```



Transaction Isolation Levels

- **Dirty read:** A transaction reads data written by another uncommitted transaction.
- **Non-repeatable read:** A transaction reads data that has been read before and finds that the data has been modified by another committed transaction.
- **Phantom read:** A phantom read occurs when, during a transaction, new rows are added or removed by another transaction.

Isolation Level	Dirty Read	Non-repeatable Read	Phantom Read
Read uncommitted	Possible	Possible	Possible
Read committed	Impossible	Possible	Possible
Repeatable read	Impossible	Impossible	Possible
Serializable	Impossible	Impossible	Impossible

HUAWEI

Contents

1. **Transaction Management**
   - Transaction Concept
   - Concurrency Control Management
   - Transaction Status
   - Clog and CSNlog
   - MVCC Mechanism
   - Subtransactions
2. Lock Mechanism

HUAWEI

About Concurrency Control

- Concurrency control refers to a series of technologies and methods that ensure transaction isolation and consistency when multiple transactions are concurrently executed. Its objectives are:
  - **Isolation:** Ensure that concurrent transactions are isolated from each other.
  - **Consistency:** Ensure that concurrent transactions do not damage the integrity constraints of the database.
  - **Throughput:** As much as possible, improve the database's concurrent processing capability to increase system throughput.

HUAWEI

Transaction Composition

- The transaction concurrency control mechanism ensures the ACID properties of the database when transactions are concurrently executed.
- In openGauss, the transaction system is divided into two layers: the upper layer (TBlockState) and the lower layer (TransState). This layered design abstracts the details when upper-layer services are being processed, flexibly supporting various client transaction statements (BEGIN, START TRANSACTION, COMMIT, ROLLBACK, and END).
  - TBlockState: represents the state of client queries, enhancing flexibility in user data operations by supporting multiple queries within a single transaction block.
  - TransState: records the state of the entire transaction from the kernel's perspective.

HUAWEI

Methods: Lock Mechanism

- Locks are one of the popular concurrency control methods. They implement concurrency control by locking database objects such as tables, rows, and pages. There are shared and exclusive locks.
  - Shared locks allow multiple transactions to read an object simultaneously.
  - Exclusive locks allow only one transaction to read and write an object at a time.
- In openGauss, you can use the LOCK TABLE statement to lock a table and use the SELECT... FOR UPDATE statement to add an exclusive lock to a row.

  - Lock a table.

  ```
  LOCK TABLE table_name IN lock_mode;
  ```

  - Add an exclusive lock to a row.

  ```
  SELECT * FROM table_name WHERE condition FOR UPDATE;
  ```


HUAWEI

Methods: Optimistic Concurrency Control

- Optimistic concurrency control is based on conflict detection. It assumes that conflicts between transactions rarely occur, allowing multiple transactions to happen at the same time. During the commit, this method checks for any conflicts.
- In openGauss, you can use versioning to implement optimistic concurrency control. Each data object has a version number. When a transaction reads data, it records the version number, and upon the commit, it checks for any changes.

- Add versioning when creating a table.

```
CREATE TABLE table_name (
  ...
) WITH (VERSION_CONTROL = ON);
```

- Record the version number when data is read.

```
SELECT * FROM table_name WHERE condition;
```


HUAWEI

Methods: Timestamp Concurrency Control

- Timestamp concurrency control is based on timestamps. Each transaction has a unique timestamp which identifies its start time. When a transaction reads data, it checks the timestamp of the data object against its own timestamp to determine if it can proceed with the read operation.
- In openGauss, you can use the SERIALIZABLE isolation level to implement timestamp concurrency control.

- Set the transaction isolation level to **SERIALIZABLE**.

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

- Execute transaction operations.

```
BEGIN;

...

COMMIT;
```


HUAWEI

Optimizing Concurrency Control Performance

- Performance optimization is essential during concurrency control to improve the system's concurrent processing capability. Here are some ways to do so:
  - **Lock granularity optimization:** Select appropriate lock granularity to minimize the lock range, thereby improving concurrency performance.
  - **Read/Write separation:** Separate read and write operations, using lower isolation levels for reads and higher isolation levels for writes to boost concurrency.
  - **Database design optimization:** Design table structures and indexes to reduce concurrency conflicts and the locking frequency.
  - **Cache optimization:** Use caching techniques to minimize database access, improving system response time and concurrency performance.

HUAWEI

Contents

1. **Transaction Management**
   - Transaction Concept
   - Concurrency Control Management
   - Transaction Status
   - Clog and CSNlog
   - MVCC Mechanism
   - Subtransactions
2. Lock Mechanism

HUAWEI

Upper Layer States

- Transaction block states
  - A transaction block state refers to the state of the entire transaction system. It reflects changes in user commands and the lower-layer execution states of transactions.
- The transaction block states include:
  - **DEFAULT**: The default state is used.
  - **STARTED**: A transaction has started.
  - **BEGIN**: Encountering the **BEGIN** command marks the start of a transaction.
  - **INPROGRESS**: A transaction block is being processed.
  - **END**: Encountering the **COMMIT** or **END** command indicates that a transaction block has ended.
  - **ABORT**: An error has occurred in a transaction, and it is waiting for the **ROLLBACK** command.
  - **ABORT_END**: An error has occurred, and the **ROLLBACK** command has been received.
  - **ABORT_PENDING**: A transaction is being processed, and the **ROLLBACK** command has been received.
  - **PREPARE**: A transaction is being processed, and the **PREPARE** command has been received.

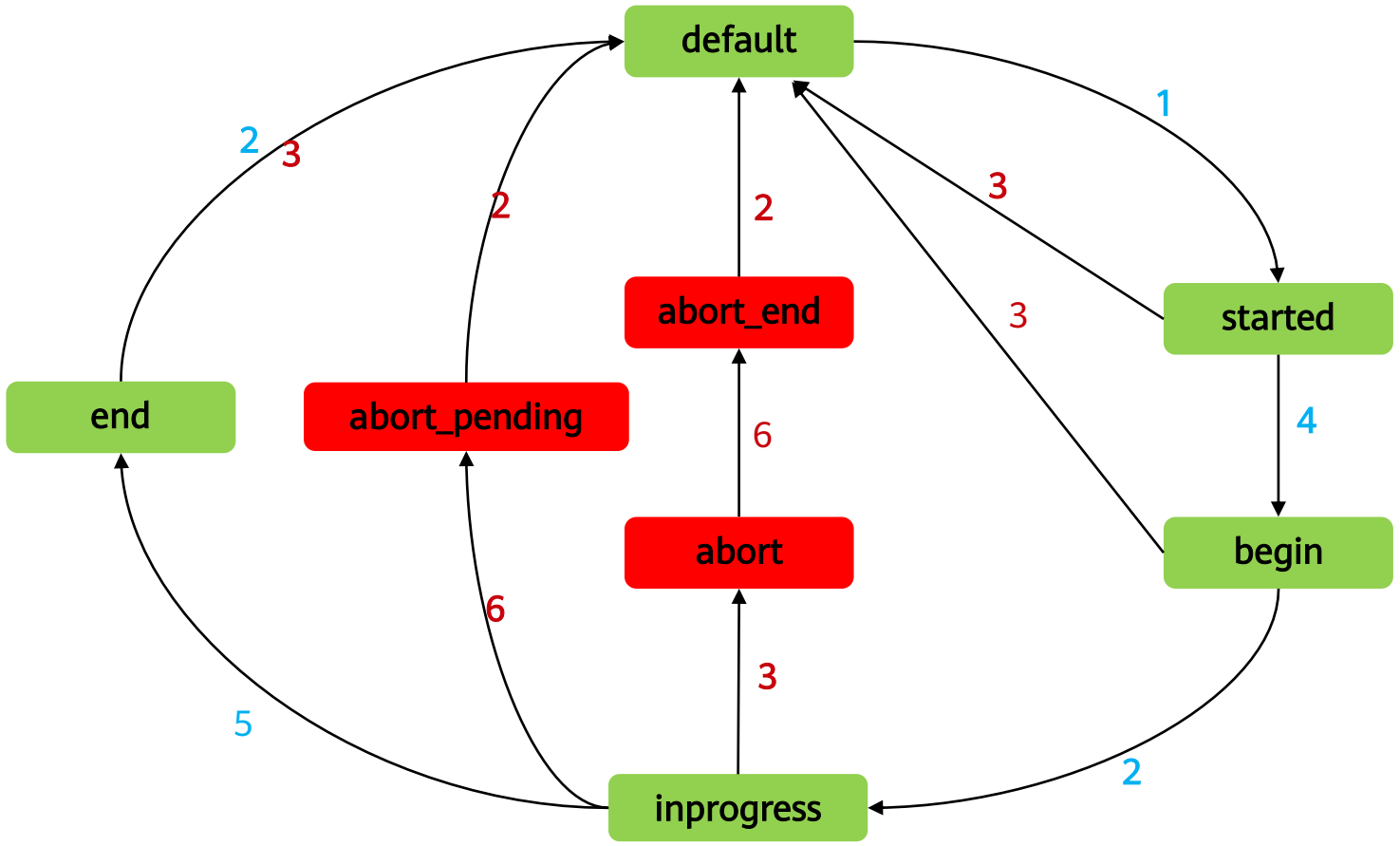
HUAWEI

Upper Layer: Transaction Block Operations

- Any statement execution in a database begins with the transaction block entry function. After the execution is complete, it exits through the transaction block exit function. If an error occurs during execution, the transaction block error handling function manages the error.
  - StartTransactionCommand: transaction block entry function, called before each statement execution
  - CommitTransactionCommand: transaction block exit function, called after each statement execution
  - AbortCurrentTransaction: error handling function, called when an error occurs
- In addition, users can run commands such as **BEGIN**, **END**, and **ROLLBACK** to change the transaction block states.
  - **BEGIN**: An explicit transaction block starts.
  - **END**: A transaction block ends after the **COMMIT** or **ROLLBACK** is delivered.
  - **ROLLBACK**: A user terminates a transaction block and rolls back the transactions.

HUAWEI

Upper Layer: Transaction State Changes



1. StartTransactionCommand
2. CommitTransactionCommand
3. AbortCurrentTransaction
4. BEGIN
5. END
6. ROLLBACK

HUAWEI

Upper Layer: Transaction State Change Description

- Red indicates that the transaction is rolled back, and green indicates that the transaction is normal.
- The preceding slide shows that if there are no exceptions, the state machine of a transaction block is in the green loop: DEFAULT --> STARTED --> BEGIN --> INPROGRESS --> END --> DEFAULT.
- For the remaining state machines, follow a similar logic to understand abnormal situations in each state:

1. If an error occurs before a transaction enters the INPROGRESS state, meaning the transaction hasn't yet started, the error is reported, and the transaction is rolled back. Resources are cleared, and the transaction returns to the DEFAULT state.

HUAWEI

Upper Layer: Transaction State Change Description

2. There are two scenarios when errors occur in the INPROGRESS state during transaction block execution:
   - Transaction execution failure: INPROGRESS -> ABORT -> ABORT_END -> DEFAULT;
   - User-initiated rollback of a successful transaction: INPROGRESS -> ABORT_PENDING -> DEFAULT
3. If an error occurs during a user commit, the transaction block moves from the **END** state to the **DEFAULT** state. Once a transaction starts, it moves away from the **DEFAULT** state and only returns to **DEFAULT** after the transaction is fully completed.
4. openGauss also supports implicit transaction blocks. When a client executes a single SQL statement, it can be automatically committed. The state machine is relatively simple: DEFAULT -> STARTED -> DEFAULT.

HUAWEI

Lower-Layer Transaction States

- The lower-layer transaction states are the real states from the perspective of the openGauss kernel. They are:
  - **DEFAULT**: This is the default state before a transaction starts.
  - **START**: A transaction starts.
  - **INPROGRESS**: A transaction is being processed. The transaction remains in this state after it starts.
  - **COMMIT/ABORT**: A transaction ends or is being rolled back.
  - **DEFAULT**: This is the default state after a transaction ends.

HUAWEI

Lower-Layer Transaction Operations

- **Starting a transaction:** Initialize the transaction.
- **Committing a transaction:** Record logs, commit the transaction, clear resources, and set the transaction state.
- **Exiting a transaction:** Release a lock, set the transaction state, record logs, and clear resources when the system encounters an error.
- **Clearing a transaction:** Clear the memory resources occupied by the transaction.

HUAWEI

Lower-Layer Functions: Transaction Processing

- Transaction processing functions allocate, recycle, and clear transaction resources based on the current upper-level state machine of the transaction.

Function	Description
StartTransaction	Starts a transaction, initializes the memory and variables, and sets the lower-layer transaction state to **INPROGRESS**.
CommitTransaction	Transits the current lower-layer state machine from **INPROGRESS** to **COMMIT**, ensures that Clogs and Xlogs are stored locally in a persistent manner, clears the corresponding transaction slot information, and then sets the lower-layer state to **DEFAULT**.
PrepareTransaction	Like the CommitTransaction function, transits the current lower-layer state machine from **INPROGRESS** to **PREPARE**, constructs a two-phase GXACT structure, creates a two-phase file, adds dummy slot information, transfers the thread lock information to the dummy slot, releases resources, and finally sets the lower-layer state machine to **DEFAULT**.
AbortTransaction	Releases LWLocks, UnlockBuffers, and LockErrorCleanup, transits the lower-layer state from **INPROGRESS** to **ABORT**, records the corresponding Clogs, clears the transaction slot information, and releases various resources.
CleanupTransaction	The current lower-layer state machine should be **ABORT**. This function is generally called after the AbortTransaction function, and it also clears some resources.
FinishPreparedTransaction	Ends a two-phase commit transaction.

HUAWEI

Lower-Layer Functions: Transaction Execution

- Transaction execution functions are called based on the corresponding state machine.

Function	Description
StartTransactionCommand	Calls the corresponding transaction execution function based on the upper-layer state machine when a transaction starts.
CommitTransactionCommand	Calls the corresponding transaction execution function based on the upper-layer state machine when a transaction ends.
AbortCurrentTransaction	Calls the longjump function, clears the corresponding resources in advance, and sets the upper-layer transaction state machine to **ABORT** when an internal error occurs in the transaction.
AbortTransaction	Releases LWLocks, UnlockBuffers, and LockErrorCleanup, transits the lower-layer state from **INPROGRESS** to **ABORT**, records the corresponding Clogs, clears the transaction slot information, and releases various resources.

HUAWEI

Lower-Layer Functions: Upper-Layer Transaction State Machine Control

- Functions for controlling the upper-layer transaction state machine

Function	Description
BeginTransactionBlock	Sets the upper-layer transaction state machine to **BEGIN** when a transaction starts explicitly.
EndTransactionBlock	Sets the upper-layer transaction state machine to **END** when a transaction is committed explicitly.
UserAbortTransactionBlock	Sets the upper-layer transaction state machine to **ABORT_PENDING** or **ABORT_END** when a transaction is rolled back explicitly.
PrepareTransactionBlock	Sets the upper-layer transaction state machine to **PREPARE** when the PREPARE statement is executed explicitly.

HUAWEI

Contents

1. **Transaction Management**
   - Transaction Concept
   - Concurrency Control Management
   - Transaction Status
   - Clog and CSNlog
   - MVCC Mechanism
   - Subtransactions
2. Lock Mechanism

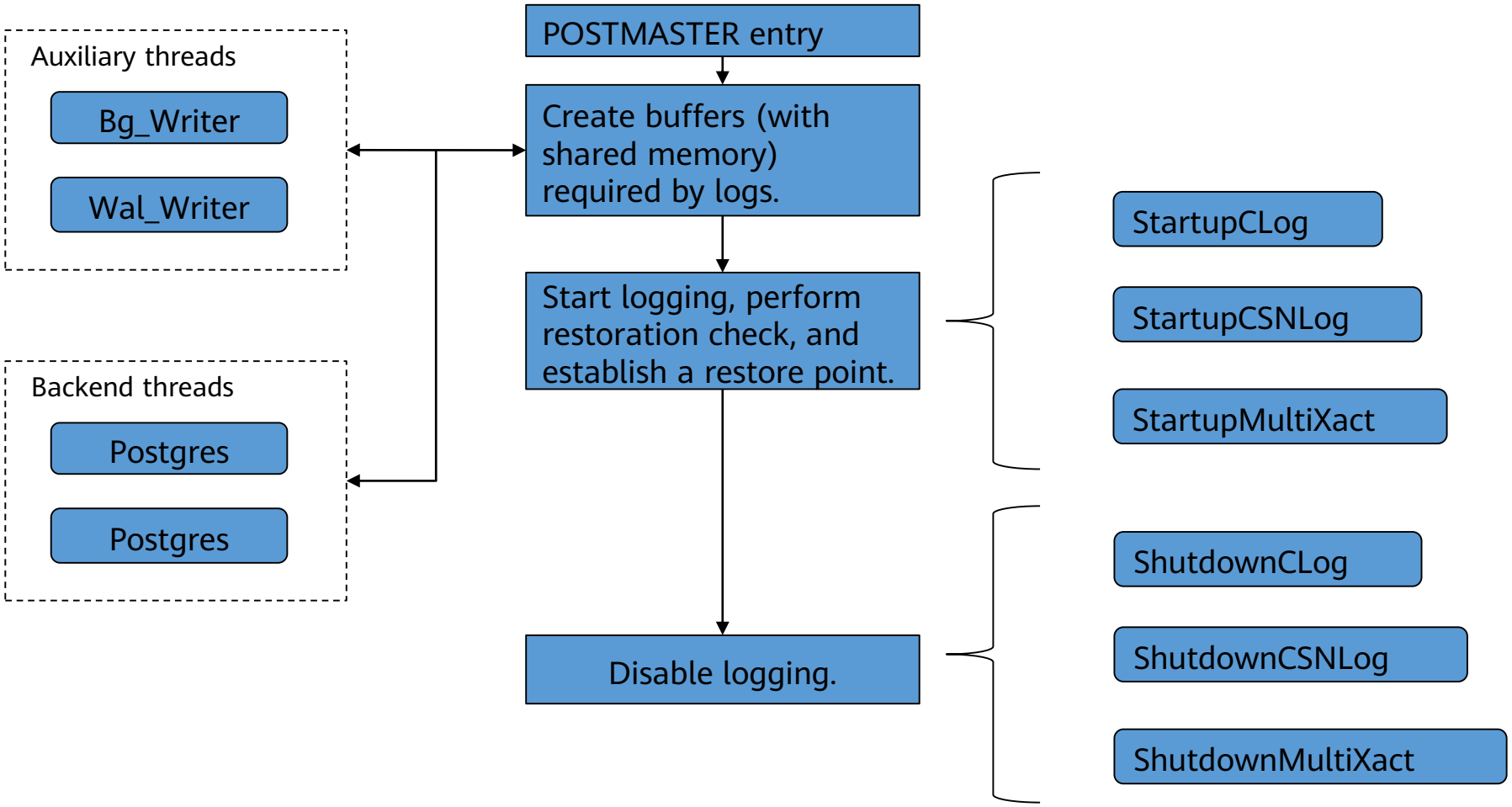
HUAWEI

Transaction Log Management

- A database mainly uses Xlogs, Clogs, and CSNlogs. Xlogs are transaction logs, while Clogs and CSNlogs are transaction commit logs.
- Logs are stored in log files. A log buffer is set to reduce the I/O overhead of writing logs to disks. Logs are first written to the log buffer of the main memory; once it's full, logs are flushed to disks as blocks.
- A database uses the same SLRU buffer to manage the Clogs, CSNlogs, and MultiXact logs.
- When the database system is started, logs are repeatedly inserted and refreshed. If the system breaks down, data restoration is required.

HUAWEI

Relationship Between Log Processing Modules



SLRU Buffer Pool

- By default, the simple least-recently-used (SLRU) buffer pool consists of eight buffers.
- Each buffer is a page mapped to an 8-KB disk block.
- The pages use the SLRU scheduling algorithm.
- There are few pages in the buffer. So, a simple linear search algorithm is used for searches.
- Logs are written to the buffer mapped to the logged page and then to the log segment file on the disk.
- Log file reading is restricted. For reading, the page containing the log record is loaded to a page in the buffer pool from which the log record can then be read.

HUAWEI

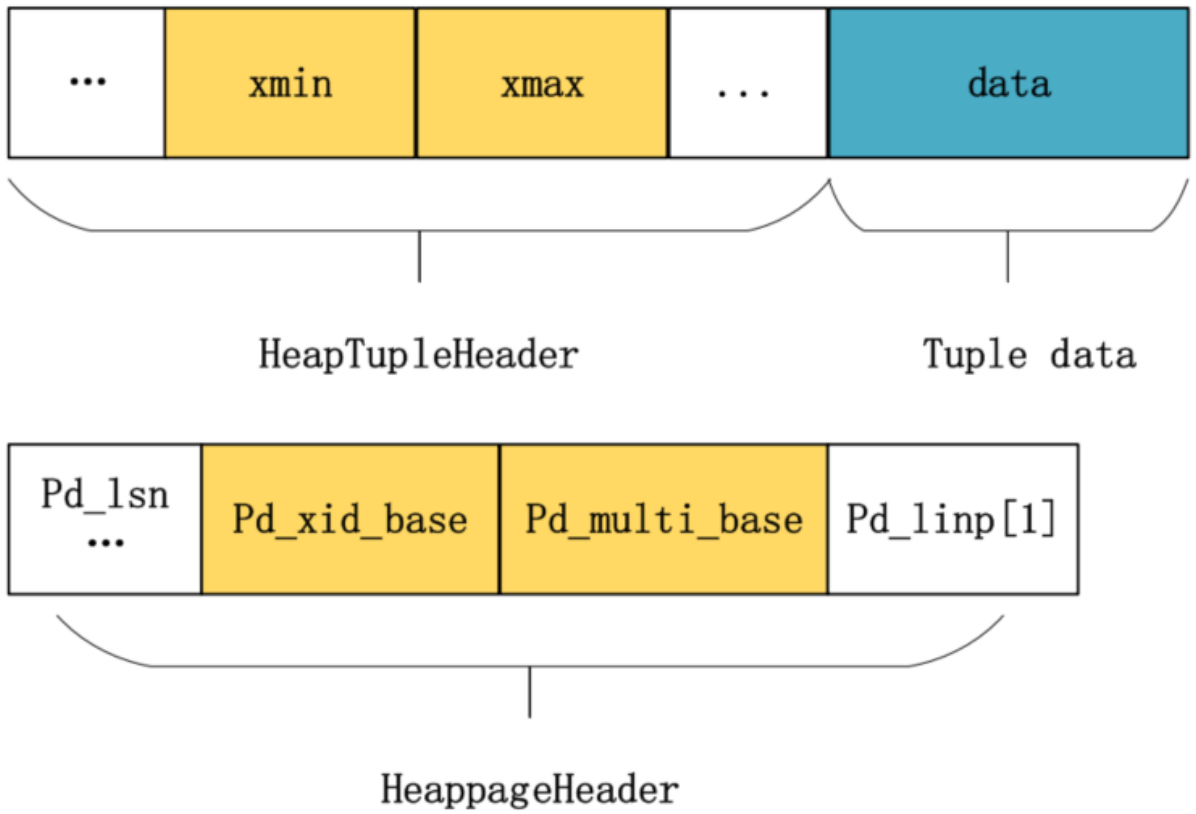
What Is a Transaction ID?

- The openGauss database allocates a unique identifier to each transaction, known as the transaction ID (XID). This XID is a uint64 monotonically increasing sequence.
- After a transaction ends, Clogs record whether the transaction is committed and CSNlogs record the sequence number of the committed transaction for visibility determination.
- openGauss allocates a unique XID to each write transaction.
  - When a transaction inserts a tuple, it writes that transaction's information to **xmin** in the tuple header.
  - When a transaction updates or deletes a tuple, it writes that transaction's information to **xmax** in the tuple.
- Currently, XID allocation uses a uint64 monotonically increasing sequence. To save space and remain compatible with earlier versions, **xmin** and **xmax** in the tuple header are divided into two parts for storage.
  - **xmin** and **xmax** in the tuple header are uint32 numbers.
  - The page header stores the current page's 64-bit xid_base. .

HUAWEI

Transaction ID Allocation

- To prevent XIDs from being consumed too quickly, openGauss allocates XIDs only to write transactions without allocating extra XIDs to read-only transactions. That is, XIDs are allocated only when they are required.
- Theoretically, 64-bit XIDs are sufficient. If a database runs 10 million transactions per second (TPS), 64-bit XIDs can be used for 580,000 years.



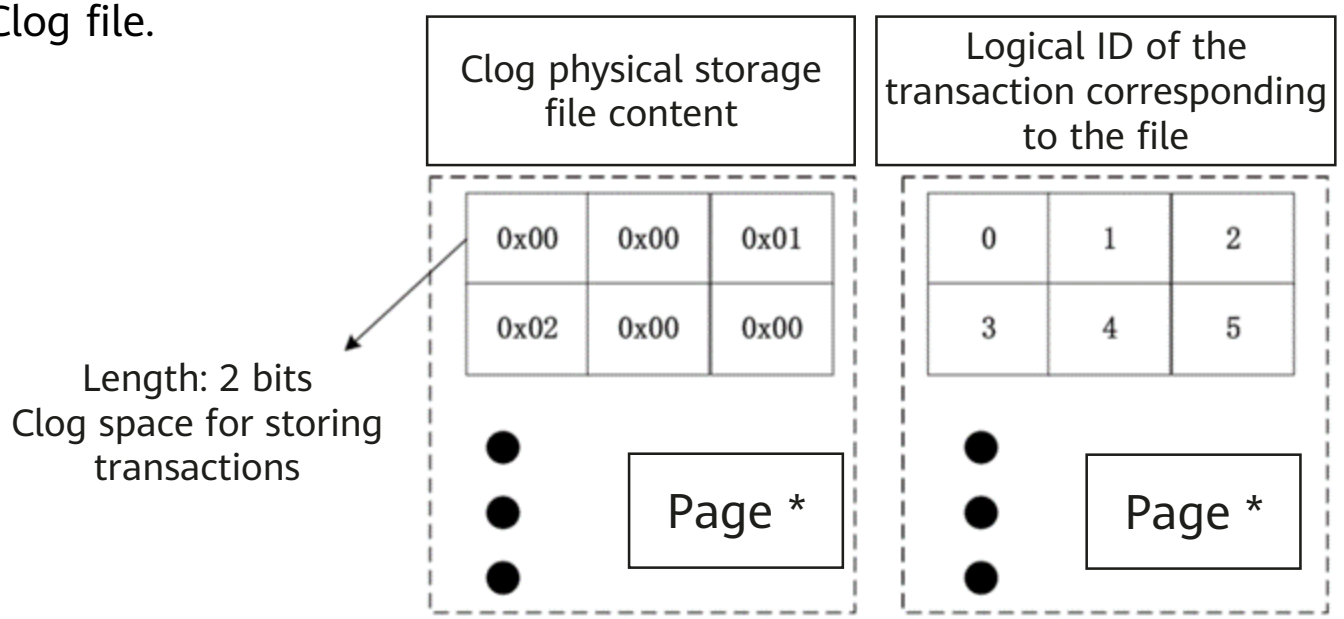
Clog and CSNlog Overview

- Clogs maintain the mapping between XIDs and CommitLogs. CSNlogs maintain the mapping between XIDs and CommitSeqNoLogs. Since memory resources are limited and there may be long transactions in the system, not all mappings can be stored in the memory. In this case, the mappings need to be written to disks as physical files. Therefore, Clog files (XID -> CommitLog Map) and CSNlog files (XID -> CommitSeqNoLog Map) are generated.
- Both Clogs and CSNlogs use the SLRU mechanism to read files and flush data to disks.

HUAWEI

Physical Organization of Clog Files

- Clogs record the commit status of XIDs. In openGauss, four bits are used to identify the status of each XID. Clogs have the following code:
  - **#define CLOG_XID_STATUS_IN_PROGRESS 0x00**: The transaction has not started or is still running.
  - **#define CLOG_XID_STATUS_COMMITTED 0x01**: The transaction has been committed.
  - **#define CLOG_XID_STATUS_ABORTED 0x02**: The transaction has been rolled back.
- The figure shows the physical organization of a Clog file.
  - Transactions 0, 1, 4, and 5 are still running.
  - Transaction 2 has been committed and transaction 3 has been rolled back.



Clog Management

- Clogs record the final states of a transaction, including:
  - Inprogress
  - Committed
  - Aborted
  - Sub_Committed
- You can locate a Clog record by using a 4-tuple **<segmentno, pageno, byte, bindex>**.
  - **segmentno**: segment number. A segment contains 32 pages.
  - **pageno**: page offset in the segment where the log record is located
  - **byte**: offset within a page
  - **bindex**: offset in a byte. Two bits express an XID state, so one byte stores the states of four XIDs.

HUAWEI

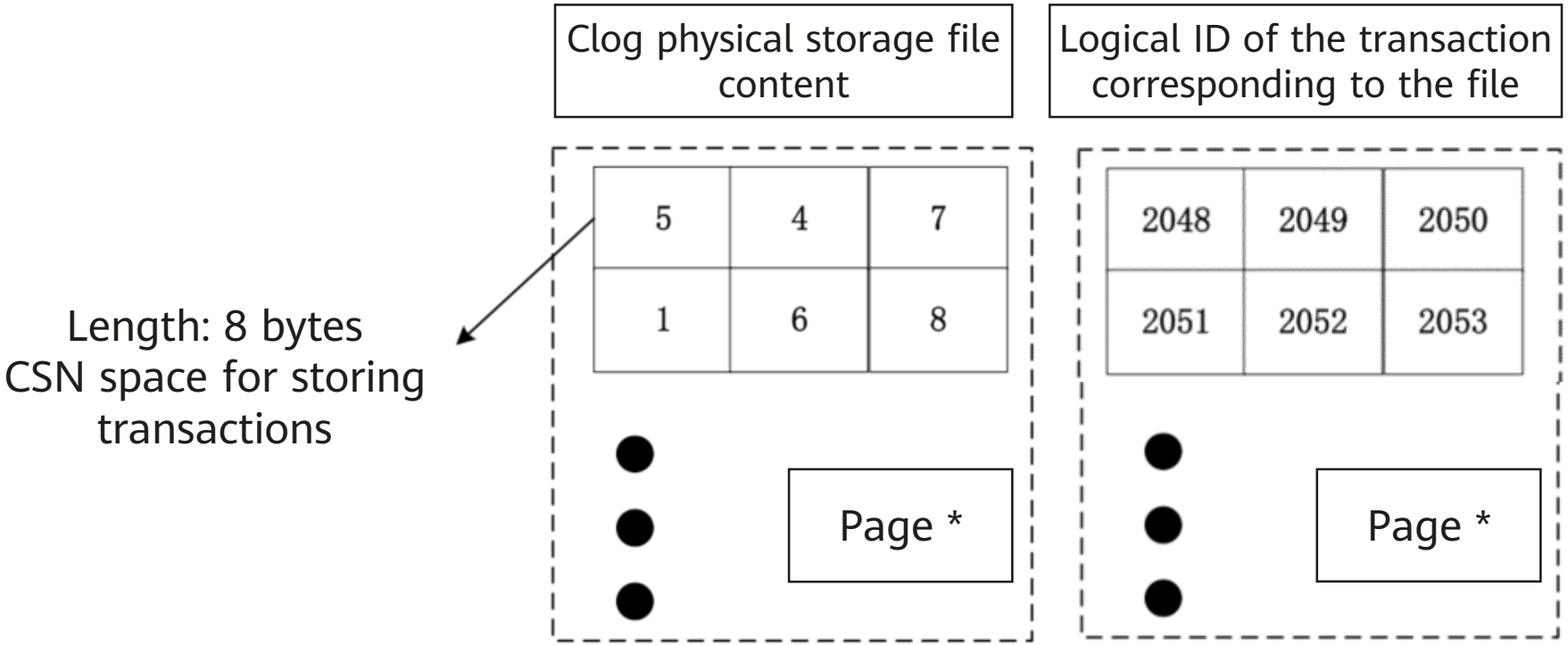
Physical Organization of CSNlog Files

- CSNlogs record the sequence number of the transaction commit. openGauss allocates an 8-byte CSN of the uint64 type to each XID. As such, an 8-KB page can store the CSNs of 1000 transactions. Like the XIDs, several special numbers are reserved for the CSNs. CSNlogs have the following code:
  - **#define COMMITSEQNO_INPROGRESS UINT64CONST(0x0)**: The transaction has not been committed or rolled back.
  - **#define COMMITSEQNO_ABORTED UINT64CONST(0x1)**: The transaction has been rolled back.
  - **#define COMMITSEQNO_FROZEN UINT64CONST(0x2)**: The transaction has been committed and is visible to any snapshot.
  - **#define COMMITSEQNO_FIRST_NORMAL UINT64CONST(0x3)**: start value of the CSN of the transaction
  - **#define COMMITSEQNO_COMMIT_INPROGRESS (UINT64CONST(1) << 62)**: The transaction is being committed.

HUAWEI

Physical Organization of CSNlog Files

- The figure shows the physical organization of a CSNlog file. The CSNs corresponding to the XIDs 2048, 2049, 2050, 2051, 2052 and 2053 are 5, 4, 7, 10, 6, and 8, respectively. That is, the transaction commit sequence is 2049 -> 2048 -> 2052 -> 2050 -> 2053 -> 2051.



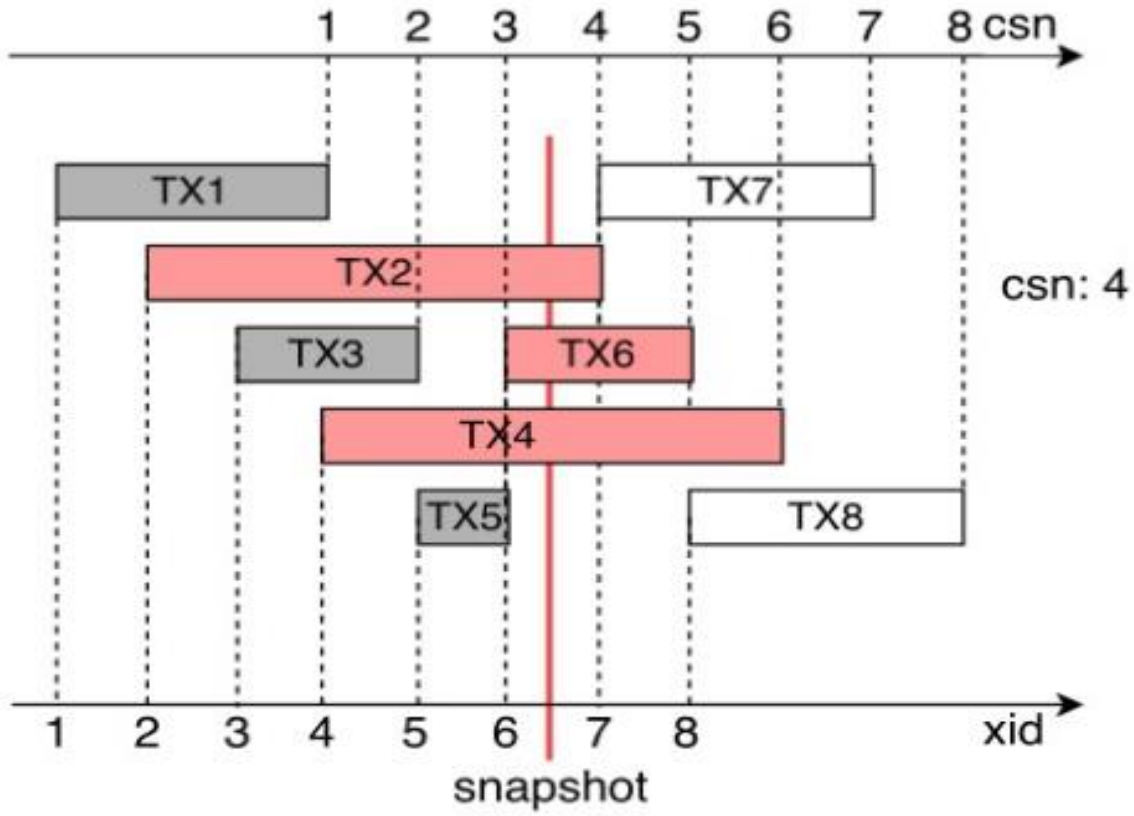
CSNlog Management: CSN

- The commit sequence number (CSN) is similar to the system change number (SCN) in the Oracle Database. A CSN is a 64-bit unsigned integer in ascending order.
- GTM maintains the CSN value. When a transaction is committed, the CSN value increases by 1. It remains unchanged at transaction rollbacks. GTM calculates the latest CSN value from the **gtm.control** file upon startup or fault recovery.
- Normally, the CSN value starts from 3. The CSN value is set when a transaction is committed. Each transaction ID is mapped to a CSN. You can check whether the tuple is visible by comparing the CSN at a transaction commit with the CSN in the snapshot captured when the transaction or statement starts.
  - When the CSN is 0, the transaction is running.
  - When the CSN is 1, the transaction is rolled back.
  - When the CSN is 2, the transaction is visible to all users.

HUAWEI

CSNlog Management: Relationship Between XID and CSN

- A CSNlog file stores the mapping between XIDs and CSNs, which is used to determine visibility.



HUAWEI

CSNlog Management: Relationship Between XID and CSN

- Each non-read-only transaction is assigned with an XID during running. Then, a CSN is given when the transaction is committed, and the mapping between the CSN and the XID is saved.
- In the graph on the previous slide, the red vertical line indicates the time when the snapshot is captured. Without considering CSNs, TX2, TX4, and TX6 are active at that time.
- If CSNs are used, the snapshot's CSN is 3. In this case, modifications to TX2, TX4, TX6, TX7, and TX8 are invisible on the snapshot because the CSNs of these transactions are 4, 6, 5, 7, and 8, respectively.

HUAWEI

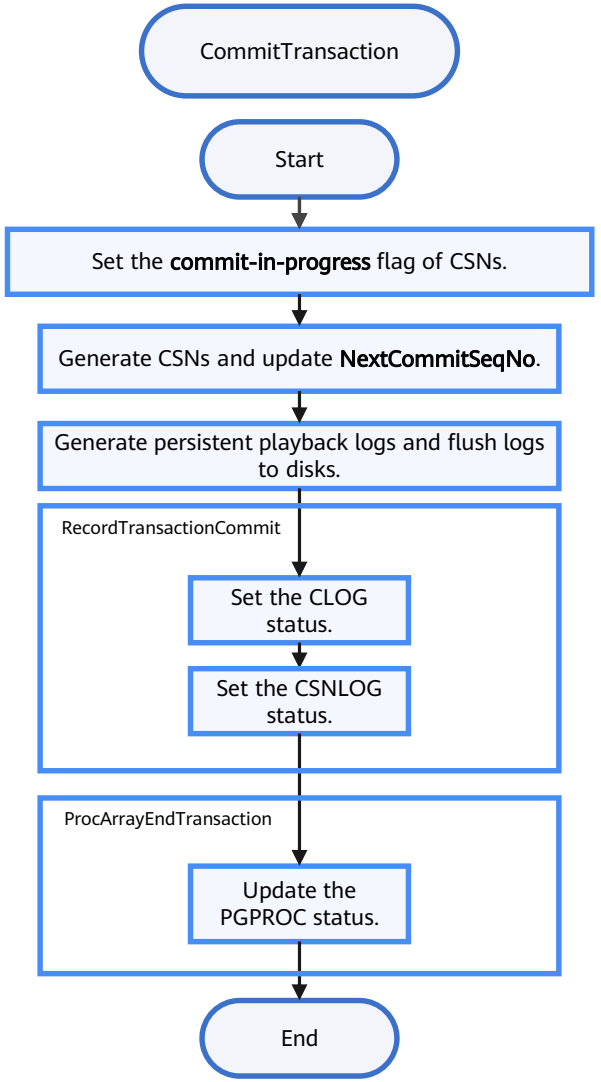
MultiXact Log Management

- MultiXact logs are used by databases to record MultiXact IDs.
- With MVCC enabled, a tuple may be associated with multiple transactions. To perform unified operations when a row-level read lock is added, transaction IDs associated with a tuple are combined and replaced by one MultiXact ID (stored in the **xmax** of the record header).
- MultiXact logs also use the SLRU buffer pool for implementation.

HUAWEI

Transaction Commit Process

- As shown in the figure, Clog and CSNlog settings are involved in transaction commit. The process is as follows:
  1. Set the **commit-in-progress** flag for the mapping between CSN and XID.
  2. Atomically update the **NextCommitSeqNo** value.
  3. Generate redo logs and write Clogs and CSNlogs.
  4. Update PGPROC to remove the corresponding transaction information from PGPROC.
  5. Set XID to **InvalidTransactionId** and **xmin** to **InvalidTransactionId**.



Contents

1. **Transaction Management**
   - Transaction Concept
   - Concurrency Control Management
   - Transaction Status
   - Clog and CSNlog
   - MVCC Mechanism
   - Subtransactions
2. Lock Mechanism

HUAWEI

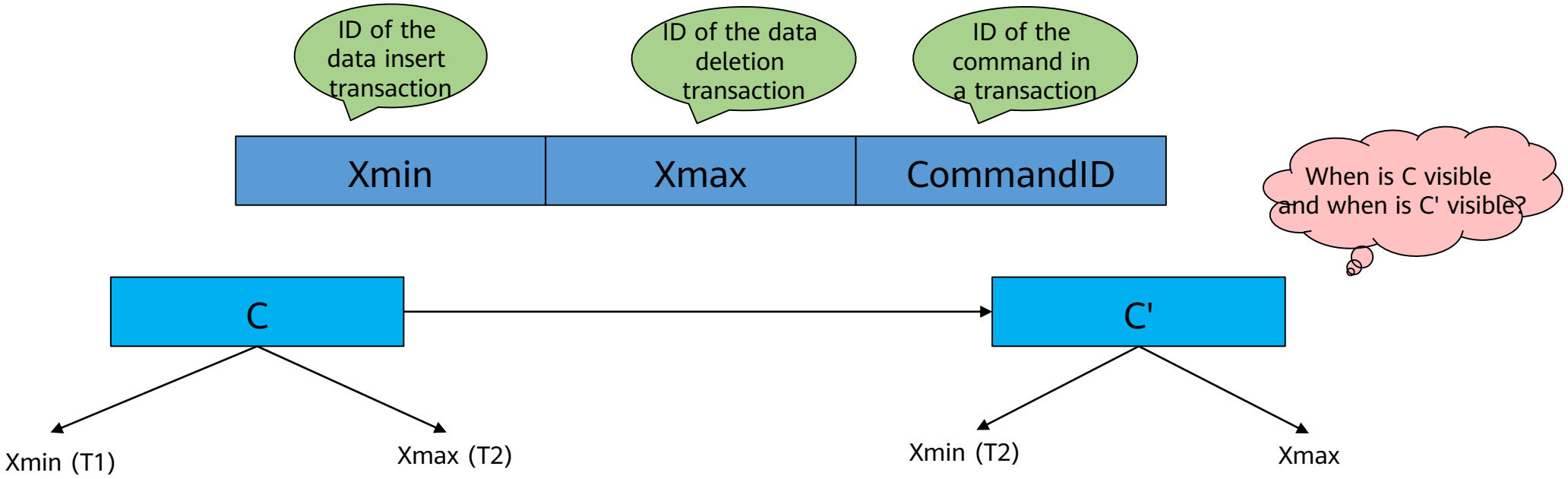
MVCC

- Multi-version snapshot control is a specific multi-version concurrency control (MVCC) implementation which is used to maintain data consistency in a database.
- When data is retrieved, only its snapshot, not the latest data state, is visible to transactions.
- Isolating each transaction session prevents data inconsistencies that could result from allowing other concurrent transactions to see the given transaction's data updates.
- MVCC is advantageous because it allows concurrent reads and writes, and read locks do not conflict with write locks, significantly improving the concurrent processing capability.
- However, only locks can control write transaction conflicts (one write transaction blocks another one).

HUAWEI

MVCC Data Structure

- In MVCC, when data is updated, it is not overwritten with a new value; instead, space is created to store a new tuple. The new and original values are both stored in the database and are identified by tuple header parameters. The figure shows the data structure recorded using MVCC:



MVCC Data Structure

- The CommandID is used to determine the visibility of multiple commands in a transaction. The result of a previous command is displayed before executing the next command.
  - When transaction T1 is not committed, records C and C' are not visible.
  - Record C is displayed when transaction T1 is committed but transaction T2 is uncommitted.
  - When both transactions T1 and T2 are committed, record C is not visible and record C' is visible.
- The transaction commitment is specific to a snapshot, that is, the status in a time window.

HUAWEI

MVCC Examples

Transaction T1	Transaction T2	Changes in Table A	Transaction T2 Query Result
BEGIN	SELECT * FROM A;	A	A
UPDATE		A->AA	
	SELECT * FROM A;		A
COMMIT			
	SELECT * FROM A;		AA

HUAWEI

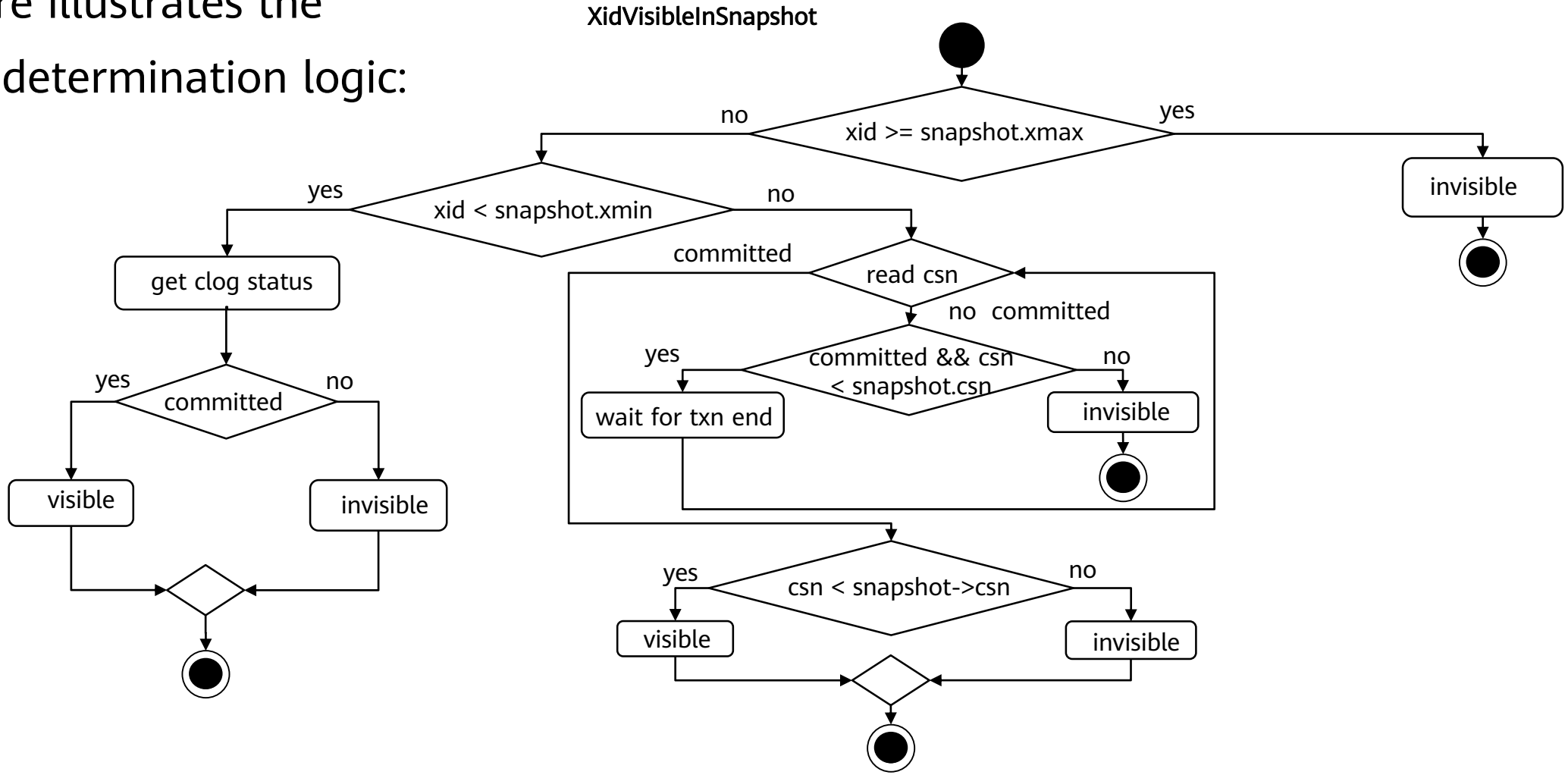
MVCC: Snapshot

- A snapshot records the active transaction commit states of a database at a specific point in time, from which you can know whether a tuple version is visible to the current snapshot.
- A snapshot contains the following information:
  - **xmin**: minimum XID among active transactions
  - **xmax**: maximum XID among committed transactions
  - **RecentGlobalXmin**: the minimum **xmin** value in a set of snapshots of active transactions
  - **Csn**: the commit sequence number of the current transaction

HUAWEI

MVCC: Snapshot

- The figure illustrates the visibility determination logic:



HUAWEI

MVCC: Snapshot

- **xmax** is used for quick filtering. When the XID is greater than or equal to the **xmax** value, the tuple is invisible.
- If XID is smaller than the **xmin** value, the transaction to which the tuples belong ended before the current transaction started. In this case, you need to query the commit state of the previous transaction in the Clog. The tuples of the previous transaction are visible only after the transaction is committed. After the transaction ends, the system first accesses the Clog of the tuple, and a flag is set in the tuple header.
- If the XID is between **xmin** and **xmax**, the CSN corresponding to the XID can be read from the CSNlog.
- If the CSN is smaller than the snapshot CSN, the tuples are visible. Otherwise, the tuples are invisible and a flag is set.
- If the CSN is in the **commit-in-progress** state:
  - If the subtransaction is in the **commit-in-progress** state, read the XID of the top-level parent transaction and go back to step 4 once the top-level parent transaction ends. More information about the subtransaction logic can be found in the following slides.
  - If no subtransactions are in the **commit-in-progress** state, but the transaction of the tuples is being committed, read the CSN again after the transaction is committed, and return to step 4.

HUAWEI

Contents

1. **Transaction Management**
   - Transaction Concept
   - Concurrency Control Management
   - Transaction Status
   - Clog and CSNlog
   - MVCC Mechanism
   - Subtransactions
2. Lock Mechanism

HUAWEI

Subtransaction Overview

- In addition to basic transaction capabilities, openGauss provides subtransaction capabilities. Specifically, it supports operations such as creating a subtransaction (SAVEPOINT), rolling back a subtransaction (ROLLBACK TO SAVEPOINT), and releasing a subtransaction (RELEASE SAVEPOINT).
- openGauss can automatically roll back to the previous savepoint when an error occurs without rolling back the entire transaction. Alternatively, openGauss can roll back to the state of a specific savepoint at any time and abandon some modifications in the transaction.
  - The SAVEPOINT syntax allows you to define a savepoint.
  - The ROLLBACK TO SAVEPOINT syntax allows rollback to a specified savepoint.
  - The RELEASE SAVEPOINT syntax allows you to release a previously defined savepoint. However, releasing a savepoint does not invalidate previous modifications.
- SAVEPOINT syntax

```
SAVEPOINT savepoint_name;
```

  - **savepoint_name**: savepoint name

HUAWEI

Differences and Relationships Between Subtransactions and Transactions

- Here are the relationships and differences between subtransactions and transactions during COMMIT and ABORT processes:
  - A subtransaction must be defined in a parent transaction. A subtransaction cannot be defined independently without defining the parent transaction.
  - A subtransaction or savepoint cannot be committed independently. When a parent transaction is committed, all subtransactions defined in the parent transaction are automatically committed.
  - If there is no subtransaction, the entire transaction must be aborted if there is an internal error or an external ROLLBACK operation is executed.
  - If a ROLLBACK operation is received from an external system, the current parent transaction and all subtransactions defined in the ROLLBACK operation are aborted.
  - If an internal error causes a ROLLBACK operation, the current subtransaction is rolled back without affecting the previously defined subtransactions.

HUAWEI

SAVEPOINT Precautions

- Savepoints can only be established inside a transaction block. Multiple savepoints can be defined within a transaction.
- The transaction cannot be rolled back to the established savepoint if a thread or process is unexpectedly terminated due to a node or connection failure or because of an error caused by inconsistencies between source and destination table structures in a COPY FROM operation. Instead, the entire transaction will be rolled back.
- In SQL, if you create a new savepoint using the SAVEPOINT command and give it a name that already exists, the previous savepoint will be automatically destroyed. In contrast, in openGauss, old savepoints are kept; however, only the most recent one will be used for rollback or release. Also, if the most recent savepoint is released, the previous savepoint will again become accessible to ROLLBACK TO SAVEPOINT and RELEASE SAVEPOINT. In addition, SAVEPOINT fully complies with the SQL standard.

HUAWEI

ROLLBACK TO SAVEPOINT

- Function:
  - Rolls back to a savepoint and implicitly destroys all savepoints that were established after the savepoint.
  - Rolls back all statements that were executed after the savepoint was established. The savepoint remains valid and can be rolled back to again in the future, if needed.
- Note:
  - You cannot roll back to an undefined savepoint. An error is reported in the syntax.
  - Cursors have somewhat non-transactional behavior with respect to savepoints. Any cursor that is opened inside a savepoint will be closed when the savepoint is rolled back. If an open cursor is affected by a FETCH statement inside a savepoint that is later rolled back, the cursor position will not be rolled back and it will continue pointing in the direction set by the FETCH statement. Closing a cursor is not undone by rolling back, either. A cursor whose execution aborts a transaction is placed in a cannot-execute state, so while the transaction can be restored using ROLLBACK TO SAVEPOINT, the cursor can no longer be used.
  - Use ROLLBACK TO SAVEPOINT to roll back to a savepoint. Use RELEASE SAVEPOINT to destroy a savepoint while keeping the effects of the commands executed after the savepoint was established.

HUAWEI

RELEASE SAVEPOINT

- Function:
  - RELEASE SAVEPOINT destroys a savepoint that was previously defined in the current transaction.
  - Destroying a savepoint makes it unavailable as a rollback point, but users will not see any other behaviors. It does not undo the effects of statements executed after the savepoint was established. To do that, use ROLLBACK TO SAVEPOINT. Destroying a savepoint when it is no longer needed allows the system to recycle some resources before the transaction end.
  - RELEASE SAVEPOINT also destroys all savepoints that were established after the named savepoint was established.
- Note:
  - You cannot release an undefined savepoint. An error is reported in the syntax.
  - If a transaction is in a rollback state, the savepoint cannot be released.
  - If multiple savepoints have the same name, only the most recently defined one is released.

Contents

1. Transaction Management
2. **Lock Mechanism**
   - Spinlock
   - LWLock
   - Regular Lock
   - Deadlock Management
   - Lock-free Atomic Operation

HUAWEI

Lock Mechanism

- There are three types of locks in a database.
  - Spinlock
    - The underlying lock, implemented through mutex, is closely related to the OS and hardware environment.
    - The spinlock locks for a short time without a wait queue or deadlock detection. It cannot be automatically released when a transaction terminates.
  - Lightweight lock (LWLock)
    - It provides exclusive access to shared data.
    - The LWLock can be either exclusive or shared.
    - The LWLock has a wait queue, but no deadlock detection. The lock is automatically released when the transaction terminates.
  - Regular lock

HUAWEI

Lock Granularity

- Table granularity
  - LockTable, UnlockTable, LockSession, UnlockSession, LockExtend, UnlockExtend
- Page granularity
  - LockPage, UnlockPage
- Tuple granularity
  - LockTuple, UnlockTuple
- Transaction granularity
  - XactLockTableInsert, XactLockTableDelete
- General object granularity
  - LockDatabaseObject, UnlockDatabaseObject

HUAWEI

Spinlock

- A spinlock is generally implemented by using a test-and-set (TAS) atomic instruction of a CPU. There are only two states: locked and unlocked.
- Only one process can hold a spinlock.
- The difference between a spinlock and a semaphore is that when a process cannot obtain resources, the semaphore puts the process in sleep block state, while the spinlock keeps the process in busy-waiting state.
- The spinlock is mainly used when the locking duration is very short, for example, modifying a flag or reading a flag field within dozens of instructions.
- When writing code, ensure that the spinlock acquisition and release are within the same function.
- Spinlocks are coded to prevent deadlocks, and there is no deadlock detection or wait queue.

HUAWEI

Spinlock-Related Operations

- As spinlocks consume CPU resources, their prolonged and improper use may lead to core dumps. openGauss updates many 32-bit, 64-bit, and 128-bit variables using compare-and-swap (CAS) atomic operations, thereby reducing the reliance on spinlocks.
- Spinlock-related operations are as follows:
  - SpinLockInit: initializes a spinlock.
  - SpinLockAcquire: acquires a spinlock.
  - SpinLockRelease: releases a spinlock.
  - SpinLockFree: destroys a spinlock and clears related resources.

HUAWEI

Contents

1. Transaction Management
2. **Lock Mechanism**
   - Spinlock
   - LWLock
   - Regular Lock
   - Deadlock Management
   - Lock-free Atomic Operation

HUAWEI

LWLock

- LWLocks are implemented by using atomic operations, wait queues, and semaphores. There are shared and exclusive LWLocks.
- Multiple processes can obtain a shared lock at the same time, but an exclusive lock can only be held by one process.
- When a process cannot obtain resources, an LWLock puts the process in sleep block state.
- LWLocks are mainly used when operations in the critical internal section take a long time. Locking and unlocking operations can cross functions but must be released immediately after use.
- LWLocks are coded to prevent deadlocks. However, the code is complex and there are various exceptions, so openGauss provides a deadlock detection mechanism to avoid LWLock deadlocks in various exceptional scenarios.

HUAWEI

LWLock-Related Functions

- LWLockAssign: requests an LWLock.
- LWLockAcquire: acquires a lock.
- LWLockConditionalAcquire: conditionally acquires a lock. If no lock is acquired, **false** is returned and the system does not keep waiting.
- LWLockRelease: releases a lock.
- LWLockReleaseAll: releases all locks. If an error occurs during a transaction, all LWLocks are rolled back and released to prevent residual LWLocks from blocking subsequent operations.

HUAWEI

Contents

1. Transaction Management
2. **Lock Mechanism**
   - Spinlock
   - LWLock
   - **Regular Lock**
   - Deadlock Management
   - Lock-free Atomic Operation

HUAWEI

Regular Lock

- Regular locks are implemented by using a hash table.
- Regular locks support multiple lock modes. The semantics and conflicts between these lock modes are defined by conflict tables.
- Regular locks lock database objects accessed by services.
- Regular locks follow the two-phase locking protocol of the database: acquiring locks during access and releasing them when a transaction is committed.
- A regular lock has a wait queue and provides a deadlock detection mechanism. When a deadlock is detected, a transaction is rolled back.

HUAWEI

SQL Statements Applicable to Different Locks

- openGauss provides eight lock levels for different statement concurrency:
  - Level-1 locks are used for the SELECT operations.
  - Level-3 locks are used for the INSERT, UPDATE, and DELETE operations.
  - Level-4 locks are used for the VACUUM and ANALYZE operations.
  - Level-8 locks are used for DDL statements.

HUAWEI

Lock Conflicts and Concurrency Control

- The table shows the conflicts and concurrency control for the eight lock levels, √ indicating that two lock operations can be performed concurrently.

Lock Level	1	2	3	4	5	6	7	8
1. AccessShareLock	√	√	√	√	√	√	√	-
2. RowShareLock	√	√	√	√	√	√	-	-
3. RowExclusiveLock	√	√	√	√	-	-	-	-
4. ShareUpdateExclusiveLock	√	√	√	-	-	-	-	-
5. ShareLock	√	√	-	-	√	-	-	-
6. ShareRowExclusiveLock	√	√	-	-	-	-	-	-
7. ExclusiveLock	√	-	-	-	-	-	-	-
8. AccessExclusiveLock	-	-	-	-	-	-	-	-

Regular Lock Conflict Table

- A regular lock is used to manage concurrent access at the table level for DML (Insert/Update/Delete/Select) and DDL operations.

No.	Lock Mode	Operation	Conflicting Mode
1	AccessShareLock	SELECT	8
2	RowShareLock	SELECT FOR UPDATE	7,8
3	RowExclusiveLock	INSERT, UPDATE, DELETE, ANALYZE	5,6,7,8
4	ShareUpdateExclusiveLock	VACUUM (non-null)	4,5,6,7,8
5	ShareLock	CREATE INDEX	3,4,6,7,8
6	ShareRowExclusiveLock	——	3,4,5,6,7,8
7	ExclusiveLock	——	2,3,4,5,6,7,8
8	AccessExclusiveLock	DROP TABLE , ALTER TABLE ,REINDEX ,CLUSTER , VACUUM FULL	All

Locks at Different Levels

- Shared and exclusive locks are the simplest and most fundamental of these eight table-level locks. A shared lock can be understood as a read lock, while an exclusive lock functions as a write lock.
  - If a transaction places a read lock on a table, other transactions cannot modify that table.
  - If a transaction places a write lock on a table, other transactions cannot write or read the data.
- The multi-version function has emerged alongside database technology development, providing ACCESS SHARE and ACCESS EXCLUSIVE locks. The keyword ACCESS in the locks is related to the multi-version read feature.
  - With the multi-version function, modifying a row of data does not affect the original row. Instead, a row is created to reflect the changes. Other users cannot see these changes until the transaction is committed.
  - Since the original data row remains unchanged, readers can still access the original data during the modification process.
  - So, data can still be modified after a transaction places an ACCESS SHARE lock on a table. Also, data can still be read after a transaction places an ACCESS EXCLUSIVE lock on a table.

HUAWEI

Locks at Different Levels

- The objects of the abovementioned locks are tables, which can lead to overly broad locking scopes, limiting concurrency in production environments. Row-level locks address this issue using mechanisms like the UPDATE lock to describe the relationship between row-level and table-level locks.
- When you want to modify a row of data in a table, you need to place a lock on the table, such as ROW SHARE or ROW EXCLUSIVE. This places either a shared or an exclusive lock on some of the rows. That's what we often call an UPDATE lock.
- The SHARE UPDATE EXCLUSIVE and SHARE ROW EXCLUSIVE locks are seldom used.
  - SHARE UPDATE EXCLUSIVE is used to protect the schema of a table from being concurrently modified and prevent the VACUUM operation from being executed on the target table.
  - SHARE ROW EXCLUSIVE prohibits any concurrent table modifications. It is an exclusive lock and can be obtained only once in a session.

HUAWEI

Table-Level Locks

- SHARE, ACCESS SHARE, ROW EXCLUSIVE, and ACCESS EXCLUSIVE are the most common table-level locks.
- The following table describes the SQL statements that place locks on tables.

Lock Mode	Description
Share	A shared lock or the CREATE INDEX statement triggers this lock.
Access Share	A shared lock or the SELECT statement triggers an ACCESS SHARE lock on the table it queries. Generally, any read-only table query triggers this lock mode.
Row Exclusive	The UPDATE lock or the UPDATE/DELETE/INSERT statements trigger this lock on a target table. Generally, statements that change table data trigger a ROW EXCLUSIVE lock on a table.
Access Exclusive	This exclusive lock ensures that only one user can access the table. Statements such as ALTER TABLE, DROP TABLE, and TRUNCATE can trigger this lock mode.

HUAWEI

Table-Level Locks

- Different locks are suitable for different SQL statements.
  - Shared locks can only be read after being added. As such, they apply only to CREATE INDEX scenarios. This mode conflicts with ROW EXCLUSIVE and ACCESS EXCLUSIVE locks. The SHARE mode protects data in a table from concurrent changes.
  - Common SELECT statements can be read and written using ACCESS SHARE statements, ensuring database concurrency in the production environment. It only conflicts with ACCESS EXCLUSIVE locks.
  - The ROW EXCLUSIVE lock applies to statements that change table data. It places exclusive locks on the corresponding rows, which conflicts with SHARE and ACCESS EXCLUSIVE modes.
  - The ACCESS EXCLUSIVE lock is at the table level and conflicts with all other lock modes.

HUAWEI

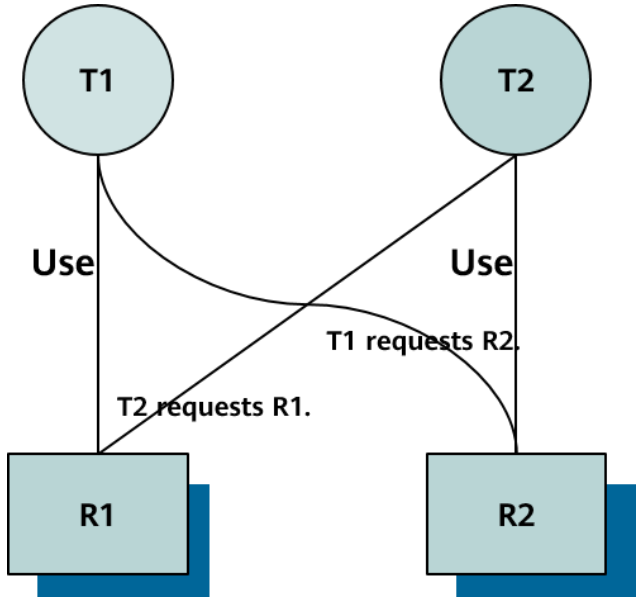
Contents

1. Transaction Management
2. **Lock Mechanism**
   - Spinlock
   - LWLock
   - Regular Lock
   - Deadlock Management
   - Lock-free Atomic Operation

HUAWEI

Deadlock Detection Mechanism

- A deadlock occurs when process B needs to access the resources of process A, but process A does not release the resources used by its lock for various reasons. As a result, the database is always blocked. As shown in the figure below, T1 uses resource R1 and requests resource R2, while T2 holds resource R2 and requests resource R1.

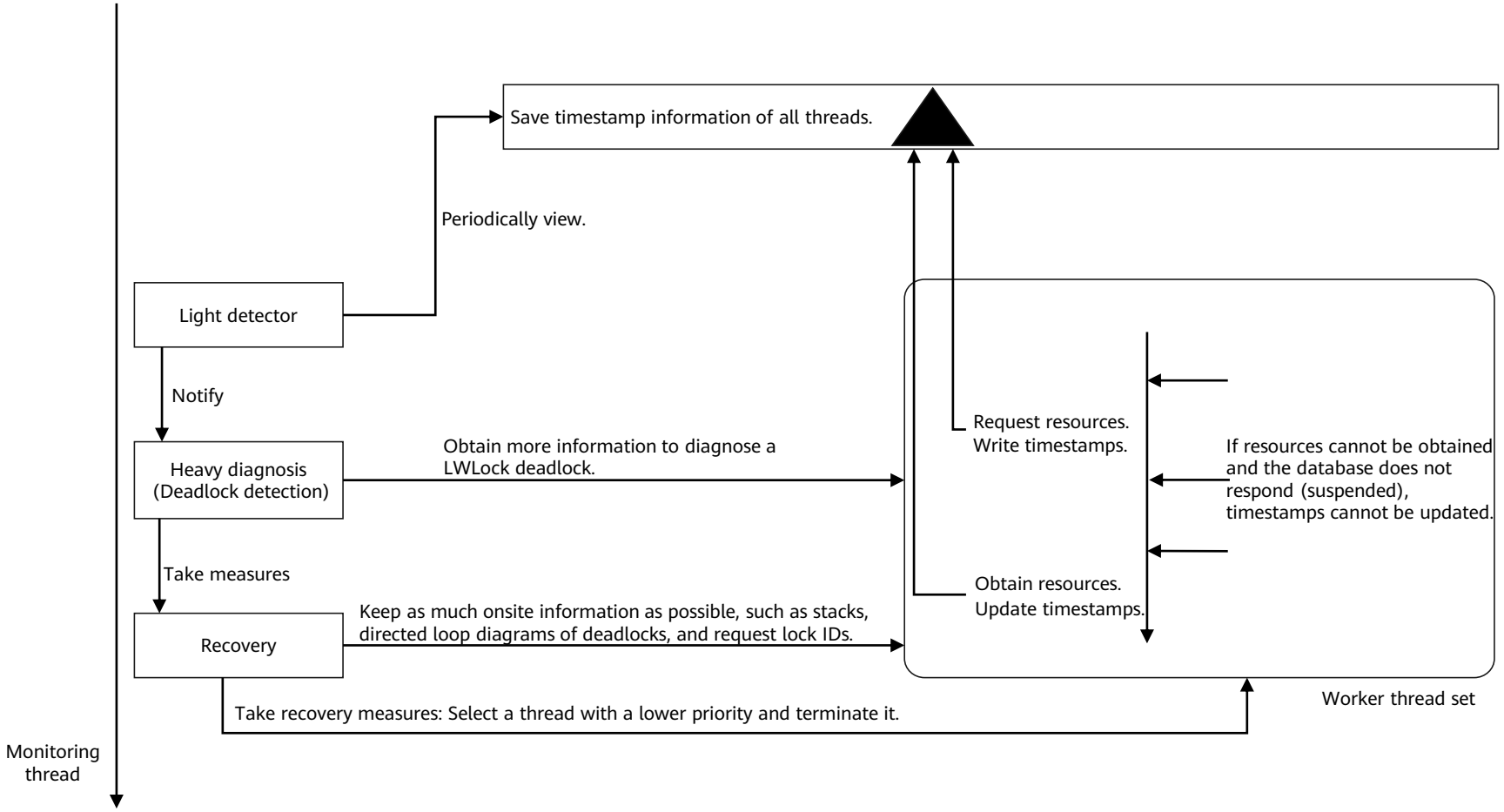


Deadlock Conditions and Handling

- Deadlocks occur under the following conditions:
  - Resource request and hold: Each process can hold one resource while requesting access to another resource.
- Common ways to handle deadlocks:
  - Interruption: Abort one of the transactions to break the circular wait.
- openGauss provides a deadlock detection mechanism for LWLock and regular locks.
- You can use PG_THREAD_WAIT_STATUS to test the blocking status of the backend thread and the auxiliary thread of the current instance.

HUAWEI

LWLock Deadlock Detection and Recovery Process



LWLock Deadlock Detection and Recovery

- openGauss uses an independent monitoring thread to detect, diagnose, and release LWLock deadlocks.
- A worker thread writes a timestamp value before successfully requesting an LWLock. After successfully obtaining the lock, the worker thread sets the timestamp to **0**.
- The monitoring thread can quickly compare the timestamp values to locate the thread that has failed to obtain the lock resource for a long time. This process is fast.
- Deadlock detection diagnosis is triggered only when a long lock wait is detected. This prevents frequent diagnoses from affecting service execution.
- Once a deadlock loop is confirmed, the monitoring thread records the deadlock information in the log and then takes recovery measures to recover the deadlock. That is, the monitoring thread selects a thread in the deadlock loop to report an error and then exit.

HUAWEI

LWLock Deadlock Detection and Recovery

- Detecting actual deadlocks is a CPU-intensive task. As such, lightweight deadlock detection uses a simpler approach to quickly assess the possibility of deadlocks. It uses a watchdog method with timestamps:
  - When a worker thread makes a lock request, it timestamps when it starts waiting in the global memory.
  - When the lock request succeeds, the timestamp is set to **0**.
  - The lock request of a deadlocked thread is in a wait state, and the timestamp is not set to 0. In addition, there is an increasing difference between the timestamp and the current time.
  - The GUC parameter **fault_mon_timeout** specifies the check interval. The default value is **5s**.
  - LWLock deadlock detection is performed at the interval specified by **fault_mon_timeout**. If the same thread with the same lock ID shows a timestamp difference that exceeds the detection interval, it triggers a deadlock detection.

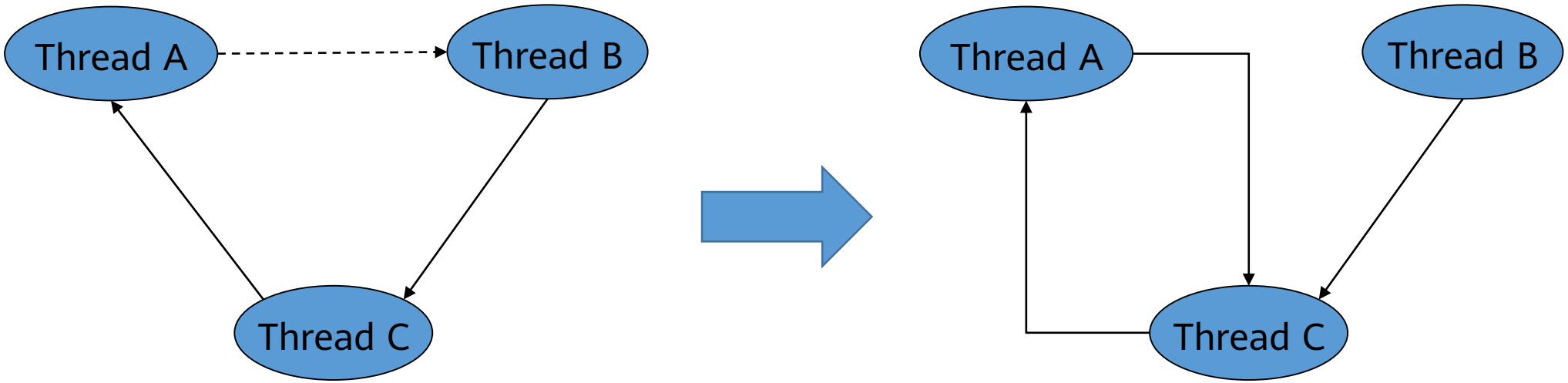
HUAWEI

LWLock Deadlock Detection and Recovery

- The functions for time statistics and lightweight detection are as follows:
  - pgstat_read_light_detect: reads the timestamp related to the thread and lock ID from the statistics structure and records the timestamp to the pointer queue.
  - lwm_compare_light_detect: compares the state to the one several seconds before detection. If it finds threads and lock IDs that may be deadlocked, **true** is returned. Otherwise, **false** is returned.
- Deadlock detection requires the following information:
  - Locks, including request and allocation information.
  - Thread, including waiting and holding information. The information is recorded in the corresponding global variables. The deadlock monitoring thread can access and determine the information.

Deadlock Detection for Regular Locks

- If there is no conflict when openGauss obtains a lock, openGauss directly locks the lock. If there is a conflict, openGauss sets a timer and waits. After the specified period, openGauss is called by the timer to detect deadlock.
- The following figure illustrates deadlock detection for regular locks.



Wait-for graph

HUAWEI

Deadlock Detection for Regular Locks

- If process T2 is behind process T1 in the lock's wait queue, and T2 and T1 need to obtain conflicting locks, there will be a soft edge from T2 to T1. If the lock request from process T2 conflicts with the lock held by process T1, there will be a hard edge.
- Generally, detection involves recursively traversing from the current vertex in the wait-for graph, following the wait edges to see if there is a cycle.
  - If a cycle contains soft edges, it indicates that both processes in the cycle are waiting for locks. In this case, reordering is attempted to resolve the deadlock conflict.
  - If there are no soft edges in the cycle, the only solution is to terminate one of the transactions in the wait-for cycle to resolve the deadlock.
- In the preceding slide, the dashed line indicates a soft edge, and the solid line indicates a hard edge. For example, if thread A is waiting for thread B, thread B is waiting for thread C, and thread C is waiting for thread A, the soft edge between thread A and thread B leads to a reordering of the wait relationships. If, after reordering, thread A is waiting for thread C and thread C is waiting for thread A, and there are no soft edges, then a deadlock is detected.

HUAWEI

Deadlock Handling

- In a database, transactions can request and release locks in any order, so it is crucial to have a mechanism for handling deadlocks.
- Deadlocks occur when two transactions hold locks on different objects and each requests the lock held by the other, leading to a deadlock conflict.
- Deadlock handling is divided into deadlock prevention, detection, and release.
  - Deadlock prevention
    - A process will enter the wait queue if it fails to request a lock. If some processes in the queue need the locks held by the current process, to minimize the risk of deadlock, this process will be inserted ahead of them in the queue.
    - When a lock is released, a process in the wait queue will be woken up. If the process requirement conflicts with another process that is ahead of it but cannot be woken up for some reason, the process will not be woken up.
    - These methods cannot completely eliminate deadlocks, so databases provide deadlock detection mechanisms.

HUAWEI

Deadlock Handling

- Deadlock detection
  - Use the pg_stat_activity view to query information about active transactions, which helps identify and resolve deadlocks. Example:

```
SELECT pid, usename, datname, query, state, query_start, backend_xid, backend_xmin FROM pg_stat_activity
WHERE state = 'active' AND query LIKE '%UPDATE%';
```

  - Use the pg_locks, pg_class, and pg_attribute views to query lock information. Example:

```
SELECT locktype, database, relation::regclass, page, tuple, virtualxid, transactionid, classid, objid, objsubid, transaction
FROM pg_locks WHERE virtualxid IS NOT NULL OR virtualtransaction IS NOT NULL;
```

  - Set **log_statement** to **'all'** to record all SQL statements to investigate deadlock causes.

```
ALTER SYSTEM SET log_statement = 'all';
```

- Deadlock release
  - Use the pg_terminate_backend function to forcibly terminate the backend process that causes the deadlock. Note that this should be used as a last resort because it interrupts an executing transaction.

```
SELECT pg_terminate_backend(pid);
```

  - Use the pg_ctl tool to restart the database. This is the last method you should use to handle severe deadlocks.

Deadlock Locating

- These methods cannot automatically locate all deadlocks, so a database provides many views to display the database's internal status for users to manually locate faults.
- The following views are often used to locate deadlocks:
  - pg_stat_activity: shows the status of each session on the current instance.
  - pg_thread_wait_status: shows the wait events of each thread on the current instance.
  - pgxc_thread_wait_status: shows the wait events of all the threads on the current instance.
  - pg_locks: shows the status of locks on the current instance.
  - pgxc_node: shows the IP addresses and ports of all the instances in the cluster.

HUAWEI

Lock Management Parameters

Parameter	Description	Value Range	Default Value
deadlock_timeout	Specifies the amount of time, in milliseconds, to wait on a lock before checking whether there is a deadlock condition. When the requested lock exceeds the preset value, the system will check for a deadlock. This parameter takes effect only for regular locks.	An integer ranging from 1 to 2147483647. The unit is ms.	1s
lockwait_timeout	Specifies the maximum wait time for a single lock, in milliseconds. If the time spent waiting for a lock exceeds the specified time, an error is reported. This parameter takes effect only for regular locks.	An integer ranging from 0 to *INT_MAX*. The unit is ms.	20 min
update_lockwait_timeout	Specifies the maximum duration, in milliseconds, that a lock waits for concurrent updates on a row to be completed when the concurrent update feature is enabled. If the time spent waiting for a lock exceeds the specified time, an error is reported. This parameter takes effect only for regular locks.	An integer ranging from 0 to 2147483647. The unit is ms.	2 min (120000 ms)
max_locks_per_transaction	Determines the average number of object locks allocated for each transaction.	An integer ranging from 10 to *INT_MAX*.	256
max_pred_locks_per_transaction	Specifies the average number of predicate locks allocated for each transaction.	An integer ranging from 10 to *INT_MAX*.	64
gs_clean_timeout	Specifies the average interval for clearing temporary tables on the primary node.	An integer ranging from 0 to 2147483. The unit is second.	1 min
partition_lock_upgrade_timeout	Specifies the timeout for attempts to upgrade an exclusive lock (read allowed) to an access exclusive lock (read/write blocked) on a partitioned table while some query statements are executed. If there are concurrent read transactions running, the lock upgrade will need to wait. This parameter sets the waiting timeout for lock upgrade attempts.	An integer ranging from –1 to 3000. The unit is second.	1800

Lock Management Parameters

Parameter	Description	Value Range	Default Value
fault_mon_timeout	Lightweight deadlock detection period. This is a SIGHUP parameter.	An integer ranging from 0 to 1440. The unit is minute.	5 min
enable_online_ddl_waitlock	Specifies whether to block DDL operations to wait for database locks to be released, such as pg_advisory_lock. This parameter is mainly used in online OM operations, and you are advised not to modify the settings.	Boolean	off
xloginsert_locks	Specifies the number of locks on concurrent write-ahead logging. This is used to improve the efficiency of writing write-ahead logs.	An integer ranging from 1 to 1000.	16
num_internal_lock_partitions	Specifies the number of internal lightweight lock partitions. It is mainly used for performance optimization in various scenarios. The content is organized in the KV format using keywords and numbers. Different types of locks are separated by commas (,). The sequence does not affect the setting result. For example, **CLOG_PART=256,CSNLOG_PART=512** is equivalent to **CSNLOG_PART=512,CLOG_PART=256**. If you set the same keyword multiple times, only the latest setting takes effect. For example, if you set **CLOG_PART** to **256** and **CLOG_PART** to **2**, the value of **CLOG_PART** is **2**.	Character string	•

Contents

1. Transaction Management
2. **Lock Mechanism**
   - Spinlock
   - LWLock
   - Regular Lock
   - Deadlock Management
   - Lock-free Atomic Operation

HUAWEI

Lock-free Atomic Operation

- openGauss encapsulates 32-bit, 64-bit, and 128-bit atomic operations, primarily to replace spinlocks and perform atomic updates on simple variables.
  - Currently, in databases or file systems, write-ahead logging (WAL) needs to insert generated log information into the log cache. To achieve high-speed logging, the log management system performs concurrent inserts by reserving global positions. Typically, two 64-bit global data position indexes are used to represent the starting and ending positions of the stored inserts, providing support for data indexes of up to 16 exabytes. To protect the global position indexes, WAL introduces a high-performance atomic lock to safeguard each log cache position. In NUMA architectures, especially Arm architectures, the performance differences in atomic lock backoff and high cross-CPU access latency make cache consistency a bottleneck for WAL concurrent cache protection.
  - WAL INSERT 128CAS lock-free critical section protection: In this approach, an atomic lock is replaced with information about the two 64-bit global data positions through 128-bit atomic operations. This eliminates the overhead associated with atomic locks, including cross-CPU access, atomic lock backoff, and cache consistency costs.
    - The global position information includes a 64-bit start address and a 64-bit end address, which are combined into a single 128-bit value. This value is reserved using CAS atomic operations without locking.
    - On Arm platforms that do not natively support 128-bit atomic operations, openGauss uses the **exclusive** command to load two 64-bit Arm data values, utilizing Arm64 assembly instructions LDXP/STXP.

HUAWEI

Quiz

1. (Single-answer question) Which of the following views is used to query the wait event of each thread on an instance?

   A. pg_thread_wait_status

   B. pg_stat_activity

   C. pgxc_thread_wait_status

   D. pg_locks

2. (True or False) The difference between a spinlock and a semaphore is that when a process cannot obtain resources, the spinlock puts the process in sleep block state, while the semaphore keeps the process in busy-waiting state.

   A. True

   B. False

HUAWEI

Summary

Through this chapter, we have gained a comprehensive and in-depth understanding of database transaction management and locking mechanisms. It covers the working principles of transactions, the definitions and transitions of different transaction states, XID allocation, and transaction-related logging. Additionally, it explores the working principles of the lock mechanisms closely related to transactions, including lock types, their uses and functions, deadlock detection and management, and lock-free atomic operations.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/en/
- openGauss Help (5.0.0)
  - https://docs-opengauss.osinfra.cn/en/docs/5.0.0/docs/GettingStarted/GettingStarted.html

Thank you.

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 2 Storage Management



HUAWEI

Foreword

- This chapter covers three aspects of storage management in openGauss: storage engines, data storage structures, and high concurrency in thread pools. The section on engines discusses row-store and column-store engines, vectorized execution, hybrid row-column storage engines, and adaptive compression. Data storage structure addresses the structure of openGauss data files, how to update and delete tuples, and the principles and use of the pageinspect plug-in. Finally, we explain the features, feature enhancements, concurrency principles, and parallel querying of high concurrency in thread pools.

HUAWEI

Objectives

- Upon completing this course, you will be able to:
  - Understand the differences between row-store and column-store engines, and when to use them.
  - Comprehend the structure of openGauss data files and how to update and delete tuples.
  - Grasp the principles and use of the pageinspect plug-in.
  - Master the concurrency principles of thread pools and related enhancement features.

HUAWEI

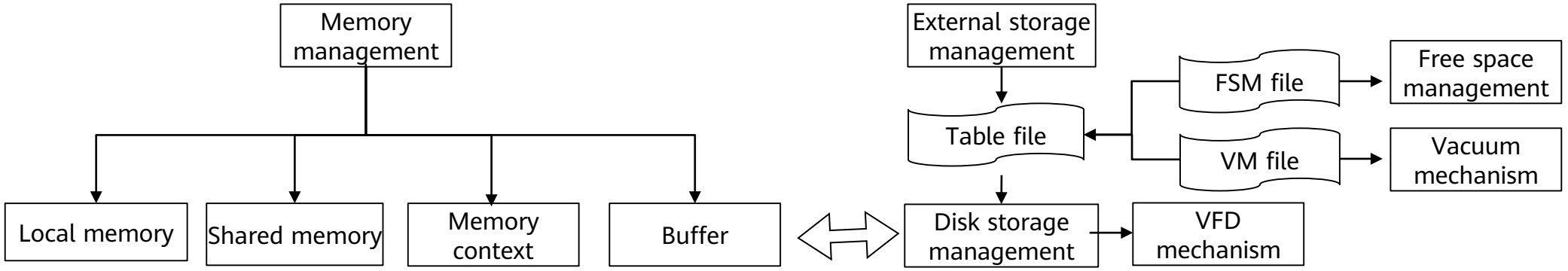
Contents

1. **Storage Engines**
2. Data Storage Structures
3. High Concurrency in Thread Pools

HUAWEI

Storage Manager Architecture

- Overview
  - A storage manager manages the interactions between memory and external storage. Its other major task is to coordinate and plan the memory.
  - Memory management includes shared and local process memory.
  - External storage management includes table files, free space, virtual file descriptor (VFD), and big data storage.



Main Tasks of the Storage Manager

- **Buffer pool management:** Database transactions frequently access data. To reduce read and write operations on disks, data is first stored in the buffer pool during transaction execution. The buffer pools include buffer pools shared between processes (shared buffer pools) and private buffer pools (local buffer pools).
- **Cache mechanism:** System data recently used by processes is cached in the private memory. Its level is higher than the buffer pool.
- **VFD management:** A VFD manages physical files, helping avoid errors caused when the OS limits the number of files opened by a process.
- **Free space management:** This function is used to quickly locate the free space in a table file to insert new data, improving space utilization.
- **Inter-process communication (IPC):** IPC is used to communicate and transfer messages between multiple background processes. It also manages shared memory.
- **Big data storage management:** This function provides large object and TOAST mechanisms. The large object mechanism is a large data storage method controlled by users. Users can call functions and insert large files into a table using SQL statements. The TOAST mechanism is automatically triggered when users insert data longer than a certain limit. Users cannot control the TOAST mechanism.

HUAWEI

Row-Store and Column-Store Tables

- Row-store table
  - Row-store tables are created by default. Data is chronologically stored in rows. This method is used when data needs to be updated frequently.

```
openGauss=# CREATE TABLE customer_t1
(
state_ID CHAR(2),
state_NAME VARCHAR2(40),
area_ID NUMBER
);
-- Drop the table.
openGauss=# DROP TABLE customer_t1;
```

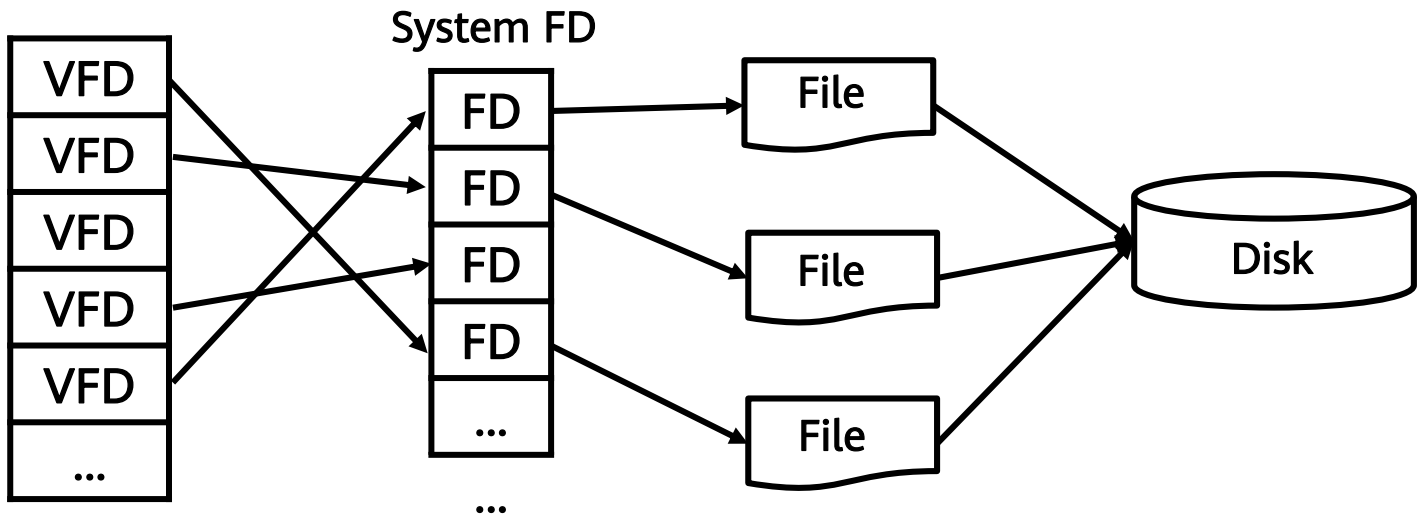
- Column-store table
  - In a column-store table, data is chronologically stored by column. Querying data in one column uses little I/O, and column-store tables occupy less storage space than row-store tables. This storage model is ideal when data is inserted in batches, is not updated frequently, and is queried for statistical analysis. A column-store table cannot be used for point queries.

```
openGauss=# CREATE TABLE customer_t2
(
state_ID CHAR(2),
state_NAME VARCHAR2(40),
area_ID NUMBER
)
WITH (ORIENTATION = COLUMN);
-- Drop the table.
openGauss=# DROP TABLE customer_t2;
```


HUAWEI

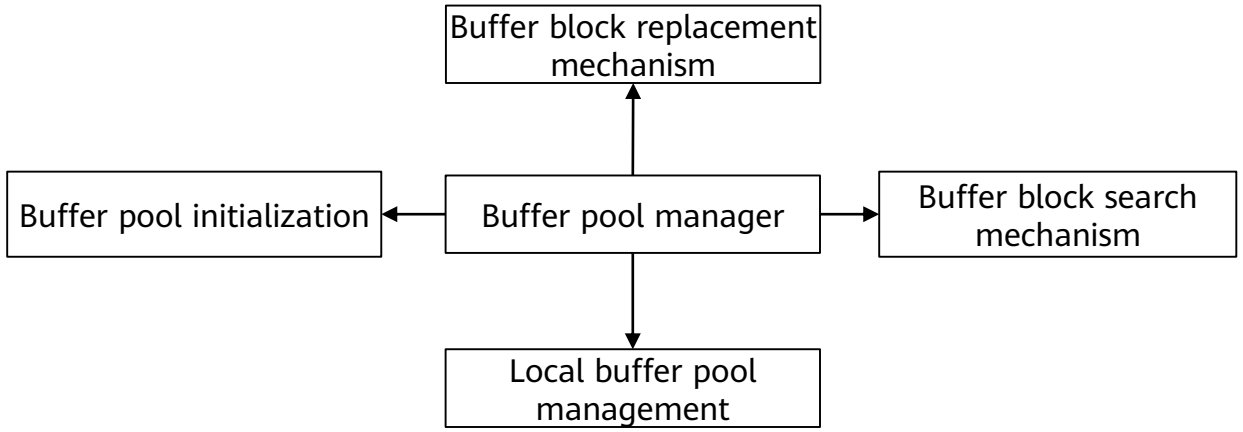
VFD Mechanism

- Implementation Principle
  - Similar to a connection pool, when a process requests to open a file, a VFD is always returned, and the actual operation of opening a physical file is encapsulated externally.
  - The OS specifies the maximum number of files a process can open simultaneously.
  - Processes using VFDs perceive that they can open an arbitrary number of files, independent of the OS restrictions.
  - When multiple processes operate on the same file, each process obtains a real file descriptor that corresponds to a VFD. The following figure shows their relationships.



Buffer Pool Management

- Overview
  - Uses: A system catalog tuple or a table tuple to be accessed cannot be found in the cache.
  - Any operations on tables, tuples, and index tables are performed in the buffer pool.
  - Buffer pool data scheduling uses disk blocks as units.
  - The disk blocks that are transferred to the buffer pool are called buffers, buffer blocks, or pages. Multiple buffers form a buffer pool. The figure shows the structure of the buffer pool management module.
- Types:
  - Shared buffer pools are used to operate common shared tables.
  - Local buffer pools are used to operate temporary tables that are visible only locally.



LRU Pool

- The last recently used (LRU) pool is used to manage all enabled VFDs. The allocation and recycling processes are as follows:
  - When opening the first file, the process initializes the VfdCache array and allocates memory space for each VFD structure.
  - Allocate a VFD by taking one from the FreeList header, open the file, and record the information about the file in the allocated VFD.
  - If there is no idle VFD in the FreeList, double the size of the VfdCache array and add the new VFD to the FreeList linked list.
  - When closing a file, insert the corresponding VFD into the FreeList header.
- LRU pool operations
  - Deleting a VFD from an LRU pool: Use the LruDelete function to delete a VFD from an LRU pool when you finish using a file and close it.
  - Inserting a VFD into an LRU pool: Use the LruInsert function to open a new VFD.
  - Deleting the VFD at the end of the LRU pool: Use the ReleaseLruFile function to delete the VFD at the end of the LRU pool.

HUAWEI

Free Space Management

- Overview
  - When a tuple is inserted into a page, the idle space of the existing page is preferentially used instead of a new data block.
  - You can use additional free space to manage modules. You can quickly check the amount of free space on each page in the table.
  - Each table has independent FSM space, which is represented as a physical file with the suffix _fsm.
- Free space mapping table
  - The FSM structure uses only one byte to record the page's free disk size

  to quickly find proper data blocks, reduce I/O overheads caused by the search, and save the FSM file space. As such, 256 types of free disk sizes can be recorded. The range of free spaces is shown on the right.

Range	Category
0–31	0
32–63	1
…–…	…
8086–8127	253
8128–8163	254
8164–8192	255

HUAWEI

FSM Page Structure

- For quick searches, FSM uses the maximum heap structure, and FSM blocks use the three-layer tree structure.
  - Layers 0 and 1 are auxiliary. The FSM block at layer 2 is used to store the idle value of each table block.
  - There is only one FSM block at layer 0, which serves as the root.
  - There may be a plurality of FSM blocks at layer 1, which are used as the child nodes of FSM blocks at layer 0.
  - There may be a plurality of FSM blocks at layer 2, which are used as the child nodes of FSM blocks at layer 1.
- A binary tree structure (big root heap) stores the records indicating the size of free space. Leaf nodes store the actual space size records. Non-leaf nodes are only used for auxiliary queries. When querying whether there is a proper data block size, you only need to compare the root nodes of the tree. This significantly speeds up queries. The following is an example of the big root heap data structure:
  - 4
  - 4 2
  - 3 4 0 2 <- The leaf node indicates the free space of the heap page.

About half of all nodes (about 4000) are leaf nodes. The page number of each relation ranges from 0 to 0xFFFFFFFE. Therefore, about 2^32/4000 FSM pages are required for storage.

FSM File Operations

- Create: An FSM file is created when it is needed. That is, the FSM file is searched for the first time a VACUUM operation is performed or to insert a tuple.
- Search: The fsm_search function uses the FSM file to search for a table block with the specified free space. For details about the algorithm, see the next slide.
- Adjust: After finding a table block with the specified free space from the FSM file and inserting data using the table block, you can modify the related information in the FSM file.

HUAWEI

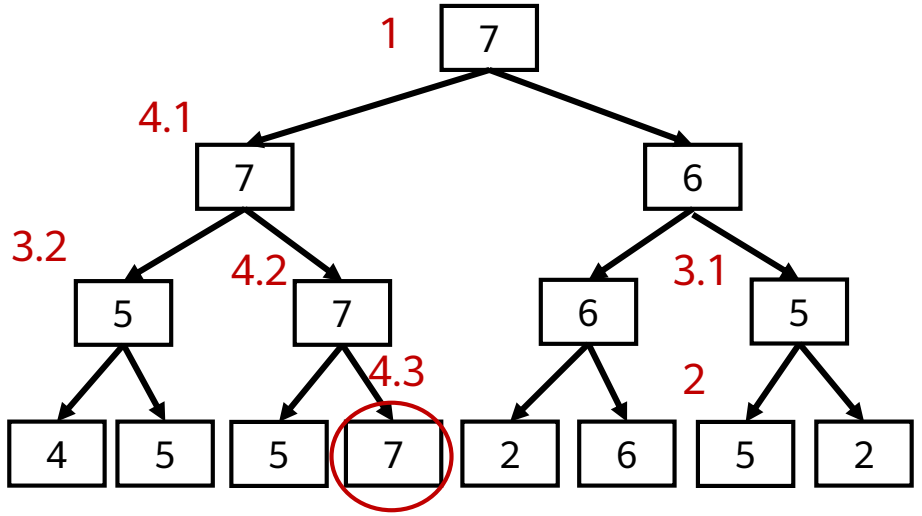
Idle Data Block Search Algorithm

- Algorithm procedure
  1. Compare the root node value. If the value to be queried is greater than the root node value, return the value, indicating that no map value meets the condition in the FSM data block. Otherwise, go to the next step.
  2. Compare the map values corresponding to the queried start position (slot). If the conditions are not met, go to the next step. Otherwise, go to step 5.
  3. Set the new query position to the parent node of the next slot (slot number + 1. The slot number indicates the sequence number of the leaf node). Then, compare the parent node with the next slot. If the condition is not met, repeat this step until the root node is found. If an intermediate node that meets the conditions is found, go to the next step.
  4. Find the leaf node that meets the conditions and go to the next step.
  5. Reset the *fp_next_slot* variable for the next query and return the slot of the leaf node.

HUAWEI

Idle Data Block Search Algorithm

- As shown in the figure on the right, it is assumed that *fp_next_slot* refers to the seventh node from the left of the bottom layer. If a table block whose free space value is **6** needs to be found, the procedure is as follows:
  - (1) **value(1,1)=7**, which is greater than 6, indicates that the FSM block has a table block that can meet the requirement.
  - (2) **value(4,7)=5**, which is less than 6. Find the parent node (3,4) on the right.
  - (3) **value(3,4)=5**, which does not meet the requirement. Find the node (3,1) and the parent node (2,1).
  - (4) **value(2,1)=7**, which is greater than 6. Find the path "node (2,1) -> node (3,2) -> node (4,4)" from node (2,1); that is, the fourth leaf node meets the condition.
  - (5) The value of the slot is **3** (= 4 – 1) because the slot number starts from 0.



Visibility Mapping Table

- Functions of the visibility map (VM)
  - Used to optimize the vacuum execution efficiency. VACUUM is not performed on all visible pages.
  - The index-only scan does not perform heap fetch on all visible pages.
- Criteria for all visible
  - There are no tuples to be deleted on the page.
  - No tuple is being inserted on the page.
  - HEAP_XMIN_COMMITTED of all tuples on the page has been set to prevent breakdown and inconsistency during asynchronous commit.
  - The xmin of all tuples on the page is less than the oldest xmin.

HUAWEI

Column-Store Engine

- Traditional row store has a low compression rate, and data must be read by row. Even if one column is queried, the entire row must be read.
- Column store has the following advantages:
  - Data in the same column is similar and is suitable for compression with a high compression ratio.
  - There can be many columns, but if only a few need to be accessed, column store can significantly reduce unnecessary I/O reads and improve performance.
  - Column-based batch data computing has a high CPU cache hit ratio and good performance.
- The column-store engine is more suitable for OLAP big data analysis.
- Column store can only read the cache. The size of each CU read is not fixed, requiring dynamic memory allocation.

HUAWEI

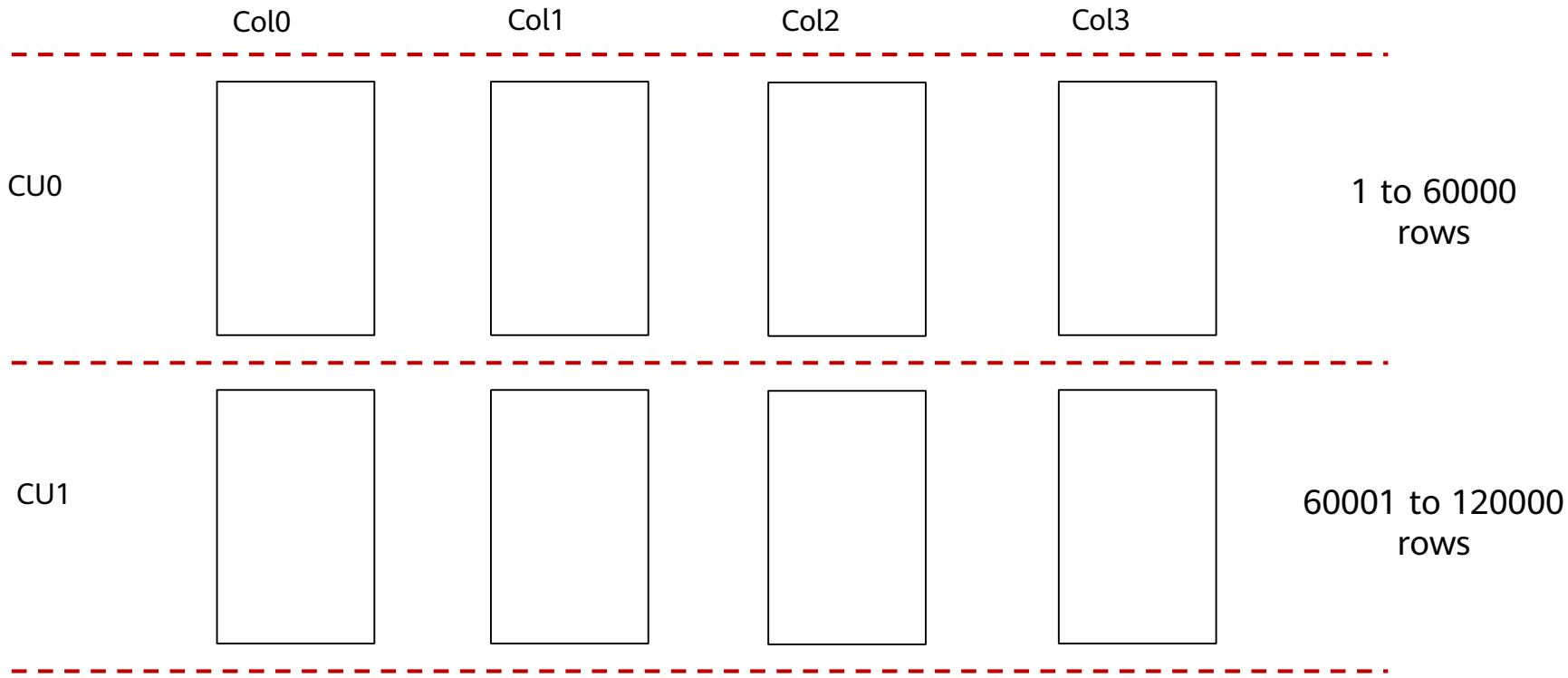
Main Concepts of Column Store

- CU
  - A compression unit (CU) is the minimum unit for column store. It is generated when data is imported and cannot be changed afterward. A single CU can store up to 60,000 rows in one column. CUs of the same column are stored consecutively in a file. When the file size exceeds 1 GB, CUs are stored in a new file.
- CUDesc
  - In a row-store table, each row records a CU, including the maximum value, minimum value, offset, and size of the CU in the file. Rows with **col_id=-10** are VCUs. **cu_pointer** records rows deleted by using **Delete BitMap** in the group of CUs (with the same **cu_id**).
- Delete Bitmap
  - When a row in a CU is deleted, the corresponding bit is set to **1**.
- Delta
  - Delta refers to small data insertions managed within an in-memory incremental merge tree, specifically for single-row inserts (INSERT INTO VALUES).
- Ctid
  - Ctid is used to identify a row in a column-store table. It consists of **cu_id** and **offset** in a CU. It is compatible with row-store tables.

HUAWEI

Column-Store CU

- Implementation
  - Each column can be separately compressed with a high compression ratio.
  - The values of each column are reconstructed into one row at a low cost.



Data Compression

- Data compression algorithms for column store
  - Data can be compressed using algorithms such as delta value encoding, dictionary, RLE, LZ4, and ZLIB. The system automatically selects a compression algorithm based on the data characteristics. The average compression ratio is 7:1. Compressed data can be accessed without decompression and is transparent to services. This allows faster access to historical data.

Category	Data Length	Compression Algorithm	Description
Fixed-length types	Integer type. Length: 1, 2, 4, or 8 bytes.	Delta+RLE hybrid compression	Delta and RLE methods of the time interval data type, and delta compression of the numeric data type
	12-byte time interval	RLE	
	64-byte name	LZ4/Zlib compression algorithm	
Variable-length types	Numeric	Delta+RLE hybrid compression	
	Character string	Dictionary+RLE hybrid compression LZ4/Zlib compression algorithm	

HUAWEI

Delta Table

- Overview
  - Column-store tables include a delta table, but it is not enabled. As a result, a new CU is generated each time a single record is imported, which deteriorates performance and reduces the effectiveness of the compression mechanism. As a result, data expansion affects query performance and disk usage.
  - To solve these issues, openGauss supports column-store delta tables. Data is stored in the delta table when a single record or a small batch of data is imported. This avoids small CUs, significantly improves performance when importing a single record to the column-store table, and resolves the data expansion problem caused by small CUs.
- Status
  - Delta table disabled: A new CU is generated each time a single record is imported. Performance deteriorates, and the compression mechanism becomes less effective. As a result, data bloat affects query performance and disk usage.
  - Delta table enabled: Data is stored in the delta table when a single record or a small batch of data is imported. This avoids small CUs, significantly improves performance when importing a single record to the column-store table, and resolves the data expansion problem caused by small CUs.

HUAWEI

Delta Table

- Objectives
  - Column-store tables support the create, read, update, and delete (CRUD) operations on the delta table.
  - Delta tables result in three times higher performance when importing a single record to a column-store table, which is only 10% less efficient than importing it into a row-store table. When many records are imported (for example, over 1 million), using delta merge significantly reduces disk space usage (the specific ratio depends on the data type).

HUAWEI

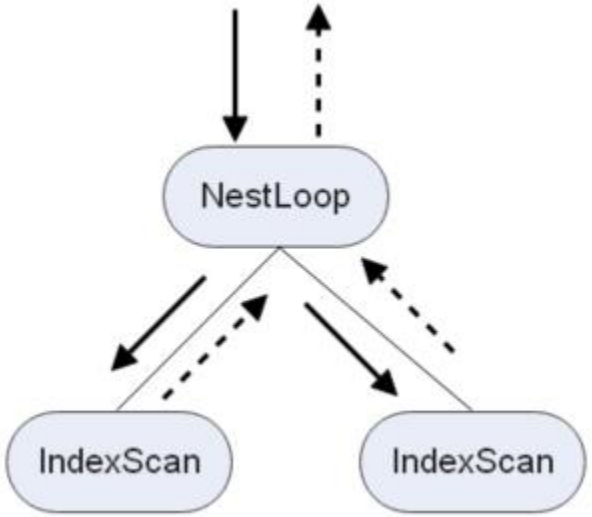
Basic Operation Implementation

- Insert
  - Delta store uses the PostgreSQL row-store table to cache the inserted single records and then periodically or manually merges the data into the stable store.
- Delete
  - The delta store uses the PostgreSQL row-store table and uses the same deletion method.
  - For each row group, a bitmap is introduced to indicate which rows are deleted. The value 1 indicates deleted.
- Update
  - Data is updated in the Delete+Insert manner. The key is to implement deletion.
- Bulkload
  - Bulkload is directly written to the stable store. If the data volume is small, the data is written to the delta store.
- Min/Max sparse index
  - Each CUDesc stores the minimum and maximum values of a CU. During a query, min/max determination can be performed based on the query condition. If the query condition is not in the range of (min, max), the CU does not need to be read.

HUAWEI

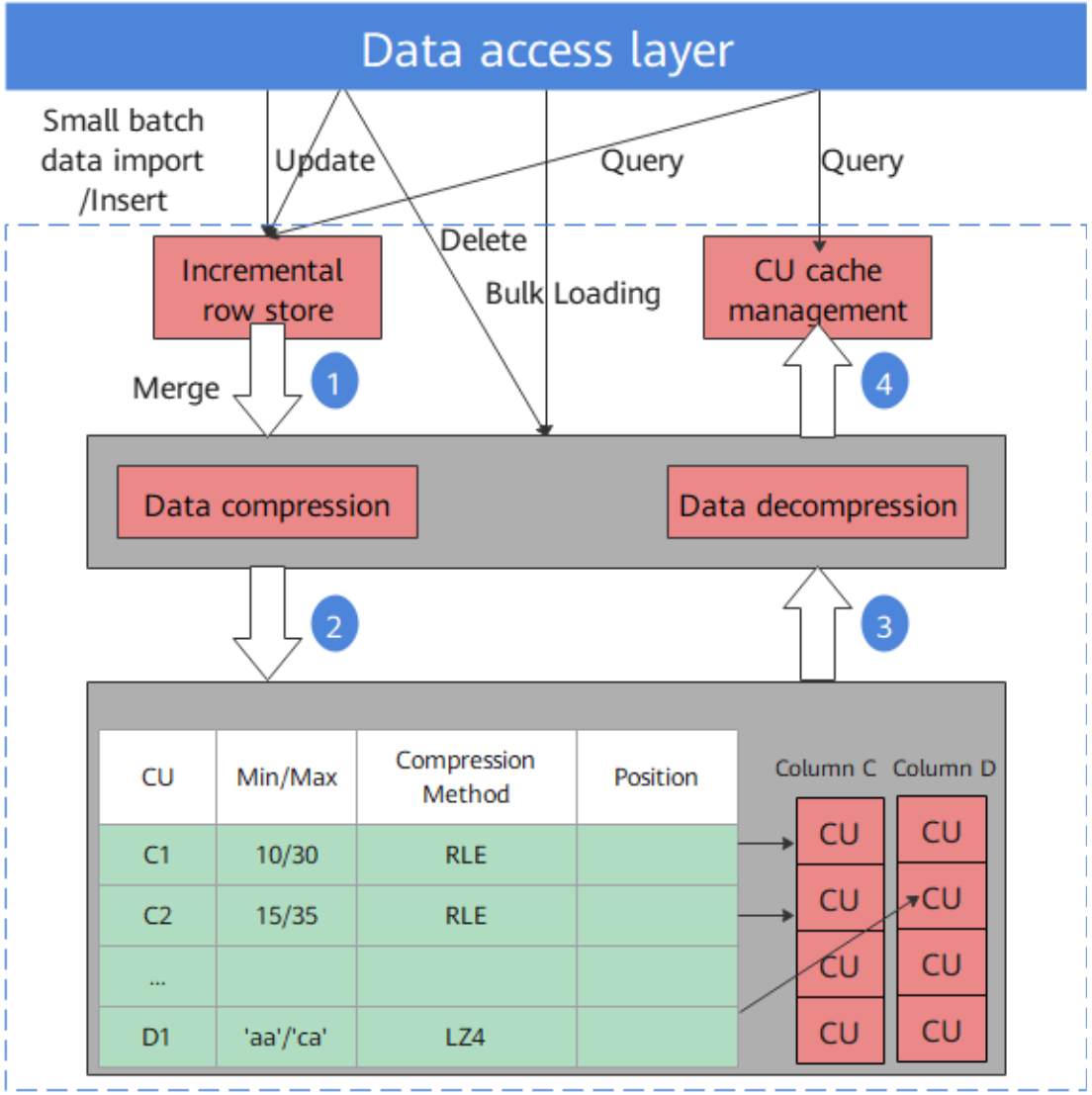
Vectorized Execution

- The figure shows a standard vectorized executor.
  - Control flow travels downward (solid lines), and data flow travels upward (dotted lines). The upper-layer node calls the lower-layer nodes for data, and each lower-layer node returns only one tuple to the upper-layer node at a time.
  - In contrast, the vectorized executor returns a batch of tuples each time, which significantly improves executor performance with the aid of the column store feature.



Hybrid Row-Column Storage Engine

- Generally, column store is used for analysis services. This means you need to perform frequent range statistics queries and batch import operations but rarely perform update, deletion, point query, and point insertion operations. The table contains many columns, which is a wide table. Only a few columns are involved in each query.
- OLTP services tend to use row store tables. In this case, we often use query, insert, delete, and update operations but rarely need to perform range statistics queries or batch import operations. There are few table columns. Most columns are queried.
- As shown in the figure, the hybrid row-column storage engine provides users with a better data compression ratio (column store), index performance (column store), and point update and point query (row store) performance.



HUAWEI

Column-Store Engine Restrictions

- Among DDL statements, only CREATE TABLE, DROP TABLE, and TRUNCATE TABLE are supported.
  - Supports partition management using DDL statements (such as ADD PARTITION, DROP PARTITION, MERGE PARTITION, and EXCHANGE).
  - Supports the CREATE TABLE LIKE statement.
  - Partially supports the ALTER TABLE syntax.
  - Does not support other functions.
- Among DML statements, only UPDATE, COPY, BULKLOAD, and DELETE are supported.
- Does not support triggers, primary keys, or foreign keys.
- Supports the Psort index, B-tree index, and GIN index.

HUAWEI

Fusion Storage Engine

- The fusion engine architecture supports a pluggable storage framework using the in-place update storage engine Ustore. It enriches transaction information by versioning indexes. Lock-free Xlog updates significantly enhance the Xlog write performance. Parallel page replay improves replay efficiency on standby. Additionally, the enterprise-grade flashback feature provides a stable environment for users to query data.
  - In-place Update Storage Engine
    - Ustore uses undo segments to address issues related to space bloat and large tuples during append updates.
  - Index Versioning
    - By versioning index tuples on pages, UB-tree can check tuple visibility at the index layer based on the MVCC mechanism. With the version information, the UB-tree independently determines whether an index tuple is dead so that the in-place update engine can implement page-level space recycling for metadata and index tables and build a space recycling mechanism independent of AutoVacuum.

HUAWEI

Fusion Storage Engine

- Parallel Page Replay
  - This feature optimizes the Ustore in-place update WAL write and Ustore DML operation parallel replay and distribution. Prefixes and suffixes are used to speed up WAL updates. Since Ustore usually replays a range of WAL pages in a DML operation, replayer threads are further classified to facilitate targeted log replay. In addition, Ustore data pages are distributed by block number (**blkno**) during the replay process to improve the degree of parallelism.
- Enterprise-Grade Flashback
  - Flashback is a database recovery technology. It enables DBAs to selectively and efficiently cancel the impact of a committed transaction and restore data incorrectly modified due to manual operations. Before flashback, a committed database modification could only be retrieved by means of backup and restoration or PITR. The restoration took several minutes or hours. With flashback technology, you can restore committed data in a database to its previous state in just seconds, regardless of the database size.

HUAWEI

Fusion Storage Engine

- Enterprise-grade flashback supports the following flashback modes:
  - Flashback query: queries a snapshot of a table at a certain time point. This feature can be used to view and logically rebuild data that was accidentally deleted or modified. The flashback query feature is based on the MVCC mechanism. You can query and retrieve an earlier version of the data.
  - Flashback table: restores a table to a specific time point. When only one table or a group of tables is logically damaged (not the entire database), this feature can be used to quickly restore the table data. Using the MVCC mechanism, the flashback table feature removes incremental data added after a specified time point and restores data that was deleted at or before that time point.
  - Flashback DROP: restores accidentally deleted tables and their auxiliary structures, including indexes and table constraints, from the recycle bin. Flashback DROP is based on the recycle bin mechanism. You can restore dropped physical table files from the recycle bin.
  - Flashback TRUNCATE: restores accidentally truncated tables, including their physical data and indexes, from the recycle bin. Flashback TRUNCATE is based on the recycle bin mechanism. You can restore truncated physical table files from the recycle bin.

HUAWEI

Contents

1. Storage Engines
2. **Data Storage Structures**
3. High Concurrency in Thread Pools

HUAWEI

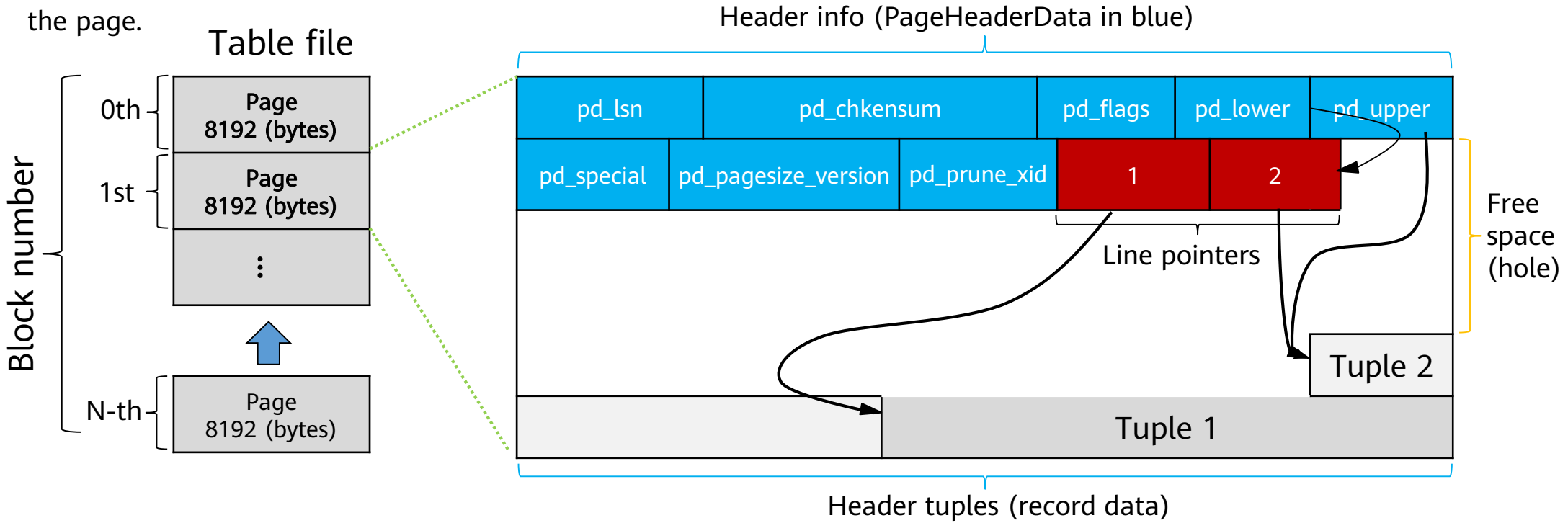
Table Page Storage Structure

- The storage file of each table is divided into several pages, with each page storing a certain amount of data.
  - At the beginning of each page, there is a PageHeaderData structure that contains metadata about the page, such as the page version number, checks, and flags.
  - Subsequent data is stored in the body of the page, organized into tuples, with each tuple having a corresponding ItemIdData structure.
    - This structure contains metadata about the tuple, such as its size and whether it has been deleted.
    - The actual data for each tuple is stored in the HeapTupleHeaderData structure, which includes metadata about the tuple, such as its version number and multi-tuple transaction ID.
- The page storage structure of openGauss tables represents an efficient physical storage method for relational data, enabling fast storage and retrieval by organizing metadata and actual data.

HUAWEI

Data File Structure

- A table file consists of a series of page blocks. A page consists of the following elements:
  - Heap tuple(s): A heap tuple is used to store specific data and is located at the bottom of the page.
  - Line pointer(s): The length of a line pointer is 4 bytes, including the pointer to the heap tuple. The top of the page contains the line pointer array.
  - Header data: indicates the page header information, which is represented by the PageHeaderData structure at the beginning of the page.



HUAWEI

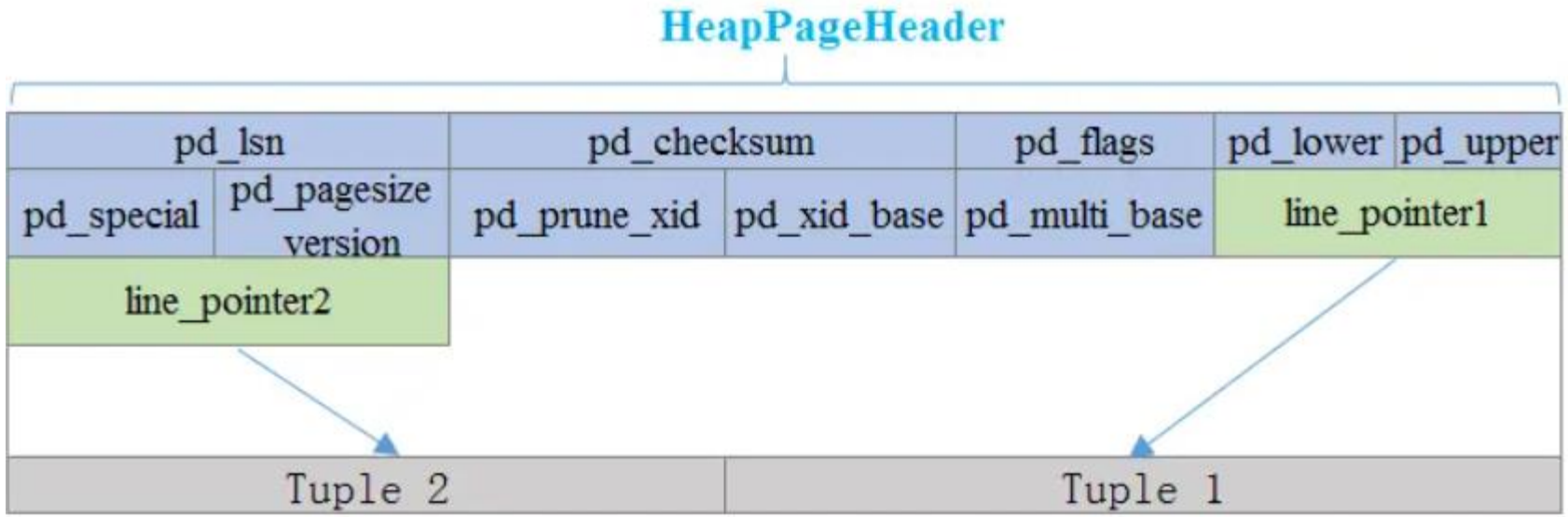
Data File Structure

- The data in openGauss tables is physically stored on non-volatile storage devices (such as disks or solid-state drives). The data in a table is distributed across *N* data files, with each data file composed of *N* pages (the default size is 8 KB, which can be specified during compile-time installation). Pages are the smallest access unit in openGauss.
- Data files (including heap tables and indexes) are divided into fixed-length pages, also referred to as blocks, with a default size of 8192 bytes (8 KB). Pages within each file are sequentially numbered starting from 0, and these numbers are known as block numbers. If a data file becomes full, openGauss extends the file by appending a new empty page at the end.

HUAWEI

Heap Table Page Structure

- In openGauss, row store uses the heap table storage method. A heap table is characterized by the unordered storage of tuples, with data being stored in the free space within pages on a "first-come, first-served" basis. Since the default media manager for row store is the disk file system, it uses a segment-page design similar to the file system, where the smallest I/O unit is a page. This design provides good I/O performance and lower I/O overhead in most scenarios. A heap table page has a default size of 8 KB, and its structure is illustrated as follows:



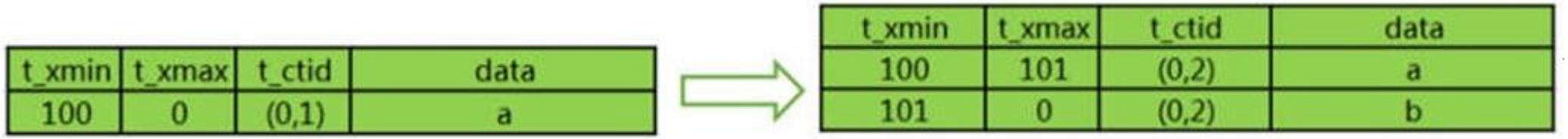
Heap Table Page Structure

- A heap table page consists of four parts:
  - PageHeaderData/HeapPageHeaderData (24 bytes)
  - An array of line pointers that point to the locations where tuples are stored
  - Free space
  - Tuple data
- On a heap table page, the page header corresponds to the HeapPageHeaderData structure.
  - **pd_multi_base** and fields before it correspond to fixed-length members that store important metadata about the entire page.
  - The field after **pd_multi_base** corresponds to a variable-length array of tuple (line) pointers. Each array member records the starting offset and length of each tuple in the page, arranged from the end of the page towards the beginning.
  - The actual tuple data is inserted starting from the end of the page and extending towards the page header.
  - Correspondingly, tuple pointers are inserted after the fixed-length members in the page header and extend towards the end of the page.
  - A hole is formed in the middle of the entire page for subsequent inserted tuples and tuple pointers.

HUAWEI

Update a Tuple

- The left of the following figure shows a tuple inserted by the transaction whose XID is 100. Since there have been no updates, **t_xmax** is set to **0**, and **t_ctid** points to itself, which is the first tuple on page 0.
- On the right is the state after the tuple has been updated by the transaction whose XID is 101. In the database, this update is equivalent to inserting a new tuple. The **t_xmax** of the original tuple is updated to **101** (XID of the transaction performing the update). The **t_ctid** pointer now refers to the newly inserted tuple (0,2), which is the second tuple on page 0. The **t_xmin** of the new tuple is set to **101** (XID of the transaction that inserted this tuple), and **t_ctid** points to (0,2), indicating no further updates and pointing to itself.



t_xmin	t_xmax	t_ctid	data
100	0	(0,1)	a

t_xmin	t_xmax	t_ctid	data
100	101	(0,2)	a
101	0	(0,2)	b

Deleting a Tuple

- The following figure shows the deletion process, indicating that the tuple is deleted by the transaction whose XID is 102. **t_xmax** is set to the XID of the transaction performing the deletion, and **t_ctid** points to itself.

t_xmin	t_xmax	t_ctid	data
100	101	(0,2)	a
101	0	(0,2)	b



t_xmin	t_xmax	t_ctid	data
100	101	(0,2)	a
101	102	(0,2)	b

HUAWEI

Tuple Data Features

- Tuple data has the following features:
  - Tuple data consists of multiple columns (attributes).
  - Each attribute can have a fixed or variable length.
  - Attributes have different byte alignments.
  - Attributes use different storage policies.
  - NULL values are not stored in tuple data.
  - The start position is 8-byte-aligned.
- In addition, heap tuples larger than about 2 KB (a quarter of 8 KB) are stored and managed using the oversized-attribute storage technique (TOAST). TOAST automatically stores such tuples in a separate dedicated area of the database, storing only references to these tuples on the original data page. This helps keep the size of the original data page within a manageable range, improving performance.

HUAWEI

TOAST

- Overview
  - TOAST is used to store the value of a large column. openGauss uses a fixed page size (typically 8 KB), and tuples cannot span multiple pages. Therefore, large column values cannot be directly stored. TOAST provides a solution that allows large column values to be compressed or split into multiple physical rows.
  - A proper configuration policy can reduce the number of I/Os and scanned blocks, improving the query speed.
- Implementation Principle
  - The TOAST mechanism is triggered when data exceeding bytes specified by *BLCKSZ*/**4** (typically 2 KB) is to be stored in a TOAST-enabled attribute.
  - The TOAST mechanism attempts to compress the data to be stored or store the data out of the line until the data is shorter than the number of bytes specified by *BLCKSZ*/**4**.
  - Mainly, variable-length data types support TOAST. The first 32 bits store the total length of the value in bytes (including the length itself).
  - TOAST uses the two most significant binary bits to identify compression and out-of-line storage. The two bits are both zeros, indicating that the value is not in TOAST mode. If the 32nd bit is 1, the value is compressed and must be decompressed before being used. If the 31st bit is 1, the value is stored out of the line. In this case, only one pointer to another location is stored. The other 30 bits represent the actual size of the data instead of the logical size after decompression or capture from out of the line.

HUAWEI

TOAST

- Four data storage policies
  - PLAIN(p): avoids compression and out-of-line storage. Only the types of data that can be stored without TOAST can be selected.
  - EXTENDED(x): allows compression and out-of-line storage. Generally, the data is compressed first. If the size is still too large, the data is stored out of the line.
  - EXTERNAL(e): allows out-of-line storage but disallows compression. This policy applies to columns that perform operations on partial data, such as character strings. This policy can achieve higher performance because the entire line of data does not need to be read and then decompressed.
  - MAIN(m): allows compression but disallows out-of-line storage. In practice, to ensure oversized data storage, out-of-line storage is used as a last resort anyway when other strategies (such as compression) cannot meet the requirements.

HUAWEI

TOAST

- TOAST table structure
  - If TOAST data is processed in out-of-line storage, the out-of-line data is divided into chunks that do not exceed the number of bytes specified by **TOAST_MAX_CHUNK_SIZE**. Each chunk is stored as an independent tuple in an associated TOAST table. The schema for each table is identical and has the following attributes.

Attribute	Type	Definition
chunk_id	oid	OID allocated to the entire TOAST data during out-of-line storage.
chunk_seq	int4	A sequence number that stores the location of the chunk in the entire TOAST data.
chunk_data	byte	Actual data of the chunk

  - Each TOAST data that is stored out-of-line is assigned an OID, which can be used to locate all data chunks in the TOAST table and reassemble it.
- Accessing out-of-line TOAST data
  - Obtain the TOAST pointer from the TOAST attribute of the table, use the pointer to find the TOAST table, use the OID of the TOAST data to find all chunks in the TOAST table, and assemble the chunks by sequence number to obtain the TOAST data.

HUAWEI

TOAST

- TOAST operations during tuple insertion or update
  - If the data to be updated or inserted is larger than the TOAST length threshold (**TOAST_TUPLE_THRESHOLD** is about 2000 bytes), the TOAST table is updated.
- Update policies
  - Traverse columns from largest to smallest. For columns that use the x or e policy, attempt to compress data (applicable only to x). If the columns are still larger than the TOAST threshold after compression, perform out-of-line storage using toast_save_datum.
  - If the tuple size is still above the TOAST threshold, continue traversing columns from largest to smallest. For columns that use the x or e policy, directly perform out-of-line storage.
  - If the tuple size remains above the TOAST threshold, traverse columns from largest to smallest and compress columns that use the m policy.
  - After the above steps, if the tuple size exceeds the TOAST main threshold (**TOAST_TUPLE_TARGET_MAIN** is about 8000 bytes), move columns that use the m policy to out-of-line storage, traversing columns from largest to smallest.

HUAWEI

TOAST

- Reading TOAST data
  - When querying data to which TOAST has been applied, extract and restore the TOAST data (TOAST data may be compressed or stored out of line).
  - TOAST data is read based on the TOAST policy used for the data.
- Implementation Process
  - If the data is stored out of line, the toast_fetch_datum function is called to obtain data chunks from the TOAST table to reassemble the data.
  - If the data is not stored out of line but is compressed, the data is returned after decompression.
  - If the data is short (not compressed or stored out of line), the data is returned directly.

HUAWEI

TOAST

- Deleting TOAST data
  - When a tuple containing TOAST data is deleted, the corresponding TOAST data also needs to be deleted.
  - For data that uses the PLAIN and MAIN policies, the TOAST data is deleted together with the tuple. No extra processing is required.
  - For data that uses the EXTENDED and EXTERNAL policies, in addition to deleting tuples, you also need to delete the data stored out of line.
- TOAST overview
  - TOAST ensures that the size of tuples in the database is sufficient for storing data in a file block.
  - TOAST must be set up as an automatic system processing mechanism, which users cannot control.
  - The TOAST mechanism is mainly used for variable-length data types.

HUAWEI

Large Object

- Supported data types
  - Binary large object (BLOB): stores non-traditional data, such as images, videos, and mixed media.
  - Character large object (CLOB): stores large single-byte character set data, such as documents.
  - Double-byte character large object (DBCLOB): stores large double-byte character set data, such as variable-length double-character graphic strings.
- Comparison between TOAST and large objects

Category	TOAST	Large Object
Triggering mechanism	Automatically triggered (when the size is greater than 2 KB)	Manually called by users
Data security	An error is reported if there are data losses.	Data can be lost and is replaced by **0**.
File storage	Unsuitable. Data needs to be converted to binary.	Files are directly stored in the large object table.
Storage mode	Out-of-line and compression mechanism	Direct storage

HUAWEI

pageinspect Plug-in and Its Functions

- The pageinspect plug-in is a contrib module inherited by openGauss from PostgreSQL. It provides low-level access to database page content, which is useful for inspecting the contents of pages in tables or indexes, diagnosing corruption, understanding storage formats, and debugging the storage layer.
- The pageinspect module offers several functions that can be used to retrieve information about page structures, including header information, tuple layouts, and the content of individual columns.
- The pageinspect plug-in provides functions to analyze the structure of openGauss storage pages.
- Commonly used functions:
  - get_raw_page (relname text, blkno int): returns a bytea.
    - Returns a copy of the page specified by **blkno** from the relational table **relname**.
    - The bytea data type is used to store binary data in openGauss. It can be used to store any binary data needed in the database.

HUAWEI

pageinspect Plug-in and Its Functions

- Commonly used functions (continued):
  - page_header (page bytea) returns a record.
    - Returns the page header. The parameter **page** should be a page copy returned by the get_raw_page function.
    - The returned record contains columns that correspond to the columns of the PageHeaderData structure.
  - heap_page_items (page bytea) returns a SETOF record.
    - Displays all row pointers on a heap page. The parameter **page** should be a page copy returned by the get_raw_page function.
    - The return value is an item pointer (ItemIdData) on the page and detailed information about the page tuple header structure (HeapTupleHeaderData).

HUAWEI

Contents

1. Storage Engines
2. Data Storage Structures
3. **High Concurrency in Thread Pools**

HUAWEI

Thread Pool Technology Overview

- openGauss implements thread pools based on the multi-thread architecture.
- The thread pool mechanism separates sessions from processing threads, ensuring that the system maintains good SLA response times even under high concurrency.
- In addition, different thread groups may be bound to different non-uniform memory access (NUMA) cores, aligning naturally with NUMA-aware CPU architectures, thereby improving overall openGauss performance.
- The idea behind thread pool technology is to pool thread resources and reuse them among different connections.
  - After the system is started, a fixed number of worker threads are started based on the current number of cores or user configurations. A worker thread serves one or more connection sessions. In this way, the session and thread are decoupled.
  - Since the number of worker threads is fixed, this design minimizes frequent thread switching under high concurrency. Sessions are scheduled and managed at the database layer.

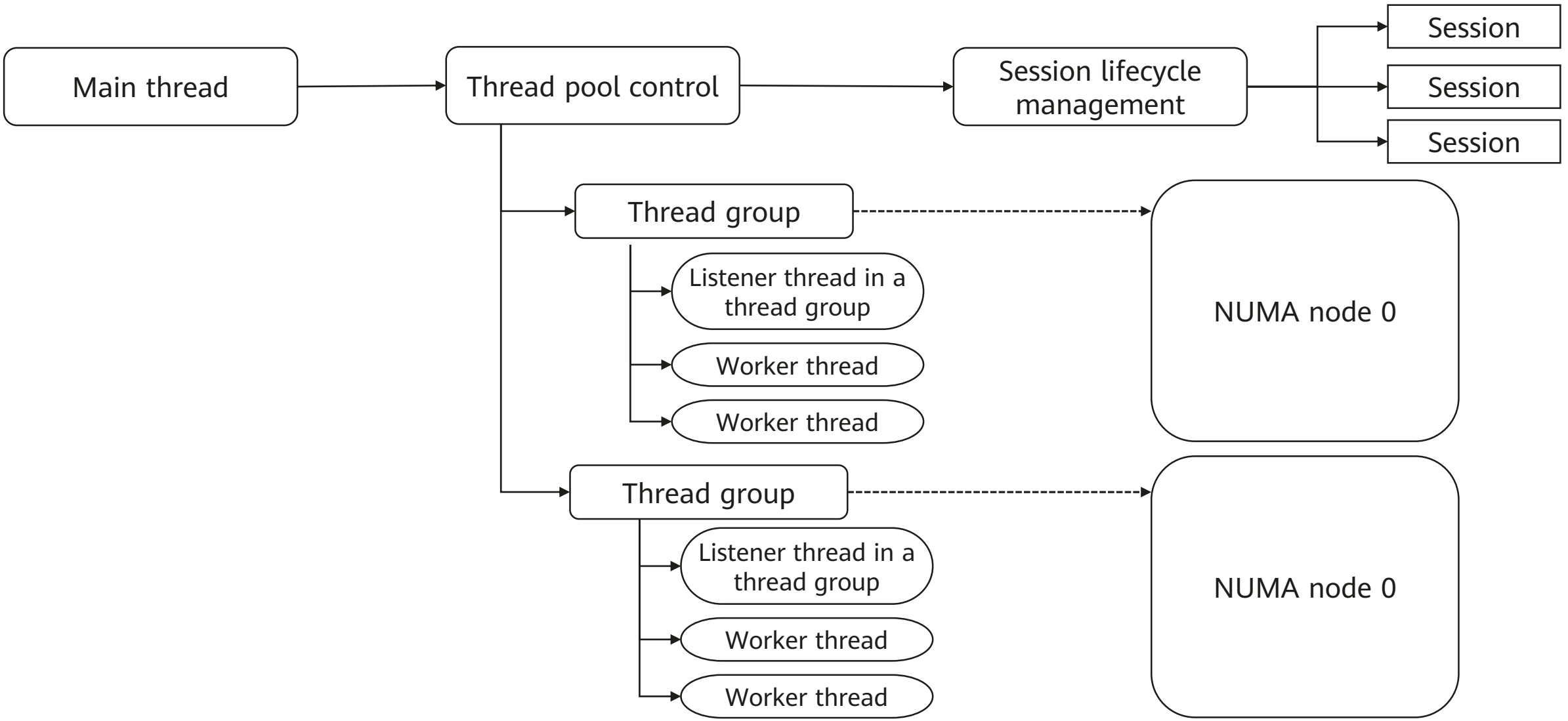
HUAWEI

High Concurrency in Thread Pools

- In OLTP, a database needs to process many client connections. As such, good processing capability in high-concurrency scenarios is crucial for databases.
- Per-thread-per-connection is the simplest processing method for external connections, in which a thread is generated for each incoming user connection. This architecture enables simple processing. However, in high-concurrency scenarios, there are too many threads, causing heavy thread switchover workloads and fierce contention for the lightweight lock. As a result, the performance (throughput) deteriorates sharply, and SLAs for user performance cannot be fulfilled.
- Thread resource pooling and reuse technology resolves this problem.
  - The idea is to pool thread resources and reuse them among different connections.
  - After the system is started, a fixed number of worker threads are started based on the current number of cores or user configurations. A worker thread serves one or more connection sessions. In this way, the session and thread are decoupled.
  - Since the number of worker threads is fixed, this design minimizes frequent thread switching under high concurrency. Sessions are scheduled and managed at the database layer.

HUAWEI

Thread Pool Mechanism Diagram



HUAWEI

Thread Pool Objects

Object	Description
Postmaster	Main thread, which listens to requests sent by clients.
ThreadPoolControler	Controls the thread pool, including initializing the thread pool and managing resources.
ThreadSessionControler	Manages session lifecycles.
ThreadPoolGroup	A thread group, which allows the flexible configuration of thread counts and core binding policies.
ThreadPoolListener	Listener thread in a thread group, which distributes and manages events.
ThreadPoolWorker	Worker thread
session	A session connected to a client
NUMA NODE	NUMA node, indicating that a thread group can be mapped to a NUMA node in the NUMA structure

HUAWEI

Thread Pool Mechanism

- (1) The client initiates a connection request to the database. The Postmaster thread receives the connection request and is awakened.
  - The Postmaster thread creates a socket for the connection and calls the ThreadPoolControler function to create a session data structure. (A socket describes the IP address and port and serves as a communication handle.)
  - The ThreadPoolControler function traverses all existing thread groups, finds the thread group with the fewest active sessions, and assigns the new session to this thread group, adding it to the thread group's epoll list. (An epoll is a Linux kernel mechanism that improves upon the poll system call, significantly increasing CPU utilization when there are many connections but few are active.)
- (2) The listener thread of the thread group is responsible for monitoring all client connections in the epoll list.
- (3) When a client initiates a task request, the listener thread is awakened.
  - The listener thread checks whether there are any idle worker threads in the current thread group.
  - If there are, it assigns the session to an idle worker thread and wakes it up. If there are no idle worker threads, the session is placed in the wait queue.

HUAWEI

Thread Pool Mechanism

- (4) When the worker thread is awakened, it reads the request from the client connection, executes the corresponding request, and returns the result.
  - When a transaction ends (committed or rolled back) or exits due to timeout, the worker thread completes its task.
  - The worker thread returns the session to the listener thread, which continues to wait for the subsequent request for that session.
  - After returning the session, the worker thread checks the session wait queue. If any sessions are waiting for a response, it retrieves a new session from the queue and continues working. If there are no waiting sessions, it marks itself as free and waits for the listener thread to wake it up.
- (5) When the client is disconnected, the worker thread is awakened, closes the connection, and cleans up the session-related structures, releasing memory and file descriptors (FDs) and other resources.
- (6) If a worker thread exits due to a FATAL error, it will unregister from the worker queue upon exit. The listener thread will then restart the worker threads until it reaches the specified number of worker threads.

HUAWEI

Parameters and Views Related to Thread Pools

- enable_thread_pool
  - Parameter description: Specifies whether to enable the thread pool function.
  - Value range: Boolean. **on** indicates that the thread pool function is enabled, and **off** indicates that the thread pool function is disabled.
  - Default value: **on**.
- thread_pool_attr
  - Parameter description: Specifies the detailed attributes of the thread pool function. This parameter is valid only when **enable_thread_pool** is enabled. Only SYSADMIN can access this parameter.
  - Value range: a string consisting of one or more characters. This parameter consists of three parts:
    - **thread_num** specifies the total number of threads in the thread pool. The value ranges from 0 to 4096.
    - **group_num** specifies the number of thread groups in the thread pool. The value ranges from 0 to 64.
    - **cpubind_info** specifies whether the thread pool is bound to a core.
- LOCAL_THREADPOOL_STATUS view
  - Displays the status of worker threads and sessions in the thread pool. This view is valid only when **enable_thread_pool** is set to **on**.

HUAWEI

Thread Pool Advantages

- The thread pool helps handle many concurrent user connections and serves as a form of traffic control. Even if there are many user connections, the backend does not need to allocate an excessive number of threads.
- Threads are OS resources, so a lot of OS resources are occupied when there are too many threads. In addition, scheduling and switching many threads cause expensive overheads.
- Without a thread pool, increasing the number of connections can gradually reduce system throughput. In addition, dividing the thread pool into thread groups helps match the NUMA CPU architecture nodes, enhancing performance in multi-core environments. Each thread group has its own listener, which helps avoid the "thundering herd problem" associated with thread pools.

HUAWEI

Quiz

1. (Single-answer question) Which of the following data storage policies is used to avoid compression and out-of-line storage?
   A. PLAIN(p)
   B. EXTENDED(x)
   C. EXTERNAL(e)
   D. MAIN(m)
2. (True or false) A column-store engine supports triggers and the following DDL statements: CREATE, DROP, and TRUNCATE TABLE, but does not support primary or foreign keys.
   A. True
   B. False

HUAWEI

Summary

- This chapter has given you a comprehensive understanding of database storage engines. You can now distinguish between and select the appropriate uses for row-store and column-store engines and have a deeper understanding of vectorized execution and adaptive compression. This chapter also covers the composition and structure of data storage, including tuple updates and deletions and the composition of data files. It also provides a detailed introduction to the features and principles of high concurrency in thread pools.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/en/
- openGauss Help (5.0.0)
  - https://docs-opengauss.osinfra.cn/en/docs/5.0.0/docs/GettingStarted/GettingStarted.html

Thank you.

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 3 Log Management



HUAWEI

Foreword

- This chapter describes openGauss log management, including the write-ahead log mechanism, checkpoint mechanism, restoration process based on redo logs, run logs, performance logs, and ways to check and clear logs.

HUAWEI

Objectives

- Upon completing this course, you will be able to:
  - Understand the fundamentals of openGauss log management.
  - Master the WAL mechanism and related contents.
  - Master the checkpoint mechanism and the redo log-based recovery process.
  - Master access permission and naming rules for run and performance logs.
  - Master how to check and clear logs.

HUAWEI

Contents

1. **WAL Mechanism**
2. Checkpoint Mechanism
3. Restoration Using Redo Logs
4. Run Logs and Performance Logs
5. Log Checking and Clearance

HUAWEI

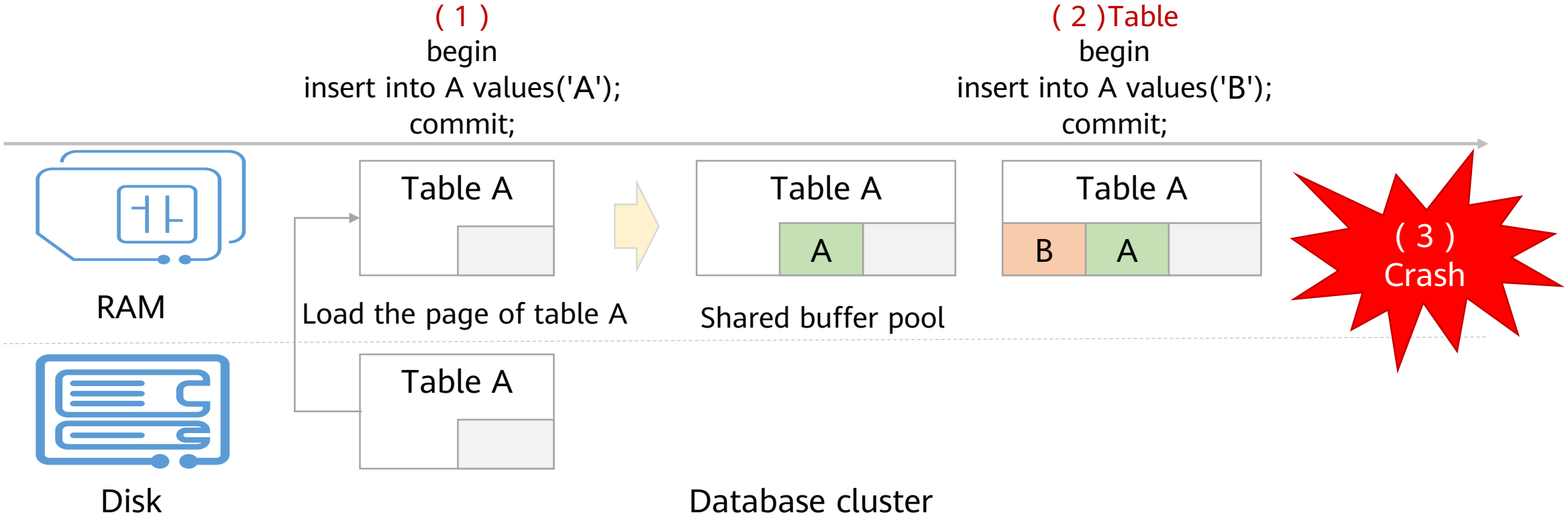
About WALs

- In a system using write-ahead logs (WALs or Xlogs), data file modifications can be written only after those modifications have been logged, that is, after WAL records describing the modifications have been flushed to permanent storage. You can use WALs to restore a database if the system crashes.
- WALs are logs used to restore a damaged database. They are also called redo logs. You are advised to routinely back up WALs. The log file is stored in the */Database node*/**pg_xlog** directory by default if a database node is used.

HUAWEI

Without WALs – How Does Data Write Work? (1)

- Almost all DBMSs implement a shared buffer pool to efficiently access relationship table pages. Assume that there is an openGauss that does not implement the WAL mechanism. Now, insert some data tuples into Table A, as shown in the following figure.



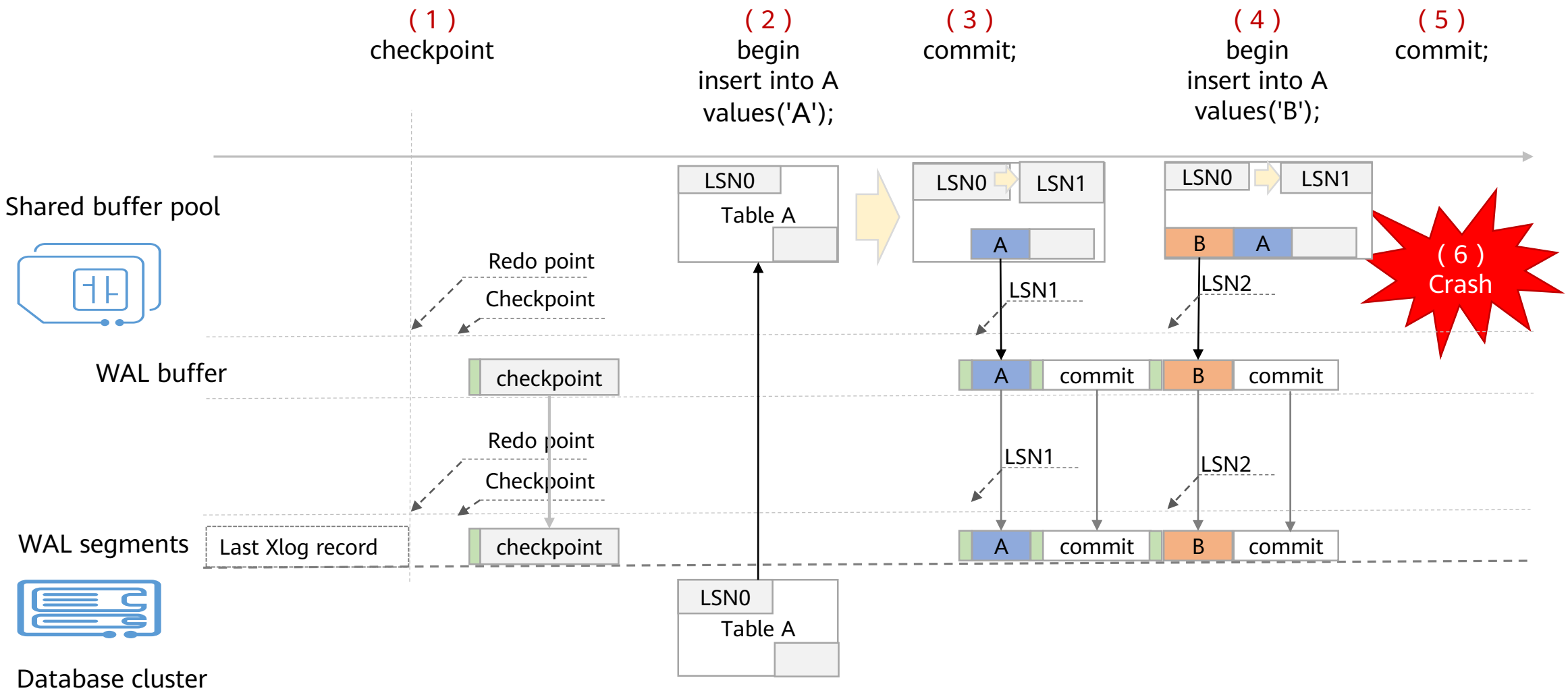
Without WALs – How Does Data Write Work? (2)

- The following is the data write process if there is no WAL:
  - When the first INSERT statement is initiated, openGauss loads the page of table A from the database cluster file to the shared buffer pool in the memory. Then, a tuple is inserted into the page. The page is not immediately written back to the database cluster file. In this case, the modified page is called a dirty page.
  - When the second INSERT statement is initiated, openGauss directly adds a new tuple to the page in the buffer pool. This page is still not written back to persistent storage.
  - If the OS or the openGauss server fails, for example, due to a power failure, all inserted data will be lost.

A database without WAL is vulnerable if the system crashes.

HUAWEI

With WALs – How Does Data Write Work? (1)



HUAWEI

With WALs – How Does Data Write Work? (2)

- The following is the openGauss data write process with WAL:
  1. The background thread named **checkpointer** periodically performs checkpointing. Each time the checkpointer is started, an Xlog record named **checkpoint record** is written to the current WAL segment. This record contains the location of the latest redo point (where the Xlog record is written at the beginning of the latest checkpoint).
  2. When the first INSERT statement is initiated, openGauss loads the page of table A from the database cluster file to the shared buffer pool in the memory, inserts a tuple into the page, and creates and writes an Xlog record at **LSN1**. Then, the LSN of table A is updated from **LSN0** to **LSN1**. In this case, an Xlog record is a pair of values consisting of header data and a complete tuple.
  3. When the transaction is committed, openGauss creates and writes an Xlog record about the commit behavior to the WAL buffer, and then flushes all Xlog records in the WAL buffer to the WAL segment file.

HUAWEI

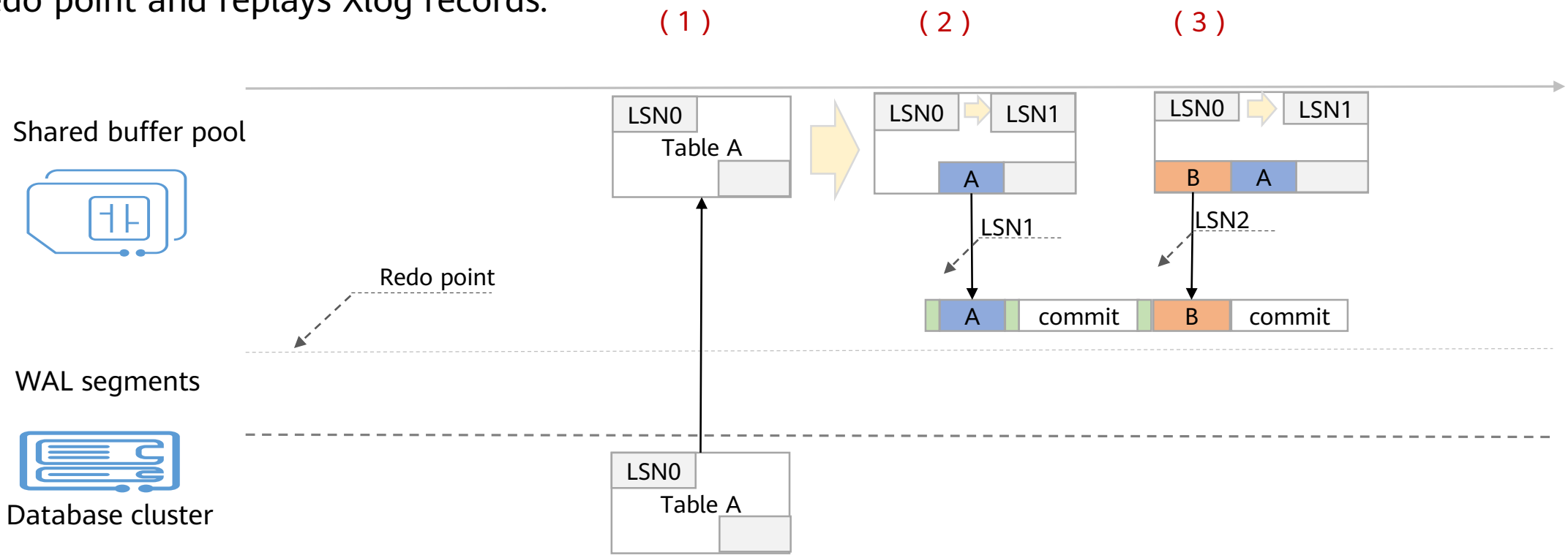
With WALs – How Does Data Write Work? (3)

4. When the second INSERT statement is initiated, openGauss inserts a new tuple into the page, creates and writes a corresponding Xlog record at **LSN2**, and updates the LSN of table A from **LSN1** to **LSN2**.
5. Now, the transaction is committed. An Xlog record corresponding to the commit is created and written to the WAL buffer, and all Xlog records starting from **LSN2** in the WAL buffer are written to the WAL segment file.
6. In case of an OS failure, the assumption is that although all data in the shared buffer is lost, all page modifications have been written to the WAL segment file as historical records.

HUAWEI

With WALs – How Does Database Restoration Work? (1)

- You do not need to perform any special operations when openGauss restores the database cluster to the status before the crash. When openGauss is restarted, it automatically enters the recovery mode, as shown in the following figure. openGauss reads the correct WAL segment files in sequence from the redo point and replays Xlog records.



With WALs – How Does Database Restoration Work? (2)

- The following is how WAL is used to restore an openGauss database:
  - openGauss reads the Xlog record of the first INSERT statement from the corresponding WAL segment file and loads the page of table A from the database disk file to the shared buffer.
  - Before replaying an Xlog record, openGauss compares the LSNs of the Xlog record and the corresponding page. The rules for replaying Xlog records are as follows:
    - If the Xlog record has the larger LSN, the data part of the Xlog record is inserted into the page, and the LSN of the page is updated to the LSN of the Xlog record.
    - If the Xlog record has the smaller LSN, you can directly read the subsequent WAL data.
  - openGauss replays other Xlog records using the same method.

HUAWEI

Database Troubleshooting Based on WAL Writing

- Scenario:
  - Are database faults always recovered based on WALs (that is, by writing WALs)?
- Conclusion:
  - No
- Possible causes:
  - The database needs to solve the problem of partial writes and implement the full-page write mechanism.

HUAWEI

Partial Writes and Corresponding Causes During Database Recovery

- What is a partial write?
  - When the database is writing dirty pages to disk files, the OS is faulty. In this case, the page data in table A is damaged (that is, only part of the page data is written to disk files), which is called a partial write.
- What causes partial writes?
  - Data is written to the database in the unit of a page. The default page size in openGauss is 8 KB. However, other parts of the storage stack may use different block sizes. For example, Linux file systems use 4 KB pages. When the database writes an 8 KB page, other layers of the storage stack may divide it into smaller blocks for management (creating atomicity problems). An 8 KB page can be divided into two 4 KB file system pages and then divided into 512-byte sectors. If the server crashes (due to a power failure or kernel error) during data writing, 8 KB data pages may be written to the database. However, partial writes occur before the server crashes.

HUAWEI

Consequences and Solutions of Partial Write Issues Caused by Database Faults

- Consequences of partial data write issues:
  - If data is being written in 8 KB blocks and the server crashes (due to a power failure or kernel error), only 4 KB of real data may be stored before the crash. During recovery, Xlog records cannot be replayed on the damaged page. As a result, the database cannot be restored to the status before the fault had occurred.
- Solution:
  - Use the full-page write mechanism to solve the partial write problem.

HUAWEI

Database Full-Page Write Mechanism

- Meaning of full-page write:
  - When each page changes for the first time after each checkpoint, openGauss writes the header data and the entire page as an Xlog record to the WAL buffer. The Xlog record that contains the entire page is called a backup block or a full-page image.
- Full-page write settings:
  - Configuration file modification: Set **full_page_write** to **on** in the **postgresql.conf** file.
  - Server tool: Run the **gs_guc** command to set **full_page_write** to **on**.
- Full-page write taking effect:
  - Once **full_page_write** is set to **on**, restart the database.

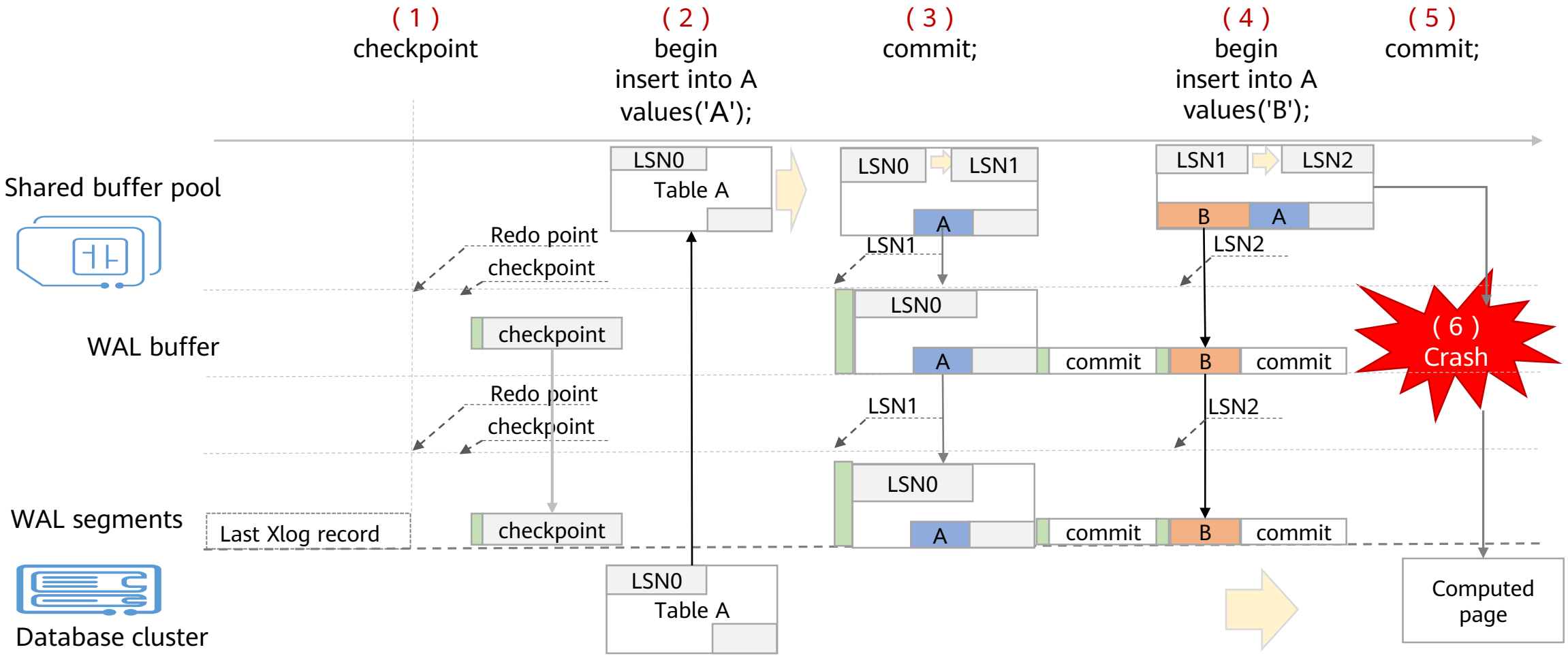
HUAWEI

Advantages and Disadvantages of the Database Full-Page Write Mechanism

- Advantages:
  - When a page is modified for the first time after a checkpoint, the entire page is written to the WAL. This ensures that the first WAL record that a page is written to contains the entire page for any recovery process without having to read a page that may be damaged from the data file.
- Disadvantages:
  - Setting **full_page_write** to **on** increases the amount of data that must be written to the WAL (that is, causes the write amplification problem of the WAL). This leads to extra I/O read and write operations and higher disk consumption in the database. For example, even if a single byte on an 8 KB page is changed, the entire page needs to be recorded in the WAL.

HUAWEI

Process of Writing Data Using the WAL Mechanism based on Full-Page Write



HUAWEI

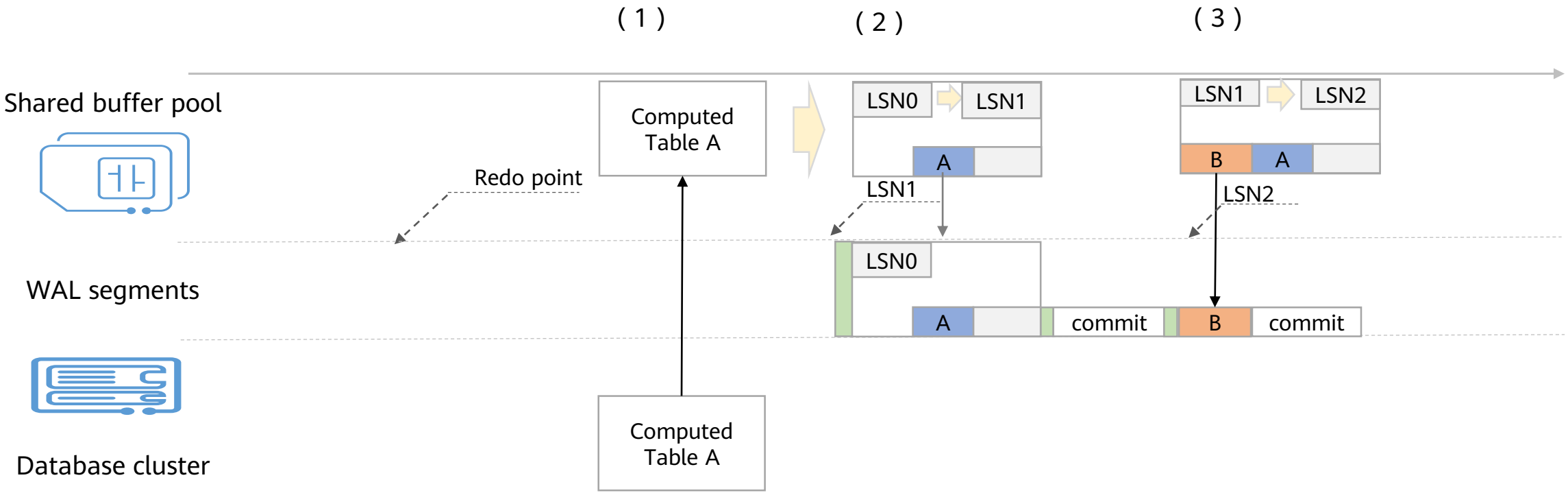
Procedure of Writing Data Using the WAL Mechanism based on Full-Page Write

- Step 1: Start a checkpoint on the checkpointer thread. The checkpoint starts the checkpoint process.
- Step 2: When the first INSERT statement is inserted, check that the Xlog has recorded the *Header data*+*Entire page* backup log because this is the first write of the page after the latest checkpoint.
- Step 3: Commit the transaction using the same operations as in the previous steps.
- Step 4: When the second INSERT statement is inserted, repeat the steps above because the Xlog record is only "*Header data*+*Inserted tuples*" instead of the entire backup block.
- Step 5: Commit the transaction using the same operations as in the previous steps.
- Step 6: To describe the full-page write effect, consider that if the OS fails while writing data to the disk file, some Table A pages on the disk may be damaged due to write problems.

HUAWEI

Restoration Process of Writing Data Using the WAL Mechanism based on Full-Page Write

- The following figure shows the database restoration process when openGauss is restarted.



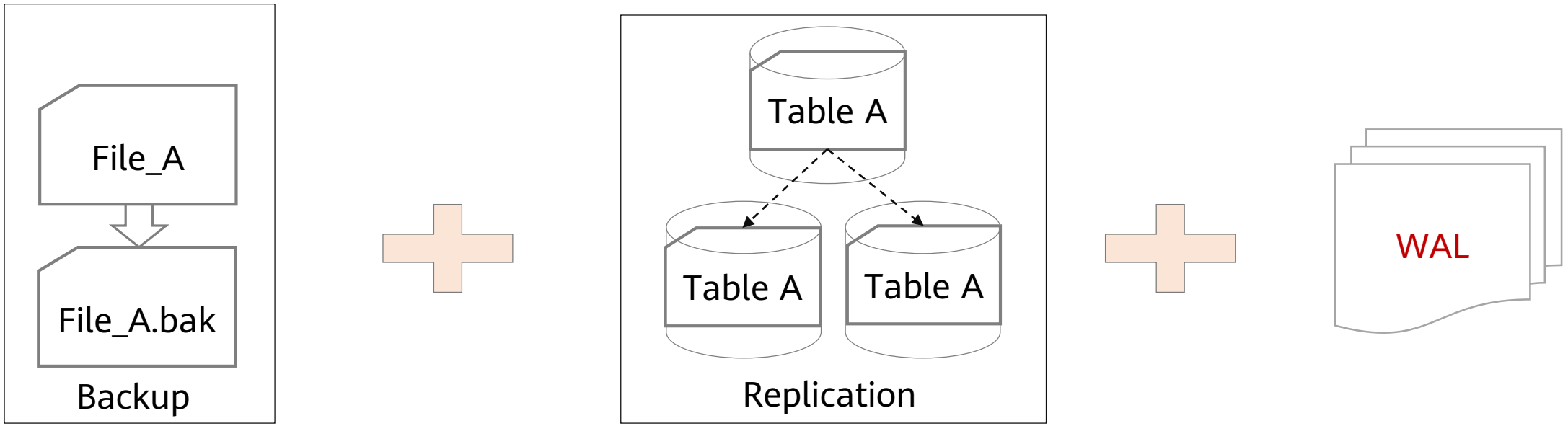
Troubleshooting of Writing Data Using the WAL Mechanism based on Full-Page Write

- Step 1: Check that openGauss has read the Xlog record of the first INSERT statement and loaded the page of table A from the database cluster directory to the shared buffer pool. According to full-page write rules, the Xlog record is a backup block.
- Step 2: When an Xlog record is a backup block, use another replay rule. That is, use the data part of the Xlog record, which is the page itself, to overwrite the current page without comparing the LSNs. Then, update the LSN of the page to **LSN1** recorded in the Xlog. As shown in the figure on the previous slide, the damaged page is overwritten by the data recorded in the Xlog and the LSN of table A is updated to **LSN1**. In this way, the damaged page is restored using its own backup block.
- Step 3: Repeat the operations in steps 1 and 2 because the second Xlog record is not a backup block.

HUAWEI

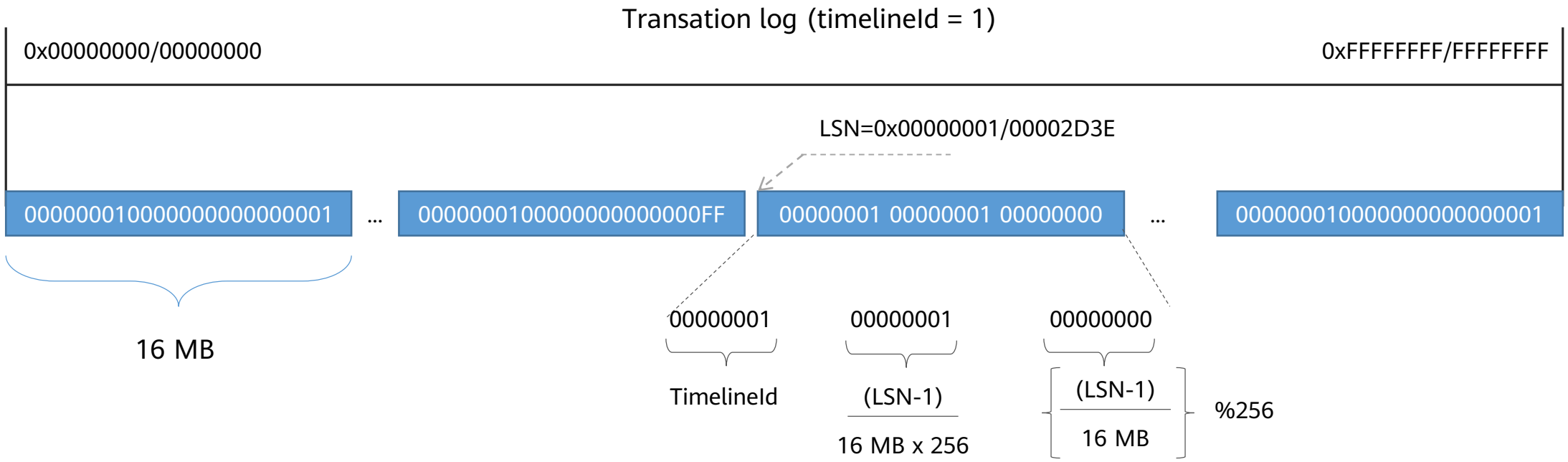
Other Database Troubleshooting Methods

- File system and media faults (such as disk damage) can also occur.
- Database troubleshooting methods:
  - Use the backup and replication mechanisms together with the WAL mechanism to ensure that the database can be restored even if a media fault occurs.



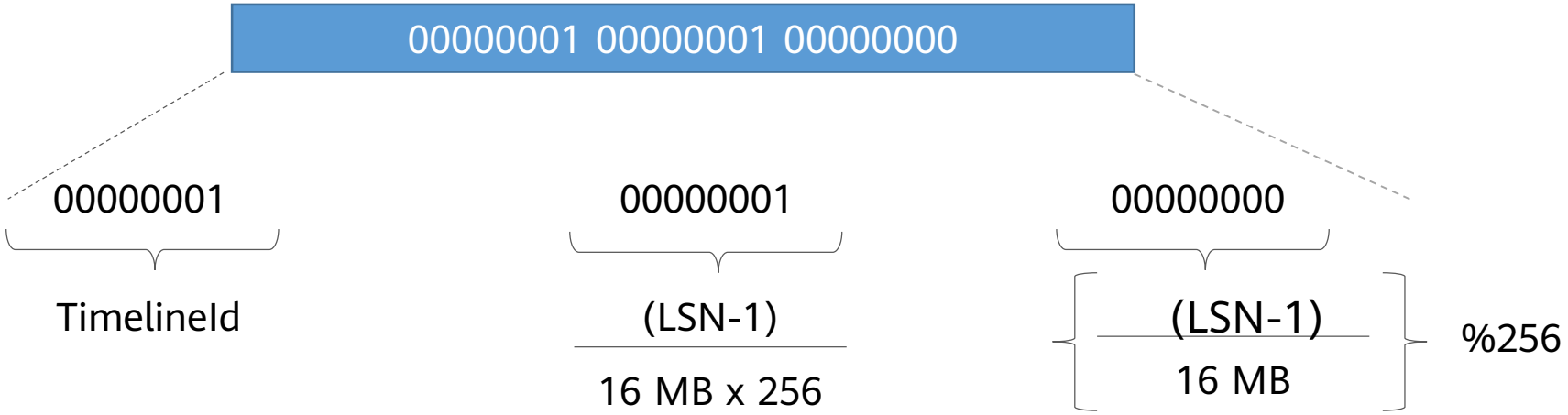
Name Encoding for WAL Files

- Logically, openGauss writes Xlog records to a virtual transaction log file using a 64-bit (8-bytes) address. The maximum length can be 16 EB, that is, 2^{16} bytes. Physically, transaction logs are divided into 16 MB files by default. Each file is called a WAL segment. The following is the WAL encoding format:



WAL Segment File – Name Encoding

- The name of a WAL segment file consists of 24 hexadecimal characters. Each name has three parts, with eight hexadecimal characters each.



- WAL segment file naming rule:
  - $$ \begin{align} \verb|*WAL segment file name*| = \verb|timelineId| + (\verb|uint32|) \frac{\verb|LSN|-1}{16\verb|M|*256}+ (\verb|uint32|)\left(\frac{\verb|LSN|-1}{16\verb|M|}\right) % 256 \end{align} $$

WAL Segment File – Components

- The name of a WAL segment file consists of 24 hexadecimal characters. Each name has three parts, with eight hexadecimal characters each.
  - The first eight bits indicate the timeline ID, which is used for PITR. In this example, the value is fixed to **00000001**.
  - The eight bits in the middle indicate the logical ID (**logid**) of a WAL. The default total size of log files corresponding to each logical ID is 256 x 16 MB (number of logical segments x WAL segment file size). That is, each logical ID contains 256 physical WAL files, each at 16 MB. For a given LSN, **logid** is calculated as follows: $\frac{(LSN-1)}{(16M*256)}$.
  - The last eight bits indicate the sequence number (**logseg**, logical segment number) of the logical ID for the current WAL segment file. The maximum number is 256. As such, only the last 2 bits are required. For a given LSN, **logseg** is calculated as follows: $\frac{(LSN-1)}{16M}\%256$.

HUAWEI

WAL Segment File – File Name Calculation Formula

- The name of the WAL segment file where the LSN is located uses the formula below. The formula considers the encoding mode and meaning of each component described previously.
  - WAL segment file name = $timelineid + \frac{LSN-1}{16M*256} + \frac{LSN-1}{16M}\%256$
- Assume that the LSN is **1/920001F8** and **timelineid** is fixed at **00000001**. The result is as follows:
  - WAL segment file name = **000000010000000100000092**

HUAWEI

WAL Segment File – Physical Organization Format

- WAL files are stored in the form of WAL segment files. Each segment is 16 MB and is divided into several pages with 8 KB per page. The name of a WAL file consists of 24 hexadecimal characters, which are divided into three parts with eight hexadecimal characters each. These indicate the timeline, the log file number, and the segment number of the log file, respectively. The timeline starts from 1, and the log file number and log file segment number start from 0. For example, the first transaction log file in the system is 00000001 00000000 00000000.

```
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 000000010000000000000001
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 000000010000000000000002
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 000000010000000000000003
-rw-------. 1 omm dbgrp  16M Mar 11 11:54 000000010000000000000004
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 000000010000000000000005
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 000000010000000000000006
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 000000010000000000000007
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 000000010000000000000008
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 000000010000000000000009
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 00000001000000000000000A
-rw-------. 1 omm dbgrp  16M Mar 11 11:49 00000001000000000000000B
drwx------. 2 omm dbgrp 4.0K Mar 11 11:49 archive_status
```


HUAWEI

WAL Segment File – Function Query

- openGauss provides built-in functions. For example, you can use pg_xlogfile_name to find the WAL segment file that contains a specific LSN or pg_current_xlog_insert_location to find the current LSN.

```
openGauss=# select pg_current_xlog_insert_location();
 pg_current_xlog_insert_location
---------------------------------
 0/40586C0
(1 row)

openGauss=# select pg_xlogfile_name('0/40586C0');
     pg_xlogfile_name
--------------------------
 000000010000000000000004
(1 row)

openGauss=#
```


HUAWEI

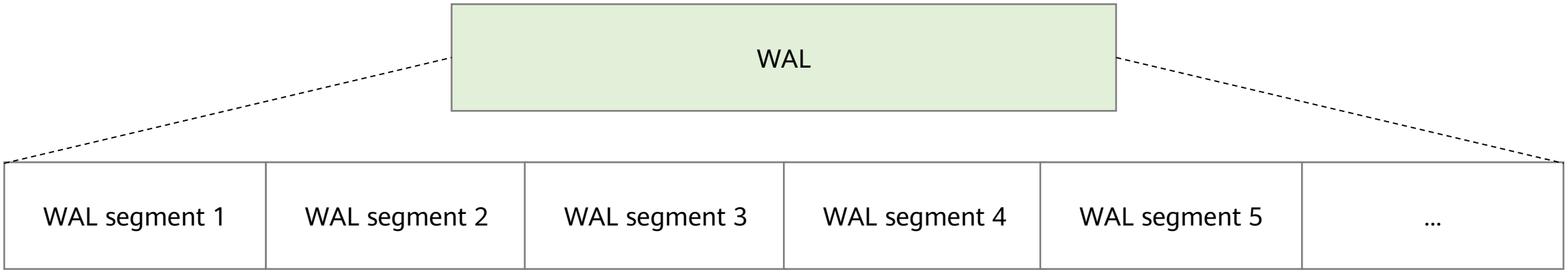
WAL Segment File – Viewing File Content

- If the primary database crashes and cannot be recovered and the standby database is properly connected, you may want to pay attention to only several tables in the database. In this case, you can use the third-party mog_xlogdump tool to view the WAL segment file content. You can also use the **-o** and **-R** parameters to parse the table data of a specified OID, where **-o** specifies the OID of a table and **-R** specifies the column type of the table.

```
[omm@standalone ~]$ mog_xlogdump -o 24582 -R int,money,inet,bool,numeric,text
/opt/huawei/install/data/dn/pg_xlog/000000010000000000000004
'insert','tuple':{'int':'1','money':'$24.24','inet':'192.168.255.132','bool':true,'numeric':'3.1415926','text':'ljfsodfo29892ifj'}
'insert','tuple':{'int':'2','money':'$928.83','inet':'10.255.132.101','bool':false,'numeric':'3.1415926','text':'vzvzcxwf2424@'}
'update',,'new_tuple':{'int':'111','money':'$982.37','inet':'10.255.132.101','bool':false,'numeric':'3.1415926','text':'vzvzcxwf
2424@'}
'update',,'new_tuple':{'int':'2','money':'$982.37','inet':'10.255.132.101','bool':false,'numeric':'3.1415926','text':'vzvzcxwf24
24@'}
```

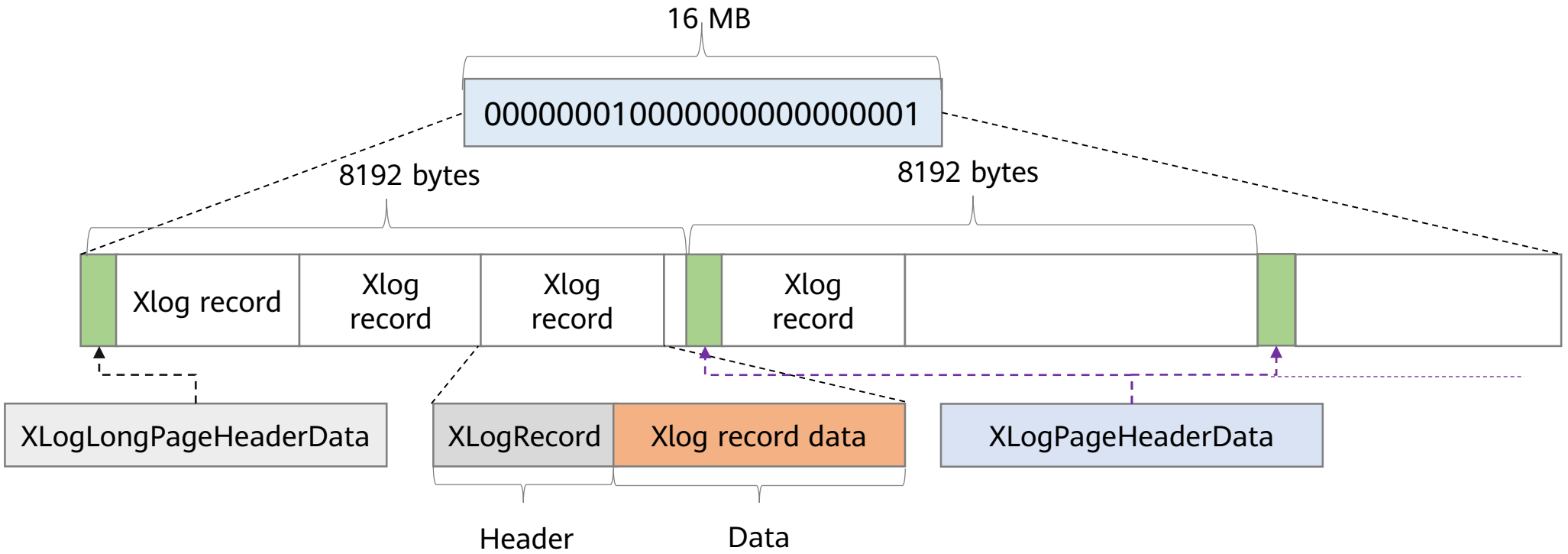
WALs – Internal Structure and Composition of Physical Files

- Logically, a WAL is a continuous file with a maximum length of 64-bit unsigned integers.
- From the perspective of physical distribution, the WAL file is composed of WAL segment files whose size is specified by **XLOG_SEG_SIZE** (16 MB by default).



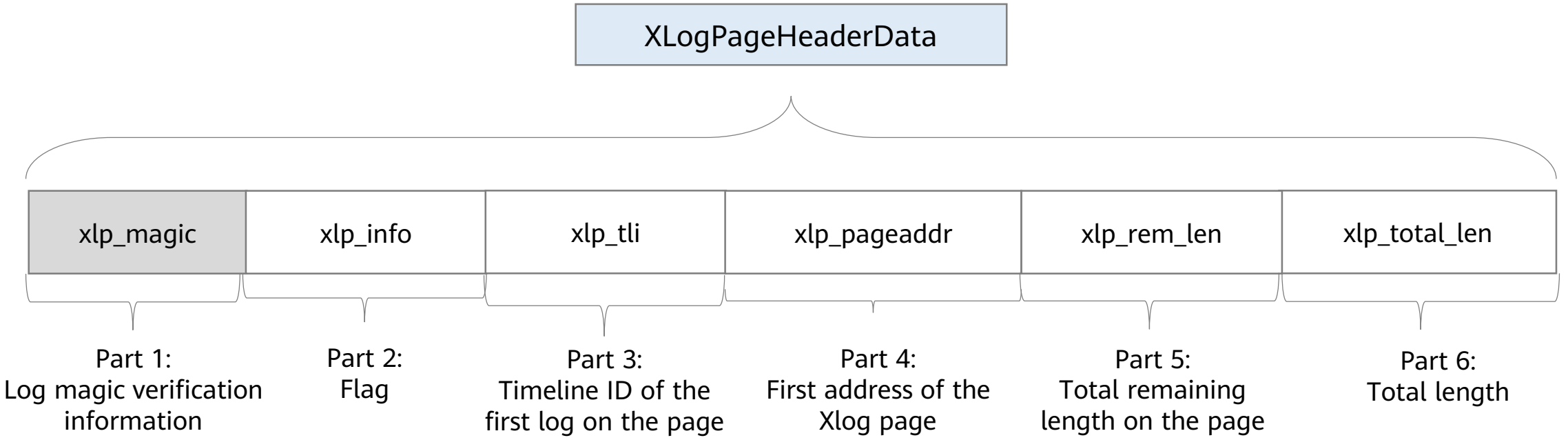
WAL Segment File – Internal Structure

- By default, a segment file is 16 MB and consists of 8 KB (8192 bytes) pages. The header data on page 1 is defined by the XLogLongPageHeaderData structure, and the headers of the other pages are defined by the XLogPageHeaderData structure. After the page header, the Xlog record is written to each page in descending order from the beginning, as shown in the following figure.



WAL Segment File – XLogPageHeaderData Structure (1)

- By default, each log segment file is divided into multiple pages of 8 KB each. The start position on each page is the page header. The page header on the first page of the entire segment file is a long page header (**XLogLongPageHeaderData**). All the other page headers (**XLogPageHeaderData**) are normal (or short).
- **XLogPageHeaderData** structure:



WAL Segment File – XLogPageHeaderData Structure (2)

- The **XLogPageHeaderData** structure is defined in the **src/include/access/xlog_basic.h** header file.

```
typedef struct XLogPageHeaderData {
    uint16 xlp_magic;       /* magic value for correctness checks */
    uint16 xlp_info;        /* flag bits, see below */
    TimeLineID xlp_tli;     /* TimeLineID of first record on page */
    XLogRecPtr xlp_pageaddr; /* XLOG address of this page */
    uint32 xlp_rem_len; /* total len of remaining data for record */
    uint32 xlp_total_len;
} XLogPageHeaderData;
```

Pay attention to the **xlp_rem_len** column, which indicates that the next page is displayed if the current page does not have enough space for the entire record. **xlp_rem_len** indicates the number of remaining bytes on the previous page. Note that **xl_rem_len** includes backup block data, which tracks **xlp_total_len** instead of **xl_len** in the initial header. Also, note that continuation data is not necessarily aligned.

HUAWEI

WAL Segment File – XLogPageHeaderData Structure (3)

- **xlp_info** in the **XLogPageHeaderData** structure indicates a flag. The related flags are defined as follows:

```
/* When record crosses page boundary, set this flag in new page's header */
#define XLP_FIRST_IS_CONTRECORD 0x0001
/* This flag indicates a "long" page header */
#define XLP_LONG_HEADER 0x0002
/* This flag indicates backup blocks starting in this page are optional */
#define XLP_BKP_REMOVABLE 0x0004
/* All defined flag bits in xlp_info (used for validity checking of header) */
#define XLP_ALL_FLAGS 0x0007
```

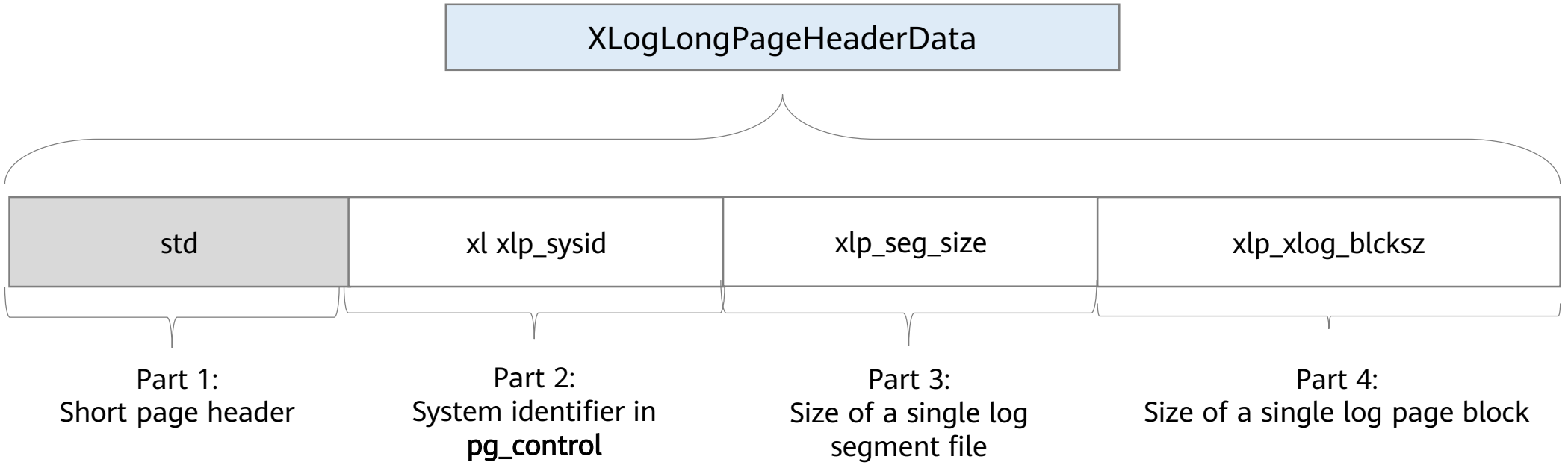
0x0001: The page contains a cross-page record, which is the last record on the previous page.

0x0002: The page is the first page of the segment file and has a long header.

0x0004: The backup data block on the page is optional.

HUAWEI

WAL Segment File – XLogLongPageHeaderData Structure (1)



- When the XLP_LONG_HEADER flag is set, other columns are stored in the page header, which is usually done only on the first page of the Xlog file. These additional columns are used to accurately identify files. This structure contains the short page header (**XLogPageHeaderData**) information.

WAL Segment File – XLogLongPageHeaderData Structure (2)

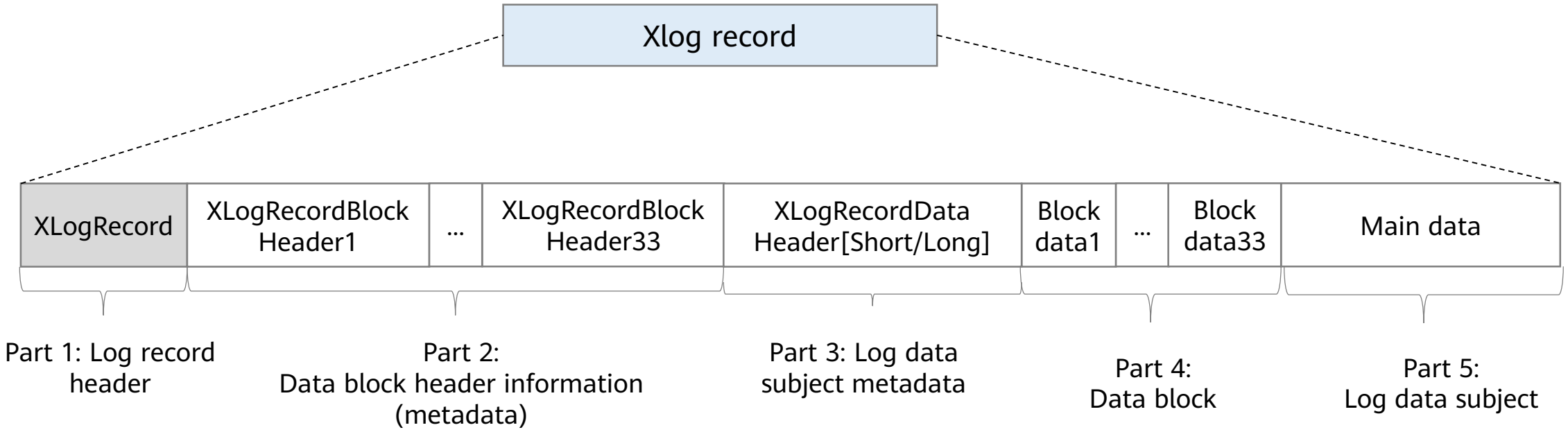
- The **XLogLongPageHeaderData** structure is defined in the **src/include/access/xlog_basic.h** header file. The source code is as follows:

```
typedef struct XLogLongPageHeaderData {
    XLogPageHeaderData std; /* standard header fields */
    uint64 xlp_sysid;       /* system identifier from pg_control */
    uint32 xlp_seg_size;    /* just as a cross-check */
    uint32 xlp_xlog_blcksz; /* just as a cross-check */
} XLogLongPageHeaderData;
```

xlp_sysid indicates the system identifier from **pg_control**, which is the same as **system_identifier** defined in **pg_control.h**. **xlp_sysid** is a unique system identifier used to ensure that it matches the generated Xlog.

WAL Segment File – Xlog Record (1)

- An Xlog record consists of a common header and specific data. The following figure shows the structure of a single Xlog record, which consists of five parts.



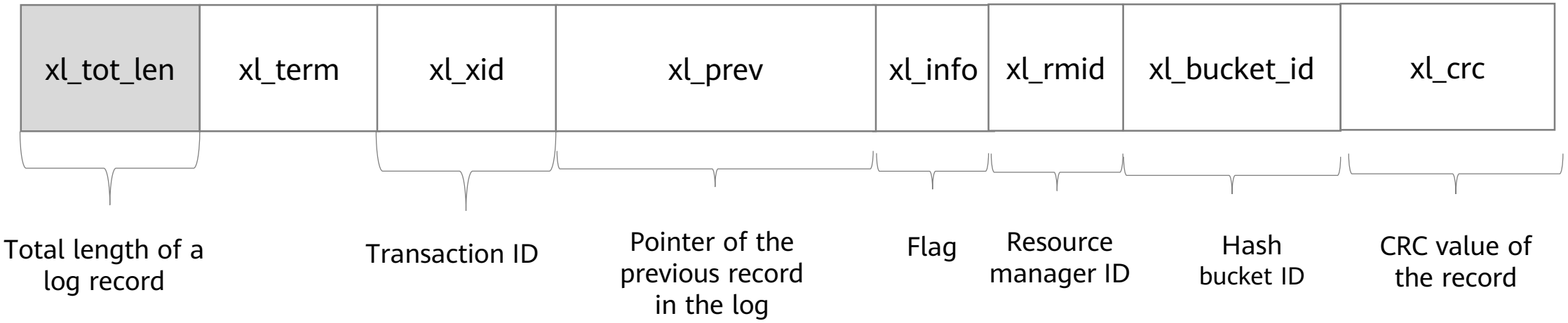
WAL Segment File – Xlog Record (2)

- A single Xlog record has the following five parts:
  - Log record header: corresponding to the **XLogRecord** structure, stores information such as the record length, primary and standby term IDs, transaction ID, start offset of the previous log record, flag, resource manager, and CRC value.
  - Metadata of 1 to 33 related pages: corresponding to the **XLogRecordBlockHeader** structure, stores information such as the page index (0 to 32), suffix of the physical file corresponding to the page, flag, and page data length. If the log does not contain the corresponding page information, this part does not exist.
  - Metadata of the log data subject: corresponding to the long or short **XLogRecordDataHeader** structure, records the special page index (to distinguish from the second part) and the length of the subject data.
  - Data on 1 to 33 related pages: If the log does not contain the corresponding page information, this part does not exist.
  - Log data subject
- When the log content is recorded, each part is close to the other, and no null character is added. If a log record does not have the corresponding page information, parts 2 and 4 are skipped.

HUAWEI

Xlog Record – XLogRecord Structure (1)

- All Xlog records have a common header, which is defined by the XLogRecord structure.
- The log record header, corresponding to the XLogRecord structure as follows, stores information such as the record length, primary and standby term IDs, transaction ID, start offset of the previous log record, flag, resource manager, and CRC value:



Xlog Record – XLogRecord Structure (2)

- The **XLogRecord** structure is defined in the **src/include/access/xlog_basic.h** header file.
- The source code is as follows:

```
typedef struct XLogRecord {
    uint32 xl_tot_len;          /* Full length of an entire record */
    uint32 xl_term;
    TransactionId xl_xid;       /* Transaction ID */
    XLogRecPtr xl_prev;         /* Pointer to the previous record in the log */
    uint8 xl_info;              /* Flag, as shown in the following: */
    RmgrId xl_rmid;             /* Resource manager of this record */
    uint2  xl_bucket_id;        /* Stores bucket ID. */
    pg_crc32c xl_crc;           /* CRC for this record */
} XLogRecord;
```

- Note: This type is followed by the XLogRecordBlockHeader and XLogRecordDataHeader types.

HUAWEI

Xlog Record – Related Resource Manager (1)

- The resource manager is a set of operations related to WAL functions (writing and replaying Xlog records). Depending on user operations, logs can be classified into several types. Each log type corresponds to a resource manager, which encapsulates their sub-types, structures, and replay logic.

Operation	Resource Management
Heap tuple	RM_HEAP, RM_HEAP2, RM_HEAP3
Index	RM_BTREE, RM_HASH, RM_GIN, RM_GIST, RM_SPGIST
Sequence	RM_SEQ
Transaction	RM_XACT, RM_MULTIXACT, RM_CLOG, RM_XLOG
Tablespace	RM_SEGPAGE, RM_DBASE, RM_TBLSPC, RM_RELMAP
Replication and hot standby	RM_STANDBY, RM_REPLORIGIN

HUAWEI

Xlog Record – Related Resource Manager (2)

- The resource manager is defined in the **src/include/access/rmgr.h** header file. Each type of resource manager is defined in the **src/include/access/rmgrlist.h** header file. Note that in the resource manager list of the **rmgrlist.h** header file, the sequence of each entry defines the value of each resource manager ID, which is stored in the WAL record. New entries should be added at the end to avoid changing existing entries.

```
/* symbol name, textual name, redo, desc, identify, startup, cleanup, undo, undo_desc info_type_name */
PG_RMGR(RM_XLOG_ID, "XLOG", xlog_redo, xlog_desc, NULL, NULL, NULL, NULL, NULL, xlog_type_name)
PG_RMGR(RM_XACT_ID, "Transaction", xact_redo, xact_desc, NULL, NULL, NULL, NULL, NULL, xact_type_name)
PG_RMGR(RM_SMGR_ID, "Storage", smgr_redo, smgr_desc, NULL, NULL, NULL, NULL, NULL, smgr_type_name)
PG_RMGR(RM_CLOG_ID, "CLOG", clog_redo, clog_desc, NULL, NULL, NULL, NULL, NULL, clog_type_name)
PG_RMGR(RM_DBASE_ID, "Database", dbase_redo, dbase_desc, NULL, NULL, NULL, NULL, NULL, dbase_type_name)
PG_RMGR(RM_TBLSPC_ID, "Tablespace", tblspc_redo, tblspc_desc, NULL, NULL, NULL, NULL, NULL, tblspc_type_name)
PG_RMGR(RM_MULTIXACT_ID, "MultiXact", multixact_redo, multixact_desc, NULL, NULL, NULL, NULL, NULL, multixact_type_name)
PG_RMGR(RM_RELMAP_ID, "RelMap", relmap_redo, relmap_desc, NULL, NULL, NULL, NULL, NULL, relmap_type_name)
PG_RMGR(RM_STANDBY_ID, "Standby", standby_redo, standby_desc, StandbyXlogStartup, StandbyXlogCleanup, \
    StandbySafeRestartpoint, NULL, NULL, standby_type_name)
PG_RMGR(RM_HEAP2_ID, "Heap2", heap2_redo, heap2_desc, NULL, NULL, NULL, NULL, NULL, heap2_type_name)
PG_RMGR(RM_HEAP_ID, "Heap", heap_redo, heap_desc, NULL, NULL, NULL, NULL, NULL, heap_type_name)
...
```


HUAWEI

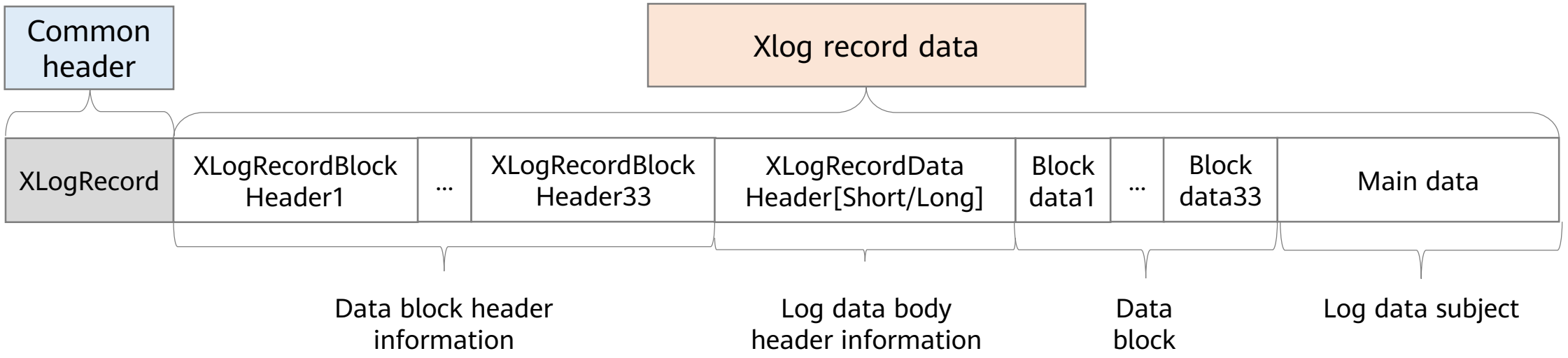
Xlog Record – Related Resource Manager (3)

- How do resource managers work?
  - If an INSERT statement is initiated, the variables *xl_rmid* and *xl_info* in the corresponding Xlog record header are set to **RM_HEAP** and **XLOG_HEAP_INSERT**, respectively. When the database cluster is restored, the heap_xlog_insert() function of the resource manager RM_HEAP is selected based on *xl_info* to replay the current Xlog record.
  - Similar to the UPDATE statement, *xl_info* in the header variable is set to **XLOG_HEAP_UPDATE**. During database restoration, the heap_xlog_update() function of the resource manager RM_HEAP is used for replay.
  - When a transaction is committed, the variables *xl_rmid* and *xl_info* in the header of the corresponding Xlog record are set to **RM_XACT** and **XLOG_XACT_COMMIT**, respectively. When the database is restored, xact_redo_commit() of **RM_XACT** replays the record.

HUAWEI

Xlog Record – Data (1)

- The data recorded in the Xlog consists of a header and data.
- The data part has zero or more block data and zero or one main data. The block data corresponds to XLogRecordBlockHeader(s), and the main data corresponds to XLogRecordDataHeader.



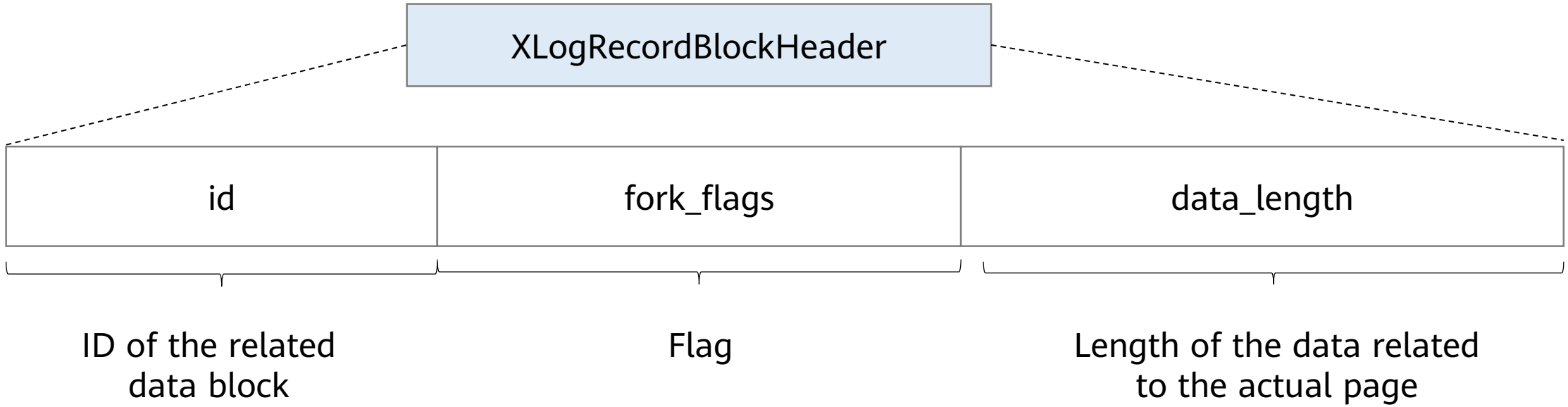
Xlog Record – Data (2)

- Header of the Xlog record data:
  - Contains at least one of the following structures: zero or more XLogRecordBlockHeader structures or zero or one XLogRecordDataHeaderShort or XLogRecordDataHeaderLong structure(s). When the Xlog record stores a full-page image (that is, a backup block), XLogRecordBlockHeader includes XLogRecordBlockImageHeader.
- The Xlog records the following types of data:
  - Backup block: records the complete page.
  - Non-backup block: Different operations correspond to different data.
  - Checkpoint record

HUAWEI

Header Information of Data Recorded in Logs – XLogRecordBlockHeader (1)

- **XLogRecordBlockHeader** is the header information of the block data appended to the Xlog record.



Header Information of Data Recorded in Logs – XLogRecordBlockHeader (2)

- XLogRecordBlockHeader is defined in the **src/include/access/xlogrecord.h** header file using the following source code:

```
typedef struct XLogRecordBlockHeader {
    uint8 id;                /* block reference ID */
    uint8 fork_flags;        /* fork within the relation, and flags */
    uint16 data_length;      /* number of payload bytes (not including page image) */
} XLogRecordBlockHeader;
```

  - **id**: page index, that is, the sequence number of the page information contained in the record.
  - **fork_flags**: flag and file name extension to which the page belongs.
  - **data_length**: length of the data related to the actual page (followed by the header structure).
    - Note 1: If the **BKPBLOCK_HAS_IMAGE** flag is **1**, the XLogRecordBlockImageHeader structure and continuous data on the page are following.
    - Note 2: If the **BKPBLOCK_SAME_REL** flag is not set, the RelFileNode structure is following.
    - Note 3: If the preceding situation does not exist, the page number is following.

HUAWEI

Header Information of Data Recorded in Logs – XLogRecordDataHeaderShort

- **XLogRecordDataHeaderShort** is used for the main data part of an Xlog record if the data length is less than 256 bytes. One byte is used to maintain the length. The structure is defined in the **src/include/access/xlogrecord.h** header file.
- Note: These structures are not currently used in code and are for documentation purposes only.

```
typedef struct XLogRecordDataHeaderShort {
    uint8 id;               /* XLR_BLOCK_ID_DATA_SHORT */
    uint8 data_length;      /* number of payload bytes */
} XLogRecordDataHeaderShort;
```

  - **id**: special XLR_BLOCK_ID_DATA_SHORT page index.
  - **data_length**: length of the short data record.

Header Information of Data Recorded in Logs – XLogRecordDataHeaderLong

- **XLogRecordDataHeaderLong** is used for the main data part of an Xlog record if the data length is greater than 256 bytes. The structure is defined in the **src/include/access/xlogrecord.h** header file.
- Note: These structures are not currently used in code and are for documentation purposes only.

```
typedef struct XLogRecordDataHeaderLong {
    uint8 id;       /* XLR_BLOCK_ID_DATA_LONG */
                    /* followed by uint32 data_length, unaligned */
} XLogRecordDataHeaderLong;
```

  - **id**: special XLR_BLOCK_ID_DATA_LONG page index

HUAWEI

Header Information of Data Recorded in Logs – XLogRecordBlockImageHeader

- XLogRecordBlockImageHeader contains additional header information when the entire page of the image is contained (that is, when **BKPBLOCK_HAS_IMAGE** is set). You can use the Xlog code (as a normal form of data compression) to check if the openGauss data page contains an unused "hole" with zero bytes. If **hole_length** is greater than 0, the "hole" has been removed from the stored data, and it is not counted in the CRC of the Xlog record. Therefore, the actual amount of block data is **BLCKSZ** minus **hole_length**, in bytes.

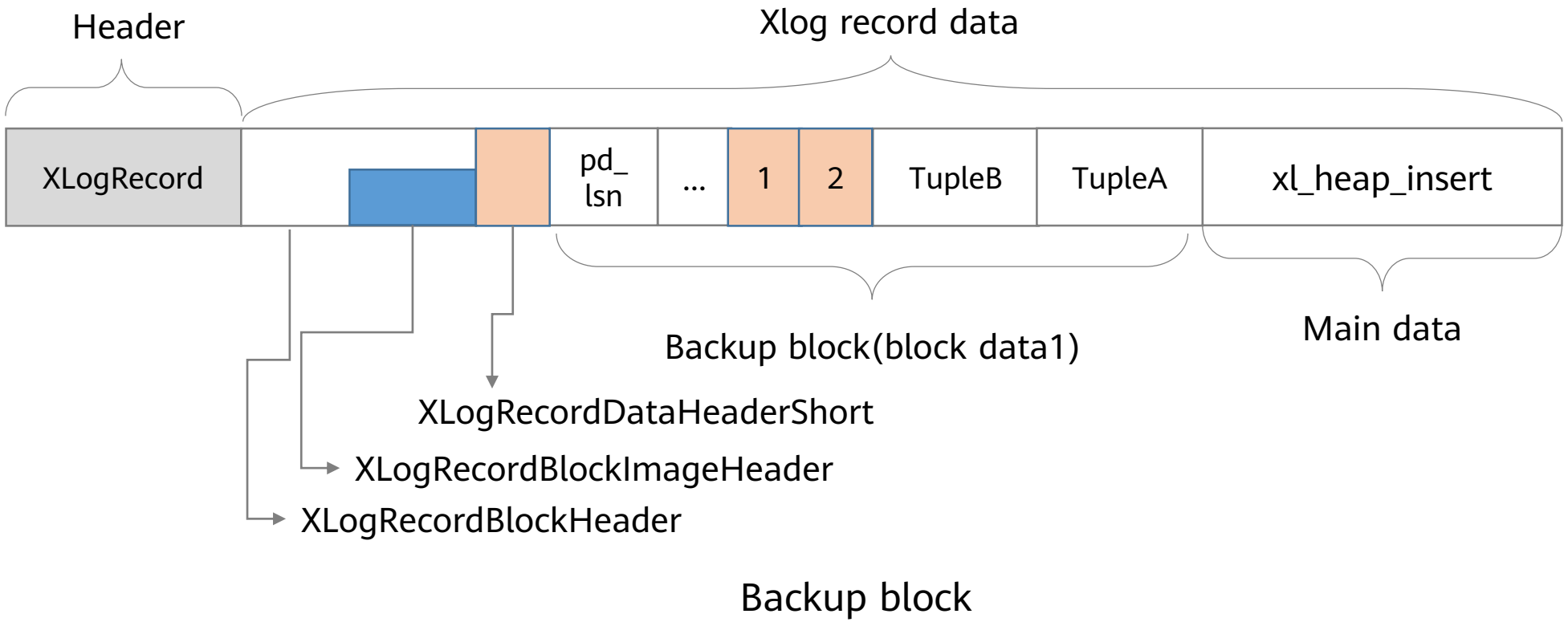
```
typedef struct XLogRecordBlockImageHeader {
    uint16 hole_offset; /* number of bytes before "hole" */
    uint16 hole_length; /* number of bytes in "hole" */
} XLogRecordBlockImageHeader;
```

  - **hole_offset**: offset of the current block.
  - **hole_length**: length of the current block.

HUAWEI

Log Recording Data – Backup Block (1)

- The following figure shows a backup block created by the INSERT statement. It consists of four data structures and a data object.



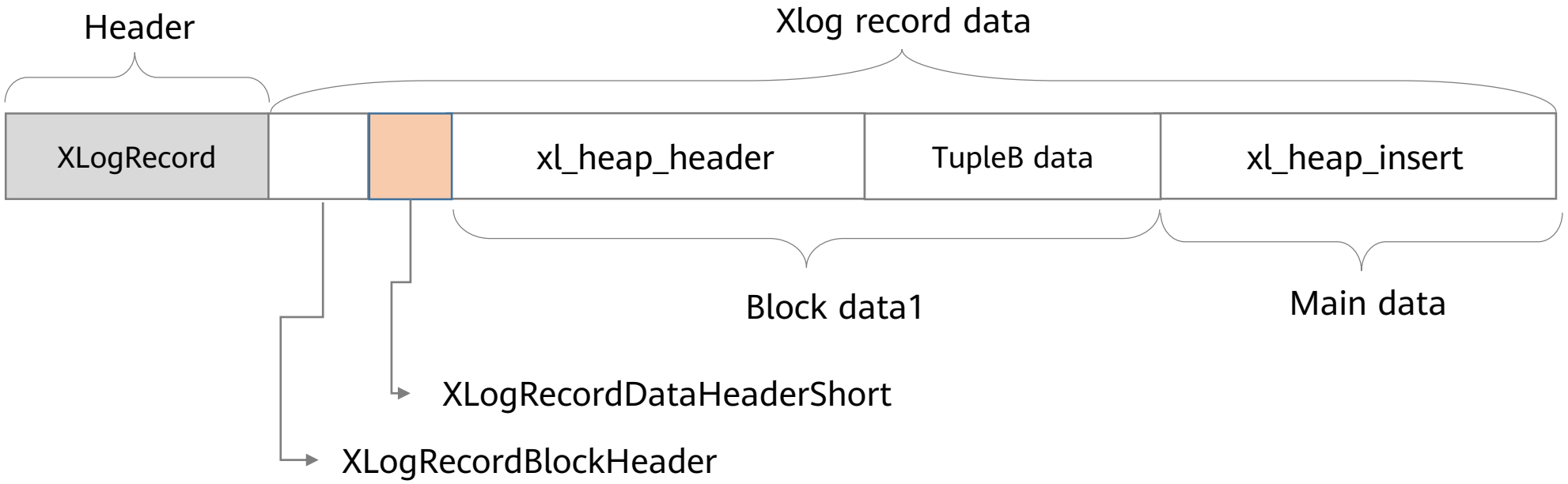
Log Recording Data – Backup Block (2)

- Four data structures and one data object of the backup block created by the INSERT statement:
  - XLogRecord structure: common header information.
  - XLogRecordBlockHeader structure: contains variables used to locate blocks in the database cluster and an XLogRecordBlockImageHeader.
  - XLogRecordDataHeaderShort structure: stores the length of the xl_heap_insert structure, which is the main data of a record.
  - Backup block: block data.
  - xl_heap_insert structure:

```
typedef struct xl_heap_insert {
    OffsetNumber offnum; /* inserted tuple's offset */
    uint8 flags;
    /* xl_heap_header & TUPLE DATA in backup block 0 */
} xl_heap_insert;
```

Log Recording Data – Non-Backup Block (1)

- The following figure shows a non-backup block created by the INSERT statement. It consists of four data structures and a data object.



Xlog record of INSERT statement

Log Recording Data – Non-Backup Block (2)

- Four data structures and one data object of the non-backup block created by the INSERT statement:
  - XLogRecord structure (common header)
  - XLogRecordBlockHeader structure
  - XLogRecordDataHeaderShort structure
  - An inserted tuple (more precisely, an xl_heap_header structure with complete inserted data)
  - xl_heap_insert structure (main data)
- XLogRecordBlockHeader contains variables used to locate a block in the database cluster. The variables specify the block into which the tuple is inserted and the length of the data part of the tuple to be inserted. XLogRecordDataHeaderShort contains the length of a new xl_heap_insert structure, which has only two values: the offset of the newly inserted tuple within the block and the visibility flag.

HUAWEI

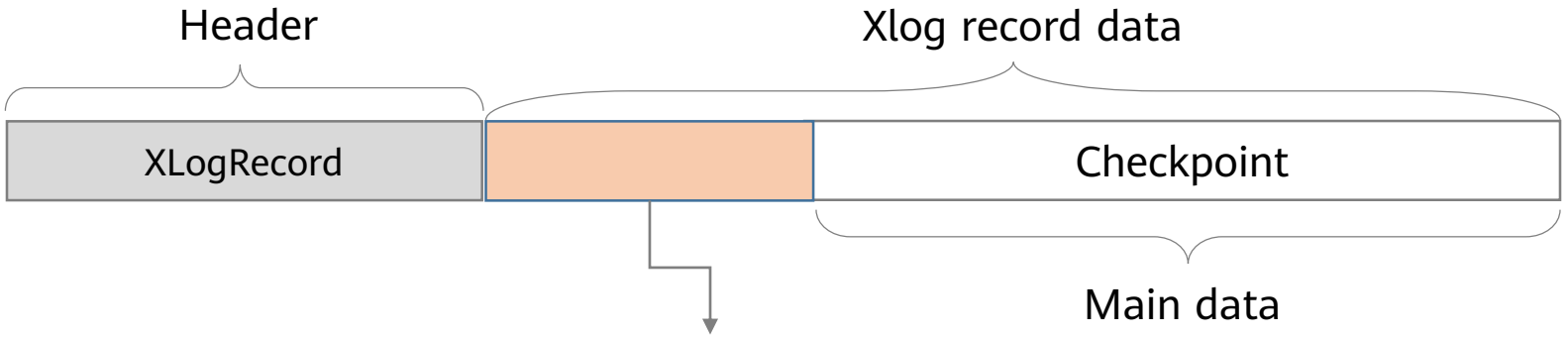
Log Recording Data – Non-Backup Block (3)

- The source code of the xl_heap_header structure is as follows:

```
/*
 * We don't store the whole fixed part (HeapTupleHeaderData) of an inserted
 * or updated tuple in WAL; we can save a few bytes by reconstructing the
 * fields that are available elsewhere in the WAL record, or perhaps just
 * plain needn't be reconstructed.  These are the fields we must store.
 * NOTE: t_hoff could be recomputed, but we may as well store it because
 * it will come for free due to alignment considerations.
 */
typedef struct xl_heap_header {
    uint16 t_infomask2;
    uint16 t_infomask;
    uint8 t_hoff;
} xl_heap_header;
```

Log Record Data – Internal Structure of Checkpoint Records (1)

- The following figure shows a non-backup block created by the INSERT statement. It consists of the following three data structures:
  - XLogRecord structure (header part)
  - XLogRecordDataHeaderShort structure (including the length of the main data)
  - Struct CheckPoint (main data)



Xlog record of a checkpoint action

HUAWEI

Log Record Data – Internal Structure of Checkpoint Records (2)

- The CheckPoint structure is defined in the **src/include/catalog/pg_control.h** header file.
- The CheckPoint structure is the body of the checkpoint Xlog record and is used for possible DR.
- The source code is shown on the right.

```
typedef struct CheckPoint {
    XLogRecPtr        redo;                 /* next RecPtr available when we began to
                                             * create CheckPoint (i.e. REDO start point) */
    TimeLineID        ThisTimeLineID;   /* current TLI */
    bool              fullPageWrites;       /* current full_page_writes */
    TransactionId     nextXid;              /* next free XID */
    Oid               nextOid;              /* next free OID */
    MultiXactId       nextMulti;            /* next free MultiXactId */
    MultiXactOffset   nextMultiOffset; /* next free MultiXact offset */
    TransactionId     oldestXid;       -    /* cluster-wide minimum datfrozenxid */
    Oid               oldestXidDB;          /* database with minimum datfrozenxid */
    pg_time_t         time;                 /* time stamp of checkpoint */
    XLogSegNo         remove_seg;       /* the xlog segno we keep during this checkpoint */
    /*
     * Oldest XID still running. This is only needed to initialize hot standby
     * mode from an online checkpoint, so we only bother calculating this for
     * online checkpoints and only when wal_level is hot_standby. Otherwise
     * it's set to InvalidTransactionId.
     */
    TransactionId     oldestActiveXid;
} CheckPoint;
```


HUAWEI

Writing WALs – INSERT (1)

- Run the **INSERT** statement to insert a tuple.

```
openGauss=insert into t1 values('A');
```

- After the statement is executed, openGauss calls the exec_simple_query() function to execute the input SQL statements, for example, INSERT, UPDATE, and DELETE, as a simple query. This function is used for queries that do not return result sets. When a query is executed, the function is blocked until the query is complete and returns a status code indicating the execution result.
- The exec_simple_query() function is defined in **src/gausskernel/process/tcop/postgres.cpp**.

```
static void exec_simple_query(const char* query_string, MessageType messageType, StringInfo msg = NULL)
{
    CommandDest dest = (CommandDest)t_thrd.postgres_cxt.whereToSendOutput;
    MemoryContext oldcontext;
    MemoryContext OptimizerContext;
    ...
}
```


HUAWEI

Writing WALs – INSERT (2)

- When the exec_simple_query function is executed, the following functions related to log writing are directly or indirectly called.

No.	API	Source File	Function
1	ExtendCLOG()	clog.cpp	Writes the current transaction status **IN_PROGRESS** to the Clog.
2	heap_insert()	heapam.cpp	Inserts a tuple, creates an Xlog record, and calls the XLogInsert function.
3	XLogInsert()	xloginsert.cpp	Writes the Xlog record of the inserted tuple to the WAL buffer and updates **pd_lsn** of the page.
4	finish_xact_command()	postgres.cpp	Executes and commits.
5	XLogInsert()	xloginsert.cpp	Writes the Xlog record of the commit behavior to the WAL buffer.
6	XLogWrite()	xlog.cpp	Writes all Xlogs in the WAL buffer to WAL files.
7	TransactionIdCommitTree()	transam.cpp	Changes the transaction status from **IN_PROGRESS** to **COMMITTED** in the Clog.

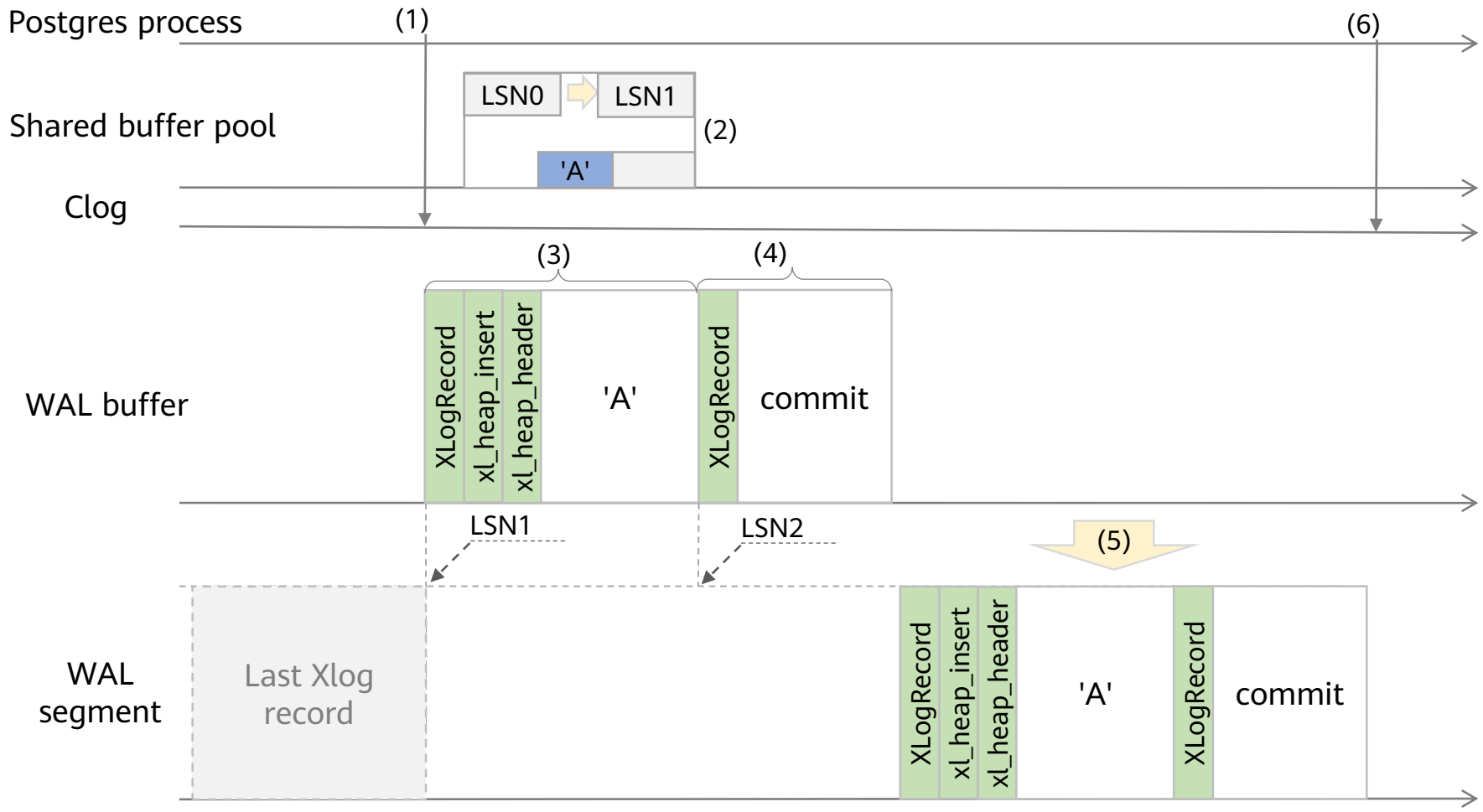
HUAWEI

Writing WALs – INSERT (3)

- The functions in the previous slide represent the main steps when writing logs.
  - Step 1: Use the ExtendCLOG() function to set the current transaction status to **IN_PROGRESS** in the Clog in the memory.
  - Step 2: Use the heap_insert() function to insert a tuple into the shared buffer, create an Xlog record, and call the XLogInsert() function.
  - Step 3: Use the XLogInsert() function to write the Xlog record created in the heap_insert() function into the **LSN1** position of the WAL buffer, and update **pd_lsn** of the page into which the tuple is inserted from **LSN0** to **LSN1**.
  - Step 4: Use the finish_xact_command() function to commit the current transaction and create an Xlog record of the commit action, and call the XLogInsert() function to write the Xlog record to **LSN2** in the WAL buffer.
  - Step 5: Use the XLogWrite() function to write all Xlog records in the WAL buffer to the WAL segment file.
  - Step 6: Use the TransactionIdCommitTree() function to set the status of the transaction in the Clog from **IN_PROGRESS** to **COMMITTED**.

HUAWEI

Writing WALs – INSERT (4)



HUAWEI

Writing WALs – INSERT (5)

- Postgres process:
  - This is a background process that periodically checks the WAL buffer and writes all Xlog records that have not been written into the WAL segment file. It prevents burst writes of Xlog records. If this process is not enabled, writing Xlog records may become a bottleneck when a large amount of data is committed at a time. The WAL writer is enabled by default and cannot be disabled. However, the check interval can be specified by the **wal_writer_delay** parameter. The default value is **200**, in ms.
- Xlog records are written into WAL segment files in the following scenarios:
  - A running transaction is committed or aborted.
  - The WAL buffer is filled with written tuples. The WAL buffer size is specified by **wal_buffers**.
  - The WAL writer process periodically writes data.

HUAWEI

Other Scenarios Where WALs Are Written

- Scenario 1:
  - An Xlog is generated for non-DML operations. For example, the COMMIT operation writes an Xlog record that contains the ID of the committed transaction.
- Scenario 2:
  - The checkpoint operation writes overview information about the checkpoint to the Xlog record.
- Scenario 3:
  - SELECT statements create Xlog records in some special cases. For example, if during SELECT statement processing, unnecessary tuples need to be deleted and necessary tuples need to be concatenated due to a heap-only tuple (HOT), the Xlog record of the corresponding page is modified and written into the WAL buffer.

HUAWEI

Other APIs Involved in Writing WALs

- A single log record uses insert (write) and read APIs. The table below lists the APIs used for a complete log insertion operation.

No.	API	Resource Management
1	XLogBeginInsert	Initializes global variables related to log insertion.
2	XLogRegisterData	Registers the subject data recorded in the log.
3	XLogRegisterBuffer/XLogRegisterBlock	Registers the log to record the meta information of the related page.
4	XLogRegisterBufData	Registers the log to record the data of the related page.
5	XLogInsert	Inserts real logs, including logs in 5.1 and 5.2.
5.1	XLogRecordAssemble	Recombines all the registered log information into a complete binary string.
5.2	XLogInsertRecord	Preoccupies the offset and length in the entire logical log, calculates the CRC is calculated, and copies the complete log record to the log shared buffer.

HUAWEI

Contents

1. WAL Mechanism
2. **Checkpoint Mechanism**
3. Restoration Using Redo Logs
4. Run Logs and Performance Logs
5. Log Checking and Clearance

HUAWEI

Checkpoint Mechanism

- Checkpoint: a point in the transaction log sequence where all data files have been updated to reflect the information in the log. All data files will be flushed to a disk.
- Function: This mechanism stores data in the database memory to disks at a certain time. openGauss periodically stores the data of both committed and uncommitted transactions to disks. The data and redo logs can be used for database restoration if a database restarts or breaks down.
- Setting transaction log checkpoints: By default, WALs place checkpoints in transaction logs at intervals. You may use gs_guc to specify the GUC parameters **checkpoint_segments**, **checkpoint_timeout**, and **incremental_checkpoint_timeout** to adjust the atomized checkpoint intervals.

HUAWEI

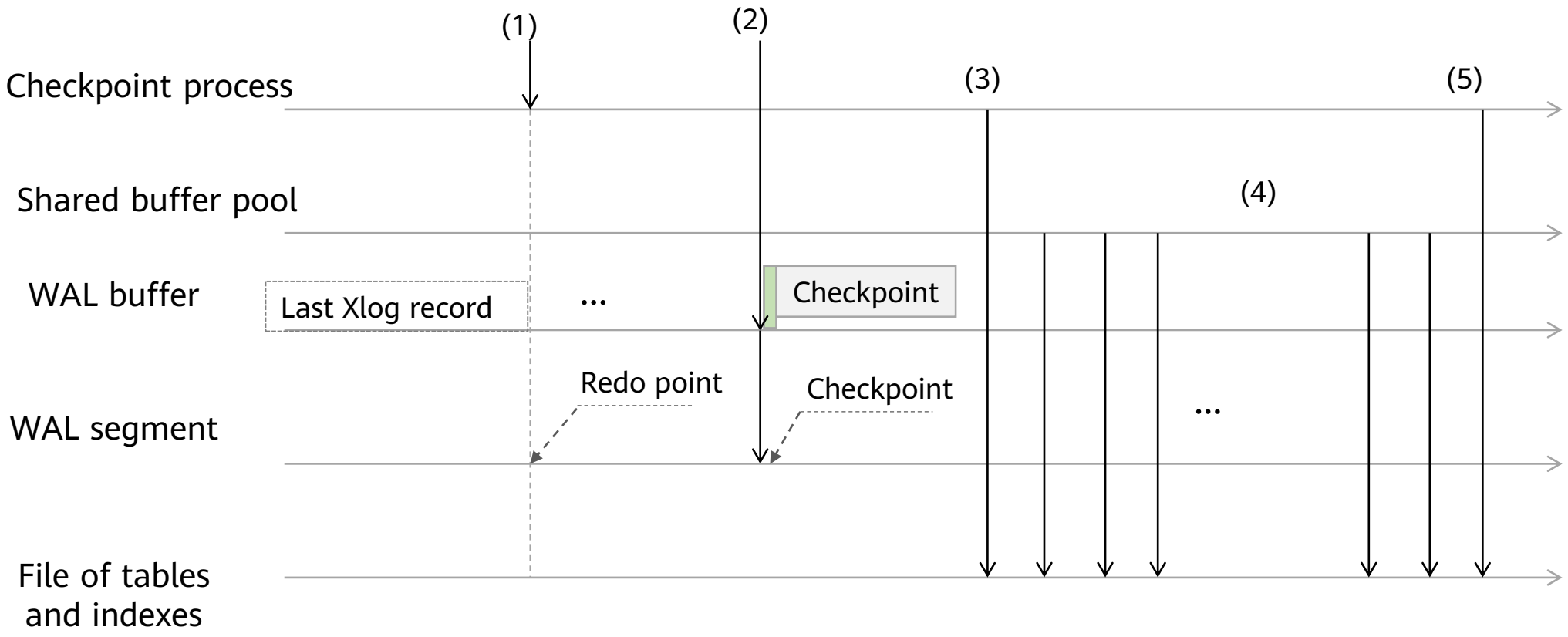
Scenarios That Trigger Checkpoint Operations

- The background checkpointer thread operates in any of the following scenarios:
  - **Scenario 1:** The execution time since the last checkpoint has exceeded the value of **checkpoint_timeout** (300 seconds by default).
  - **Scenario 2:** The total size of WAL segment files in the **pg_xlog** directory exceeds the value of **max_wal_size** (**1GB** by default, that is, 64 files).
  - **Scenario 3:** The database server is shut down in fast mode.
  - **Scenario 4:** The checkpoint command is executed manually.
- Precautions
  - Only system administrators and O&M administrators can call the checkpoint.
  - The checkpoint command forces an immediate checkpoint without waiting for a regular checkpoint scheduled by the system.

HUAWEI

Checkpoint Execution – Process

- The checkpoint process is as follows:



Checkpoint Execution – Main Steps

- **Step 1**: Store the redo point in the memory when the checkpoint process starts. The redo point is the write location recorded in the Xlog at the latest checkpoint and where database restoration will begin.
- **Step 2**: Write the Xlog record (checkpoint) corresponding to the checkpoint to the WAL buffer. The data part of the record is defined by the CheckPoint structure and contains some variables, for example, the location of the redo point in step 1. In addition, the location where a checkpoint record is written is literally called a checkpoint.
- **Step 3**: Flush all data in the shared memory (for example, the content of a Clog) into persistent storage.
- **Step 4**: Gradually write all dirty pages in the shared buffer pool to the storage.
- **Step 5**: Update the **pg_control** file, which contains some basic information, such as the location of the last checkpoint.

HUAWEI

Checkpoint Mechanism – Checkpointer Background Thread

- Checkpointer is a background process in openGauss. It flushes data from the memory to disks to ensure data durability and consistency. The Checkpointer thread is automatically scheduled after a certain amount elapses since the last checkpoint, and can also send a signal to perform the requested checkpoint.
- Checkpointer functions:
  - Data durability and consistency: Checkpointer periodically flushes dirty pages in the memory to disks to ensure data durability and consistency.
  - Memory space: When writing dirty pages to disks, Checkpointer releases the modified pages from the memory so that other operations can use the space.
  - Controlling system crash restoration: If the database crashes, Checkpointer helps perform system restoration to ensure data consistency.

HUAWEI

Checkpoint Mechanism – Checkpointer Background Thread Execution

- The process of Checkpointer is as follows:
  - Checkpointer periodically checks dirty pages (that is, pages that have been modified but have not been written to disks) in the database. If the number of dirty pages reaches a certain threshold or a certain period of time has passed, Checkpointer triggers a checkpoint.
  - Before a checkpoint starts, Checkpointer sends a signal to all background processes, requesting them to stop modifications on the database. This prevents the database status from changing during the checkpoint.
  - Checkpointer writes dirty pages in the memory to disks and updates the corresponding metadata (such as LSNs) to ensure data durability and consistency.
  - After the checkpoint is complete, Checkpointer sends a signal to all background processes, requesting them to continue to modify the database.

HUAWEI

Checkpoint Mechanism – pg_control File

- **pg_control** is a key file in openGauss. It stores the metadata of the database cluster, including the database version, status, location of the last checkpoint, and system identifier. If the file is damaged, the system cannot determine the starting point of the restoration, and the database cannot be restored.
- In openGauss, the **pg_control** file is usually stored in the *$GAUSSDATA*/**pg_control** directory. When the database starts, the system reads information in the **pg_control** file to determine the database status and version and perform necessary initialization operations. Therefore, the integrity and accuracy of the **pg_control** file are critical for openGauss to run normally.
- The **pg_control** file stores the following information related to database restoration:
  - **State**: status of the database server when the latest checkpoint starts. There are seven states, including: **starting up** indicates that the database is being started; **shutdown** indicates that the database was shut down properly; **in production** indicates that the database is running.
  - **Latest checkpoint location**: LSN location of the latest checkpoint in the log file.
  - **Prior checkpoint location**: LSN location of the previous checkpoint in the log file.

HUAWEI

Checkpoint Mechanism – pg_control Content

- The **pg_control.h** header file describes the **pg_control** content. Part of the content is as follows:
  - CheckPoint structure: describes the main part of the CheckPoint in the Xlog record.
  - CheckPointNew: new structure, including the old checkpoint and transaction commit sequence number **CommitSeqNo**
  - CheckPointPlus structure: adds the length to distinguish between checkpoints from different versions.
  - Xlog information value corresponding to the Xlog resource manager, for example, **XLOG_CHECKPOINT_SHUTDOWN=0x00**
  - ControlFileData structure: defines the main **pg_control** content.
  - Physical size of the **pg_control** file, for example, **PG_CONTROL_SIZE=8192 byte (8KB)**
  - Default file size updated by the LSN of an Xlog, for example, **PG_LSNXLOGFLUSHCHK_FILESIZE=512 byte**

HUAWEI

Checkpoint Mechanism – CheckPoint Structure Source Code

- **pg_control** retains the latest checkpoint copy for possible troubleshooting. The source code is as follows:

```
typedef struct CheckPoint {
    XLogRecPtr          redo;                 /* next RecPtr available when we began to
                                               * create CheckPoint (i.e. REDO start point) */
    TimeLineID          ThisTimeLineID;    /* current TLI */
    bool                fullPageWrites;    /* current full_page_writes */
    TransactionId       nextXid;              /* next free XID */
    Oid                 nextOid;                 /* next free OID */
    MultiXactId         nextMulti;            /* next free MultiXactId */
    MultiXactOffset     nextMultiOffset; /* next free MultiXact offset */
    TransactionId       oldestXid;            /* cluster-wide minimum datfrozenxid */
    Oid                 oldestXidDB;             /* database with minimum datfrozenxid */
    pg_time_t           time;                  /* time stamp of checkpoint */
    XLogSegNo           remove_seg;       /*the xlog segno we keep during this checkpoint*/
    /*
     * Oldest XID still running. This is only needed to initialize hot standby
     * mode from an online checkpoint, so we only bother calculating this for
     * online checkpoints and only when wal_level is hot_standby. Otherwise
     * it's set to InvalidTransactionId.
     */
    TransactionId     oldestActiveXid;
} CheckPoint;
```


HUAWEI

Checkpoint Mechanism – Viewing pg_control Content

- openGauss provides the pg_controldata tool to read the **pg_control** file. The read content is as follows:

```
pg_control version number:            923
Catalog version number:               201611171
Database system identifier:           5765045736331161 41
Database cluster state:               in production
pg_control last modified:             Mon 25 Mar 2024 08:50:17 PM CST
Latest checkpoint location:           0/244D4A8
Prior checkpoint location:            0/244D388
Latest checkpoint's REDO location:    0/244D428
Latest checkpoint's TimeLineID:       1
Latest checkpoint's full_page_writes: off
Latest checkpoint's NextXID:          19960
Latest checkpoint's NextOID:          15731
Latest checkpoint's NextMultiXactId:  2
Latest checkpoint's NextMultiOffset:  0
Latest checkpoint's oldestXID:        12780
Latest checkpoint's oldestXID's DB:   15725
Latest checkpoint's oldestActiveXID:  19960
Latest checkpoint's remove lsn:       0/1
Time of latest checkpoint:            Mon 25 Mar 2024 08:50:17 PM CST
Minimum recovery ending location:     0/0
```

```
Backup start location:                0/0
Backup end location:                  0/0
End-of-backup record required:        no
Current wal_level setting:            hot_standby
Current max_connections setting:      5050
Current max_prepared_xacts setting:   800
Current max_locks_per_xact setting:   256
Maximum data alignment:               8
Database block size:                  8192
Blocks per segment of large relation: 131072
WAL block size:                       8192
Bytes per WAL segment:                16777216
Maximum length of identifiers:        64
Maximum columns in an index:          32
Maximum size of a TOAST chunk:        1996
Date/time type storage:               64-bit integers
Float4 argument passing:              by value
Float8 argument passing:              by value
Database system TimeLine:             1
```


HUAWEI

Contents

1. WAL Mechanism
2. Checkpoint Mechanism
3. **Restoration Using Redo Logs**
4. Run Logs and Performance Logs
5. Log Checking and Clearance

HUAWEI

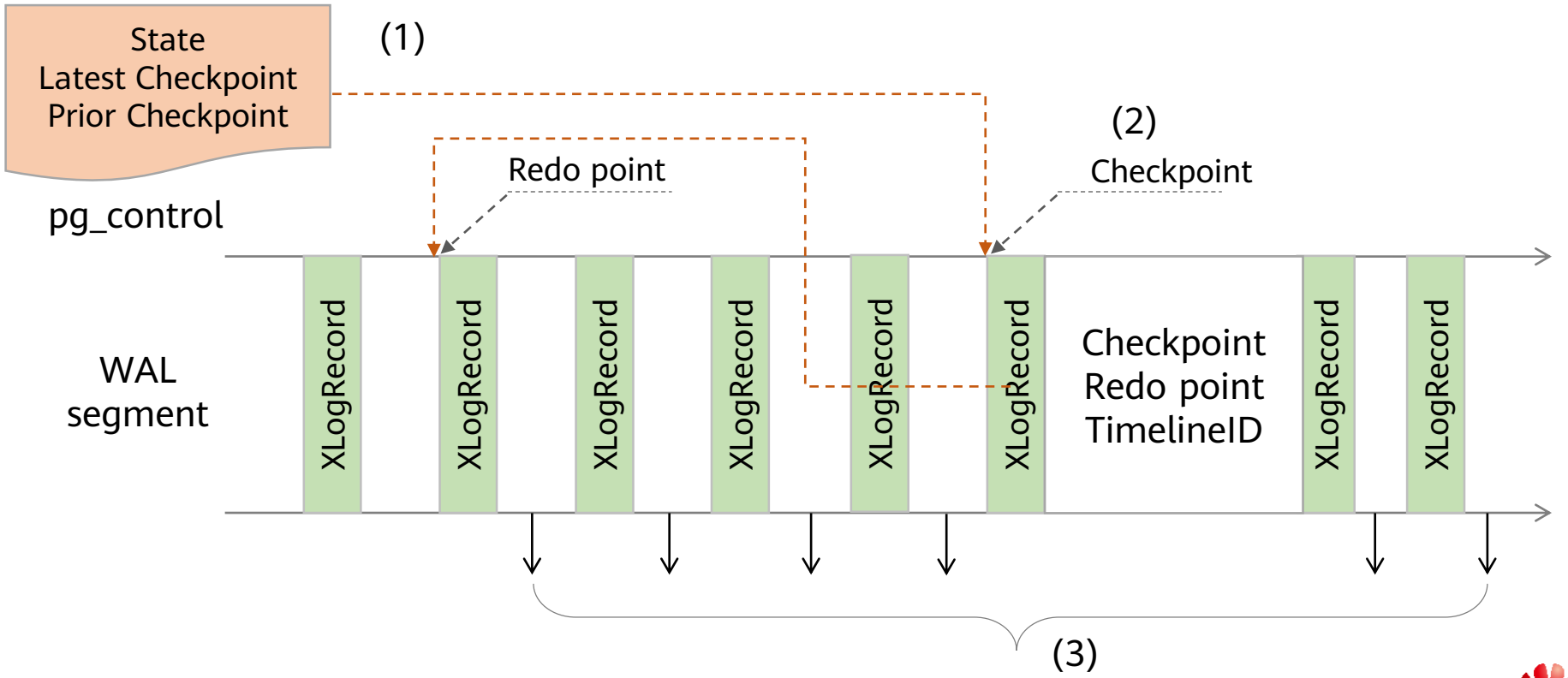
Redo Log Overview

- Redo logs: record the operations performed on the database. The log contains the information required to perform the operations again. If a database is faulty, redo logs can be used to restore the database to its original state.
- WALs: logs used to restore a damaged database. They are also called redo logs. You are advised to routinely back up WALs.
- Redo log copy: In primary and standby mode, when data is written to the primary node, redo logs are automatically synchronized to the standby node. A success message indicates a successful write operation. However, if the write operation fails, the entire transaction fails. In this way, data is stored in both the primary and standby nodes, ensuring data reliability across the database.
- Parallel redo log replay: The primary node usually writes logs using multiple threads. If the standby node replays logs using a single thread, the gap between the standby and primary nodes may be widening. Using multiple threads to replay logs in parallel can improve the log replay speed.

HUAWEI

Starting Restoration Based on Redo Logs (1)

- openGauss uses redo logs for restoration. If the database server crashes, openGauss restores the database cluster by chronologically replaying Xlog records in WAL segment files from redo points. When the system starts, it first reads the **pg_control** file and starts the restoration process.



HUAWEI

Starting Restoration Based on Redo Logs (2)

- The procedure for starting the restoration is as follows:
  - openGauss reads all items in the **pg_control** file during startup. If the value of **state** is **in_production**, openGauss enters the recovery mode because the database did not shut down properly. If the value of **state** is **shut_down**, openGauss starts up normally.
  - openGauss reads the latest checkpoint from the appropriate WAL segment file, writes the location of the record in the **pg_control** file, and obtains the redo point from the checkpoint. If the latest checkpoint is invalid, openGauss reads the previous checkpoint. If both records are unreadable, self-recovery is abandoned.
  - A proper resource manager is used to chronologically read and replay Xlog records, starting from the redo point at the last location of the latest WAL segment file. When it encounters an Xlog record that belongs to a backup block, it overwrites the page of the corresponding table regardless of its LSN. In other cases, the Xlog record (of a non-backup block) is replayed only when the LSN of the record is greater than **pd_lsn** of the corresponding page.

HUAWEI

Reading the Latest Checkpoint Records

- Redo logs are crucial for openGauss restoration. During the restoration, the latest checkpoint records need to be read to ensure database consistency. The process of reading the latest checkpoint records is as follows:
  1. Open the redo log file and locate the last checkpoint record.
  2. Read the redo start location column in the checkpoint record. This column indicates the location of the latest redo log.
  3. Use the redo start location to begin the restoration based on redo logs.
- In openGauss, the checkpoint process periodically writes checkpoint records to ensure that the database can be quickly restored if it breaks down.

HUAWEI

Using a Proper Resource Manager to Restore Data

- To use a proper resource manager to restore data, perform the following steps:
  1. Verify that the database is faulty and stop the database service.
  2. Use a proper resource manager to open the data directory of the database and find the redo log file.
  3. Run the **pg_resetxlog** command to initialize the redo log file to a new log file and generate a new control file.
  4. Start the database service and restore data. During the restoration, the database automatically reads the content in the redo log file and restores the data to the status when the fault had occurred.

HUAWEI

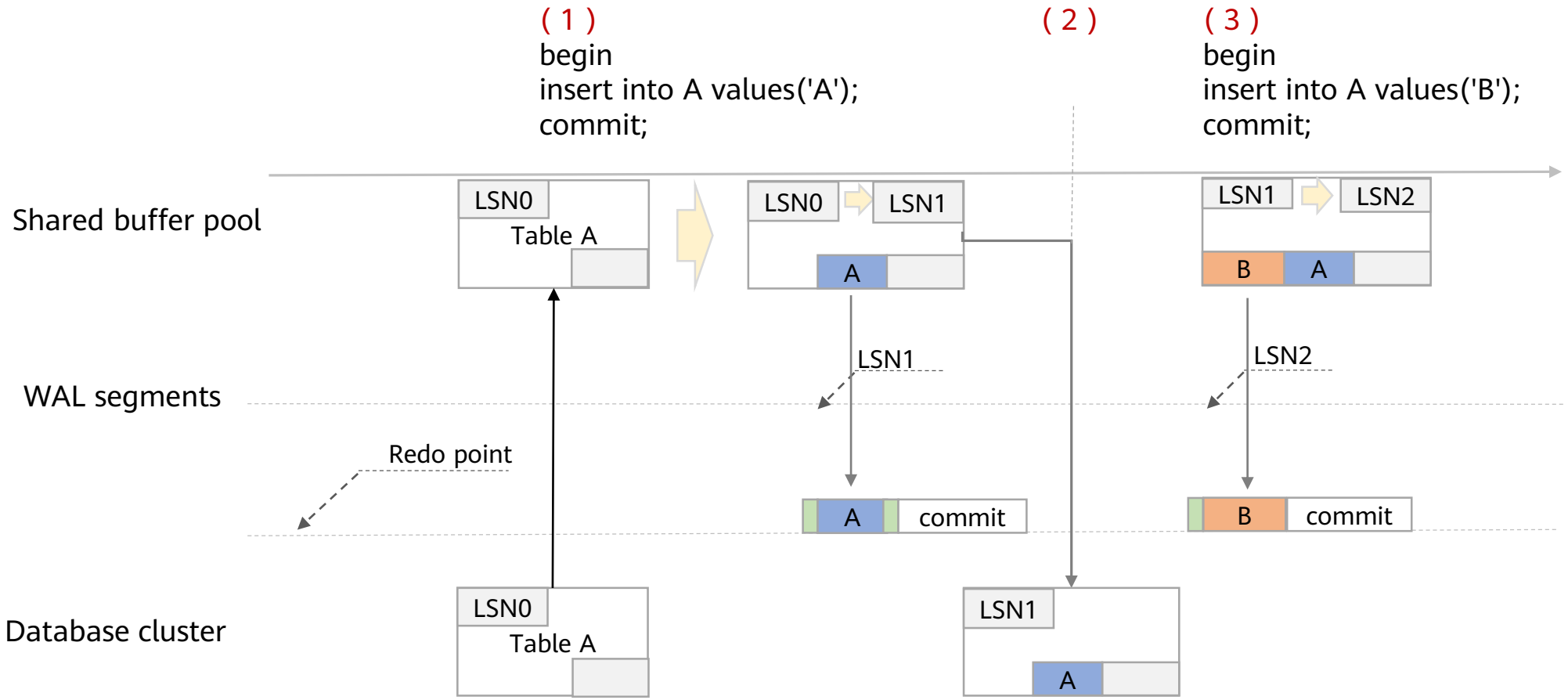
LSN Comparison Overview

- Objective
  - The LSN recorded in the Xlog of a non-backup block is compared with **pd_lsn** of the corresponding page.
- Necessity
  - If the replay sequence of non-backup blocks is incorrect or non-backup blocks are replayed multiple times, the database cluster will be inconsistent. In short, a redo (replay) operation on a non-backup block is not idempotent. Therefore, in order to preserve the correct replay order, non-backup block records are replayed only if their LSN is greater than **pd_lsn** of the corresponding page.
  - Since the redo operation of the backup block is idempotent, the backup block can be replayed repeatedly regardless of its LSN.

HUAWEI

LSN Comparison Example – Inserting Data While the Background Writer Thread is Working (1)

- The following figure shows the process of inserting data into table A.



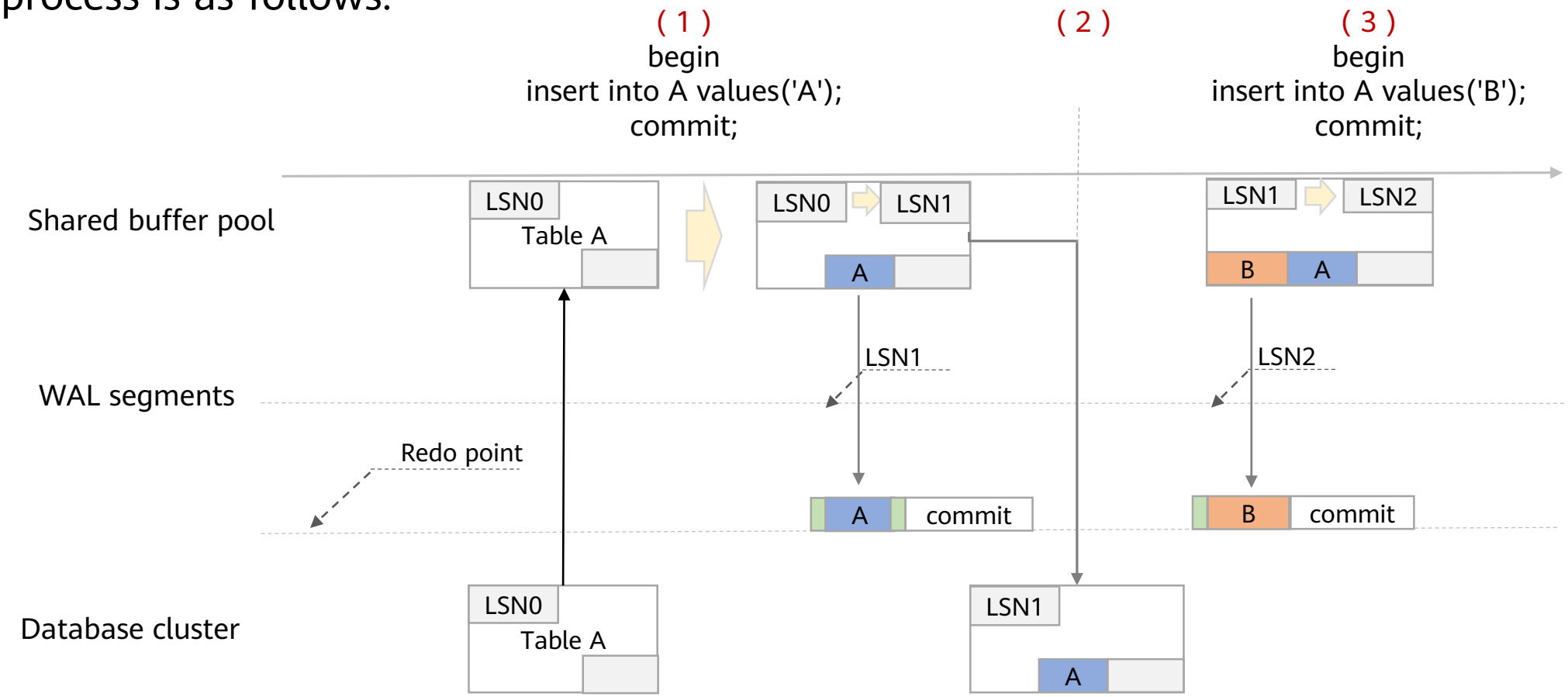
LSN Comparison Example – Inserting Data while the Background Writer Thread is Working (2)

- Inserting data into table A:
  - openGauss inserts a tuple into table A and writes an Xlog record to **LSN1**.
  - The background writer process writes the page of table A to the storage. In this case, **pd_lsn** of this page is **LSN1**.
  - openGauss inserts a new tuple into table A and writes an Xlog record to **LSN2**. The modified page has not been written to the storage.
- Note: The page of table A has been written to the storage in a batch.

HUAWEI

LSN Comparison – Database Restoration Example (1)

- Shut down the database in immediate mode, and then start the database. The restoration process is as follows:



LSN Comparison – Database Restoration Example (2)

- Shut down the database in immediate mode, and then start the database.
  - openGauss loads the first Xlog record and the page of table A but does not replay it because the LSN of the record is not greater than that of table A (both values are **LSN1**). It is clear that record replay is not required.
  - Then, openGauss replays the second Xlog record because the LSN (**LSN2**) of the record is greater than the LSN (**LSN1**) of table A.
- If the replay sequence of non-backup blocks is incorrect or non-backup blocks are replayed multiple times, the database cluster will be inconsistent. In short, a redo (replay) operation on a non-backup block is not idempotent. Therefore, in order to preserve the correct replay order, non-backup block records are replayed only if their LSN is greater than **pd_lsn** of the corresponding page. Since the redo operation of the backup block is idempotent, the backup block can be replayed repeatedly regardless of its LSN.

HUAWEI

Database Restoration – PITR

- Point-in-time recovery (PITR) refers to the combination of physical backup and WAL recovery in openGauss. A physical backup, also called a base backup, is a snapshot taken when a database cluster is running.
- Function:
  - The basic backup and archive log data is used to restore the database to any point in time after the basic backup.
- Working principles:
  - Based on the basic backup, you can redo the Xlog records in archive logs. PITR helps database administrators restore the database to the state before a misoperation. For example, if you mistakenly delete all table data, you can use PITR to restore the data.

HUAWEI

Database Restoration – PITR Conditions

- To perform PITR, the following conditions must be met:
  - Full data files have been physically backed up.
  - WAL files have been archived.
- Physical backup:
  - Physical backup refers to copying entire physical files of the database to the backup storage device. Physical backup usually includes all data, control, log, and configuration files, which are copied to the backup device. Physical backup is fast, and the restoration is convenient. However, the backup file is large, which is inconvenient for cross-platform restoration.
- Logical backup:
  - Logical backup is a process of exporting logical data and objects from a database to a backup file. Logical backup usually includes logical objects (such as tables, views, stored procedures, and triggers) and their data. Logical backup files are relatively small, which facilitates cross-platform restoration. However, backup and restoration are relatively slow.

HUAWEI

Database Restoration – PITR Process

- The PITR process is as follows:
  - Replace the target database directory with the physical backup files.
  - Delete all files in the database directory **pg_xlog**.
  - Copy archived WAL files to the **pg_xlog** directory.
  - Create the **recovery.conf** file in the database directory and specify the database restoration degree.
  - Restart the database, connect to the database, and check whether it is restored to the expected state.
  - If the database is in the expected status, call pg_xlog_replay_resume() to allow the primary node to provide external services.

HUAWEI

Contents

1. WAL Mechanism
2. Checkpoint Mechanism
3. Restoration Using Redo Logs
4. **Run Logs and Performance Logs**
5. Log Checking and Clearance

HUAWEI

System Logs

- Description
  - System logs include those generated by database nodes when openGauss is running, and those generated when openGauss is deployed.
- Function
  - If an error occurs while openGauss is running, you can use system logs to locate the cause and troubleshoot it.
- Log storage directory
  - Run logs of database nodes are stored in the corresponding folders in the **/var/log/gaussdb/***Username***/pg_log** directory.
  - The logs generated during database installation and deployment are stored in the corresponding folders in **/var/log/gaussdb/***Username***/om**.

System Logs – File Naming Format

- Log naming rules:
  - Database node run logs
    - **postgresql-Creation time.log**. By default, a new log file is generated at 0:00 every day, or when the latest log file exceeds 16 MB or a database instance (database node) is restarted.
  - Logs generated during database installation
    - Preinstallation logs: **gs_preinstall-***Creation time***.log**, for example, **gs_preinstall-2024-03-25_203913.log**.
    - Installation logs: **gs_install-***Creation time***.log**, for example, **gs_install-2024-03-25_204212.log**.
  - Database startup and stop logs
    - **gs_ctl-***Creation time***.log**, for example, **gs_ctl-2024-03-25_204315-current.log**.
    - Logs whose names contain **current** are current log files; if they do not, then they are historical logs. When a process is called for the first time, a log file whose name contains **current** is created. If the size of this file exceeds 16 MB, the file is renamed using the historical log format, and a new log file is generated.

System Logs – Log Content Format

- Log content description
  - By default a line in a database node log is arranged in the following format:
    - Date+Time+Time zone+Username+Database name+Session ID+Log level+Log content
  - By default, a line in CM Agent, CM Server, and OM Monitor logs is arranged in the following format:
    - The SYSTEM_CALL log records the tool commands called by the CM Agent when the system is running in the following format: Time+Time zone+Session ID+Log content.
    - By default, a line in a key_event log is arranged in the following format: Time+Thread ID+Thread name: Key event type+Arbitration object instance ID+Arbitration details.

HUAWEI

System Logs – Database Run Logs (1)

- Database logs record the operations (starting, running, and stopping) performed on servers. Database users can view logs to quickly locate faults, find their root causes, and rectify the faults accordingly.
  - Log file location: You can check **log_directory** in the configuration file **postgresql.conf** or use the **gsql** tool to run the **show log_directory** command to obtain the log file location. By default, logs are stored in the *$GAUSSLOG* directory.
  - Log files are named in the **postgresql-%Y-%m-%d_%H%M%S.log** format, in which **%Y** indicates the year, **%m** indicates the month, **%d** indicates the date, and **%H%M%S** indicates the hour, minute, and second. Log names help you easily find a log file at a specific time. For example, **postgresql-2015-06-08_202702.log** represents the logs generated at 20:27:02 on June 8, 2015.

HUAWEI

System Logs – Database Run Logs (2)

- Log file content: The database obtains **log_line_prefix** of run log files from the configuration file **postgresql.conf** upon startup. The prefix of a run log file can be customized. The default setting is *%m %p*, where *%m* indicates the timestamp (ms) when the log is generated, and *%p* indicates the thread that outputs the log, that is, the thread ID. Next to the prefix is the log level and content. Users can adjust the log level by setting **log_min_messages**. The default minimum log level is **Warning**.
- The following is an example of the log content:

```
2015-06-12 20:53:58.421 CST 140521357960960 WARNING: 0: can not connect to node 16385
2015-06-12 20:53:58.421 CST 140521357960960 WARNING: 0: failed to acquire connection from datdnode 16385 for thread 140521267787520
2015-06-12 20:53:58.421 CST 140521267787520 WARNING: 20971524: incomplete message from client:77,9
2015-06-12 20:53:58.421 CST 140521267787520 WARNING: 20971524: failed to receive connDefs at the time:1.
2015-06-12 20:53:58.421 CST 140521267787520 ERROR: 20971524: failed to getpooled connections
```


HUAWEI

System Logs – Examples of Database Run Logs

- View the location of the run logs.

```
openGauss=# show log_directory;
         log_directory
-------------------------------
 /var/log/omm/omm/pg_log/dn_6001
(1 row)
```

- View log files in the log directory.

```
[omm@standalone dn_6001]$ ll -h
-rw-------. 1 omm dbgrp  42K Mar 25 20:54 postgresql-2024-03-25_204316.log
-rw-------. 1 omm dbgrp  30K Mar 25 21:07 postgresql-2024-03-25_205749.log
-rw-------. 1 omm dbgrp 241K Mar 25 23:59 postgresql-2024-03-25_213027.log
-rw-------. 1 omm dbgrp  31K Mar 26 00:40 postgresql-2024-03-26_002954.log
...
```

- View the content of a log file, which is generated when the database is restarted.

```
2024-03-28 21:53:28.187 66057658.10000 [unknown] 139668688594688 dn_6001 0 dn_6001 00000  0 [BACKEND] LOG:  [Alarm Module]alarm checker started.
2024-03-28 21:53:28.188 66057658.10000 [unknown] 139668671813376 dn_6001 0 dn_6001 00000  0 [BACKEND] LOG:  reaper backend started.
2024-03-28 21:53:28.260 [MOT] <TID:13445/04095> <SID:-----/-----> [INFO]     <System>           Startup: All core services Initialized successfully
StartupXLOG: biggest_lsn_in_page is set to FFFFFFFF/FFFFFFFF, enable
2024-03-28 21:53:28.277 66057658.6125 [unknown] 139667938735872 dn_6001 0 dn_6001 00000  0 [BACKEND] LOG:  database system timeline: 29
```

System Logs – Logs Related to Database Installation and Deployment

- The openGauss installation and deployment process consists of pre-installation and installation, both of which generate installation logs. Generally, related logs are stored in the *$gaussdbLogPath*/*Username*/**om** directory. Four types of logs are stored in this directory. Pay attention to the following log types:
  - Preinstallation logs: **gs_preinstall-***Creation time***.log**, for example, **gs_preinstall-2024-03-25_203913.log**.
  - Installation logs: **gs_install-***Creation time***.log**, for example, **gs_install-2024-03-25_204212.log**.

```
[omm@standalone om]$ ll -lh
total 880K
-rw-------. 1 omm dbgrp  11K Mar 25 20:43 gs_install-2024-03-25_204212.log
-rw-------. 1 omm dbgrp 485K Mar 27 23:32 gs_local-2024-03-25_204005.log
-rw-------. 1 omm dbgrp  27K Mar 28 22:09 gs_om-2024-03-28_100603.log
-rw-------. 1 omm dbgrp 8.4K Mar 25 20:41 gs_preinstall-2024-03-25_203913.log
```

System Logs – Logs Generated During Database Preinstallation

- openGauss preinstallation logs can be used to check whether preinstallation is successful. You can use these logs to locate exceptions during preinstallation and quickly analyze their root causes.
- Sample logs generated during successful preinstallation:

```
[2024-03-25 20:39:13.728073][2648][gs_preinstall(initGlobals:431)][gs_preinstall][LOG][Step1]:Parsing the configuration file.
[2024-03-25 20:39:13.744978][2648][gs_preinstall(initGlobals:447)][gs_preinstall][LOG][Step1]:Successfully parsed the configuration file.
[2024-03-25 20:39:35.689664][2700][gs_preinstall(initGlobals:431)][gs_preinstall][LOG][Step1]:Parsing the configuration file.
[2024-03-25 20:39:35.704715][2700][gs_preinstall(initGlobals:447)][gs_preinstall][LOG][Step1]:Successfully parsed the configuration file.
...
[2024-03-25 20:40:06.079024][2700][gs_preinstall][DEBUG]:Creating OS user and create trust for user
[2024-03-25 20:40:08.904796][2700][gs_preinstall][DEBUG]:Setting the flag for creating user's trust.
[2024-03-25 20:40:09.056070][2700][PreinstallImpl.py(getUserPasswd:147)][gs_preinstall][LOG][Step3]:Please enter password for cluster user.
[2024-03-25 20:40:14.427800][2700][PreinstallImpl.py(getUserPasswd:150)][gs_preinstall][LOG][Step3]:Please enter password for cluster user again.
[2024-03-25 20:40:19.235874][2700][gs_preinstall][DEBUG]:Successfully created [omm] user on all nodes.
[2024-03-25 20:40:19.255942][2700][gs_preinstall][LOG]:Generate cluster user password files successfully.
[2024-03-25 20:40:19.766926][2700][gs_preinstall][LOG]:Successfully created [omm] user on all nodes.
```


HUAWEI

System Logs – Logs Generated During Database Installation

- openGauss installation logs can be used to check whether installation is successful. You can use these logs to locate exceptions during installation and quickly analyze their causes.
- Sample logs generated during successful installation:

```
[2024-03-25 20:42:12.143681][5263][gs_install(initGlobals:109)][gs_install][LOG][Step1]:Parsing the configuration file.
[2024-03-25 20:42:12.237121][5263][gs_install(initGlobals:126)][gs_install][DEBUG][Step1]:Successfully parsed the configuration file.
[2024-03-25 20:42:12.516412][5263][InstallImpl.py(checkGaussenvFlag:193)][gs_install][LOG][Step2]:Successfully checked preinstall on every node.
[2024-03-25 20:42:12.702619][5263][InstallImplOLAP.py(checkClusterStatus:565)][gs_install][DEBUG][Step3]:Successfully checked the cluster status.
[2024-03-25 20:42:12.708241][5263][InstallImpl.py(prepareBackDir:237)][gs_install][LOG][Step4]:Successfully created the backup directory.
[2024-03-25 20:42:12.708360][5263][gs_install][LOG]:begin deploy..
[2024-03-25 20:42:12.708418][5263][gs_install][DEBUG]:Installing application
[2024-03-25 20:42:18.226497][5263][gs_install][LOG]:Installing applications on all nodes.
[2024-03-25 20:43:25.587727][5263][gs_install][LOG]:Successfully started cluster.
[2024-03-25 20:43:25.609800][5263][gs_install][LOG]:Successfully installed application.
[2024-03-25 20:43:25.609938][5263][gs_install][LOG]:end deploy..
...
```


HUAWEI

Operation Logs (1)

- Operation logs are generated when database tools are used by a database administrator or called by openGauss. If openGauss is faulty, you can use the operation logs to backtrack user operations on the database and reproduce the fault.
  - Log storage directory:
    - The default directory is *$GAUSSLOG*/**bin**. If the environment variable *$GAUSSLOG* does not exist or its value is empty, the log information generated for a tool will be displayed but not recorded in the log file of the tool. The default value of *$GAUSSLOG* is **/var/log/gaussdb/***Username*.
    - Note: When a database is deployed using the OM script, the log path is **/var/log/gaussdb/***Username*.

Operation Logs (2)

- Log names have the following formats:
  - *Tool name*-*Log creation time*.log
  - *Tool name*-*Log creation time*-current.log

  Tool name-*Log creation time*.log is a historical log file, and *Tool name*-*Log creation time*-current.log is a current log file. If the size of a log file exceeds 16 MB, the next time the tool is called, the current file is renamed in the historical log format, and a new log file is generated.

  For example, gs_guc-2015-01-16_183728-current.log is renamed as gs_guc-2015-01-16_183728.log, and gs_guc-2015-01-17_142216-current.log is generated.

- Maintenance suggestions:
  - You are advised to periodically dump expired logs to save disk space and prevent important logs from being lost.

Operation Logs – Logs Related to Database Startup and Stop

- Database management provides functions such as startup, stopping, querying, primary/standby switchover, rebuilding, and status monitoring. The log information is stored in the *$GAUSSLOG*/**bin** directory. For example, the logs stored in the **gs_ctl** directory record database startup and stop information.
  - View the location of the log, for example:

```
[omm@standalone ~]$ cd /var/log/omm/omm/bin/gs_ctl
```

  - View the log files in the log directory, for example:

```
[omm@standalone gs_ctl]$ ll -h
-rw-------. 1 omm dbgrp 34K Mar 28 21:53 gs_ctl-2024-03-25_204315-current.log
...
```

Operation Logs – Content of Logs Related to Database Startup and Stop

- Sample content of a log file generated when the database is restarted:

```
[2024-03-28 23:37:20.930][41436][][gs_ctl]: gs_ctl stopped ,datadir is /opt/huawei/install/data/dn
[2024-03-28 23:37:23]
[2024-03-28 23:37:23.863][42296][][gs_ctl]: gs_ctl started,datadir is /opt/huawei/install/data/dn
[2024-03-28 23:37:23.894][42296][][gs_ctl]: waiting for server to start...
.....
[2024-03-28 23:37:28.970][42296][][gs_ctl]:  done
[2024-03-28 23:37:28.970][42296][][gs_ctl]: server started (/opt/huawei/install/data/dn)
[2024-03-28 23:37:30]
[2024-03-28 23:37:30.722][43095][][gs_ctl]: gs_ctl query ,datadir is /opt/huawei/install/data/dn
 HA state:
        local_role                : Normal
        static_connections        : 0
        db_state                  : Normal
        detail_information        : Normal
 Senders info:
No information
...
```


HUAWEI

Operation Logs – Database GUC Logs

- The GUC-related log information of the database, such as setting GUC parameters and their status, is stored in the *$GAUSSLOG*/**bin/gs_guc** directory.
  - View the location of the log, for example:

```
[omm@standalone ~]$ cd /var/log/omm/omm/bin/gs_guc
```

  - View the log files in the log directory, for example:

```
[omm@standalone gs_guc]$ ll -h
-rw-------. 1 omm dbgrp 6.1K Mar 25 20:43 gs_guc-2024-03-25_204312-current.log
-rw-------. 1 omm dbgrp 16M Mar 24 20:43 gs_guc-2024-03-24_20442.log
...
```

Operation Logs – Content of Database GUC Logs

- Sample content of database GUC logs:

```
...
expected instance path: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: listen_addresses='localhost,192.168.56.28': [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: local_bind_address='192.168.56.28': [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: port=26000: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: cstore_buffers=16MB: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: max_connections=5000: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: shared_buffers=256MB: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: work_mem=64MB: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: maintenance_work_mem=128MB: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: data_replicate_buffer_size=128MB: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: enable_data_replicate=off: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: max_wal_senders=16: [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: application_name='dn_6001': [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: log_directory='/var/log/omm/omm/pg_log/dn_6001': [/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: audit_directory='/var/log/omm/omm/pg_audit/dn_6001':
[/opt/huawei/install/data/dn/postgresql.conf]
gs_guc set: checkpoint_segments=64: [/opt/huawei/install/data/dn/postgresql.conf]
```


HUAWEI

Performance Access Logs

- Performance logs focus on the access performance of external resources. They record the status of physical resources and access performance by external resources such as disks. When a performance issue occurs, you can locate the cause using performance logs, which significantly improves troubleshooting efficiency.
  - Log storage directory
    - Database node performance logs are stored in the directories under *$GAUSSLOG*/**gs_profile**.
  - Log naming rules
    - Database node performance logs use the following naming format: **postgresql-***Creation time*.**prf**. By default, a new log file is generated at 0:00 every day or when the latest log file exceeds 20 MB or a database instance (database node) is restarted.
  - Log content
    - Line arrangement of a database node log: *Host name*+*Date*+*Time*+*Instance name*+*Thread number*+*Log content*

Permissions to Call Performance Logs (1)

- In openGauss, performance log access permissions depend on the log storage location and the file system permission settings of the OS.
  - File system permissions:
    - Log files comply with the file permission model of the OS. Generally, log files can be read only by database service accounts and users with sufficient permissions (such as database administrators or users in corresponding user groups). Ensure that permissions on log directories and files are set to allow access for relevant users or processes.
  - Log directory settings:
    - In the openGauss configuration file (usually **postgresql.conf**), you can set **log_directory** to specify the log storage location. The database service account must have the write permission on the directory, and users who need to read the logs must have the read permission on the directory.

HUAWEI

Permissions to Call Performance Logs (2)

- **Security:** Performance logs may contain sensitive information (such as executed SQL statements and performance data). Therefore, only authorized users and system administrators can access performance logs.
- **Log archiving and management:** Logs that need to be stored or analyzed for a long time may be archived to other systems or services. In this case, properly set the access permissions for these archive systems or services to protect log data security and privacy.
- **Audit and monitoring:** To ensure security and compliance, operations (such as reading, modifying, and deleting) on accessing performance logs must be audited and monitored to confirm that all access behaviors are reasonable and authorized.

- To manage the access permissions of openGauss performance logs, you need to consider the OS permission settings, database configuration options, and security and compliance requirements. Proper permission management ensures that only authorized users can access these logs while protecting sensitive information from unauthorized access.

HUAWEI

System Views Related to Performance Logs (1)

- In openGauss, performance analysis often depends on various system views that provide detailed performance data and statistics during running. The following system views are closely related to performance logs.

View	Function
pg_stat_activity	Displays the information about the current active processes, including the query, start time, and status of each process.
pg_stat_database	Displays database-level statistics, such as the number of transaction commits, number of rollbacks, number of read blocks, and hit ratio.
pg_stat_user_tables	Displays the access statistics of user tables, such as the number of sequential scans, number of index scans, number of inserts, number of updates, and number of deletes.
pg_stat_user_indexes	Displays the statistics on the usage of user-defined indexes, including the number of index scans and reads.
pg_statio_user_tables	Displays user table I/O statistics, such as the number of block reads and hits.
pg_statio_user_indexes	Displays user index I/O statistics, such as the number of index block reads and hits.

HUAWEI

System Views Related to Performance Logs (2)

- These views are important tools for understanding and analyzing openGauss database performance. They help database administrators diagnose performance problems, monitor database activities, analyze query performance, and optimize database configurations. By periodically checking the data in these views, you can effectively monitor the database status and quickly detect potential performance bottlenecks.

View	Function
pg_stat_bgwriter	Displays statistics about the background writer process, including the number of buffer writes and clearances.
pg_stat_statements	Displays performance statistics of a single SQL statement, such as the number of calls, total execution time, and total number of rows, if the **pg_stat_statements** module is enabled.
pg_statio_user_tables	Displays user table I/O statistics, such as the number of block reads and hits.
pg_statio_user_indexes	Displays user index I/O statistics, such as the number of index block reads and hits.

HUAWEI

Contents

1. WAL Mechanism
2. Checkpoint Mechanism
3. Restoration Using Redo Logs
4. Run Logs and Performance Logs
5. **Log Checking and Clearance**

HUAWEI

Log Checking and Clearance

- In openGauss, logs are important files used to record database operations. To ensure the stability and security of the database, you need to periodically check and clear logs, particularly in the following scenarios.

Periodic backup

Helps you restore the database and prevent data losses.



Periodic clearing

Helps you release disk space and improve database performance.

Periodic checks

Helps you detect exceptions, such as errors and warnings.

Periodic database backup

Helps you restore data and prevent data losses.

HUAWEI

Checking OS Logs

- You should check OS logs monthly to detect and prevent potential OS problems.
- To check system logs, perform the following steps:
  - Go to the directory where the log files are stored and run the following command to view the OS log files:

```
cat /var/log/gaussdb/Username/pg_log/ postgresql-xxxxx.log
```

Pay attention to words like "kernel", "error", and "fatal" in the logs generated within the last month and handle the problems based on the alarm information.

```
2024-04-01 18:01:06.357,"dn_6001","omm","metadatabase",140302676322048,"::1:49328",660a85e2.5060,3,"",2024-04-01 18:01:06 CST,10/46810,0,0,"BACKEND",LOG,00000,"statement: SELECT NSPNAME FROM PG_NAMESPACE WHERE NSPNAME LIKE 'pg_temp_%'",,,,,,,,,"gs_clean"
2024-04-01 18:01:06.357,"dn_6001","omm","metadatabase",140302676322048,"::1:49328",660a85e2.5060,4,"",2024-04-01 18:01:06 CST,10/46811,0,0,"BACKEND",LOG,00000,"statement: SELECT SESSIONID, TEMPID, TIMELINEID FROM PG_DATABASE D, PG_STAT_GET_ACTIVITY_FOR_TEMPTABLE() AS S WHERE S.DATID = D.OID AND D.DATNAME = 'metadatabase'",,,,,,,,,"gs_clean"
```


HUAWEI

Checking Database Run Logs (1)

- A database may still run despite certain operational errors. However, data may be inconsistent before and after the error. Therefore, you are advised to check database run logs monthly to quickly detect potential problems.
- Prerequisites
  - The host used for collecting logs is running properly, and the network connection is normal. The mutual trust relationship between the database and the installation user is normal.
  - An OS tool (for example, gstack) that the log collection tool depends on has been installed. If it is not installed, an error message is displayed, and this collection item is skipped.
- Procedure
  - Log in to the primary database node as the OS user **omm**.

HUAWEI

Checking Database Run Logs (2)

- Run the following command to collect database logs:

```
gs_collector --begin-time="20160616 01:01" --end-time="20160616 23:59"
```

- In the command, **20160616 01:01** indicates the start time of the log and **20160616 23:59** indicates the end time of the log.
- Based on command output in the previous step, access the related log collection directory, decompress collected database logs, and check these logs.
- Assume that the collected logs are stored in **/opt/gaussdb/tmp/gaussdba_mppdb/collector_20160726_105158.tar.gz**.

```
tar -xvzf /opt/gaussdb/tmp/gaussdba_mppdb/collector_20160726_105158.tar.gz
cd /opt/gaussdb/tmp/gaussdba_mppdb/collector_20160726_105158
```


HUAWEI

Checking Database Run Logs – Example

- Run the **gs_collector** command together with parameters **--begin-time** and **--end-time**.

```
gs_collector --begin-time="20160616 01:01" --end-time="20160616 23:59"
```

If information similar to the following is displayed, the logs have been archived:

```
Successfully collected  files
All results are stored in /tmp/gaussdba_mppdb/collector_20160616_175615.tar.gz.
```

- Run the **gs_collector** command together with parameters **--begin-time**, **--end-time**, and **-h**.

```
gs_collector --begin-time="20160616 01:01" --end-time="20160616 23:59" -h plat2
```

If information similar to the following is displayed, the logs have been archived:

```
Successfully collected  files
All results are stored in /tmp/gaussdba_mppdb/collector_20160616_190225.tar.gz.
```


HUAWEI

Clearing Run Logs

- Many run logs are generated when the database is running, occupying huge disk space. You are advised to clear expired run logs and retain logs generated within one month. The procedure is as follows:
  - Log in to the primary database node as the OS user **omm**.
  - Clear logs.
    - Back up logs generated one month ago to other disks.
    - Access the directory where the logs are stored.
    - Access the corresponding sub-directory and run the following command to delete logs generated one month ago.

      ```
      cd $GAUSSLOG
      ```

    - The naming convention of the log file is **postgresql-***Year-Month-Day_HHMMSS*.

      ```
      rm Log name
      ```

Quiz

1. (Multiple-answer question) Which of the following can cause Xlog records to be written into WAL segment files? ( )

   A. A transaction is committed or terminated.

   B. The WAL buffer is full.

   C. The WAL write thread periodically writes data.

   D. The exec_simple_query() function is called and executed.

HUAWEI

Summary

- This chapter describes the write-ahead log mechanism, checkpoint mechanism, restoration process based on redo logs, run logs, performance logs, and ways to check and clear logs.
- This chapter focuses on the WAL mechanism, checkpoint mechanism, and recovery based on redo logs.

HUAWEI

More Information

- openGauss Community:
  - https://opengauss.org/en/
- Huawei Talent:
  - https://e.huawei.com/en/talent/cert/#/careerCert
- Related books:
  - *openGauss Database Core Technologies*

Recommendations

- Huawei Career Certification
  - HCIA-openGauss
  - HCIP-openGauss
- Advanced learning map of the openGauss Community
  - https://opengauss.org/zh/advanced/

Thank you.

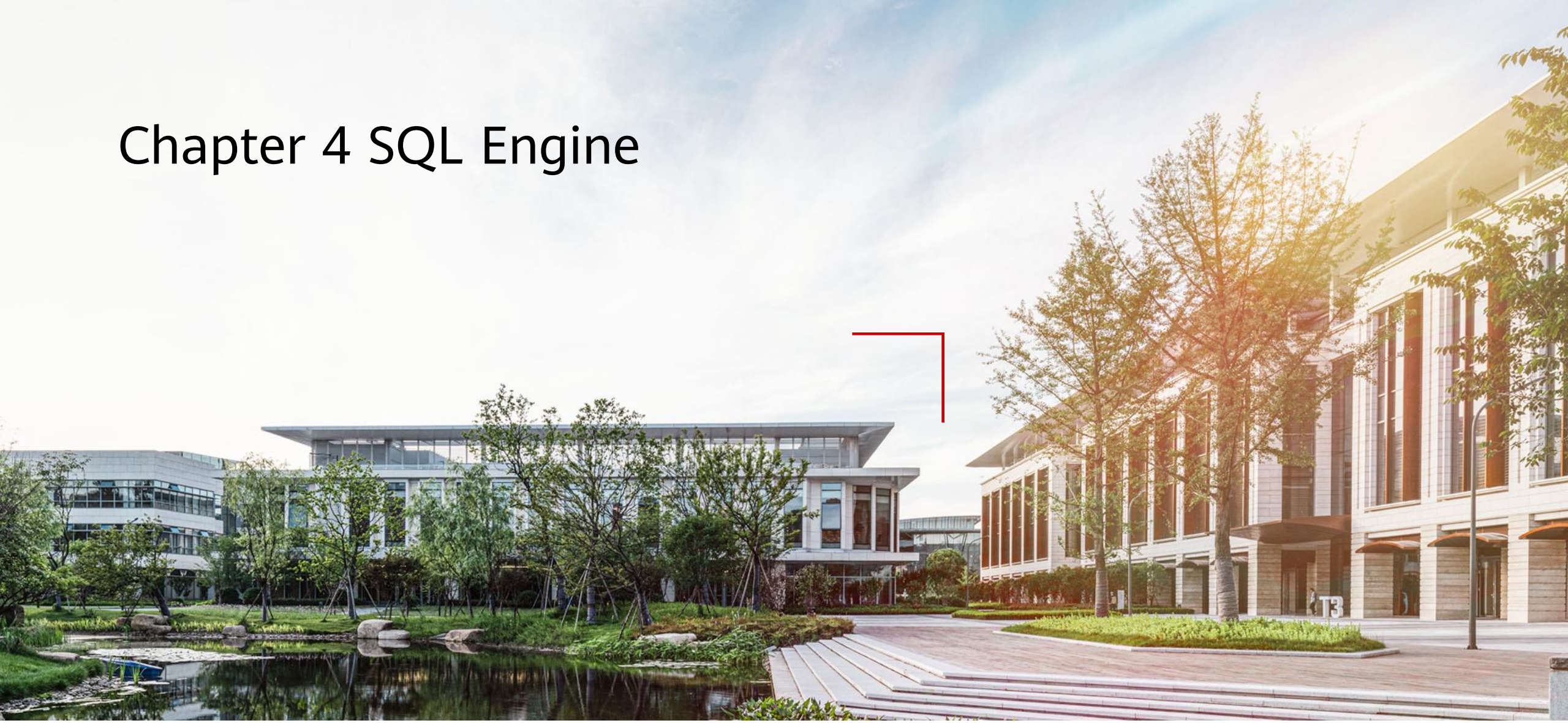
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 4 SQL Engine



HUAWEI

Foreword

- This chapter covers the following information about the SQL execution engine:
  - Table scan, table join, table control, and core operators of the materialization and stream classes
  - Query execution process, steps, and operators
  - Execution plans, usage, and examples

HUAWEI

Objectives

- Upon completing this course, you will be able to:
  - Describe how to execute a query.
  - Master the classification of core operators and the working principles of some operators.
  - Understand the meaning of each component of the execution plan.
  - Parse different types of SQL execution plans.

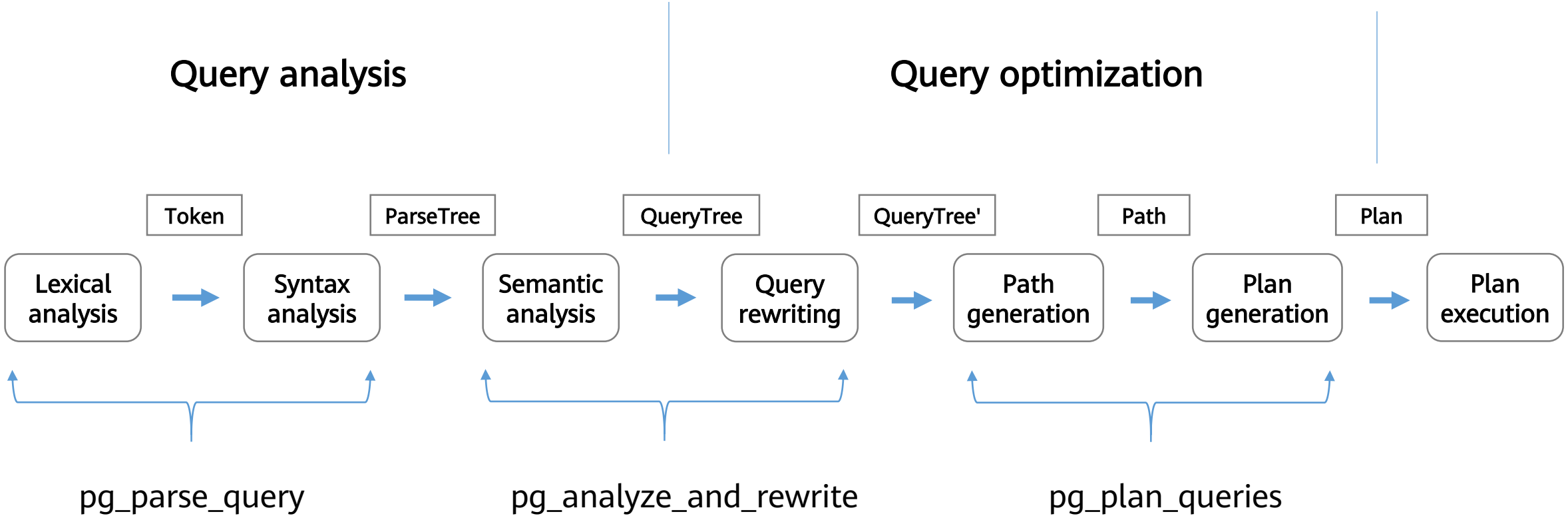
HUAWEI

Contents

1. **Query Execution Process**
2. Core Operator Principles
3. Execution Plans

HUAWEI

SQL Statement Execution Process



HUAWEI

Execution Procedure (1)

- The SQL parsing process is as follows:
  - **Lexical analysis:** The lexical analyzer lex is used to split SQL statements entered by users into word (token) sequences and identify keywords, identifiers, and constants.
  - **Syntax analysis:** The yacc analyzer checks whether the word (token) sequence parsed by the lexical analyzer meets the SQL syntax rules.
  - **Semantic analysis:** This is a logical phase of the SQL parsing process. The main task is to check the context-related properties based on the correct syntax. This phase determines the visibility of elements such as table names, operators, and types, and checks the semantic validity.
  - **Query rewriting:** This phase may involve replacing a view with a query of a base table or applying some specific SQL rules to modify a query. For example, if there is a view expansion or row-level trigger, the syntax tree will be transformed during this phase. These rules are predefined and indicate how the database modifies the syntax tree under specific conditions.

HUAWEI

Execution Procedure (2)

- **Query optimization:** During this phase, the query plans are optimized based on estimated costs and resource usage. The output query plan defines in detail how to perform database operations, including the data access path (such as scan method), join method (such as hash join and nested loop join), and other operations (such as aggregation and sorting).
- **Plan generation:** The selected query plan is received from the query optimization phase. This plan, usually in a logical form, introduces the main operations needed to execute the query but is not specific to the actual execution algorithm or physical operations. In this phase, the optimizer's output is transformed into a detailed sequence of steps that the database engine will follow during execution.
- **Plan execution:** The execution engine executes SQL commands based on the steps defined in the plan. For example, INSERT operations usually involve assigning new data rows to the appropriate locations in the table and updating any related indexes.

HUAWEI

Contents

1. Query Execution Process
2. **Core Operator Principles**
   - Scan Operators
   - Control Operators
   - Materialization Operators
   - Join Operators
   - Stream Operators
3. Execution Plans

HUAWEI

Scan Operators (1)

- Depending on the operation, there are several types of node operators in the openGauss execution plan: scan, control, materialization, join, and stream.
  - A scan node extracts data from the underlying data source, which may come from the file system or network (distributed query). Generally, a scan node is located on a leaf node of the execution tree and functions as the data input source of the execution tree PlanTree.

Operator	Description	Scenario
SeqScan	Sequentially scans the row-store engine.	This is a basic scan operator used to sequentially scan ordinary physical row-store tables based on the scan direction.
CstoreScan	Scans the column-store engine.	This is a basic scan operator used to scan ordinary column-store tables.
SubQueryScan	Scans the subquery.	This operator uses the result set of a subquery as the input of the current query.
BitmapHeapScan BitmapIndexScan	Obtains a tuple using a bitmap.	You can use indexes to scan and obtain the result bitmap that meets conditions. The bitmap marks the offset of the tuple that meets the condition on the page.
TidScan	Obtains a tuple by tuple ID.	This operator filters tuples from **table.ctid** by tuple ID. For example, you can use **where table.ctid = tid OR table.ctid IN (tid1, tid2, tid3, ...)**.

HUAWEI

Scan Operators (2)

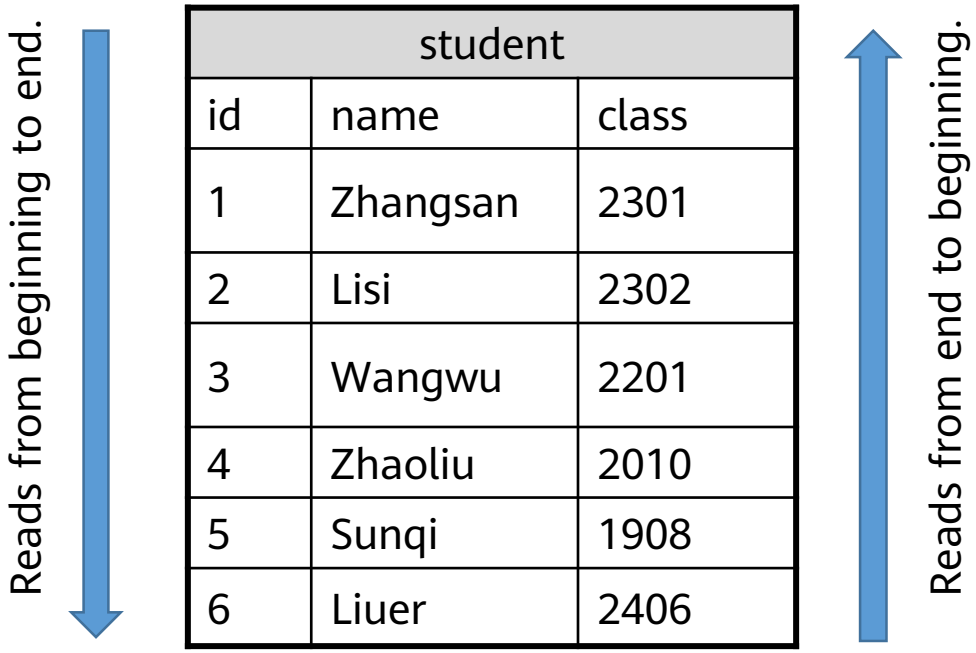
Operator	Description	Scenario
IndexScan	Scans data by index.	The IndexScan operator scans tables when indexes are created for the attributes involved in the selection condition.
IndexOnlyScan	Returns a tuple directly from an index.	The index column completely overwrites the target list of the query. In this scenario, the index result can be directly returned without accessing the base table again.
CteScan	Scans common table expressions (CTEs).	The output of CTEs is treated as a set for subsequent relational calculation.
FunctionScan	Scans functions.	The output of functions is treated as a set for subsequent relational calculation.
ForgeinScan	Scans foreign tables.	This operator queries FDWs of foreign data sources such as HDFS, GDS, or OBS when a query is made on a foreign table.
WorkTableScan	Scans the intermediate result sets.	This operator scans the split-out intermediate result sets during the query; for example, an intermediate result set generated in each iteration of the RecursiveUnion operator.
ValueScan	Scans the VALUE list.	This operator scans the given tuple set in values() clauses.

HUAWEI

SeqScan Operators (1)

1. Description

- The sequential scan operator is the most universal among all scan operators. It scans a table in a certain direction (forward or backward). For a SeqScan execution node, the corresponding code source file is **nodeSeqScan.cpp**.
- In the following figure, if the table contains *n* rows, the time complexity of the sequential scan is **O(*n*)**.



student		
id	name	class
1	Zhangsan	2301
2	Lisi	2302
3	Wangwu	2201
4	Zhaoliu	2010
5	Sunqi	1908
6	Liuer	2406

SeqScan Operators (2)

2. Typical scenarios

- The table has no index and needs to be scanned.
- The table has indexes, but most of the data in the table needs to be scanned.

3. Performance analysis

Scenario	Performance Deterioration Cause	Solution
Tables with a large amount of data	A sequential scan is performed from beginning to end or vice versa. A small cache will result in many disk I/O operations. Also, even if the cache can store data in all tables, the time complexity is $O(n)$. When n is large, the efficiency is still low.	1. Allocate a large shared_buffers space to improve the cache hit ratio. 2. If a small amount of data is obtained from a table, use the index scan of the time complexity $O(\log(n))$, that is, add indexes to the columns using the WHERE condition. 3. If a large amount of data is obtained from a table, use the parallel sequential scan whose time complexity is $O(n/dop)$. *dop* indicates the degree of parallelism.

HUAWEI

IndexScan Operators (1)

1. Description

- In an index scan, the database uses the index column specified by a statement to retrieve rows by traversing the index tree.
- When the database scans an index for a value, the value to be searched for can be found after *n* I/Os occur, where *n* is the height of the B-tree index.
- Generally, table data is retrieved. The database reads the index block sequentially, finds the corresponding index key value, and then reads the corresponding table tuple based on the TID corresponding to the index key.
- In scenarios with a large amount of data, IndexScan is more efficient than SeqScan.
- The code source file for the IndexScan plan node is **nodeIndexScan.cpp**.

HUAWEI

IndexScan Operators (2)

2. Typical scenarios

Typical Scenario	Description
Querying a specific row	When a query statement contains a WHERE clause whose conditions can be matched by index columns, openGauss uses the index scan to search for rows that meet the conditions.
Sorting	When a query statement contains the ORDER BY clause whose columns can be sorted by index, openGauss uses the index scan for sorting.
Aggregation	When a query statement contains the GROUP BY clause whose columns can be grouped by index, openGauss uses the index scan for aggregation.
Join	When a query statement contains a JOIN operation whose columns can be matched by index, openGauss uses the index scan to perform the join operation.

HUAWEI

IndexScan Operators (3)

3. Performance analysis

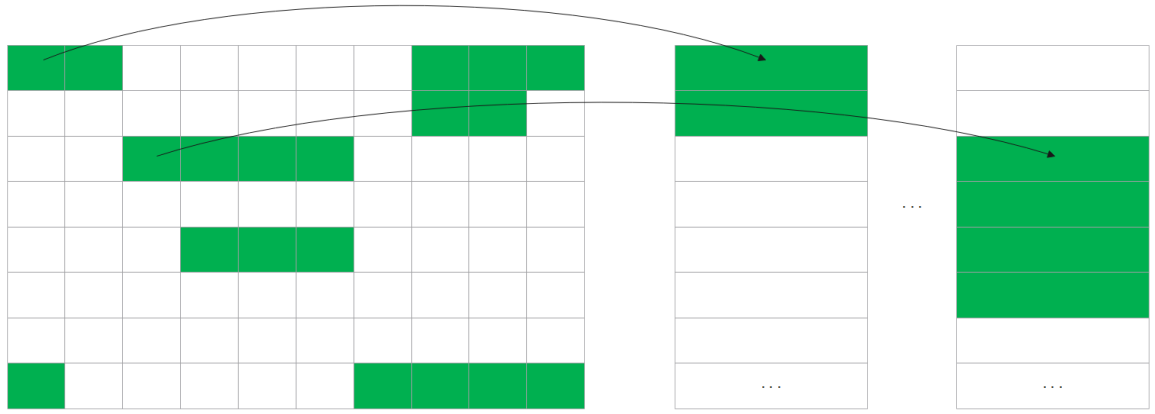
Scenario	Performance Deterioration Cause	Solution
Uneven data distribution	If the data in a table is not evenly distributed, an index scan may cause a large amount of random I/Os, reducing query efficiency.	No solution is available. You are advised to avoid this scenario.
Incorrect data type in the index column	If the index column contains an incorrect data type, for example, a long string, an index scan may cause a large amount of I/Os.	Replace the data with a correct type.
Repetitive data in the index columns	If index columns have highly repetitive data, an index scan may cause a large number of I/Os.	Select a column with a high filtering degree as the index.
Incorrect query conditions	If the query conditions are incorrect, for example, using the LIKE operator or non-prefix matching, an index scan may cause a large number of I/Os.	Modify the query criteria.
Index columns with too much data	If there is too much data in the index columns, an index scan may cause a large number of I/Os.	For a composite index, you are advised to delete the columns that do not improve the index.

HUAWEI

BitmapHeapScan and BitmapIndexScan Operators (1)

1. Description

- (1) BitmapHeapScan: The lower layer of this operator contains the bitmap index scan, which is used to scan the bitmap returned by the lower-layer operator to determine which tuples meet certain conditions, obtain the tuples, and return them to the upper layer.
- (2) BitmapIndexScan: This operator uses the index scan to form a bitmap based on the TIDs of tuples that meet the filtering conditions and returns the bitmap to the upper layer.
- The BITMAPAND or BITMAPOR operation can be performed among multiple result bitmaps scanned by multiple tables. Compared with the index scan, the BITMAPAND or BITMAPOR operation needs to compare the tuples filtered between tables, making filtering faster.



Bitmap returned by the lower layer of the operator

Actual storage tuples of the physical storage pages

HUAWEI

BitmapHeapScan and BitmapIndexScan Operators (2)

2. Typical scenarios

- Prerequisites: The GUC parameter **enable_bitmapscan** is **on**.
- The optimizer uses the bitmap index scan based on cost estimation and needs to return tuples that meet certain conditions. In this case, the upper layer usually uses the bitmap heap scan to filter tuples by scanning bitmaps and return the tuples.
- The bitmap index scan is often used when multiple tables are associated, indexes are available on the associated columns of one or more tables, and there is enough data in some or all tables.

3. Performance analysis

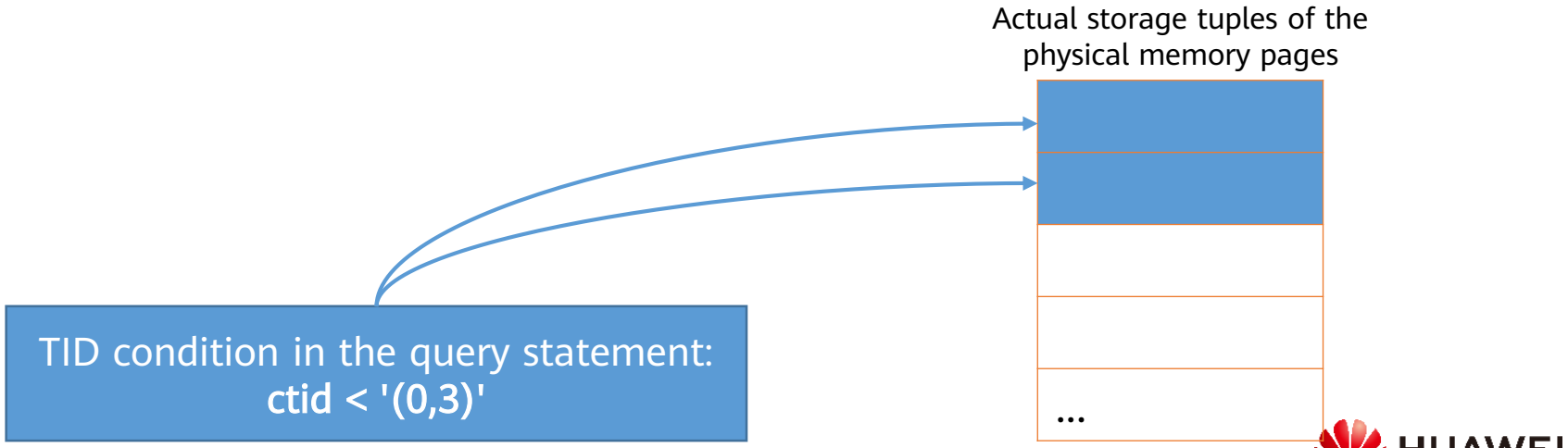
- The bitmap index scan requires extra time and storage overhead to generate and store bitmaps.

HUAWEI

TidScan (1)

1. Description

- This is a row number scan operator, which uses the row number (**ctid**) to filter and return the tuples.
- openGauss uses Astore for storage by default. Data is stored in a heap page by row. In addition to the column value, the corresponding row number is also stored in the B-tree index. This enables quick searches by row number in openGauss.
- The row number format is (*page_number*, *item_number*).
- The value of *page_number* starts from **0**, and the value of *item_number* starts from **1**.
- As shown in the figure, the corresponding tuple in the disk or cache is found based on the given **ctid** value.



TidScan (2)

2. Typical scenarios

- **Prerequisites:** The GUC parameter **enable_tidscan** is **on**.
- You can perform a quick search by specifying a valid row number if a table supports TidScan. Currently, openGauss selects TidScan only when the WHERE condition is =. IN is not supported.
- **Supported scenarios:** Astore row-store tables, Astore partitioned tables, Ustore row-store tables, and Ustore partitioned tables.
- **Unsupported scenarios:** Astore column-store tables, Astore column-store partitioned tables, Astore column-store compressed tables, and Astore column-store partitioned compressed tables.

HUAWEI

TidScan (3)

3. Performance analysis

- Scenario
  - You need to use TidScan to scan the entire table, and the **targetlist** column is **ctid**.
- Performance deterioration causes
  - openGauss does not automatically create indexes for the **ctid** column. If TidScan needs to scan the entire table, it may be better to use the IndexOnlyScan instead.
- Solution
  - Add an index to the **ctid** column of the table and use IndexOnlyScan instead of TidScan.

HUAWEI

ValueScan Operators

1. Description

- Reads the list of values specified in the VALUES clause and returns it as a set of virtual rows to the query planner. These virtual rows can be used by other scanners or operators, such as hash join or merge join.

2. Typical scenarios

- The ValueScan operator provides a quick and simple way to specify a set of values without reading data from a table. This is useful during testing or debugging, or when you need to insert a small amount of data.

HUAWEI

Contents

1. Query Execution Process
2. **Core Operator Principles**
   - Scan Operators
   - Control Operators
   - Materialization Operators
   - Join Operators
   - Stream Operators
3. Execution Plans

HUAWEI

Control Operators

- Generally, control operators do not map algebra operators. They are introduced by executors to complete some special processes.

Operator	Description	Scenario
Result	Handles processes with only one result or if the filter condition is a constant.	Handle condition expressions that require only one calculation or scenarios where INSERT contains only one VALUES clause.
ModifyTable	Performs INSERT, UPDATE, or DELETE operations.	Insert, update, and delete data during a query.
Append	Appends multiple relationship sets.	Perform UNION or UNION ALL.
MergeAppend	Appends multiple sequential relationship sets.	Perform UNION or table inheritance.
RecursiveUnion	Performs recursive subqueries.	Perform recursive.
Limit	Limits the output result sets.	Execute an SQL statement with LIMIT.
BitmapAnd	Performs the AND operation on the bitmap.	Multiple attribute constraints contain indexes and are connected by AND.
BitmapOr	Performs the OR operation on the bitmap.	Multiple attribute constraints contain indexes and are connected by OR.
Rownum	Specifies the row number operator.	Execute an SQL statement with ROWNUM.

HUAWEI

Contents

1. Query Execution Process
2. **Core Operator Principles**
   - Scan Operators
   - Control Operators
   - Materialization Operators
   - Join Operators
   - Stream Operators
3. Execution Plans

HUAWEI

Materialization Operators (1)

- Materialization operators generally refer to algorithm requirements. When operator logic processing is performed, lower-layer data needs to be cached. It is impossible to predict the amount of data lower-layer operators will return. As such, you need to consider the possibility that the memory will not be able to hold all the data.

Operator	Description	Scenario
Materialize	Materialization operators	The node result set is cached for scanning again.
Sort	Sorting lower-layer data	ORDER BY, Merge Join, SortAgg, MergeAppend, and UNIQUE deduplication.
Group	Grouping sorted lower-layer data	Group operations using GROUP BY.
Agg	Grouping lower-layer data without sorting it	Aggregate functions such as COUNT, SUM, AVG, MAX, and MIN; DISTINCT clause; UNION deduplication; GROUP BY clause.

HUAWEI

Materialization Operators (2)

Operator	Description	Scenario
Unique	Deduplicating lower-layer data	DISTINCT clause or UNION deduplication
Hash	Caching lower-layer data to a hash table	HashTable construct or hash join operators
SetOp	Caching lower-layer data for processing set operations such as INTERSECT	INTERSECT/INTERSECT-ALL or EXCEPT/EXCEPT-ALL operations
WindowAgg	Window functions	Statements that contain window functions, for example, row_number() OVER (PARTITION BY xxx) and avg(xxx) OVER (PARTITION BY xxx)
LockRows	Processing row-level locks	SELECT ... FOR SHARE/UPDATE operations

HUAWEI

Aggregation Operators

- Aggregation operators are used to perform operations that contain aggregate functions. The corresponding code source file is **nodeAgg.cpp**.
- They support three processing policies: common aggregation (ungrouped aggregation calculation), sort aggregation, and hash aggregation.
  - Sort and hash aggregation calculations contain the GROUP BY clause, but ungrouped aggregation calculations do not.
  - Common aggregation can be regarded as a special case of group aggregation (each tuple corresponds to a group).
  - Common and sort aggregations use the agg_retrieve_direct function to obtain tuples while hash aggregation uses the agg_retrieve function.
- The WindowAgg operator is used to aggregate tuple windows. The corresponding code source file is **nodeWindAgg.cpp**. The WindowAgg operator has similar functions and modes to the aggregation operator. The main difference is that WindowAgg operators process tuples that are limited to the same partition (window), while aggregation operators process tuples for the entire table (GROUP BY partition).

HUAWEI

Contents

1. Query Execution Process
2. **Core Operator Principles**
   - Scan Operators
   - Control Operators
   - Materialization Operators
   - Join Operators
   - Stream Operators
3. Execution Plans

HUAWEI

Join Operators

- Join operators are used to cope with the most common association operations in the database.
- There are three types.

Operator	Description	Scenario
Nested loop join	Joins two relations by using two nested loops for low layers.	INNER, LEFT OUTER, SEMI JOIN, or ANTI JOIN
Merge join	Joins data from two input streams into a single combined output stream.	INNER, LEFT-OUTER-JOIN, RIGHT-OUTER-JOIN, FULL-OUTER-JOIN, SEMI JOIN, or ANTI JOIN
Hash join	Joins data from two input streams through hashing.	INNER, LEFT-OUTER-JOIN, RIGHT-OUTER-JOIN, FULL-OUTER-JOIN, SEMI JOIN, or ANTI JOIN

HUAWEI

Nested Loop Join Operators (1)

1. Description

- A nested loop join is the simplest join method, which is implemented in all relational database systems. It compares the data in two tables to check whether the join conditions are met.
- In openGauss, a nested loop join scans the inner table for each row in the outer table searching for rows that meet the join conditions. This is like two nested loops, giving the operation its name. The outer loop traverses the outer table, and the inner loop traverses the inner table.
- The time complexity of a nested loop join is O(n x m), where n and m indicate the numbers of rows in the two tables. If the inner table can be scanned by using indexes, the time complexity can be reduced to O(n log m).

HUAWEI

Nested Loop Join Operators (2)

2. Typical scenarios

- Any of the inner or outer table is very small.
- An inner table quickly locates the row that meets the condition (for example, an index has been created for the join column of the inner table) based on the join condition.
- A nested loop join can be executed under any join conditions if it has no restrictions.

3. Performance analysis

- A nested loop join is not efficient at processing large tables, especially when there is no index between two large tables because it might need a large amount of disk I/Os and CPU time.

HUAWEI

Hash Join Operators (1)

1. Description

- A hash join is an efficient join method that depends on the hash technology. During a hash join, openGauss selects one of the two tables (usually small tables) and creates a hash table based on join conditions.
- The key of the hash table is the join column of the small table, and the other columns of the small table are the value. Then, the hash value of the join column is calculated for each row in the large table. After this, the hash table is searched for a matching row.
- This method is used when smaller tables can be stored in the memory. In this way, the total cost is the sum of the costs of accessing the two tables.
- The time complexity of a hash join is O(n + m), where n and m indicate the numbers of rows in the two tables. However, the performance may degrade if the inner table is too large for the memory storage, as it will require additional disk I/O operations.

HUAWEI

Hash Join Operators (2)

2. Typical scenarios

- The two tables differ significantly in size and the join operation is performed.
- The join columns of the two tables are equi-join (for example, using the = operator).

3. Performance analysis

- One of the main issues faced by hash joins is when a table too large for the memory storage. This requires openGauss to write some of the hash tables to disks, significantly reducing the query performance.
- A hash join can return result rows only after a hash table is constructed. Unlike a nested loop join, a hash join can return result rows immediately after being executed.

HUAWEI

Merge Join Operators (1)

1. Description

- A merge join is an efficient join method that depends on sorting operations. During a merge join, openGauss sorts the join columns of the two tables and scans both tables at the same time to find matching rows.
- This method supports the join of sorting result sets. The corresponding code source file is **nodeMergeJoin.cpp**.
- The time complexity of a merge join is O(n + m), where n and m indicate the numbers of rows in the two tables. However, if a sorting operation is required, the time complexity of the sorting operation may reach max(O(log n), O(log m)), which is usually more time-consuming than a direct merge join operation.
- The merge join is seldom used in openGauss. The optimizer usually prefers hash joins, even if the two tables to be joined have been sorted.
- Generally, a hash join is more effective than a sort-merge join. However, if tuples have already been sorted, they do not need to be sorted again, making a sort-merge join more effective.

HUAWEI

Merge Join Operators (2)

2. Typical scenarios

- The two tables are similar in size.
- The join columns of two tables have been sorted, for example, by index.
- In addition to the equi-join (for example, using the = operator), the join condition also includes the range query (for example, using the <, <=, >, and >= operators).

3. Performance analysis

- Main performance issue: If the join columns of the two tables have not been sorted, openGauss needs to first sort the join columns for each table, which may require a lot of CPU time and disk I/O.

HUAWEI

Contents

1. Query Execution Process
2. **Core Operator Principles**
   - Scan Operators
   - Control Operators
   - Materialization Operators
   - Join Operators
   - Stream Operators
3. Execution Plans

HUAWEI

Stream Operators

- openGauss uses stream operators for parallel execution in producer-consumer mode. These new operators are a subtype of stream operators.

Operator	Description	Scenario
Local Gather	Aggregates data of parallel threads within an instance.	Parallel execution
Local Redistribute	Redistributes data based on the distribution key across threads within an instance.	Parallel execution
Local Broadcast	Broadcasts data to each thread within an instance.	Parallel execution
Local RoundRobin	Distributes data in polling mode across threads within an instance.	Parallel execution

HUAWEI

Contents

1. Query Execution Process
2. Core Operator Principles
3. **Execution Plans**

HUAWEI

openGauss Execution Plan Overview

- What is the execution plan of openGauss?
  - An openGauss execution plan is generated based on query conditions and table structures when the database system executes SQL statements. It provides guidance for the database system to perform query operations.
  - An execution plan includes each operation step and their sequence, the index used, and the table access mode of a query statement. It is the key to optimizing query performance. By analyzing the execution plan, you can learn about the execution status of query statements, find query performance bottlenecks, and optimize performance.
  - An execution plan has a node tree structure. Each node represents a basic database operator used during execution. In the default output mode (**Normal**), nodes at different layers of the node tree are distinguished by indentation. That is, nodes are output from the top node, and the left and right subtree nodes of the top node are indented before being output.
  - An execution plan shows how the tables referenced by the SQL statement will be scanned, for example, using a plain sequential scan or an index scan. If multiple tables are referenced, what JOIN algorithms will be used to join the required rows from each input table.
  - The most critical part of the display is the estimated statement execution cost, which is the planner's guess at how long it will take to run the statement.
  - The **ANALYZE** option not only plans a statement but executes it. This informs the actual execution time statistics displayed, including the total time elapsed within each plan node (in milliseconds) and the total number of rows returned. This helps check if the planner’s estimates are similar to actual results.

HUAWEI

Execution Plan Syntax Format

- To print an openGauss execution plan, use the following syntax format:

```
EXPLAIN [ ( option [, ...] ) ] statement;
statement indicates an SQL statement. The syntax of the option clause is as follows:
        ANALYZE [ boolean ] | ANALYSE [ boolean ] |
    VERBOSE [ boolean ] |
    COSTS [ boolean ] |
    CPU [ boolean ] |
    BUFFERS [ boolean ] |
    TIMING [ boolean ] |
    PLAN [ boolean ] |
    FORMAT { TEXT | XML | JSON | YAML }
```

- In addition to configuring different display formats for execution plans, you can use different EXPLAIN syntax to display execution plan information in detail.
  - EXPLAIN *statement*: only generates an execution plan and does not execute. *statement* indicates SQL statements.
  - EXPLAIN ANALYZE *statement*: generates and executes an execution plan, and displays the execution summary. This adds actual execution time statistics to the display, including the total time elapsed within each plan node (in milliseconds) and the total number of rows returned.
  - EXPLAIN PERFORMANCE *statement*: generates and executes an execution plan, and displays all execution information.

Execution Plan Option Parsing (1)

Option	Function	Value Range
ANALYZE boolean | ANALYSE boolean	Displays the actual runtime and other statistics of an SQL statement.	**TRUE** (default): displays them. **FALSE**: does not display them.
VERBOSE boolean	Displays additional information about the plan.	**TRUE** (default): displays it. **FALSE**: does not display it.
COSTS boolean	Specifies the amount of memory required by internal sort operations and hash tables before they write data to temporary disk files.	If the physical memory specified by **work_mem** is insufficient, additional operator calculation data will be written into temporary tables based on query characteristics and the degree of parallelism. This reduces performance five- to ten-fold, and slows down the query response time from seconds to minutes.
CPU boolean	Displays CPU usage information.	**TRUE** (default): displays it. **FALSE**: does not display it.

HUAWEI

Execution Plan Option Parsing (2)

Option	Function	Value Range
BUFFERS boolean	Displays information about the buffer usage.	**TRUE**: displays it. **FALSE** (default): does not display it.
TIMING boolean	Displays the actual startup time and time spent on the output node.	**TRUE** (default): displays them. **FALSE**: does not display them.
PLAN boolean	Specifies whether to store the execution plan in the **PLAN_TABLE** table. When this option is enabled, the execution plan is stored in **PLAN_TABLE** and is not displayed on the current screen. As such, this option cannot be used together with other options.	**TRUE** (default): stores the execution plan in **PLAN_TABLE** and does not display it on the screen. If the plan is stored successfully, "EXPLAIN SUCCESS" is returned. **FALSE**: does not store the execution plan but prints it on the screen.

HUAWEI

Execution Plan Option Parsing (3)

Option	Function	Value Range
FORMAT	Specifies the output format.	**TEXT**, **XML**, **JSON**, and **YAML** Default value: **TEXT**
PERFORMANCE	Prints all relevant execution information.	None

Notes:

- The statement is executed when the **ANALYZE** option is used.
- If you want to use EXPLAIN ANALYZE on an INSERT, UPDATE, DELETE, CREATE TABLE AS, or EXECUTE statement without the statement affecting your data, you can roll back the transaction.
  - START TRANSACTION;
    EXPLAIN ANALYZE statement;
    ROLLBACK;

HUAWEI

Key Parameters in the Execution Plan

- Output parameters in the first parenthesis
  - **cost**: startup cost and total estimated cost of the operator.
  - **rows**: estimated number of last output rows of the operator.
  - **width**: average width of each record.
- Output parameters related to **actual** in the second parenthesis
  - **time**: time when the first data record is returned and time when all data is returned (that is, the end time of the operator).
  - **rows**: actual number of output rows of the operator.
  - **loops**: number of calls on operators.

HUAWEI

EXPLAIN Display Information – actual Parameters

- Parameters related to **actual**:

```
(actual time=460.745..677.468 rows=100000 loops=1)
```

  - **actual time**: actual execution time. **460.745** indicates the time when the current operator obtained the first data record, and **677.468** indicates the time when all data records were obtained.
  - **actual rows**: number of actual returned rows.
  - **loops**: number of executions.

HUAWEI

EXPLAIN Display Information – Hash Join Parameters

- Parameters related to hash join:

```
Buckets: 32768 Batches: 1 Memory Usage: 1kB
```

  - **Buckets**: number of actual buckets in the hash join calculation.
  - **Batches**: number of actual blocks in the hash join calculation.
    - If the value of **Batches** is **1**, hash calculation is performed in the memory and no data is written to disks.
    - If the value of **Batches** is greater than 1, data is spilt to disks. The number of batches is the number of temporary files that are actually used.
  - **Memory Usage**: memory usage in the operator.

HUAWEI

EXPLAIN Display Information – HashAgg and Sort Parameters

- Parameters related to HashAgg:

  Max File Num: 512 Min File Num: 512

  - **File Num**: number of temporary files used by the HashAgg operator when data is spilled to disks.
  - Parameters related to CPU:
    - **inc cyc**: number of CPU cycles from the current node, including all subtrees.
    - **ex c/r**: average number of CPU cycles required to process a single tuple.
    - **ex cyc**: number of CPU cycles of the current node excluding the left and right subtrees.

- Parameters related to sort:
  - **Sort Method**: sorting method. The value in the memory can be **top-*N* heapsort** or **quicksort**.
  - **spaceType**: The value can be **Disk** or **Memory**. **Disk** indicates that data is spilled to disks, and **Memory** indicates that data is sorted in the memory.
  - **spaceUsed**: size of the memory used for sorting in the memory.

HUAWEI

Execution Plan Case 1

- Example:

```
CREATE TABLE t1(a INT, b INT);
INSERT INTO t1 SELECT x,x FROM generate_series (1,10) x; -- Insert 10 data records.
ANALYZE t1;
EXPLAIN ANALYZE SELECT * FROM t1 WHERE a=1;
                                  QUERY PLAN
-------------------------------------------------------------------------------------------
 Seq Scan on t1  (cost=0.00..1.12 rows=1 width=8) (actual time=0.037..0.044 rows=1 loops=1)
   Filter: (a = 1)
   Rows Removed by Filter: 9
 Total runtime: 0.238 ms
(4 rows)
```

  - **Seq Scan** corresponds to the full table scan operator. **cost** indicates the cost of the operator, whose value **0.00** indicates the startup cost (cost of returning the first data record), and **1.12** indicates the total cost. **rows** indicates the estimated size of the result set of the operator.
  - **actual time** indicates the actual execution time of the operator, whose value **0.037** indicates the time when the first record is returned, and **0.044** indicates the execution time of the entire operator. **rows** indicates the actual number of rows, and **loops** indicates the number of repetitions.
  - **Rows Removed by Filter** indicates the number of rows filtered out by the filter criteria. **Total runtime** indicates the actual execution time of the executor.

Execution Plan Case 2 (1)

- The following SQL execution plan is used as an example:

```
openGauss=# explain analyze select * from t1, t2 where t1.salary = t2.salary ;
                                                    QUERY PLAN
-------------------------------------------------------------------------------------------------------------------
 Hash Join  (cost=283.00..36861.94 rows=1267394 width=50) (actual time=21.124..1182.786 rows=499219 loops=1)
   Hash Cond: (t1.salary = t2.salary)
   ->  Seq Scan on t1  (cost=0.00..15155.00 rows=1000000 width=23) (actual time=0.163..497.455 rows=1000000 loops=1)
   ->  Hash  (cost=158.00..158.00 rows=10000 width=27) (actual time=20.606..20.606 rows=10000 loops=1)
         Buckets: 32768  Batches: 1  Memory Usage: 586kB
         ->  Seq Scan on t2  (cost=0.00..158.00 rows=10000 width=27) (actual time=0.131..9.472 rows=10000 loops=1)
 Total runtime: 1242.081 ms
(7 rows)
```


HUAWEI

Execution Plan Case 2 (2)

- Layer 1: **Seq Scan on t2**
  - Table scan operator: scans the table **t2** using Seq Scan. At this layer, data in table **t2** is read from a buffer or disk and then transmitted to the upper-layer node for calculation.
- Layer 2: **Hash**
  - Hash operator: constructs a hash bucket based on the data transmitted from the lower layer and calculates the hash value to prepare for subsequent hash join operations.
- Layer 3: **Seq Scan on t1**
  - Table scan operator: scans table **t1** using Seq Scan. At this layer, data in table **t1** is read from a buffer or disk and then transmitted to the upper-layer node for hash join calculation.
- Layer 4: **Hash Join**
  - Join operator: joins data in tables **t1** and **t2** using the hash join method and outputs the result data.

HUAWEI

Quiz

1. (Single-answer question) Which of the following operators broadcasts data to each thread in an instance?

   A. Gather

   B. RoundRobin

   C. Broadcast

   D. Redistribute

2. (True or False) The ValueScan operator provides a quick and simple way to specify a set of values by reading data from a table. This is useful during testing or debugging, or when you need to insert a small amount of data.

   A. True

   B. False

HUAWEI

Summary

- This chapter describes the openGauss SQL execution engine based on the complete SQL execution process. It focuses on core operators and SQL execution plans, including scan, join, and control operators. In addition, it covers SQL execution plan cases and EXPLAIN usage, precautions, and content interpretation methods.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/en/
- openGauss Help (5.0.0)
  - https://docs-opengauss.osinfra.cn/en/docs/5.0.0/docs/GettingStarted/GettingStarted.html

Thank you.

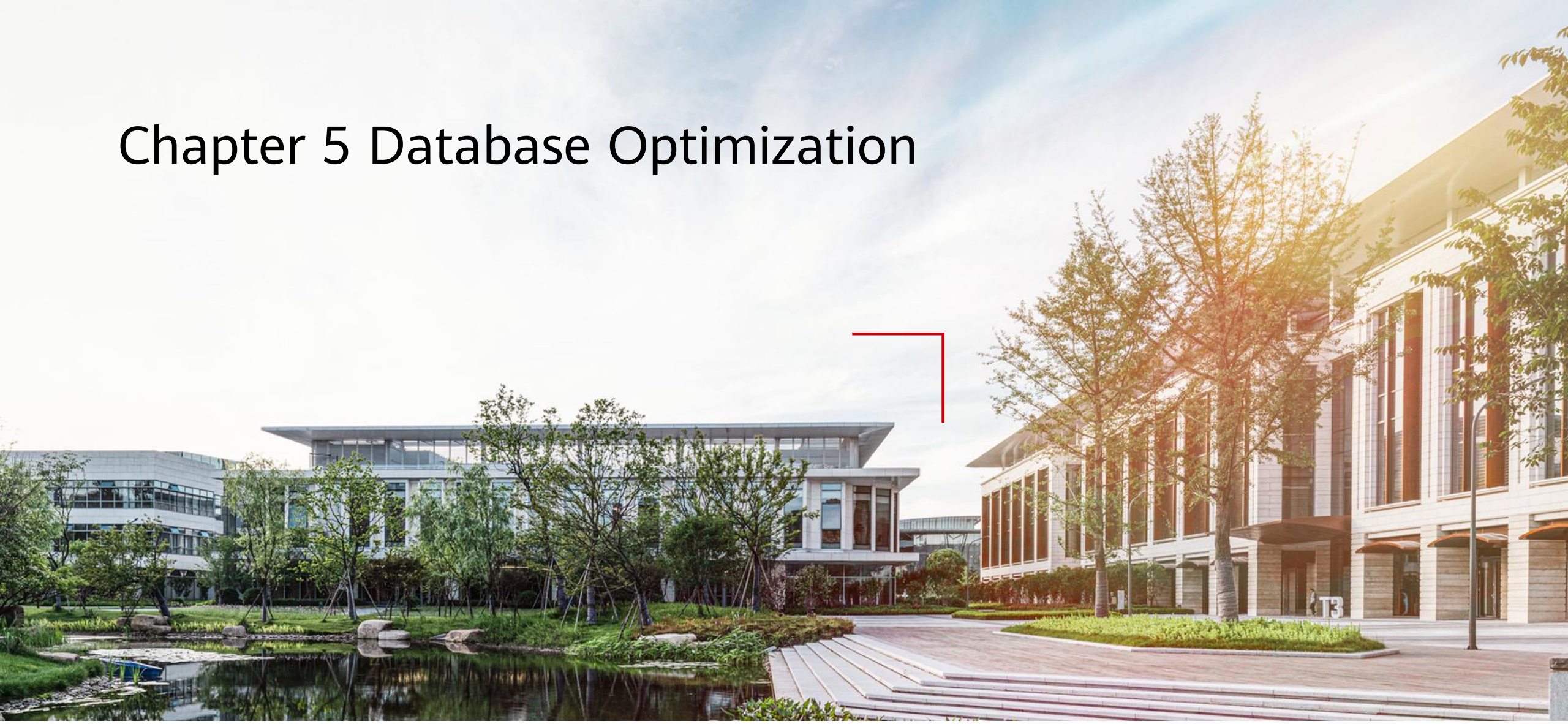
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 5 Database Optimization



HUAWEI

Foreword

- This chapter describes how to optimize database performance, which is a multi-faceted process. We will focus on the basics — optimizing the overall database performance and SQL statements. The former covers servers and the database system. The latter includes optimizing operators, plan hints, indexes, or table structures, using the cost-based optimizer (CBO) or subqueries, diagnosing SQL performance, and rewriting SQL statements (with optimization cases).

HUAWEI

Objectives

Upon completing this course, you will be able to:

- Describe the basic performance optimization guidelines and process.
- Diagnose database performance problems and analyze performance bottlenecks.
- Have a solid command of openGauss optimization features and methods.
- Use proper performance optimization methods to solve database performance problems in real-life projects.

Contents

1. **Performance Optimization Guidelines and Process**
2. Optimizing the Overall Database Performance
3. SQL Optimization

HUAWEI

Performance Optimization Guidelines

- Analyze the bottlenecks in system hardware resources and SQL statements and optimize the database performance accordingly.
- Multiple factors need to be considered during openGauss performance optimization. As such, optimization personnel must have a wide and in-depth understanding of databases, storage systems, networks, OSs, and even server kernel architectures.
- Performance optimization sometimes requires restarting the openGauss database, which may interrupt services. So, you need to first submit an operation window request to the relevant management department. You should only restart the openGauss database after the request is approved.

HUAWEI

Factors Affecting Performance

- **Hardware resources**
  - CPU, memory, disk, and network
- **Software resources**
  - OS, including its parameter configuration, drivers, and database software
- **Load and system scale**
  - Data volume, concurrency, and maximum throughput
- **Database design**
  - Database architecture design, table structure design, index design, query design, and database parameter configurations

HUAWEI

Optimization Process — Analyzing System Performance

1. **CPU**: Run commands such as **top** to check whether the CPU usage is too high (generally, over 80%).
2. **Memory**: Run commands such as **free** to check the memory usage, including how much physical memory is being used and if there is too little physical memory.
3. **Disk I/O**: Run commands such as **iostat** to check the disk usage. Disk performance becomes a bottleneck when over 90% of the disk space is in use. If there are many INSERT or UPDATE operations in SQL statements, check whether the disk is full.
4. **network**: Run commands such as **ping** and **sar** to check whether the network latency and bandwidth are causing bottlenecks. Ideally, the network latency should be less than 0.1 ms, and the bandwidth usage should be less than 10%.

HUAWEI

Optimization Process — Analyzing Basic Parameter Settings (1)

- Before analyzing slow SQL statements, review some important GUC parameter settings. Performance deterioration is usually related to improper parameter settings.

Parameter	Description	Best Practices
max_process_memory	Specifies the maximum physical memory of a database node.	Adjust the value based on the physical memory size: (Physical memory size – **vm.min_free_kbytes**) x 0.7. That is, **max_process_memory** = Physical memory size x 0.665.
shared_buffers	Specifies the size of the shared memory used by the openGauss database.	You are advised to set **shared_buffers** to a value below 40% of the memory. Set **shared_buffers** to a large value for row store and to a small value for column store. For column-store tables: Memory of a single server/Number of database nodes on the server x 0.4 x 0.25.
work_mem	Specifies the amount of memory to be used by internal sort operations and hash tables before they write data into temporary disk files.	If the physical memory specified by **work_mem** is insufficient, additional operator calculation data will be written into temporary tables based on query characteristics and the DOP. This can lead to five to ten times lower performance and slow down query responses from seconds to minutes.

HUAWEI

Optimization Process — Analyzing Basic Parameter Settings (2)

- max_process_memory
  - This parameter is used to prevent node OOM when memory usage increases, ensuring system reliability. **vm.min_free_kbytes** indicates the OS memory reserved for the kernel to receive and send data. Its value is at least 5% of the total memory.
  - If this parameter is set to a value greater than the server's physical memory, an OS OOM problem may occur.
  - If multiple database instances are deployed on a host, you need to set the parameter to a smaller value.

HUAWEI

Optimization Process — Analyzing Basic Parameter Settings (3)

- shared_buffers
  - The value of **shared_buffers** must be an integer multiple of **BLCKSZ**. Currently, **BLCKSZ** is set to **8kB**. That is, the value of **shared_buffers** must be an integer multiple of 8 KB. The minimum value changes according to **BLCKSZ**.
  - If **shared_buffers** is set to a larger value, increase the value of **checkpoint_segments** because more time is required to write lots of new or changed data.
  - In resource pooling mode, if the primary and standby nodes run services at the same time, set this parameter to a larger value. If this parameter is set to a small value, the primary and standby nodes may frequently interact with each other, increasing the network load. In severe cases, the database may respond slowly or even be suspended.

HUAWEI

Optimization Process — Analyzing Basic Parameter Settings (4)

- work_mem
  - For complex serial queries, each query requires five to ten associated operations. Set **work_mem** using the following formula: **work_mem** = 50% of the memory/10.
  - For simple serial queries, each query requires two to five associated operations. Set **work_mem** using the following formula: **work_mem** = 50% of the memory/5.
  - For concurrent queries, set **work_mem** using the following formula: **work_mem** = **work_mem** for serial queries/Number of concurrent SQL statements.
  - **work_mem** also restricts bitmap scan hash tables but does not strictly control data spilling to disks. In the case of complete lossify, each time the hash table occupies 1 MB memory, it corresponds to a 16 GB page of BitmapHeapScan (32 GB for Ustore). After the upper limit of **work_mem** is reached, memory usage increases linearly with the data access traffic following this ratio.

HUAWEI

Optimization Process — Analyzing SQL Performance

If the basic configuration parameters are normal, analyze the SQL performance in depth.

1. Collect all table statistics associated with the SQL statements.
2. View the execution plan to find out the cause.
3. Review and modify the table definitions.
4. For the EXPLAIN or EXPLAIN PERFORMANCE information, check the difference between the actual row and the estimated row. If they are very different, the statistics are incorrect.
5. Analyze the predicates and indexes of the execution plan to check whether the indexes are correctly created and properly used.
6. Check whether the hash index or sort overflows. If there is an overflow, temporary files are generated on the disk.
7. If the execution plan cannot be optimized further, analyze the flame graph and cold flame graph.

HUAWEI

Querying High-Load SQL Statements (1)

- Some SQL statements in the system run for a long time and do not end. These statements consume a lot of system performance.

  1. Query the statements in the system that have been running for a long time.

     ```
     SELECT current_timestamp - query_start AS runtime, datname, username, query FROM pg_stat_activity
     where state != 'idle' ORDER BY 1 desc;
     ```

     - The command output lists the query statements in descending order by their execution duration. The first record represents the statement that is taking the longest to execute. There are SQL statements called by the system and SQL statements run by users. Determine which statements are taking too long based on the given situation.
     - If the current system is busy, you can set the value of **current_timestamp** minus **query_start** to be greater than a threshold. This will identify query statements that are taking longer than the threshold to execute.

     ```
     SELECT query FROM pg_stat_activity WHERE current_timestamp - query_start > interval '1 days';
     ```

  2. Set the parameter **track_activities** to **on**.

     ```
     SET track_activities = on;
     ```

     - The database system collects running information about active queries only if the parameter is set to **on**.

HUAWEI

Querying High-Load SQL Statements (2)

3. View the running query statements. The pg_stat_activity view is used as an example.

```
SELECT datname, usename, state FROM pg_stat_activity;
 datname  | usename | state  |
----------+---------+--------+
 postgres |   omm   | idle   |
 postgres |   omm   | active |
(2 rows)
```

   - If the **state** column is **idle**, the connection is idle and requires a user to enter a command.

4. Analyze the status of the query statements that have been running for a long time.
   - If a query statement is normal, wait until its execution completes.
   - If a query statement is blocked, run the following command to view this query statement:

```
SELECT datname, usename, state, query FROM pg_stat_activity WHERE waiting = true;
```

   - The command output lists a blocked query statement because it requested a lock resource that may be occupied by another session and is waiting for the lock to be released.
   - Note: Only when the query is blocked by internal lock resources, **waiting** is **true**. In most cases, blocking happens when query statements are waiting for lock resources to be released. However, query statements may be blocked due to write operations and timers. Such blocked queries are not displayed in the pg_stat_activity view.

HUAWEI

Checking Blocked Statements

- When the database is running, certain service scenarios cause blocked query statements, which run for too long. In this case, you can forcibly terminate the abnormal session.

1. View the blocked query statements and details about the tables and schemas that block them.
   - The information is shown in the query statement on the right.
   - The query returns the thread ID, user information, and query status, and information about the tables and schemas that cause blocking.

```
SELECT w.query as waiting_query,
w.pid as w_pid,
w.usename as w_user,
l.query as locking_query,
l.pid as l_pid,
l.usename as l_user,
t.schemaname || '.' || t.relname as tablename
from pg_stat_activity w join pg_locks l1 on w.pid = l1.pid
and not l1.granted join pg_locks l2 on l1.relation = l2.relation
and l2.granted join pg_stat_activity l on l2.pid = l.pid join
pg_stat_user_tables t on l1.relation = t.relid
where w.waiting;
```

2. Run the following command to terminate the required session, where **139834762094352** is the thread ID:

```
SELECT PG_TERMINATE_BACKEND(139834762094352);
```

   - If the information is similar to the example below, the session is successfully terminated. If the command output is similar to the example below, a user is attempting to terminate the session, and the session will be reconnected rather than terminated.

```
PG_TERMINATE_BACKEND
----------------------
 t
(1 row)
```

```
FATAL:  terminating connection due to administrator command
FATAL:  terminating connection due to administrator command
The connection to the server was lost. Attempting reset: Succeeded.
```


HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. **Optimizing the Overall Database Performance**
   - Optimizing the Server
     - Configuring the Vectorized Executor
     - Configuring Ustore
     - Configuring SMP
3. SQL Optimization

HUAWEI

Optimizing the Server — Overview

- Server parameters and configurations directly affect overall database performance. As such, you need to set server parameters based on service scenarios.
- Different server models have different parameter items and paths. This section uses the Kunpeng 920 server as an example to describe key server parameters based on a high-performance use case for the openGauss database.
- Check whether the OS parameters are set to the suggested values using gs_check. If not, modify them as required.

HUAWEI

Setting Memory and Network Parameters

- Memory parameters
  - In the **sysctl.conf** file, change the value of **vm.extfrag_threshold** to **1000** (reference value). If the file does not contain memory parameters, add them manually.
    - vim /etc/sysctl.conf
  - Run the following command for the modification to take effect:
    - sysctl -p
- Network parameters
  - In the **sysctl.conf** file, modify the network parameters. If the file does not contain such parameters, add them manually.
    - vim /etc/sysctl.conf
  - Run the following command for the modification to take effect:
    - sysctl -p

HUAWEI

Optimizing the Server — BIOS Parameters

- After modifying BIOS parameters, you need to restart the server. This operation can be performed only on a physical server and when related permissions are available.
- The parameters and their paths vary according to the server model. For details, see the server usage guide.

Parameter Path	Parameter Description
Advanced->Performance Config->Power Policy	Power policy of the server. The default value is **Efficiency**. If the CPU performance is closely related to the computing power, you are advised to set the parameter to **Performance**. Once set, the system runs at a nominal frequency without dynamic frequency adjustment, which increases the power consumption and improves the CPU performance.
Advanced >PCIe Config >CPU X PCIe - Port X	Use cases: testing NVMe PCIe SSDs or 25GE/100GE NICs Impact: Data is transferred between the PCIe device and the memory used by the CPU through the PCIe bus. **Max Payload Size** is the maximum amount (in bytes) of data that can be transferred each time. The size is in direct proportion to the transfer efficiency of the PCIe link. The larger the value of this parameter, the higher the bandwidth usage of the PCIe link.
Advanced->Memory Config -> NUMA	Controls whether to enable non-uniform memory access (NUMA). The CPU core runs faster when accessing the local NUMA memory but slows down when accessing memory outside the node. You are advised to enable this function (default setting).

HUAWEI

Optimizing the Server — OS Parameters

Parameter Path	Parameter Description
vm.dirty_ratio	When the percentage of dirty pages (relative to all available memory, that is, idle memory pages + recyclable memory pages) reaches **dirty_ratio**, write is used to call the flusher thread, which can wake up the kernel, to write back dirty page data until the percentage of dirty pages is less than the value of **dirty_ratio**. In this case, write calling is blocked, and the hard disk I/O is suspended. The default value of the parameter is 20%.
vm.dirty_background_ratio	When the percentage of dirty pages (relative to all available memory, that is, idle memory pages + recyclable memory pages) reaches **dirty_background_ratio**, write is used to call the flusher thread, which can wake up the kernel, to write back dirty page data until the percentage of dirty pages is less than the value of **dirty_background_ratio**. However, unlike in **dirty_ratio**, write calling is not blocked, and disk I/Os are not suspended. The default value of the parameter is **10%**.
vm.dirty_bytes and vm.dirty_background_bytes	Same as **vm.dirty_ratio** and **vm.dirty_background_ratio**. The only difference is that the memory capacity is controlled. The default value is **0**.
vm.dirty_expire_centisecs	Sets the expiration period (in milliseconds), after which cached content will be flushed to disks. The flusher thread of the kernel is periodically woken up to check the retention period of dirty page content. If the period exceeds the value of this parameter, dirty data is asynchronously written back to disks. The default value is **3000**, that is, 30 seconds.
vm.dirty_writeback_centisecs	Sets the interval for waking up the flusher kernel thread. This thread is used to write dirty pages back to disks. The unit is ms. The default value is **500**, that is, 0.5s.

HUAWEI

Optimizing the Server — Network Optimization (1)

1. Increase the MTU to increase throughput.
   - If all network packets to be transmitted are large (more than 1500 bytes), increase the MTU. The default MTU is 1500 bytes, and the maximum MTU is 9000 bytes.
   - The size of the package transmitted between the database server and client varies according to the service scenario. As such, set this parameter based on your database requirements.
   - Setting method:
     - (1) Instant setting (invalid after the system is restarted):
       - ifconfig ethx mtu 9000
     - (2) Permanent setting:
       - Run the command **vi /etc/sysconfig/network-scripts/ifcfg-$(Interface)**, set **MTU** to **"9000"**, and restart the network service.

Optimizing the Server — Network Optimization (2)

2. Set the interrupt coalescing parameter.
   - Generally, an interrupt is generated each time a NIC sends or receives a data packet. However, frequently sending or receiving data causes ongoing CPU interruptions, resulting in poor system performance.
   - If the interrupt is triggered too late, the CPU cannot process data in time. This leads to a delay that affects upper-layer services. Set this parameter based on service requirements. To reduce CPU usage, set this parameter to a larger value. If it is not essential to reduce CPU usage because the delay is low, set this parameter to a smaller value.
   - **Setting method:**
     - (1) Run the **ethtool -c ethX** command to view the parameter (**ethX** indicates the NIC name).
     - (2) Run the **ethtool -C ethX <params>** command to set the parameter.
   - For example, **eth3** is the NIC name and *N* is the actual value.
     - ethtool -C eth3 adaptive-rx off adaptive-tx off rx-usecs N rx-frames N tx-usecs N tx-frames N

HUAWEI

Optimizing the Server — Network Optimization (3)

3. Bind NIC interrupts to cores.
   - Binding NIC interrupts to cores applies when the network load is heavy or NIC interrupts need to be bound to specific CPU cores.
   - For multi-core servers, you need to configure multiple queues for NICs on the server and client. The recommended configuration is 12 interrupt queues for NICs on servers and 48 for NICs on clients. Set this parameter based on your service requirements.
   - (1) Disable irqbalance.
     - If the irqbalance service is not disabled, it will assign the interrupt to another CPU core even if the bound network is interrupted. In this case, you need to stop the service.
       - service irqbalance stop
   - (2) Configure IRQ core binding.
     - Separate the CPU that processes network interrupts from the CPU that processes database services to avoid mutual interference.

HUAWEI

Optimizing the Server — Data Disk Partitioning

- If there are multiple data disks, you can store Xlogs on different disks to improve the Xlog write speed of the database.
- Multiple tablespaces can be created in the openGauss database, which means that different table structures can be placed on different disks, and the database can perform read/write operations on disks faster.
- When the disk I/O is the performance bottleneck, data partitioning becomes very effective.
- Example:

```
mv  $DATA_NODE/pg_xlog    $XLOG
mv  $DATA_NODE/pg_location/tablespace2   $TABLESPACE2
mv  $DATA_NODE/pg_location/tablespace3   $TABLESPACE3

ln  -svf  $XLOG   $DATA_NODE/pg_xlog
ln  -svf  $TABLESPACE2   $DATA_NODE/pg_location/tablespace2
ln  -svf  $TABLESPACE3   $DATA_NODE/pg_location/tablespace3
```


HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. **Optimizing the Overall Database Performance**
   - Optimizing the Server
   - Configuring the Vectorized Executor
   - Configuring Ustore
   - Configuring SMP
3. SQL Optimization

HUAWEI

Database System Feature Optimization — Configuring the Vectorized Executor (1)

- Vectorized executor highlights:
  - More data is read in one batch at a time, saving I/O resources.
  - There are many records in a batch, and the CPU cache hit rate increases.
  - In pipeline mode, the functions are called infrequently.
  - Data is processed in batches, which is efficient.
- Column-store tables perform better in complex analysis queries but do not perform well in inserting and updating data. This means such tables cannot be used for services that need to insert and update data frequently.
- openGauss provides the vectorized executor to improve the query performance of row-store tables in complex analytical queries. You can set the GUC parameter **try_vector_engine_strategy** to convert query statements containing row-store tables into vectorized execution plans for execution.

HUAWEI

Database System Feature Optimization — Configuring the Vectorized Executor (2)

- Converting a row-store table to vectorized execution does not work for all types of queries.
- Vectorized execution has the most gains when query statements contain operations such as expression calculation, multi-table join, and aggregation. In these cases, converting statements to vectorized execution can lead to significant performance improvements.
- Theoretically, if a row-store table is converted to vectorized execution, it increases the overhead and the performance deteriorates. However, when used for expression calculation, join, and aggregation, performance improves.
- Performance gains from vectorized execution should outweigh performance losses from overhead during the conversion to truly improve overall performance.

HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. **Optimizing the Overall Database Performance**
   - Optimizing the Server
   - Configuring the Vectorized Executor
   - Configuring Ustore
   - Configuring SMP
3. SQL Optimization

HUAWEI

Ustore Design Principles

- Ustore, also called the in-place update storage engine, is a new storage model added to the openGauss kernel.
- Earlier versions of the row storage engine used by openGauss use append update, which performs well in service addition, deletion, and heap-only tuple (HOT) updates (that is, updates on the same page). However, recycling is not efficient in non-HOT updates across data pages. Therefore, Ustore emerges as the solution.
- Design principles
  - The valid data from the latest version and junk data from old versions are stored separately. Valid data is stored on the data page and junk data is centralized into an independent undo space. This prevents the data space from bloating due to frequent updates, and the junk data is recycled more efficiently.
  - Ustore adopts the NUMA-aware undo subsystem design, which enables the undo subsystem to be effectively expanded on the multi-core platform. With multi-version index technology, the engine can delete indexes and maximize the reuse of storage space.
  - Ustore works with the undo space to implement more efficient and comprehensive flashback query and recycle bin mechanisms, quickly roll back misoperations, and provide abundant enterprise-grade functions for openGauss.

HUAWEI

Core Ustore Advantages

- High performance
  - Services with different loads, such as insertion, update, and deletion, have relatively balanced resource usage.
  - In-place updates are recommended in frequent update scenarios, featuring higher and more stable performance. This works well for typical OLTP service scenarios that require short transactions, frequent updates, and high performance.
- Efficient storage
  - Supports in-place updates to the maximum extent, saving a lot of space.
  - Rollback segments and data pages are stored separately to provide more efficient and stable I/O usage. The undo subsystem uses the NUMA-aware design to provide better multi-core scalability. Undo space is centrally allocated and recycled, improving reuse efficiency and storage space usage.
- Fine-grained resource control
  - Ustore provides multi-dimensional transaction monitoring. It monitors transactions based on their duration, the undo space size used by a single transaction, and the overall undo space limit to prevent exceptions and unexpected behaviors. This allows database administrators to regulate and restrict the use of database system resources.
- Ustore provides stable performance when data is frequently updated, enabling service systems to run more stably and adapt to more service scenarios and workloads. This is especially true for core financial services that have higher performance and stability requirements.

HUAWEI

Ustore — Creating Tables

- Ustore contains undo logs. Before creating a Ustore table, you need to set **undo_zone_count** in **postgresql.conf**. This parameter indicates the number of undo log resources and must be set to **16384**, that is, **undo_zone_count=16384**. Once set, restart the database.
  - **postgresql.conf** configuration: **undo_zone_count=16384**
  - Method 1: Specify the storage engine type when creating a table.
    - create table test(id int, age int, name varchar(10)) with (storage_type=ustore);
  - Method 2: Specify the Ustore by configuring a GUC parameter.
    - 1. Before starting a database, set **enable_default_ustore_table** to **on** in **postgresql.conf** to specify that Ustore is used by default when a user creates a table. **postgresql.conf** configuration: **enable_default_ustore_table=on**
    - 2. Create the table: **create table test(id int, age int, name varchar(10));**

HUAWEI

Ustore — Creating Indexes

- Ustore only supports UB-tree indexes, which are developed exclusively for Ustore.
- Example: Add a UB-tree index to the **age** column in the **test** table.
  - Method 1: If the index type is not specified, a UB-tree index is created by default.
    - create index ubt_idx on test(age);
  - Method 2: When creating an index, use the USING keyword to set the index type to **ubtree**.
    - create index ubt_idx on test using ubtree(age);

HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. **Optimizing the Overall Database Performance**
   - Optimizing the Server
   - Configuring the Vectorized Executor
   - Configuring Ustore
   - Configuring SMP
3. SQL Optimization

HUAWEI

Parallel Query

- The Symmetric Multi-Processing (SMP) technology of openGauss uses the computer's multi-core CPU architecture to implement multi-thread parallel computing, capitalizing on CPU resources to improve query performance.
- Complex queries take a long time, and the system concurrency is low. SMP is used to implement operator-level parallel execution, which effectively speeds up queries, improving query performance and resource utilization.
- SMP implementation: For query operators that can be executed in parallel, data is sliced, multiple worker threads are started for computation, and then the results are summarized and returned to the front end. A data interaction operator named Stream is added to the SMP architecture to implement data interaction between multiple worker threads, ensuring the query's accuracy and integrity.
- SMP improves performance through operator parallelism and occupies more system resources, including CPU, memory, and I/O.
- Actually, SMP consumes resources to save time. When used appropriately and there are enough resources, it improves system performance; otherwise, performance may deteriorate.
- SMP applies to analytical query scenarios where a single query takes a long time and the service concurrency is low. SMP can reduce query delay and improve system throughput. However, in a high transactional concurrency scenario, a single query has a short delay. In this case, using SMP increases query delay and reduces system throughput.

HUAWEI

When to Use SMP

- Operators support parallelism, as detailed in the plan:
  - Scan: Sequentially scans ordinary row-store tables, partitioned row-store tables, ordinary column-store tables, and partitioned column-store tables.
  - Join: hash join and nested loop join
  - Agg: HashAgg, SortAgg, PlainAgg, and WindowAgg (which supports only PARTITION BY and does not support ORDER BY).
  - Stream: Local Redistribute and Local Broadcast
  - Others: Result, Subqueryscan, Unique, Material, Setop, Append, and VectoRow
- SMP-specific operators: To execute queries in parallel, Stream operators (a new sybtype of Stream operators) are added for SMP data exchanges.
  - Local Gather: aggregates data of parallel threads within an instance.
  - Local Redistribute: redistributes data based on the distribution key across threads within an instance.
  - Local Broadcast: broadcasts data to each thread within an instance.
  - Local RoundRobin: distributes data in polling mode across threads within an instance.

HUAWEI

SMP Configuration Instance

- The following parallel execution plan is used as an example.

```
 id |                    operation
----+-------------------------------------------------
  1 | ->  Sort
  2 |     ->  HashAggregate
  3 |         ->  Streaming(type: LOCAL GATHER dop: 1/4)
  4 |             ->  HashAggregate
  5 |                 ->  Seq Scan on lineitem
(5 rows)
```

- In this plan, the Scan and HashAgg operators are processed in parallel, and the Local Gather operator is added for data exchange. Operator 3 is a Local Gather operator. "dop: 1/4" indicates that the DOP of the sender thread is 4 and the DOP of the receiver thread is 1. That is, the lower-layer HashAggregate operator 4 is executed based on the DOP 4, the upper-layer operators 1 and 2 are executed in serial mode, and operator 3 aggregates the data of parallel threads within the instance.
- You can view the parallelism of each operator in the DOP information displayed by the Stream operator.

HUAWEI

When Not to Use SMP

- Scenarios where parallel execution does not apply:
  - Index scan
  - Merge join
  - WindowAgg order by
  - cursor
  - Queries in stored procedures and functions
  - Queries of subplans and initplans, and operators that contain subqueries
  - Queries that contain the median operation
  - Queries on global temporary tables
  - Updates of materialized views

HUAWEI

Resource Impact on SMP Performance

- The SMP architecture uses abundant resources to obtain time. Executing plan parallelism adds resource consumption, including the CPU, memory, and I/O resources. As the DOP increases, resource consumption grows. If these resources become a bottleneck, SMP cannot improve performance, and the overall performance of databases instances may deteriorate. Here is how SMP affects the following resources.
- CPU resources
  - For regular customers whose system CPU usage is low, using the SMP architecture will capitalize on the CPU resources to improve system performance. If the database server has few CPU cores and the CPU usage is already high, enabling the SMP feature may deteriorate the system performance due to resource contention among threads.
- Memory resources
  - Parallel queries consume more memory, but the memory usage of each operator is still restricted by **work_mem** and other parameters. Assuming that **work_mem** is 4 GB and the DOP is 2, the maximum memory usage of each thread in parallel is 2 GB. When **work_mem** is small or the system does not have enough memory, using SMP may spill data to disks, deteriorating performance.
- I/O resources
  - A parallel scan increases I/O resource consumption. It can improve scan performance only when there are enough I/O resources.

HUAWEI

Other Factors Affecting SMP Performance

- Besides resource factors, there are other factors that impact SMP performance, such as uneven data distribution in a partitioned table and system DOP.
- Data skew
  - Severe data skew deteriorates SMP performance. For example, if the join column has a lot more data compared with other values, the parallel thread will also have a lot more data than others after hash redistribution, resulting in the long-tail issue and poor SMP performance.
- System DOP
  - The SMP feature uses more resources. In a high concurrency scenario, there may not be enough remaining resources. So, enabling SMP will result in severe resource competition among queries. Once resource competition occurs, it will deteriorate system performance, regardless of whether it is for CPU, I/O, or memory resources.

HUAWEI

Configuring the SMP — Procedure

- To use SMP, perform the following steps:
  - (1) Observe the current system workload. If there are enough system resources (the resource usage is less than 50%), go to the next step. Otherwise, exit the system.
  - (2) Set **query_dop** to **1** (default value). Use EXPLAIN to generate an execution plan and check whether the plan includes scenarios where SMP applies. If it does, go to step (3).
  - (3) Set **query_dop** to *value* or **1** regardless of the resource usage and plan characteristics. Before the query statement is executed, set **query_dop** to an appropriate value. After the statement is executed, set **query_dop** to disable the query. For example:

```
SET query_dop = 4;
SELECT COUNT(*) FROM t1 GROUP BY a;
SET query_dop = 1;
```


HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. Optimizing the Overall Database Performance
3. **SQL Optimization**
   - Similar Operator Selection and Plan Hints
   - Indexes
   - Optimizing Table Structures
   - CBO
   - Diagnosing SQL Performance
   - Optimizing Subqueries and Rewriting SQL Statements

HUAWEI

Selecting an Operator — Scan Operators

- Selection criteria:
  1. If the table contains a small amount of data, you are advised to select SeqScan because it does not require additional operations such as index maintenance.
  2. If a table contains a lot of data, the query speed in descending order is as follows:
     - BitmapScan > IndexOnlyScan > IndexScan > SeqScan
  - Refer to sequence above to choose the fastest operator for your use case.

HUAWEI

Selecting Operators — Join Operators

- Selecting the most efficient join mode:
  - (a) If both tables contain a small amount of data, use Nested Loop.
  - (b) If one table contains a lot of data and the other contains little data, use hash join with the smaller table as the inner table.
  - (c) If both tables contain a similar and large amount of data, use merge join.
- Selecting the join sequence:
  - (a) You are advised to select the smaller table as the inner table.
  - (b) In multi-table join, use the table with a smaller join result set as the inner table.

HUAWEI

Controlling an Execution Plan by Using GUC Parameters

- In the openGauss database, using GUC parameters and plan hints can control an execution plan.
- In specific scenarios, you can set GUC parameters or add plan hints to influence the SQL execution plan. Selecting a correct execution plan helps improve SQL performance.
- Examples of setting GUC parameters:
  - Enable bitmap scan.
    - **set enable_bitmapscan to on;** or **set enable_bitmapscan=on;**
  - Disable bitmap scan.
    - **set enable_bitmapscan to off;** or **set enable_bitmapscan=off;**
  - Viewing the parameter status.
    - **show enable_bitmapscan;**

HUAWEI

Controlling an Execution Plan by Using GUC Parameters — Scan Operators

Operator	GUC Parameter	Description
SeqScan	enable_seqscan	It is impossible to suppress sequential scans entirely, but setting this variable to **off** encourages the optimizer to choose other methods if available.
IndexScan	enable_indexscan	Specifies whether the optimizer uses index scan.
IndexOnlyScan	enable_indexonlyscan	Specifies whether the optimizer uses index-only scan.
BitmapScan	enable_bitmapscan	The bitmap scan applies only in the query condition where **a > 1 and b > 1** and indexes are created on columns **a** and **b**. Sometimes, it does not perform as well as other scans.
TidScan	enable_tidscan	Specifies whether the TidScan operator is used.

HUAWEI

Controlling an Execution Plan by Using GUC Parameters — Join Operators

Operator	GUC Parameter	Description
Nested Loop	enable_nestloop	(1) Specifies whether the optimizer uses nested loop join. If it is **"on"**, the optimizer preferentially uses nested loop join. If it is **"off"**, the optimizer preferentially uses other methods, if any. (2) This parameter is **"on"** by default. Change the value as required. (3) Generally, nested loop join performs worse than the other join methods (merge join and hash join). You are advised to set this parameter to **off**.
Hash Join	enable_hashjoin	Specifies whether the optimizer uses hash join.
Merge Join	enable_mergejoin	Specifies whether the optimizer uses merge join.

HUAWEI

Controlling an Execution Plan by Using Plan Hints — Introduction

- (1) In plan hints, you can specify a join order, join and scan operations, and the number of rows in a result to optimize an execution plan and improve query performance. Plan hints are normally used when you already know the optimal SQL execution plan or want to stabilize the SQL execution plan for optimized performance.
- (2) openGauss provides the SQL patch function. You can create an SQL patch to make hints take effect without modifying service statements.
- (3) A plan hint can be specified after a SELECT keyword in the following format:
  - /*+ <plan hint>*/
- (4) You can specify multiple hints for a query plan and separate them by spaces. A hint specified for a query plan does not apply to its subquery plans. To specify a hint for a subquery, add the hint following the "SELECT" of this subquery.
  - select /*+ <plan_hint1> <plan_hint2> */ from t1, (select /*+ <plan_hint3> */ from t2) where 1=1;
  - Here, <*plan_hint1*> and <*plan_hint2*> are the query's hints, and <*plan_hint3*> is its subquery's hint.

HUAWEI

Supported Plan Hints

- The current version supports the following plan hints:
  - Join order hints (leading)
  - Join operation hints that are commonly used, excluding the semi join, anti join, and unique plan hints.
  - Hints specifying the number of rows in the result set.
  - Stream operation hints
  - Certain scan operation hints, namely table scan, index scan, and index-only scan
  - Sublink name hints
  - Hints specifying not to expand subqueries
  - Hints for inner table materialization
  - Bitmap scan hints
  - Hints of the Agg method

Plan Hint Type (1)

1. Join order hint
   - Syntax format:
     - leading(join_table_list): Only the join order is specified.
     - leading((join_table_list)): specifies both the join order and the sequence of inner and foreign tables. The sequence of inner and foreign tables takes effect only at the outermost layer.
     - **leading(t1 t2 t3 t4 t5)**: **t1**, **t2**, **t3**, **t4**, and **t5** are joined. The join order and outer/inner tables are not specified.
     - **leading((t1 t2 t3 t4 t5))**: **t1**, **t2**, **t3**, **t4**, and **t5** are joined in sequence. The table on the right is used as the inner table in each join.
     - **leading((t1 (t2 t3 t4) t5))**: First, **t2**, **t3**, and **t4** are joined, and the outer/inner tables are not specified. Then, the result is joined with **t1**, and **(t2 t3 t4)** is used as the inner table. Finally, the result is joined with **t5**, and **t5** is used as the inner table.
     - **leading((t1 (t2 t3) t4 t5)) leading((t3 t2))**: First, **t2** and **t3** are joined and **t2** is used as the inner table. Then, the result is joined with **t1**, and **(t2 t3)** is used as the inner table. Finally, the result is joined with **t4** and then **t5**, and the table on the right in each join is used as the inner table.
   - Example:
     - select /*+ leading((((((store_sales store) promotion) item) customer) ad2) store_returns) leading((store store_sales))*/ i_product_name product_name ...
     - First, **store_sales** and **store** are joined and **store_sales** is the inner table. Then, the result is joined with **promotion**, **item**, **customer**, **ad2**, and **store_returns** in sequence.

HUAWEI

Plan Hint Type (2)

2. A join operation hint specifies the join method, which can be nested loop, hash join, or merge join.
   - Syntax format:
     - [no] nestloop|hashjoin|mergejoin(table_list)
     - Parameter description: **no** indicates that the specified hint will not be used for a join. **table_list** specifies the tables to be joined. The values are the same as those of **join_table_list** but contain no parentheses for specifying priorities.
     - For example, **no nestloop(t1 t2 t3)** means that nested loop join is not used for joining **t1**, **t2**, and **t3**. The three tables may be joined in either of two ways: Join **t2** and **t3**, and then **t1**; join **t1** and **t2**, and then **t3**. This hint takes effect only for the last join. If necessary, you can hint other joins. For example, you can add **no nestloop(t2 t3)** to join **t2** and **t3** first to forbid the use of nested loop join.
   - Example:
     - select /*+ nestloop(store_sales store_returns item) */ i_product_name product_name ...
     - Nested loop is used for the last join between **store_sales**, **store_returns**, and **item**.

HUAWEI

Plan Hint Type (3)

3. A rows hint specifies the number of rows in an intermediate result set. Both absolute values and relative values are supported.
   - Syntax format: rows(table_list #|+|-|* const)
     - **#**, **+**, **-**, and ***** are operators used for hinting the estimation. **#**: The original estimation is used without any calculation. **+**, **-**, and *****: The original estimation is calculated using these operators. The minimum calculation result is 1. **table_list**: The tables to be joined. The values are the same as those of **table_list** in the join hint.
     - *const* can be any non-negative number and supports scientific notation.
     - Example: **rows(t1 #5)**: The result set of **t1** is five rows.
     - Example: **rows(t1 t2 t3 *1000)**: Multiply the result set of joined **t1**, **t2**, and **t3** by 1000.
   - Example:
     - select /*+ rows(store_sales store_returns *50) */ i_product_name product_name ...
     - Multiply the result set of joined **store_sales** and **store_returns** by 50.

Plan Hint Type (4)

4. A scan operation hint specifies the scan method, which can be **tablescan**, **indexscan**, or **indexonlyscan**.
   - Syntax format:
     - [no] tablescan|indexscan|indexonlyscan(table [index])
     - **no**: The specified hint will not be used for scanning. *table* specifies the table to be scanned. You can specify only one table. It is preferred that the value is a table alias (if any) instead of a table name.
     - *index* indicates the index for the indexscan or indexonlyscan hint. You can specify only one index.
     - The indexscan and indexonlyscan hints can be used only when the specified index belongs to the table.
   - Example:
     - select /*+ indexscan(item i) */ i_product_name product_name ...
     - This hint means **item** is scanned based on index **i**.
5. A sublink name hint specifies the name of the sublink block.
   - Syntax format: blockname (table) *table* specifies the name you have specified for a sublink block.
     - Example: explain select /*+nestloop(store_sales tt) */ * from store_sales where ss_item_sk in (select /*+blockname(tt)*/ i_item_sk from item group by 1);
     - **tt** indicates the sublink block name. After being pulled up, the sublink is joined with the outer-query table **store_sales** by using nested loop.
     - An outer query uses the blockname hint only when the corresponding sublink is not pulled up. Currently, only the Agg equivalent join, IN, and EXISTS sublinks can be pulled up. This hint is usually used together with the hints described before.

HUAWEI

Common Plan Hints

Hint Type	Syntax	Description
Join order	leading(join_table_list)	**join_table_list** specifies the tables to be joined. The values can be table names or table aliases. If a subquery is pulled up, the value can also be the subquery alias. Separate the values with spaces. You can add parentheses to specify the join priorities of tables. For example, (1) **leading(t1)** indicates that **t1** is a driving table. (2) **leading((t1 (t2 t3) t4 t5))** **leading((t2 t3))**: First, **t2** and **t3** are joined and **t3** is used as the inner table. Then, the result is joined with **t1**, and **(t2 t3)** is used as the inner table. Finally, the result is joined with **t4** and then **t5**, and the table on the right in each join is used as the inner table.
Join operation	[no] nestloop|hashjoin|mergejoin(table_list)	These hints specify the join method, which can be nested loop join, hash join, or merge join.
Rows	rows(table_list #|+|-|* const)	These hints specify the number of rows in an intermediate result set. Both absolute values and relative values are supported.
Access path	[no]tablescan|indexscan|indexonlyscan(table [index])	These hints specify a scan operation, which can be tablescan, indexscan, or indexonlyscan.

HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. Optimizing the Overall Database Performance
3. **SQL Optimization**
   - Similar Operator Selection and Plan Hints
   - Indexes
   - Optimizing Table Structures
   - CBO
   - Diagnosing SQL Performance
   - Optimizing Subqueries and Rewriting SQL Statements

HUAWEI

Index Optimization Principles

1. Implementation

- For efficient queries, the implementation structure of an index in openGauss is usually a B+ tree. The data in some common query columns of a table is used to build a tree structure, and each query's performance is optimized by trading space off for time.

2. Query process

- (1) The index is scanned first, the location (pointer) of the record that meets the requirements is found in the index, and then the specific page in the table is located to obtain the corresponding tuple. That is, the index is used first, and then the table data is read.
- (2) As the number of queried data records increases, so does the number of times that the index tree is accessed. This deteriorates the performance of an index scan compared to a full table scan because the index scan needs to scan the table data and the index tree.

3. Suggestions for creating indexes

- Create an index on the WHERE condition and **ORDER BY** column with high selectivity. In openGauss, if the selectivity is lower than 30%, that is, 70% or more data is returned as a result set, the optimizer may scan the entire table instead of the index.

HUAWEI

Index Invalidation

- What is index invalidation?
  - Index invalidation indicates that, in a specific SQL syntax or scenario, a full table scan with poorer performance is selected instead of a better index scan. The impact of index invalidation is more obvious for large tables.
- Which syntaxes will cause an index to become invalid?
  - Among single-table query operations in openGauss, the following may cause an index to become invalid:
    - (1) Using functions on the index column
    - (2) Performing calculations on the index column
    - (3) Implicit type conversion of the index column
    - (4) OR statement in the WHERE condition
    - (5) Using fuzzy match conditions on the index column

HUAWEI

Index Invalidation — Example (1)

- Create table **t1** and indexes for the first column **c1** and the third column **c3**.

```
create table t1(c1 int primary key, c2 int, c3 text unique);
```

- The indexes become invalid in the following situations:
  1. select * from t1 where c1 = nvl(c2, 0);
     - A function is used on the index column: Since the condition following the index column **c1** is a function, the index scan cannot be selected. Instead, the full table scan is selected.
     - Optimization method: Avoid using functions on index columns.
  2. select * from t1 where c1 + 1 = 10;
     - Calculation is performed on the index column: Although **c1** is an index column, an index scan cannot be performed due to a calculation on the index column.
     - Optimization method: Do not perform calculations on index columns.
  3. select * from t1 where c1 = 6::numeric; select * from t1 where c3 = 6;
     - Implicit type conversion of the index column. **c1** on the left of the first equal sign (=) is implicitly converted. That is, **c1** is converted to numeric type and then compared with the value on the right of the equal sign (=). Therefore, the index cannot be used, and the statement scans the entire table. The same applies to the two records. The coordinate of the equal sign is text, and the coordinate on the right is int. In this case, implicit conversion occurs. The text in the **c3** column is converted into int, and then compared with 6 on the right of the equal sign.
     - Optimization method: Convert the type on the right of the equal sign to the type on the left of the equal sign: **select * from t1 where c1 = 6; select * from t1 where c3 = 6::text;**
     - Index invalidation may be caused by implicit type conversion in equality or other conditions (such as <, >, <>, and like).

HUAWEI

Index Invalidation — Example (2)

- Indexes become invalid in the following situations:
  4. select * from t1 where c1 = 10 or c2 = 20;
     - OR statement in the WHERE condition: In the WHERE clause, if the condition columns connected by OR are an index column and a non-index column, the index becomes invalid. If indexes used before and after the OR condition are composite indexes and they contain two columns, the indexes become invalid.
  5. select * from t1 where c3 like '%test%';
     - Fuzzy match conditions are used for an index column. If like %xx, like xx %, or like %xx % is used for an index column, the index becomes invalid.

HUAWEI

Index Invalidation — Other Scenarios

- A global index is created for a partitioned table. After the partition is deleted, the index becomes invalid.
  - Generally, if the keyword **LOCAL** is not specified when an index is created for a partitioned table, the created index is a global index. For a partitioned table, if a global index is created and a partition is deleted from the partitioned table, the index becomes **UNUSABLE**. If you do not rebuild the index, the index will remain invalid.
  - Optimization method:
    1. Use local instead of global indexes when creating partition key indexes to prevent index invalidation caused by partition deletion.
    2. After a table is created, do not change the table structure to avoid index invalidation. You can also add the UPDATE GLOBAL INDEX statement when deleting partitions.

       ```
       alter table index_part drop partition p1 update global index;
       ```

    3. In this case, you can run the rebuild command to rebuild the index.

       ```
       reindex index idx_part;
       ```


HUAWEI

Introduction to TABLE ACCESS BY INDEX ROWID for Index-Only Scans

- Index-only scans
  - The index-only scan in openGauss is an optimization technology that responds to queries using only indexes without accessing the main table data. This operation can significantly improve query performance and reduce the I/O overhead.
- TABLE ACCESS BY INDEX ROWID for index-only scans
  - In some cases, index-only scans still require TABLE ACCESS BY INDEX ROWID. TABLE ACCESS BY INDEX ROWID indicates that after a matched row is found in the index, all row data needs to be obtained through the main table's physical address. This is because the query cannot be completed accurately by only relying on the information from the index tree, and all row data needs to be obtained to determine whether the data is valid.
  - Note that TABLE ACCESS BY INDEX ROWID may require extra I/O overhead. Therefore, during optimization, you need to consider the overhead of index-only scans where TABLE ACCESS BY INDEX ROWID is used to obtain the optimal query performance.

HUAWEI

TABLE ACCESS BY INDEX ROWID for Index-Only Scans — Use Cases

- Why do index-only scans need TABLE ACCESS BY INDEX ROWID?
  1. If the index column contains **NULL** values, TABLE ACCESS BY INDEX ROWID verifies the existence of **NULL** values.
  2. If the index column contains variable-length data types (such as VARCHAR), TABLE ACCESS BY INDEX ROWID obtains complete variable-length data.
  3. In Astore, if the index data is modified, index-only scans first search for the visibility map (VM) to determine the visibility of the data in the index column. If the visibility cannot be determined, the transaction information of the entire row is accessed by TABLE ACCESS BY INDEX ROWID to determine the row's visibility.

HUAWEI

Overusing TABLE ACCESS BY INDEX ROWID During Index-Only Scans

- How do I determine whether TABLE ACCESS BY INDEX ROWID is performed too many times during index-only scans?
  - Use EXPLAIN ANALYZE to print the execution plan. Check the **Heap Fetched** field of the index-only scan
    - o

```
openGauss=# explain analyze select c1 from index_lookup where c1 < 10000;
                                                        QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------
-----
Index Only Scan using idx_lookup on index_lookup  (cost=0.00..262.12 rows=9920 width=4) (actual
time=0.023..3.230 rows=9999 loops=1)
  Index Cond: (c1 < 10000)
  Heap Fetches: 9999
Total runtime: 4.150 ms (4 rows)
```

s.

      - The information in red indicates the number of times that TABLE ACCESS BY INDEX ROWID is performed. You can see that it is performed in each query, which means the performance depends on the disk I/O and fluctuates greatly.

HUAWEI

Preventing the Overuse of TABLE ACCESS BY INDEX ROWID During Index-Only Scans

- How do I prevent the overuse of TABLE ACCESS BY INDEX ROWID for index-only scans?
  1. This problem is usually caused by using Astore to store tables. If Ustore is used, the problem can be avoided.
  2. When Astore is used, many update, deletion, and insertion operations change row visibility. Accessing the VM alone cannot solve the visibility problem. The transaction information of the actual row in the table needs to be read to determine whether the row is visible. In this case, you can use VACUUM ANALYZE *<my_table_name>* to update the VM information for the table and make all rows visible to the current transaction.
  3. If the overhead of VACUUM ANALYZE is too high, you can use plan hints instead of index-only scans to avoid the overhead caused by VM access and TABLE ACCESS BY INDEX ROWID.

     ```
     select/*+ no indexonlyscan(index_lookup idx_lookup)*/ c1 from index_lookup where c1 < 10000;
     ```


HUAWEI

Index Bloat (1)

- Why does index bloat occur?
  - Index bloat refers to a B-tree index structure for openGauss.
  - In a B-tree index structure, each version's row data in the table corresponds to a live index entry that is visible to all rows. When **VACUUM** is executed, the index entries corresponding to dead tuples in the B-tree index are cleared, but the space occupied by the index is not recycled. Instead, the index space is set as reusable. New index data that meets the conditions can overwrite the index space that can be reused.
  - As more of such operations are performed, no more new index data that meets the conditions can overwrite the reusable index space. As a result, the index gradually occupies more space, resulting in an index bloat. This phenomenon is common when Astore is used.

HUAWEI

Index Bloat (2)

- What performance problems will index bloat cause?
  - Index bloat is inevitable to some extent, so some index bloat is normal. However, excessive index bloat may cause the following performance problems:
    1. More pages need to be scanned during range scans using indexes.
    2. The openGauss database uses the memory space of shared buffers to cache index pages. Index bloat increases memory usage, causing memory waste.
    3. Fewer index entries are stored on each index page. This adds more layers to the index tree, reducing the efficiency of index queries.

HUAWEI

Index Bloat (3)

- How do I check the size of the index space?
  - Run the meta-command **\di+** to check the size of the index space.
    - \di+ my_index
- How do I solve the index bloat problem?
  - (1) Rebuild indexes.
    - reindex index my_index
  - (2) Use VACUUM FULL.
    - vacuum full my_table
  - (3) In a composite index, if too many combined index columns are created for a large table, the index space bloats, deteriorating the index performance. You can delete the unnecessary composite index columns to reduce index space and ensure index scan efficiency.

HUAWEI

Too Many Dead Tuples During Index Scan

- 1. What is a dead tuple?
  - When Astore is used in openGauss, a DELETE operation does not remove the corresponding row data from the storage space. Instead, the data is marked as deleted in the **xmax** field in the header of the row data.
  - Also, the UPDATE operation in Astore is split into two operations: INSERT and then DELETE. The tuples marked as deleted also become dead tuples.
- 2. How do dead tuples impact index scans?
  - The tablespace bloats when there are too many dead tuples. However, ordinary index scans require TABLE ACCESS BY INDEX ROWID to view data. With more dead tuples, TABLE ACCESS BY INDEX ROWID is performed more times during index scans, affecting the index scan performance.
- 3. How do I solve the problems caused by too many dead tuples?
  - You can use VACUUM to recycle dead tuples.
  - You are advised to use VACUUM ANALYZE to recycle dead tuples and update optimizer statistics to obtain a better execution plan.

HUAWEI

Improper Index Selection (1)

- Why are some indexes selected improperly?
  - In openGauss, the bottom-layer scan operator is selected based on cost estimation. In addition, data in the table is evenly distributed by default.
  - When data is not evenly distributed, for example, due to improper index selection, the openGauss cost estimation will not be accurate. As a result, the wrong bottom-layer scan operator is selected.
- Improper index selection scenarios:
  - (1) The cost estimation is inaccurate because multiple indexes are created on a table.
  - (2) Multiple indexes join two tables, so the costs are not accurately estimated during the join process.

HUAWEI

Improper Index Selection (2)

- How do I address improper index selection?
  - Normally, this problem is difficult to detect and locate. It is easily mistaken as low index efficiency.
  - Generally, to locate an index that is not properly selected, you need to know which index should be selected. If using the same query on the same data in environment A is more efficient than in environment B, you need to compare the two execution plans to check for an improperly selected index.
  - To solve the problem, you can only manually specify the index through hint optimization.

HUAWEI

Columns That Require Index Creation

- An index is a data structure that helps quickly search for data in a database. As such, it usually performs better when searching for a small amount of data and is less efficient than a full table scan when searching for a large amount of data. In addition, the index incurs certain storage space overhead. So, you need to specify whether an index or full table scan is more suitable for a column. Generally, indexes work well for the following types of columns:
  - (1) Columns with high selectivity (usually over 70%) in the WHERE condition
  - (2) Join columns with a small join result set
  - (3) Sorting columns with a large sorting result set
  - (4) Grouping columns with a large grouping result set
  - (5) Aggregation columns with a large aggregation result
- For query statements with many WHERE conditions, you can create a composite index for the three columns with the highest selectivity. A composite index should not contain too many columns because it will bloat rapidly when many insertion, update, and deletion operations are performed. This increases the index space and lowers efficiency.
- Creating proper indexes based on the preceding rules can effectively improve the query efficiency of the openGauss database.

HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. Optimizing the Overall Database Performance
3. **SQL Optimization**
   - Similar Operator Selection and Plan Hints
   - Indexes
   - Optimizing Table Structures
   - CBO
   - Diagnosing SQL Performance
   - Optimizing Subqueries and Rewriting SQL Statements

HUAWEI

Principles of Optimizing Table Structures

- Table structure optimization is a crucial SQL process that involves the following:
  1. **Selecting data types**: Select the most suitable data type for storage.
     - For example, if a column stores only values ranging from 0 to 255, using tinyint saves more storage space than using int.
  2. **Using suitable indexes**: Indexes can significantly speed up queries.
     - However, too many indexes slow down insertion, update, and deletion operations because indexes need to be updated each time data changes. Select the appropriate indexes based on your service requirements.
  3. **Normalization and denormalization**: Normalization eliminates data redundancy but may increase query complexity.
     - Denormalization can reduce query complexity but may introduce data redundancy. Decide the one that works best for your service requirements.
  4. **Table partitioning and splitting**: When a table contains a lot of data, you can use table partitioning or splitting to speed up queries.
     - Partitioning is to physically divide a table into multiple parts, each of which can be independently indexed and queried. Splitting is to divide a large table into multiple small tables. Each small table has its own index.
  5. **Avoiding foreign keys**: Foreign keys ensure data consistency and integrity, but foreign key constraints affect performance.

HUAWEI

Selecting Storage Modes

- Optimizing ordinary tables
  - To optimize ordinary tables (non-partitioned tables), you can select suitable data types (use proper storage modes) or indexes, or use normalization or denormalization.
- Selecting storage modes
  - During database design, there are several key factors in table design that will have a significant impact on the database's query performance and data storage. A good table design reduces I/O operations and minimizes memory usage, improving the query performance.
  - Selecting a storage mode is the first step in defining a table. Customer service attributes determine the table's storage mode. Use the following table to select the best option for your use case.

Storage Mode	Scenario
Row-store	Point queries (simple index-based queries that only return a few records) Scenarios requiring frequent addition, deletion, and modification operations
Column-store	Statistics analysis query, in which operations, such as group and join, are performed many times.

Selecting Data Types

1. **Select data types that facilitate data calculation.**
   - Generally, calculating integers (including common comparison calculations, such as =, >, <, ≥, ≤, and ≠ and GROUP BY) is more efficient than strings and floating-point numbers.
2. **Select data types with a short length.**
   - Data types with a short length reduce both the data file size and the memory used for computing, improving the I/O and computing performance.
3. **Use the same data type for a join.**
   - You are advised to use the same data type for a join.
4. Do not use numeric data columns in a table if high precision is not required. Calculating numeric data in openGauss is much less efficient than float or double precision.

HUAWEI

Using Partitioned Tables

- Sometimes, a table is too large to be stored in the memory, or only the last part of the table contains hotspot data, while the rest is historical data. In this case, partitioned tables can be used.
- 1. What is a partitioned table?
  - A partitioned table is divided into smaller easy-to-manage segments based on certain rules. Though partitioning is a logical table concept, it often results in physical data separation.
- 2. What are the advantages of partitioned tables?
  - (1) High query performance: The system queries only the concerned partitions rather than the whole table, improving the query efficiency.
  - (2) High availability: If a partition is faulty, data in the other partitions is still available.
  - (3) Easy maintenance: To fix a partitioned table, you only need to fix the partition that is faulty.

HUAWEI

Classifying Partitioned Tables

- openGauss supports level-1 and level-2 partitioned tables. Level-1 partitioned tables include range, interval, list, and hash partitioned tables. Level-2 partitioned tables include nine combinations of any two — range, list, and hash partitioned tables.

  1. **Range partitioned table**: Data is mapped to each partition by range. The partition key used to identify a range is specified when a partitioned table is created. This is the most popular type of partitioning. The partition key is usually a date. For example, sales data is partitioned by month.
  2. **Interval partitioned table**: a special range partitioned table. Compared with a range partitioned table, an interval partitioned table adds interval value definitions. Partitions can be automatically created based on the interval values.
  3. **List partitioned table**: Key values contained in the data are stored in different partitions, and the data is mapped to each partition in sequence. The key values contained in the partitions are specified when the partitioned table is created.
  4. **Hash partitioned table**: Data is mapped to each partition in sequence based on the internal hash algorithm.
  5. **Level-2 partitioned table**: This is a partitioned table obtained by randomly combining range, list, and hash partitioning. Both level-1 and level-2 partitions can be defined in the preceding three ways.

HUAWEI

Local Indexes and Global Indexes of Partitioned Tables (1)

1. Global index
   - A global index is an index for an entire partitioned table.
     - create index on my_part_tbl(c1) global;
2. Local index
   - A local index is an index for a single partition in a partitioned table.
     - create index on my_part_tbl(c1) local;
3. If a cross-partition query is involved, local indexes cannot be used. Use only global indexes.
4. In terms of performance, frequent insertion and update operations increase the maintenance overhead of global indexes.

HUAWEI

Using Partial Clustering Keys (PCKs)

- PCK is a column-store technology. It can minimize or maximize sparse indexes to quickly filter base tables. PCKs can be used to specify multiple columns. However, you should not specify more than two columns.
- Use the following principles to specify PCKs:
  - The specified PCKs must be restricted by simple expressions in base tables. Such constraints are usually represented by *col op const*, in which *col* indicates the column name, *op* indicates operators (including =, >, >=, <=, and <), and *const* indicates constants.
  - Select columns that are frequently selected (to filter much more undesired data) in simple expressions.
  - List the less frequently selected columns at the top.
  - List enumerated columns at the top.

HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. Optimizing the Overall Database Performance
3. **SQL Optimization**
   - Similar Operator Selection and Plan Hints
   - Indexes
   - Optimizing Table Structures
   - CBO
   - Diagnosing SQL Performance
   - Optimizing Subqueries and Rewriting SQL Statements

HUAWEI

Optimizers

- Objective of optimizers: Select the optimal plan from several alternative execution plans based on cost estimation, including rule-based query rewriting (logical optimization) and cost-based query optimization (physical optimization).
- The rule-based optimizer (RBO) is about logic optimization.
  - Relational algebra is used to implement equivalent transformation of query statements, and the query is simplified logically.
  - Traditionally, query rewriting is based on rules. However, cost-based query rewriting has emerged as SQL languages have become more complex, and there are more query rewriting rules.
- The cost-based optimizer (CBO) is about physical optimization.
  - For the simplest query output by logical optimization, the CBO lists various feasible execution plans, compares their costs, and provides the optimal plan.
  - Generally, as more tables are added, the number of possible plans increases exponentially. As such, it is unrealistic to list all possible plans.
  - The execution time may differ by thousands of times for different execution plans. So, physical optimization is critical to query performance.

HUAWEI

CBO

- openGauss generates execution plans based on cost estimation.
- The CBO estimates the execution cost based on data statistics, cardinality estimation, and operator cost models, and selects the plan with the lowest cost for SQL execution.
  - Data statistics include the number of rows in a table, column width, NULL record ratio, distinct values, and MCVs.
  - Cardinality estimation estimates the number of rows in the intermediate result of each operator, including the number of rows with filter criteria and in the join result. The two parts are obtained based on correct selectivity.

HUAWEI

CBO — Common Statistics Problems

- In openGauss, the CBO uses statistics as the source data to generate an execution plan.
- If no statistics are available or they are out of date, the execution plan may seriously deteriorate, leading to low performance.
- Common statistics problems are:
  - (1) The estimated value of **rows** is larger than the actual value.
    - This usually happens when a small amount of data is inserted into an empty table. In this case, statistics are incorrectly estimated, leading to a larger estimated value of **rows**. However, the execution plan cannot be optimized after the modification.
  - (2) The estimated value of **rows** is slightly different from the actual value.
    - This often occurs during centralized deployment because the statistics can be dynamically scanned. When the table data is not evenly stored, it may cause relative deviation. You can add hints to determine whether the problem is caused by inaccurate statistics.
  - (3) Plan hints affect the selection of an execution plan.
    - In some SQL statements, hints are added to correct statistics, changing the execution plan. In this case, incorrect operator selection may be mistaken as the cause of a statistics problem. You need to determine the real cause by checking if the data volume is related to the execution plan selection.
    - In this scenario, you can create different volumes of data and add hints to ensure statistical accuracy. This will help you determine whether the root cause is related to statistics.

HUAWEI

CBO — Identifying Statistics Problems

- To check whether an SQL statement contains statistics problems, perform the following steps:
  1. Run the **EXPLAIN ANALYZE** statement to check whether the estimated number of **rows** differs a lot from the actual number. If it does, the statistics may be incorrect.
  2. Change the execution statement to **SELECT COUNT(*) FROM** *xxx* to check whether the judgment in step 1 is correct.
  3. Query the corresponding system catalog to check whether statistics are collected or updated promptly. If not, manually update the statistics for optimization.
  4. If you find that the statistics are incorrect or efficiency does not improve after performing step 3, you can add hints to check whether the execution plan is optimized and the delay is reduced. If there is a significant improvement, incorrect statistics are the likely cause.

HUAWEI

CBO — Updating Statistics

- There are two basic methods for identifying and rectifying statistics problems: adding hints and manually updating statistics.
  - Generally, manually collecting statistics applies to tables that are frequently used. Adding several lines of SQL statements may improve the performance of the entire script. However, you need to consider whether performance will deteriorate after adding SQL statements. If the ANALYZE statement is added to a loop, it may cause a severe delay and deteriorate the overall performance.
  - Adding hints applies to complex SQL statements. It is a simple method with few side effects. However, hints may not work if there are many execution plans with abnormal statistics in scripts or if a lot of data is frequently added to or deleted from the base table.
- Update statistics:

1. Update the statistics about a table or the entire database.

```
ANALYZE tablename;
ANALYZE;
```

2. Perform statistics-related operations on multiple columns.

```
ANALYZE tablename ((column_1, column_2));
ALTER TABLE tablename ADD STATISTICS ((column_1, column_2));
ALTER TABLE tablename DELETE STATISTICS ((column_1, column_2));
```


HUAWEI

Optimizing CBO Parameters (1)

GUC Parameter	Description
var_eq_const_selectivity	Specifies whether the optimizer uses histograms to calculate the selectivity of integer constants.
partition_page_estimation	Specifies whether to optimize the page estimation of partitioned table pages based on the pruning result. Only partitioned table pages and local index pages are included. Global index pages are not included. Formula: Number of pages after estimation = Total number of pages in the partitioned table x (Number of partitions after pruning/Total number of partitions)
partition_iterator_elimination	Specifies whether to eliminate the partition iteration operator to improve execution efficiency when the partition pruning result of a partitioned table is a partition.
enable_functional_dependency	Specifies whether function dependency statistics are used. If this parameter is set to **on**, the statistics about multiple columns generated by ANALYZE contain function dependency statistics, which are used to calculate the selectivity. If it is set to **off**, the statistics about multiple columns generated by ANALYZE do not contain function dependency statistics, so they are not used to calculate the selectivity.

HUAWEI

Optimizing CBO Parameters (2)

GUC Parameter	Description
seq_page_cost	Specifies the optimizer's cost estimate for a sequential disk page fetch. Default value: **1**
random_page_cost	Specifies the optimizer's cost estimate for a non-sequential disk page fetch. Default value: **4**
cpu_tuple_cost	Specifies the optimizer's cost estimate for processing each row during a query. Default value: **0.01**
cpu_index_tuple_cost	Specifies the optimizer's cost estimate for processing each index entry during an index scan. Default value: **0.005**

HUAWEI

CBO Case Analysis (1)

- To score (estimate) feasible physical execution costs, select the optimal plan, and output it to the executor, pay attention to the following:
  - Number of rows in the result set
    - Indicates the estimated number of rows in the result set of the current layer when the optimizer performs estimation.
  - Executing cost estimation
    - It is represented by two numbers. The first number is the estimated cost of the first tuple, and the second number is the estimated total cost of all tuples. The total cost includes the cost of all sub-operators.
  - Join path finding and selection
    - Identifies the operator used by the executor to perform the calculation at the current layer.

```
openGauss=# explain select * from t1 join t2 on t1.a=t2.a;
                              QUERY PLAN
-----------------------------------------------------------------------
 Hash Join  (cost=270.00..552.50 rows=10000 width=16)
   Hash Cond: (t1.a = t2.a)
   ->  Seq Scan on t1  (cost=0.00..145.00 rows=10000 width=8)
   ->  Hash  (cost=145.00..145.00 rows=10000 width=8)
         ->  Seq Scan on t2  (cost=0.00..145.00 rows=10000 width=8)
(5 rows)
```


HUAWEI

CBO Case Analysis (2)

- Row estimation
  - Purpose: Estimate the size of the result set during the connection to prepare for cost estimation.
- Run the **analyze** command to collect base table statistics, including:
  - Size of each data table, including the number of rows and pages.
  - Statistics for each table column, including distinct values, MCVs, and histograms.
    - Distinct values, which record the number of different values in the column
    - Proportion of **NULL** values
    - MCVs, which record skew information
    - Histograms, which record data distribution

HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. Optimizing the Overall Database Performance
3. **SQL Optimization**
   - Similar Operator Selection and Plan Hints
   - Indexes
   - Optimizing Table Structures
   - CBO
   - Diagnosing SQL Performance
   - Optimizing Subqueries and Rewriting SQL Statements

HUAWEI

WDR

- WDR is a performance collection and analysis tool provided by openGauss. You can learn about the overall running status of the database from the report.
  - A performance report is generated by summarizing and collecting statistics based on WDR snapshot data tables.
  - By default, the initial user or monitoring administrator can generate reports.
- Through WDR snapshots, you can periodically obtain the system performance view and snapshots.
  1. This function is enabled using the GUC parameter **enable_wdr_snapshot**. By default, this function is disabled.
  2. Only the performance views whose schema is dbms_perf are collected.
  3. The default collection interval is 60 minutes, which can be adjusted by the GUC parameter **wdr_snapshot_interval**.
  4. The default retention period is one day, which can be adjusted by the **wdr_snapshot_retention_days**.
  5. You can call the system function create_wdr_snapshot to manually generate snapshots.

HUAWEI

Generating a WDR

- You can manually create a snapshot if there is only one snapshot in the database or if you want to view the database's current monitoring data. This command is only available to users with the SYSADMIN permission.
  1. Query the generated snapshot and obtain **snapshot_id**.
     - select * from snapshot.snapshot;
  2. (Optional) Manually create a snapshot.
     - select create_wdr_snapshot();
  3. Generate a performance report.
     - a. Generate a formatted performance report file.
       - \a \t \o <file_path>
     - b. Write the queried information to a performance report.
       - select generate_wdr_report(begin_snap_id bigint, end_snap_id bigint, report_type cstring, report_scope cstring, node_name cstring);

Parameters of the generate_wdr_report Function

- **begin_snap_id**: specifies the ID of a snapshot when a query starts, which is specified by **snapshot_id** in the **snapshot.snaoshot** table.
- **end_snap_id**: specifies the ID of a snapshot when a query ends. By default, the value of **end_snap_id** is greater than that of **begin_snap_id** (**snapshot_id** in the **snapshot.snaoshot** table).
- **report_type**: specifies the type of the generated report. For example, **summary** indicates summarized data, **detail** indicates detailed data, and **all** indicates summarized and detailed data.
- **report_scope**: specifies the range of the generated report. The value can be **cluster** or **node**.
- **cluster**: database-level information
- **node**: node-level information
- **node_name**: When **report_scope** is set to **node**, set this parameter to the name of the corresponding node. (To obtain a node name, run the **select * from pg_node_env** command. When **report_scope** is set to **cluster**, the parameter can be omitted, left blank, or set to **NULL**.)

HUAWEI

Viewing WDRs

- Summary:
  1. Most time-consuming wait events and event types
  2. CPU, I/O, and memory statistics
- Detail:
  1. Time model, SQL statistics, SQL details, and wait events
  2. Statistics on cache, I/Os, applications, and objects (tables, indexes, and bad blocks).
  3. System configuration

Introduction to Slow SQL Diagnosis

- Slow SQL diagnosis records information about all jobs whose execution time exceeds the threshold **log_min_duration_statement**.
- Slow SQL diagnosis provides table-based and function-based query APIs. You can query the execution plan, start time, end time, query statement, row activity, kernel time, CPU time, execution time, parsing time, compilation time, query rewriting time, plan generation time, network time, I/O time, network overhead, and lock overhead.
- Slow SQL diagnosis in openGauss features enhanced security. All information is anonymized.

HUAWEI

Querying Slow SQL Statements

1. Check the execution information about the SQL statements in the database instance.
   - select * from dbe_perf.get_global_full_sql_by_timestamp(start_timestamp, end_timestamp);
2. Check the execution information about the slow SQL statements in the database instance.
   - select * from dbe_perf.get_global_slow_sql_by_timestamp(start_timestamp, end_timestamp);
3. Check the execution information about the SQL statements on the current node.
   - select * from statement_history;

HUAWEI

Limitations of Slow SQL Diagnosis

1. The SQL tracing information is based on the normal execution logic. The tracing information may be inaccurate if SQL statements fail to be executed.
2. Restarting a node may cause data loss on the node.
3. If you exit a session immediately after SQL statements are executed, the session data that is not updated to the system catalog may be lost.
4. The number of SQL statements to be collected is specified by a GUC parameter. If the actual number of statements exceeds the threshold, new SQL statement execution information will not be collected.
5. The maximum number of bytes a single SQL statement can collect for lock event details is specified by a GUC parameter. If the number of bytes exceeds the threshold, new lock event details will not be collected.
6. The SQL statement information is updated asynchronously. So, after a query statement is executed, the related view function result is slightly delayed.
7. Certain metric information (such as row activities, cache I/O, and time distribution) depends on the dbe_perf.statement view. If the number of records in the view exceeds the preset size (depending on the GUC parameter **instr_unique_sql_count**), related metrics may not be collected.
8. Functions related to the statement_history table and the **details** column in the view are in binary format. To parse the detailed information, use the pg_catalog.statement_detail_decode(details, 'plaintext', true) function.

HUAWEI

Contents

1. Performance Optimization Guidelines and Process
2. Optimizing the Overall Database Performance
3. **SQL Optimization**
   - Similar Operator Selection and Plan Hints
   - Indexes
   - Optimizing Table Structures
   - CBO
   - Diagnosing SQL Performance
   - Optimizing Subqueries and Rewriting SQL Statements

HUAWEI

Introduction to Subqueries and Sublinks

- What are subqueries and sublinks?
  - When an application runs an SQL statement to operate the database, many subqueries are used because they are clearer than table join. Especially in complicated query statements, subqueries have more complete and independent semantics, which makes SQL statements clearer and easier to understand. As such, subqueries are very popular.
  - In openGauss, subqueries can also be called sublinks based on where they are in SQL statements.
    1. **Subquery**:
       - Corresponds to a range table (RangeTblEntry) in the query parse tree. That is, a subquery is a SELECT statement following immediately after the FROM keyword.
    2. **Sublink**:
       - Corresponds to an expression in the query parse tree. That is, a sublink is a statement in the WHERE or ON clause or in the target list.
    3. A subquery is essentially a range table, while a sublink is essentially an expression.

HUAWEI

Types of Subqueries (1)

- Generally, there are non-correlated and correlated subqueries.
  1. Non-correlated subqueries
     - Subquery execution is independent of the attributes of the outer query. In this way, a subquery can be executed before outer queries.
       - select t1.c1, t1.c2 from t1 where t1.c1 in (select c2 from t2 where t2.c2 in (2,3,4));
  2. Correlated subqueries
     - Subquery execution depends on some attributes of outer queries. In the following example, **t1.c1** in the **t2.c1 = t1.c1** condition is a correlated attribute. Such a subquery depends on outer queries and needs to be executed once for each outer query.
     - Correlated subqueries are peculiar because they are not complete in and of themselves; their closure contains some parameters provided by outer queries.
       - select t1.c1,t1.c2 from t1 where t1.c1 in (select c2 from t2 where t2.c1 = t1.c1 AND t2.c2 in (2,3,4));

HUAWEI

Types of Subqueries (2)

There are different types of subqueries based on the data they generate:

1. Scalar subquery
   - Outputs a result table with only one row and one column. This scalar value is its result. If the result is empty (0 lines), a **NULL** value is output. However, more than one row of results is not allowed, and a runtime exception occurs.
   - Scalar queries can appear in any place that contains scalars, such as SELECT and WHERE clauses.
   - Example 1: A scalar query in a WHERE clause: select t1.a from t1 where 100 < (select sum(b) from t2 where t2.c = t1.c);
   - Example 2: A scalar query in a SELECT clause: select t1.a ,(select t2.b from t2 where t1.a = t2.a) as b from t1;
2. Subquery for existence testing
   - EXISTS subquery, which returns a Boolean value. If it appears in WHERE clauses, this is the semi-join. Of course, it may appear anywhere you can put a Boolean value.
   - Example: select t1.a from t1 where t1.b = 8 and EXISTS (select * from t2 where t1.c = t2.c);

HUAWEI

Types of Subqueries (3)

3. Set comparison subquery
   - Refers to an IN, SOME, or ANY query. It returns a Boolean value, which may appear anywhere a Boolean value can be placed. The common form can be **x = SOME(Q)** (equivalent to **x IN Q**) or **X <> ALL(Q)** (equivalent to **x NOT IN Q**).
   - Example: select t1.a from t1 where t1.b <> all (select t2.b from t2);
   - The preceding example is a non-correlated subquery for set comparison.

HUAWEI

Optimizing Subqueries (1)

- Case 1: Rewrite an SQL statement to eliminate a correlated subquery.

```
SELECT    t1.name,
 (SELECT t2.name FROM t2 WHERE t2.id = t1.id) as department_name
FROM     t1;
```

  - To improve query performance, the subquery in the SQL statement above is deleted, and the statement is modified as follows:

```
select
    t1.name, t2.name as department_name
from t1 left  join  t2
on t1.id = t2.id;
```

  - In this case, we replace the subquery to be executed for each data record with a left outer join to reduce the number of query operations.
  - Summary: Using JOIN is usually more efficient than subqueries because the database engine can optimize JOIN operations better. The JOIN operation is usually performed at the beginning of a query. It first puts together and then filters the data of correlated tables. A subquery may be executed once in each step of a query. The difference is especially obvious if there are more correlated external query columns and a larger volume of data. However, it should be noted that JOIN is not always better. So, each optimization should be determined based on your needs and the query execution plan.

HUAWEI

Optimizing Subqueries (2)

- Case 2: Rewrite an SQL statement to optimize a subquery for existence testing.

```
SELECT    t1.name,  t1.addr
 FROM t1
WHERE EXISTS ( SELECT 1 FROM t2 WHERE t2.id = t1.id AND t2.location = 'abc');
```

  - To improve query performance, modify the SQL statement above as follows:

```
SELECT    t1.name,  t1.addr
 FROM t1
 INNER JOIN  T2
  ON  t1.id = t2.id
WHERE t2.location = 'abc';
```

  - Summary: In this case, the EXISTS subquery is replaced by INNER JOIN, improving the query efficiency.

HUAWEI

Optimizing Subqueries (3)

- Case 3: Rewrite an SQL statement to optimize a set comparison subquery.

```
SELECT    t1.name,  t1.addr
 FROM t1
WHERE t1.id IN ( SELECT t2.id FROM t2 WHERE t2.location = 'abc');
```

  - To improve query performance, modify the SQL statement above as follows:

```
SELECT    t1.name,  t1.addr
 FROM t1
 INNER JOIN  t2
  ON  t1.id = t2.id
WHERE t2.location = 'abc';
```

  - Summary: In this case, JOIN also makes execution more efficient because it avoids executing subqueries on each row of employee data.

HUAWEI

Types of Sublinks

- What are the different types of sublinks?
  - (1) exist_sublink: corresponds to the EXIST and NOT EXIST statements.
  - (2) any_sublink: corresponds to the *op* ALL(SELECT...) statement. *op* can be the IN, <, >, or = operator.
  - (3) all_sublink: corresponds to the *op* ALL(SELECT...) statement. *op* can be the IN, <, >, or = operator.
  - (4) rowcompare_sublink: corresponds to the RECORD *op* (SELECT...) statement.
  - (5) expr_sublink: corresponds to the (SELECT with single targetlist item...) statement.
  - (6) array_sublink: corresponds to the ARRAY (select...) statement.
  - (7) cte_sublink: corresponds to the with query (...) statement.

HUAWEI

Optimizing Sublinks (1)

- To optimize a sublink in openGauss, a subquery is pulled up to join with tables in outer queries, preventing the subquery from being converted into a plan involving subplans and broadcast.
- You can use the EXPLAIN statement to check whether a sublink is converted into such a plan.
- Sublink-release scenarios supported by openGauss:
  1. An IN-Sublink has no related condition: It cannot contain columns in the table queried at the upper layer or volatile functions.
  2. EXIST-Sublink contains related conditions: The WHERE clause must contain a column in the outer query. Other parts of the subquery cannot contain the column. Other restrictions include:
     - The subquery must contain the FROM clause.
     - The subquery cannot contain the WITH clause.
     - The subquery cannot contain aggregate functions.
     - The subquery cannot contain a SET, SORT, LIMIT, WindowAgg, or HAVING operation.
     - The subquery cannot contain volatile functions.

HUAWEI

Optimizing Sublinks (2)

3. An equality correlated query containing aggregate functions is pulled up.
   - The WHERE condition of the subquery must contain a column from the outer query. An equality comparison must be performed between this column and related columns in the tables of the subquery. These conditions must be connected using AND. Other parts of the subquery cannot contain the column from the outer query. Other restrictions are:
     - The expression in the WHERE condition of the subquery must use columns in the tables.
     - There must only be one output column after the SELECT keyword of the subquery. The output column must be an aggregate function (for example, MAX), and the parameter (for example, **t2.c2**) of the aggregate function cannot be columns of a table (for example, **t1**) in outer queries. The aggregate function cannot be COUNT.
     - The subquery must be a FROM clause.
     - The subquery cannot contain a GROUP BY, HAVING, or SET operation.
     - The subquery can only be an inner join.
     - The target list of the subquery cannot contain a function that returns a set.
     - The WHERE condition of the subquery must contain a column from the outer query. An equality comparison must be performed between this column and related columns in the tables of the subquery. These conditions must be connected using AND. Other parts of the subquery cannot contain the column from the outer query.

HUAWEI

Optimizing Sublinks (4)

4. Pulling up a sublink in the OR clause; take the following SQL statement as an example:

```
select a, c from t1 where t1.a = (select avg(a) from t3 where t1.b = t3.b) or exists (select * from t4 where t1.c = t4.c);
```

- If the WHERE condition contains an EXIST correlated sublink connected by OR, here is how to pull it up:

  (1) In the WHERE condition, the **opExpr** in the OR clause is **t1.a = (select avg(a) from t3 where t1.b = t3.b)**.

  (2) The **opExpr** contains a subquery. If the subquery can be pulled up, it is rewritten as **select avg(a), t3.b from t3 group by t3.b**, generating the NOT NULL condition **t3.b is not null**, which replaces the **opExpr**. In this case, the SQL statement changes to:

```
select a, c from t1 left join (select avg(a) avg, t3.b from t3 group by t3.b)  as t3 on (t1.a = avg and t1.b = t3.b)
where t3.b is not null or exists (select * from t4 where t1.c = t4.c);
```

  (3) Extract the EXISTS sublink **exists (select * from t4 where t1.c = t4.c)** from the OR clause to check whether the sublink can be pulled up. If so, it is converted into **select t4.c from t4 group by t4.c**, generating the NOT NULL condition **t4.c is not null**. In this case, the SQL statement changes to:

```
select t1.a, t1.c from t1 left join (select avg(a) avg, t3.b from t3 group by t3.b) as t3 on (t1.a = avg and t1.b = t3.b)
left join (select t5.c from t5 group by t5.c) as t5 on (t1.c = t5.c) where t3.b is not null or t5.c is not null;
```


HUAWEI

Optimizing Sublinks (5)

- Sublink-release scenarios not supported by openGauss:
  - No other sublinks can be pulled up except those described previously. In this case, a correlated subquery is planned as the combination of "subplan+broadcast". As a result, if inner tables have a lot of data, query performance may be poor.
  - During a correlated subquery, if the inner and outer tables are joined, the subquery cannot be pulled up. You need to rewrite the outer query into a statement with the WITH clause and then perform the correlated query. Example:

```
select distinct t1.a, t2.a
from t1 left join t2 on t1.a=t2.a and not exists (select a,b from test1 where test1.a=t1.a and test1.b=t2.a);
```

  - The modification is as follows:

```
with temp as
(
        select * from (select t1.a as a, t2.a as b from t1 left join t2 on t1.a=t2.a)
)
select distinct a,b
from temp
where not exists (select a,b from test1 where temp.a=test1.a and temp.b=test1.b);
```


HUAWEI

Optimization by Rewriting SQL Statements (1)

SQL statements can be optimized using certain rules based on the SQL execution mechanism and a lot of practice. The goal is to enable the database to execute SQL statements faster and obtain correct results. You can follow these rules to improve service query efficiency.

1. Replace UNION with UNION ALL.

   UNION eliminates duplicate rows while merging two result sets, but UNION ALL merges the two result sets without deduplication. If the service logic proves that the two sets do not overlap, you can use UNION ALL instead of UNION to improve performance.

2. Add the NOT NULL constraint to the join columns.

   If there are many **NULL** values in the join columns, you can add the filter criterion IS NOT NULL to filter data in advance to improve the join efficiency.

3. Replace NOT IN with NOT EXISTS.

   Nested Loop Anti Join must be used to implement NOT IN, and Hash Anti Join is required for NOT EXISTS. If no **NULL** value exists in the join columns, NOT IN is equivalent to NOT EXISTS.

4. Use HashAgg.

   If a plan involving groupAgg and SORT operations generated by the GROUP BY statement is not performing well, you can set **work_mem** to a larger value to generate a hashagg plan, which does not require sorting and improves performance.

HUAWEI

Optimization by Rewriting SQL Statements (2)

5. Replace functions with CASE statements.

   CASE statements perform better than functions. You can change a function that can be pushed down to a CASE expression as required.

6. Do not use functions or expressions for indexes.

   Using functions or expressions for indexes will stop indexing and enable full table scan.

7. Do not use operators !=, <, or > , NULL, OR, or implicit parameter conversion in WHERE clauses.

8. Split complex SQL statements.

   You can split an SQL statement and save the execution result to a temporary table if the SQL statement is too complex to be optimized using the solutions above, including but not limited to the following scenarios:

   - The same subquery is involved in multiple SQL statements of a job, and the subquery contains a lot of data.
   - An incorrect estimation of plan cost causes a small hash bucket in a subquery. For example, the actual number of rows is 10 million, but only 1000 rows are in the hash bucket.
   - Functions such as substr and to_number cause incorrect measures for subqueries containing a lot of data.

HUAWEI

Quiz

1. (Multiple-answer question) Which of the following are characteristics of the vectorized executor?
   A. More data is read in one batch at a time, saving I/O resources.
   B. In pipeline mode, functions are called frequently.
   C. There are many records in a batch and the CPU cache hit rate increases.
   D. Data is processed in batches, which is efficient.
2. (True or false) The SMP feature improves performance through operator parallelism and occupies more system resources, including CPU, memory, and I/O.
   A. True
   B. False

HUAWEI

Summary

- This chapter describes openGauss performance optimization in terms of the system and SQL statements, which correspond to traditional static and dynamic optimization, respectively. Many factors should be considered during performance optimization. Key OS configuration parameters need to be modified based on actual service requirements because the performance of complex query statements depends a lot on such parameters.

HUAWEI

Recommendations

- openGauss Community
  - https://opengauss.org/en/
- openGauss Help (5.0.0)
  - https://docs-opengauss.osinfra.cn/en/docs/5.0.0/docs/GettingStarted/GettingStarted.html

Thank you.

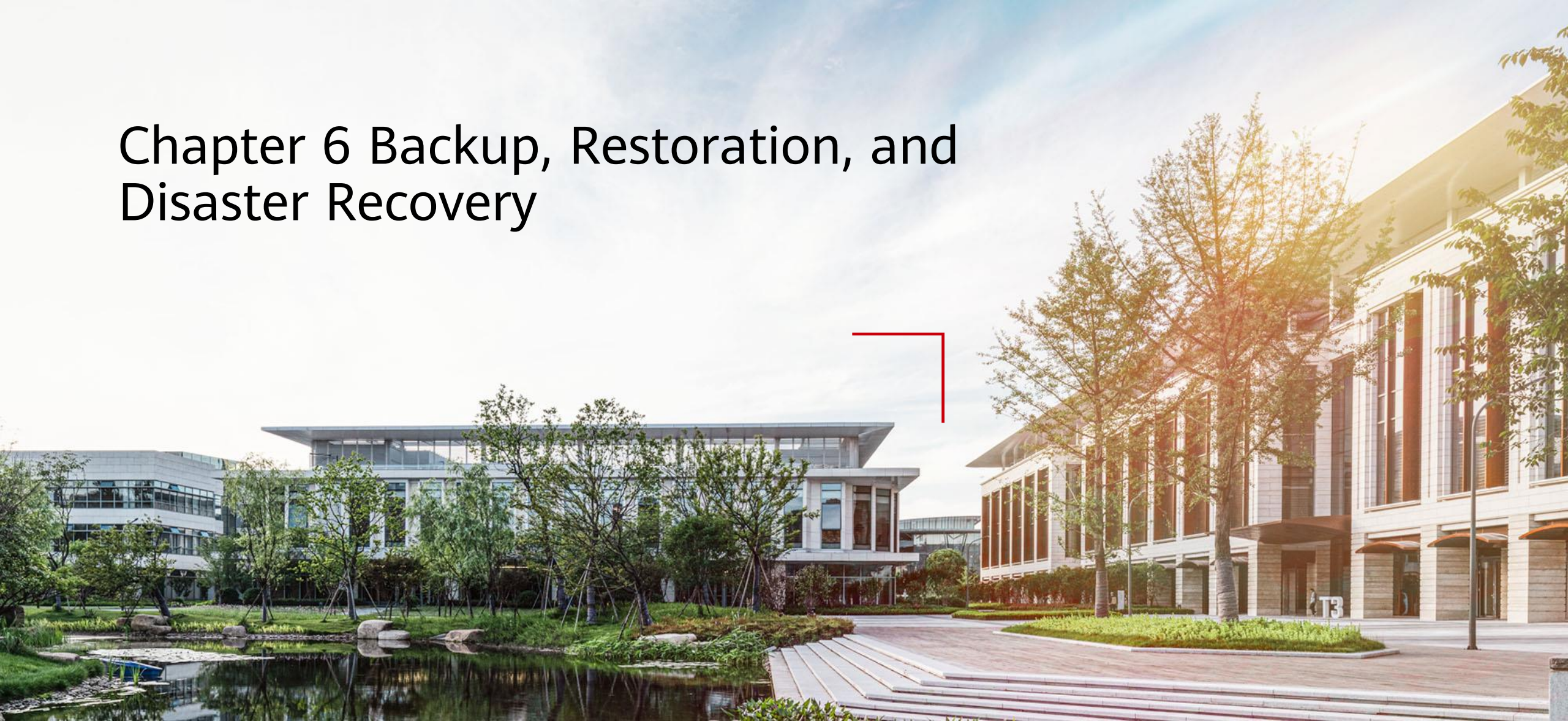
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 6 Backup, Restoration, and Disaster Recovery



HUAWEI

Foreword

- Data is a core enterprise asset in today's information-driven society. Data security and integrity directly impact operational efficiency, service continuity, and accuracy in strategic decision-making. However, data faces various risks in the face of increasingly complex information environments. As such, it is critical to develop scientific, efficient, and reliable mechanisms for database backup and restoration, which will ensure the security and continuity of enterprise data assets.
- This chapter describes the basic principles, key technologies, and best practices of database backup and restoration, helping you enhance your core knowledge in this field and data protection and emergency recovery capabilities.

HUAWEI

Objectives

- Upon completing this course, you will be able to:
  - Describe the backup and restoration feature.
  - List flashback instances.
  - Distinguish between logical and physical backup.
  - Describe two-city three-DC DR.

Contents

1. **Backup and Restoration Overview**
2. Logical Backup and Restoration
3. Physical Backup and Restoration
4. Flashback
5. Two-City Three-DC DR

HUAWEI

Backup and Restoration

- For database security, openGauss provides three types of backup and restoration, multiple backup and restoration solutions, and data reliability assurance mechanisms during backup and restoration.
- The three types of backup and restoration are logical backup and restoration, physical backup and restoration, and flashback.
- Backing up and restoring configuration files
  - Static configuration files are essential for openGauss to function properly because they define the openGauss topology structure and primary/standby relationship. The static configuration files generated by gs_om can replace damaged configuration files to ensure that the openGauss database runs normally.

HUAWEI

Backing Up and Restoring Configuration Files — Procedure

- Log in to the primary database node as the OS **omm** user.
- Generate configuration files in a specified directory on the current server.

```
gs_om -t generateconf -X /opt/software/openGauss/clusterconfig.xml --distribute
```

  - **/opt/software/ openGauss /clusterconfig.xml** is the XML configuration file used during openGauss installation.

- (Optional) Replace the damaged static configuration files in the **/opt/gaussdb/app/bin** directory of the three hosts.
  - Take one host as an example:

```
mv /opt/huawei/wisequery/script/static_config_files/cluster_static_config_SIA1000056771
/opt/gaussdb/app/bin/cluster_static_config
```


HUAWEI

Backing Up and Restoring Configuration Files — Example

- Run the following commands on any openGauss host to generate configuration files:

```
gs_om -t generateconf -X  /opt/software/openGauss/clusterconfig.xml --distribute
Generating static configuration files for all nodes.
Creating temp directory to store static configuration files.
Successfully created the temp directory.
Generating static configuration files.
Successfully generated static configuration files.
Static configuration files for all nodes are saved in
/opt/huawei/Bigdata/mppdb/wisequery/script/static_config_files.
Distributing static configuration files to all nodes.
Successfully distributed static configuration files.
```

- Open the generated configuration file directory that contains three new files.

```
cd /opt/huawei/Bigdata/mppdb/wisequery/script/static_config_files
ll
total 456
-rwxr-xr-x 1 omm dbgrp 155648 2016-07-13 15:51 cluster_static_config_plat1
-rwxr-xr-x 1 omm dbgrp 155648 2016-07-13 15:51 cluster_static_config_plat2
-rwxr-xr-x 1 omm dbgrp 155648 2016-07-13 15:51 cluster_static_config_plat3
```


HUAWEI

Contents

1. Backup and Restoration Overview
2. **Logical Backup and Restoration**
3. Physical Backup and Restoration
4. Flashback
5. Two-City Three-DC DR

HUAWEI

Logical Backup and Restoration

- Logical backup and restoration logically export data to back it up. This method can dump data that is backed up at a certain point in time , and restore it to that moment. It does not back up the data processed between the time a failure occurs and the last backup. As such, it works best when the data does not change often. If misoperations damage the data, it can be quickly restored from logical backup. To use logical backup, restore all the data in a database, rebuild the database, and import the backup data. This method is not recommended for databases requiring high data availability because it takes a long time to restore data. However, logical backup is a popular approach to migrating and transferring data because it can be performed on any platform.
  - gs_dump
  - gs_dumpall
  - gs_restore

HUAWEI

gs_dump

- openGauss provides the gs_dump tool to export database information. You can export a database or its objects (such as schemas, tables, and views), excluding objects in the recycle bin. The database can be the default **postgres** database or a user-defined database.
- gs_dump is executed by OS user **omm**.
- When gs_dump is used to export data, other users can still access (read and write) the openGauss database.
- gs_dump can export complete and consistent data. For example, if gs_dump is enabled to export database A at T1, the database data at T1 will be exported, but the modifications after T1 will not.
- The columns generated by executing gs_dump are not dumped.

HUAWEI

Precautions for gs_dump

- Do not modify the files and contents exported using the **-F c/d/t** formats. Otherwise, the restoration may fail. You can carefully modify the files exported using the **-F p** format if necessary.
- To ensure data consistency and integrity, gs_dump acquires a shared lock on the table to be dumped. If another transaction has acquired the shared lock on the table, gs_dump waits until this lock is released and then locks the table for dumping. The dump will fail if the table cannot be locked within the specified time. You can customize the timeout period to wait for the lock to be released by specifying the **--lock-wait-timeout** option.
- Stored procedures and functions cannot be exported in encrypted mode.
- Syntax:

```
gs_dump [OPTION]... [DBNAME]
```


HUAWEI

Parameters of gs_dump

Parameter	Description
-U	Username for database connection
-W	User password for database connection
-f	Folder to store exported files
-p	TCP port or the local UDS file extension on which the server is listening for connections
dbname	Name of the database to be exported
-F	Format of the file to be exported. **-F** values: **p** is plain text; **c** is custom archive; **d** is directory archive; **t** is tar archive.
-n	Names of schemas to be exported. The data of the specified schemas will also be exported. Single schema: Enter **-n** *schemaname*. Multiple schemas: Enter **-n** for each schema.
-t	Table (or view, sequence, foreign table) to be exported. You can use multiple **-t** options or wildcard characters to specify the tables. Single table: Enter **-t** *schema.table*. Multiple tables: Enter **-t** *schema.table* for each table.
-T	List of tables, views, sequences, or foreign tables that cannot be dumped. You can use multiple **-T** options or wildcard characters to specify the tables.

HUAWEI

gs_dump – Examples

- Example 1: Use gs_dump to export the full information from the **postgres** database. The exported **MPPDB_backup.sql** file is in plain-text format.

```
gs_dump -U omm -W Bigdata@123 -f backup/MPPDB_backup.sql -p 37300 postgres -F p
```

- Example 2: Use gs_dump to export the information from the **postgres** database, excluding the information from the table specified in the **/home/MPPDB_temp.sql** file. The exported **MPPDB_backup.sql** file is in plain-text format.

```
gs_dump -U omm -W Bigdata@123 -p 37300 postgres --exclude-table-
file=/home/MPPDB_temp.sql -f backup/MPPDB_backup.sql
```


HUAWEI

gs_dumpall

- openGauss provides the gs_dumpall tool to export all openGauss database information, including the data stored in the default **postgres** database, user-defined databases, and common global objects of all openGauss databases.
- gs_dumpall internally calls gs_dump. For details about the diagnosis information, see "gs_dump".
- Once gs_dumpall is restored, run **ANALYZE** on each database to get useful statistics from the optimizer.
- All tablespace directories must be empty before gs_dumpall restoration. If the databases are not in their default locations, database creation will fail.
- The context information, syntax, and parameters are the same as gs_dump.

HUAWEI

gs_dumpall — Precautions

- Do not modify any exported files. Otherwise, restoration may fail.
- To ensure data consistency and integrity, gs_dumpall acquires a shared lock on the table to be dumped. If another transaction has acquired the shared lock on the table, gs_dumpall waits until this lock is released and then locks the table for dumping. The dump will fail if the table cannot be locked within the specified time. You can customize the timeout period to wait for the lock to be released by specifying the **--lock-wait-timeout** option.
- When exporting, gs_dumpall reads all database tables. As such, you need to connect to the database as an openGauss administrator to export a complete file. When you use gsql to execute SQL scripts, administrator permissions are also required to add users and user groups and create databases.
  - Example: Use gs_dumpall to export all database information in one go.

```
gs_dumpall -f backup/bkp2.sql -p 37300
```


HUAWEI

gs_restore

- openGauss provides the gs_restore tool to import data or files that have been exported using gs_dump. It can also be used to import files exported by gs_dump.
- gs_restore is executed by OS user **omm**.
- gs_restore features:
  - Import data to a database.
    - If a database is specified, data is imported into the database. You must specify a password to connect to the database for parallel import.
  - Import data to a script.
    - If no database is specified, a script containing the SQL statement to rebuild the database is created and written to a file or standard output. This script output is equivalent to the plain-text output of gs_dump.
- Command syntax: gs_restore [OPTION]... FILE

HUAWEI

gs_restore - Parameters

Parameter	Description
-U	Username for database connection
-W	User password for database connection
-p	TCP port or the local UDS file extension on which the server is listening for connections
-d	Connects to the database specified by **dbname** and imports data directly into the database.
-e	Exits if an error occurs when you send the SQL statement to the database. The default setting is to ignore the tasks with errors and continue. A series of errors are displayed at the end of the import.
-c	Cleans (Deletes) existing database objects in the database to be imported before rebuilding them.
-s	Imports only object definitions in schemas and does not import data. Sequence values will not be imported either.

HUAWEI

gs_restore Examples

- Example 1: Run **gs_restore** to import all object definitions from the exported file of the whole **postgres** database to the **backupdb** database.

```
gs_restore -U jack /home/omm/backup/MPPDB_backup.tar -p 8000 -d backupdb -s -e -c
```

- Example 2: Run **gs_restore** to import data and all object definitions of the **postgres** database from the **MPPDB_backup.dmp** file (custom format).

```
gs_restore backup/MPPDB_backup.dmp -p 8000 -d backupdb
```

- Example 3: Run **gs_restore** to import all object definitions of the **postgres** database to the **backupdb** database. Before the import, complete definitions and data exist in the **postgres** database. After the import, all object definitions are in the **backupdb** database and no data exists in the table.

```
gs_restore /home/omm/backup/MPPDB_backup.tar -p 8000 -d backupdb -s -e -c
```


HUAWEI

Contents

1. Backup and Restoration Overview
2. Logical Backup and Restoration
3. **Physical Backup and Restoration**
4. Flashback
5. Two-City Three-DC DR

HUAWEI

Physical Backup and Restoration

- Physical backup and restoration copies physical files from the primary node to the standby node in the unit of data block units. A database can be restored using backup files, such as data and archive log files. A physical backup is useful when you need to quickly back up and restore the entire database and is not costly when properly planned.
  - gs_backup
  - gs_basebackup
  - Restoring data from backup files
  - Point-in-time recovery (PITR)
  - gs_probackup

HUAWEI

gs_backup

- Problems and exceptions may occur when the openGauss database is running. The database provides gs_backup for backup, to restore important data, and to display help and version information.
- Prerequisites:
  - The openGauss database can be connected.
  - During the restoration, backup files exist in the backup directory on each node.
  - gs_backup is executed by OS user **omm**.

HUAWEI

gs_backup — Syntax Examples

- Back up the primary database node.

```
gs_backup -t backup --backup-dir=BACKUPDIR [-h HOSTNAME] [--parameter] [--binary] [--all] [-l LOGFILE]
```

- Restore the primary database node.

```
gs_backup -t restore --backup-dir=BACKUPDIR [-h HOSTNAME] [--parameter] [--binary] [--all] [-l LOGFILE] [--force]
```

- Example 1: Use the gs_backup script to back up the primary database node.

```
gs_backup -t backup --backup-dir=/opt/software/gaussdb/backup_dir -h plat1 --parameter
```

- Example 2: Use the gs_backup script to restore the primary database node.

```
gs_backup -t restore --backup-dir=/opt/software/gaussdb/backup_dir -h plat1 --parameter
```


HUAWEI

gs_basebackup

- Problems and exceptions may occur when the openGauss database is running. The database provides gs_basebackup for basic physical backup, where the database's binary files on the server are copied using a replication protocol. You need to use the system administrator account to remotely use gs_basebackup. gs_basebackup supports hot backup and compressed backup.
- Syntax:

```
gs_basebackup -? | --help
```

- Example 1:

```
gs_basebackup -D /home/test/trunk/install/data/backup -h 127.0.0.1 -p 21233
```


HUAWEI

gs_basebackup — Prerequisites

- The openGauss database can be connected.
- User permissions are not revoked during the backup.
- Replication connection is allowed in the **pg_hba.conf** file, and the connection is established by a system administrator.
- If the Xlog transmission mode is **stream**, the value of **max_wal_senders** must be configured to at least **1**.
- If the Xlog transmission mode is **fetch**, the **wal_keep_segments** parameter must be set to a large value to prevent logs from being removed before the backup ends.
- During the restoration, backup files exist in the backup directory on all the nodes. If backup files are lost on any node, copy them from another node.

HUAWEI

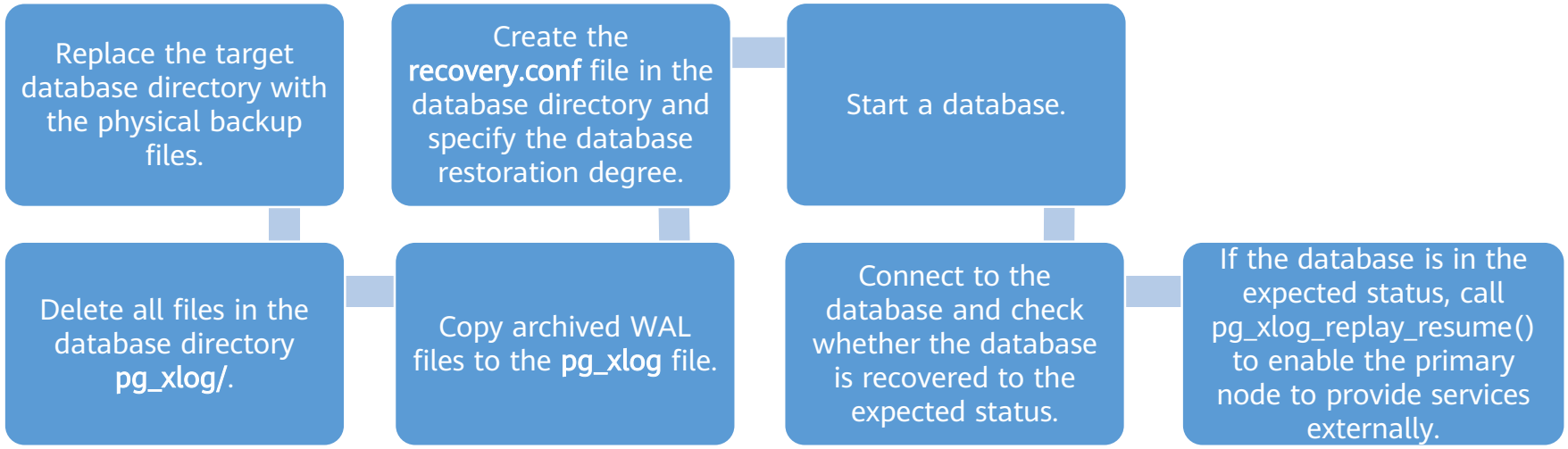
Restoring Data from Backup Files

- When the database is faulty, you need to restore data from backup files. gs_basebackup backs up the database in binary mode. As such, you can directly copy and replace the original files or start the database from the backup.
  - If the current database instance is running, a port conflict may occur when you start the database from the backup file. In this case, you need to modify the **port** parameter in the configuration file or specify a port when starting the database.
  - If the current backup file is generated for a primary/standby database, you may need to modify the replication connections between the primary and standby nodes, such as **replconninfo1** and **replconninfo2** in the **postgre.conf** file.
  - If the **data_directory** parameter in the **postgresql.conf** configuration file is enabled and configured, and the backup directory is used to start the database, the database will fail to start because the value of **data_directory** is different from the backup directory. You can change the value of **data_directory** to a new data directory or comment out this parameter.

HUAWEI

PITR

- When a database breaks down or needs to be rolled back to a previous state, the PITR feature of openGauss can be used to restore the database to any point in time after the backup and generate archive data.
- The prerequisite is to back up full data files and archive WAL files.
- PITR Process



Configuring the recovery.conf File (1)

Configure archive recovery:

- restore_command = string
  - Example:
```
restore_command = 'cp /mnt/server/archivedir/%f %p'
```
- archive_cleanup_command = string
  - Example:
```
archive_cleanup_command = 'pg_archivecleanup /mnt/server/archivedir %r'
```
- recovery_end_command = string
  - This parameter is optional and is used to declare a **shell** command that is executed only when the recovery is completed. **recovery_end_command** provides a cleanup mechanism for future replication and recovery.

HUAWEI

Configuring the recovery.conf File (2)

Set recovery targets.

- recovery_target_name = string
  - Example: `recovery_target_name = 'restore_point_1'`
- recovery_target_time = timestamp
  - Example: `recovery_target_time = '2020-01-01 12:00:00'`
- recovery_target_xid = string
  - Example: `recovery_target_xid = '3000'`
- recovery_target_lsn = string
  - Example: `recovery_target_lsn = '0/0FFFFFF'`

HUAWEI

gs_probackup

- The gs_probackup tool is used to manage openGauss database backup and restoration. It periodically backs up openGauss instances so that the database server can be restored if there is a database fault.
  - Physical backup for databases deployed in standalone or primary/standby mode
  - Backing up contents in external directories, including script files, configuration files, log files, and dump files
  - Incremental backup, periodic backup, and remote backup
  - Settings on the backup retention policy

HUAWEI

gs_probackup — Syntax Examples

- Print the gs_probackup version.

```
gs_probackup -V|--version gs_probackup version
```

- Create a backup for a specified database instance.

```
gs_probackup backup -B backup-path --instance=instance_name -b backup-mode
```

- Restore a specified instance from the backup copy in the **backup-path** directory. If you specify an instance you wish to restore, gs_probackup will look for its latest backup and restore it to the specified recovery target. Otherwise, the latest backup of any instance is used.

```
gs_probackup restore -B backup-path --instance=instance_name
```

- Delete a specified backup or delete backups that do not meet the current retention policy.

```
gs_probackup delete -B backup-path --instance=instance_name
```


HUAWEI

Contents

1. Backup and Restoration Overview
2. Logical Backup and Restoration
3. Physical Backup and Restoration
4. **Flashback**
5. Two-City Three-DC DR

HUAWEI

Flashback

- Flashback query
  - The flashback query uses the MVCC mechanism, allowing you to retrieve and query data from an earlier version of the database.
- Flashback table
  - Using the MVCC mechanism, the flashback table feature removes incremental data added after a specified point in time and restores the data that was deleted at or before that moment at the table level.
- Flashback DROP/TRUNCATE
  - Flashback DROP/TRUNCATE is based on the recycle bin mechanism. You can restore physical table files from the recycle bin to restore the dropped/truncated tables.

HUAWEI

Flashback

- Flashback is part of a database recovery technology. You can use it to selectively cancel the impact of a committed transaction and restore data if incorrect manual operations are performed. Before flashback, committed database modifications could only be restored using backup and restoration or PITR, which would take several minutes or even hours. Using flashback, you can restore committed data to its previous state in just seconds, regardless of the database size.
  - Flashback query
    - With flashback query, you can query a snapshot of a table at a certain point in time. You can use this feature to view and logically rebuild data that was erroneously deleted or modified. The flashback query uses the MVCC mechanism, allowing you to retrieve and query data from an earlier version of the database.

HUAWEI

Flashback Query — Syntax Example

- Syntax

```
{[ ONLY ] table_name [ * ] [ partition_clause ] [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
[ TABLESAMPLE sampling_method ( argument [, ...] ) [ REPEATABLE ( seed ) ] ]
[TIMECAPSULE { TIMESTAMP | CSN } expression ]
|( select ) [ AS ] alias [ ( column_alias [, ...] ) ]
|with_query_name [ [ AS ] alias [ ( column_alias [, ...] ) ] ]
|function_name ( [ argument [, ...] ] ) [ AS ] alias [ ( column_alias [, ...] | column_definition [, ...] ) ]
|function_name ( [ argument [, ...] ] ) AS ( column_definition [, ...] )
|from_item [ NATURAL ] join_type from_item [ ON join_condition | USING ( join_column [, ...] ) ]}
```

- Example

```
SELECT * FROM t1 TIMECAPSULE TIMESTAMP to_timestamp ('2020-02-11 10:13:22.724718', 'YYYY-MM-
DD HH24:MI:SS.FF');
SELECT * FROM t1 TIMECAPSULE CSN 9617;
```


HUAWEI

Flashback Table — Syntax Example

- Flashback table restores a table to a specific point in time. This feature can help you quickly restore table data when just one or several tables are logically damaged, but the rest of the database is intact. Using the MVCC mechanism, flashback table removes incremental data added after a specified point in time and restores the data that was deleted at or before that moment at the table level.
- Syntax: TIMECAPSULE TABLE table_name TO { TIMESTAMP | CSN } expression

```
TIMECAPSULE TABLE t1 TO TIMESTAMP to_timestamp ('2020-02-11 10:13:22.724718', 'YYYY-MM-DD HH24:MI:SS.FF');
TIMECAPSULE TABLE t1 TO CSN 9617;
```


HUAWEI

Flashback DROP/TRUNCATE — Syntax Examples

- Flashback DROP restores tables that are dropped by mistake and their auxiliary structures, such as indexes and table constraints, from the recycle bin.
- Flashback TRUNCATE restores tables that are truncated by mistake by restoring the physical data of the truncated tables and indexes from the recycle bin.
- Flashback DROP/TRUNCATE is based on the recycle bin mechanism. You can restore physical table files from the recycle bin to restore the dropped/truncated tables.
- Flash back a dropped table.

```
TIMECAPSULE TABLE { table_name } TO BEFORE DROP [RENAME TO new_tablename]
```

- Flash back a truncated table.

```
TIMECAPSULE TABLE { table_name } TO BEFORE TRUNCATE
```


HUAWEI

Contents

1. Backup and Restoration Overview
2. Logical Backup and Restoration
3. Physical Backup and Restoration
4. Flashback
5. **Two-City Three-DC DR**

HUAWEI

Two-City Three-DC DR

- To implement cross-region DR, you need to deploy one primary database instance and one standby database instance. They are usually deployed in two cities far away from each other. You can optionally use storage media for full and incremental data synchronization between database instances. If a regional fault occurs in the primary (production) database instance and data cannot be restored, you can promote the standby database instance to primary so it can take over services.
- openGauss provides a remote DR solution based on streaming replication.

HUAWEI

Remote DR Solution Based on Streaming Replication

- Feature specifications
  - The network latency within the database instances or those for DR must be less than or equal to 10 ms, and the network latency between the primary and standby database instances must be less than or equal to 100 ms. The DR can run normally within the range of the required network latency. Otherwise, the primary and standby database instances will be disconnected.
  - When the network bandwidth is not a bottleneck and parallel replay is enabled for the standby database instance for DR, different hardware specifications support different log generation speeds for the primary database instance. The RPO and RTO can be ensured only under these log generation speeds.
  - A certain amount of data can be lost when the standby database instance is promoted to primary. RPO is less than or equal to 10 seconds.
  - When the standby database instance is normal, the RTO for promoting it to primary is less than or equal to 10 minutes. However, when it is degraded, the RTO goes up to 20 minutes.

HUAWEI

GUC Parameter Settings That Affect DR Performance Specifications

- Impact of checkpoint-related parameter settings
  - The DR performance metric described in "Feature specifications" is measured when the parameters related to checkpoints are set to default values.
  - When **enable_incremental_checkpoint** is set to **on**, the maximum interval between automatic WAL checkpoints is determined by the value of **incremental_checkpoint_timeout**. If the default value is not used and you set it to a larger value, many logs will need to be replayed when the instance is restarted. As a result, the specified RTO cannot be ensured.
- Impact of parameter settings related to ultimate RTO
  - To enable ultimate RTO, ensure that each host has more logical CPUs than the number of extra threads started after ultimate RTO is enabled. Otherwise, threads may contend for CPU resources. As a result, some operations in the DR process will take a long time and will not be able to meet the specified DR performance specifications. The formula for calculating the number of extra threads started after ultimate RTO is enabled is: (**recovery_parse_workers** x (**recovery_redo_workers** + 2) + 5) x Number of DN instances on each host.

HUAWEI

Establishing a DR relationship

- Evaluate the workload of the primary database instance before establishing a DR relationship.
  - Data volume
  - Log generation rate

Tool	Function
gs_sdr	API for establishing the DR relationship
	Promotes a standby database instance to primary (during a failover).
	Clears DR information of the primary database instance.
	Supports planned switchover.
	Queries the DR status of the primary and standby database instances.
	Upgrades the primary and standby database instances in the DR state during a major version upgrade.
	Upgrades the primary and standby database instances in the DR state during a minor version upgrade.

HUAWEI

Troubleshooting — Exceptions When Establishing DR Relationships

Symptom	Cause and Solution
The following error is returned when the primary database instance establishes a DR relationship: "Result exception error : Failed to do check main standby connection. Because Waiting timeout: XXs".	**Cause**: If the primary database instance has a lot of data or the remote network bandwidth is low, the primary database instance may exit the DR relationship establishment due to a timeout before the standby database instance finishes copying the data. **Solution**: If the DR relationship is being or has been established on the standby database instance, you can directly reenter the DR relationship establishment process of the primary database instance. The primary database instance will enter the state of waiting to connect to the standby database instance again. Before re-entering the process, you can estimate the timeout interval again and set the timeout interval parameter based on the data volume of the primary database instance and the remote network bandwidth. If the standby database instance also fails to establish the DR relationship, rectify the fault in the standby database instance and then establish the DR relationship again.
The primary DN in the primary cluster is switched over. As a result, the DR relationship fails to be established.	**Cause**: The primary DN of the primary cluster is switched over. The standby cluster is disconnected from the primary cluster for data build. As a result, the establishment fails. **Solution**: Check whether a switchover is manually performed in the primary cluster. If it is, stop the switchover. If not, ignore the exception. Issue the establishment command again.

Troubleshooting — Exceptions in Failover

Symptom	Cause and Solution
A faulty node of the standby database instance does not participate in the failover.	**Cause**: The node is disconnected from the standby database instance due to server breakdown or network interruption. **Solution**: After the faulty node is recovered, add it back to the database instance. Modify the parameters related to the database instance DR mode in the CM Server and CM Agent, and switch back to the primary database instance configuration. `gs_guc set -Z cmserver -N all -I all -c "backup_open = 0" gs_guc set -Z cmagent -N all -I all -c "agent_backup_open=0" gs_guc set -Z cmagent -N all -I all -c "disaster_recovery_type= 0"` Connect to the faulty node, query the CM Server and CM Agent process IDs, and run the **kill -9** command to kill the processes. The processes will be restarted by the OM Monitor to make the setting of CM Server and CM Agent parameters take effect. Run **cm_ctl start-n NODEID -D DATADIR** to manually restore the node.

HUAWEI

Troubleshooting — Exceptions in Planned Switchover

Symptom	Cause and Solution
Result exception error: Failed to generate switchover barrier before switchover	**Cause**: After the primary database instance receives a planned switchover command, a switchover barrier is generated before the primary database instance is demoted to standby. This is a switchover prerequisite, which ensures that the logs of all DN shards of the primary and standby database instances are stopped at the barrier. If a switchover barrier fails to be generated, for example, because of network jitter in the primary database instance, the planned switchover will be canceled. **Solution**: After the switchover command executed on the standby database instance times out and exits, you can perform the planned switchover again on the primary and standby database instances. If logs fail to be truncated after multiple switchover attempts, you will need to further analyze the log files related to streaming DR.
Result exception error: Failed to do check switchover_barrier on all main standby dn and cn. Because check timeout: XXs	**Cause**: After receiving the planned switchover command, the standby database instance checks whether the switchover barrier is received on the main standby DN before being promoted to primary. This is a switchover prerequisite, which ensures that the logs of the primary and standby DNs are stopped at the barrier. **Solution**: After the switchover command executed on the primary database instance times out and exits, you can perform the planned switchover again on the primary and standby database instances.

HUAWEI

Troubleshooting — Exceptions in Database Instances of the Standby Cluster for DR

Symptom	Cause and Solution
The CM Agent on a node of the standby cluster is faulty. The DN status on the node is **Unknown**. The status of some main standby DNs is **Main Standby Need repair(Connecting)**.	**Cause**: The CM Agent on the node is faulty. The DN status on the node cannot be reported to the CM Server, and the DN instance is displayed as **Unknown**. If there is a main standby instance on the node, it will be switched over. The original main standby instance is normal and has a normal streaming replication relationship with the primary DN in the primary database instance. However, the primary DN in the primary database instance allows only one main standby connection. As a result, the new main standby instance cannot be connected to the primary DN in the primary database instance, and the instance status is displayed as **Main Standby Need repair(Connecting)**. **Solution**: Check the CM Agent alarm ALM_AI_AbnormalCMSProcess of the standby cluster and try to restore the faulty CM Agent. After the fault is rectified, the connection to the new main standby instance will be restored. If the faulty CM Agent cannot be recovered quickly, run the **gs_ctl stop -D DATADIR** or **kill** command to manually stop the DN process on the node and recover the node.

Quiz

1. When gs_dump or gs_dumpall is used to export data, other users can access or perform read/write operations on the openGauss database. ( )

   A. True

   B. False

HUAWEI

Summary

- This chapter describes backup and restoration as well as DR used in the openGauss database, including logical backup and restoration, physical backup and restoration, flashback, and two-city three-DC DR in terms of their basic concepts, usage methods, and application scenarios. Upon completing this chapter, you should understand the backup and restoration methods and DR solutions used in the openGauss database.

HUAWEI

Thank you.

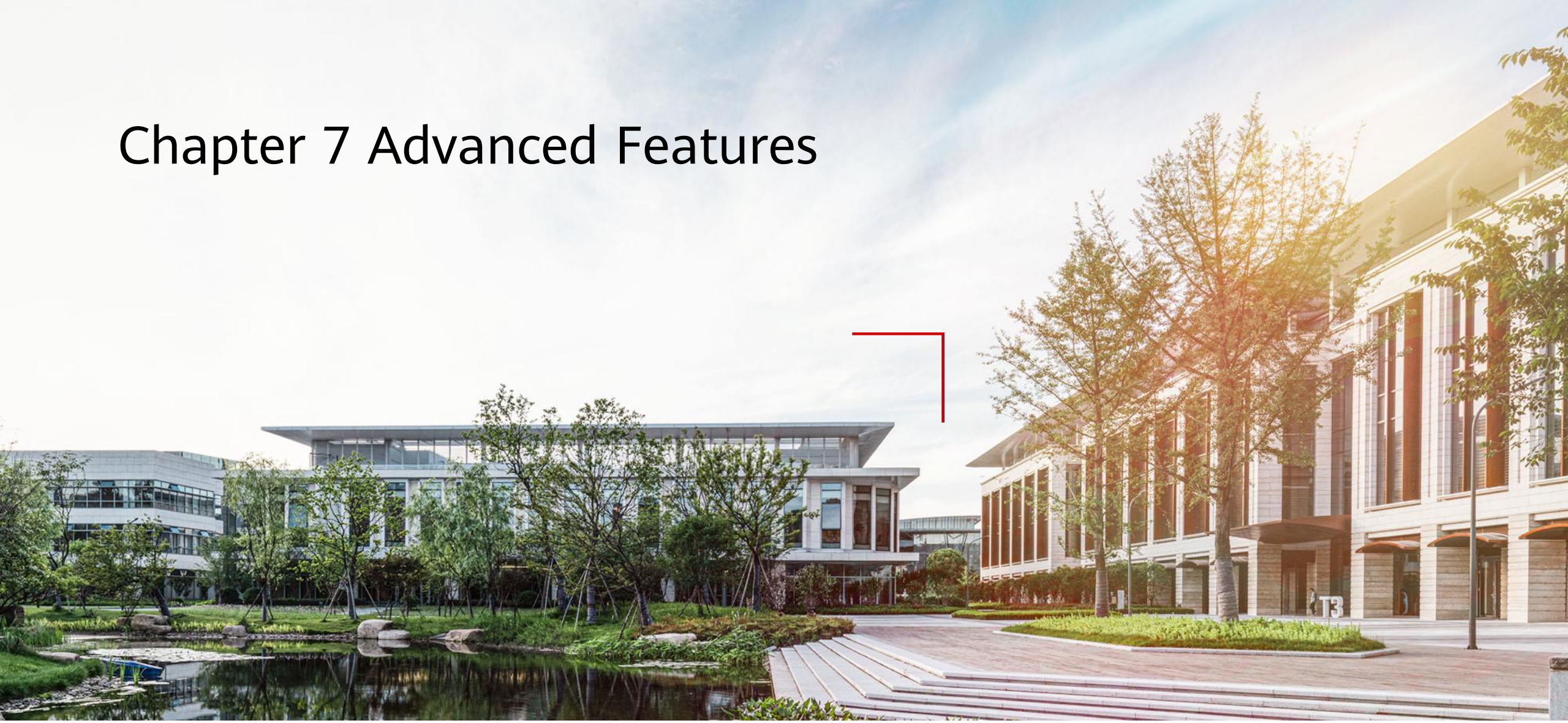
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 7 Advanced Features



HUAWEI

Foreword

- This chapter provides an overview of the advanced features of the openGauss database. These include NUMA architecture optimization and resource pooling, AI capabilities, fusion storage engine (introduction of the in-place update storage engine), Paxos-based high availability implemented using the Distributed Consensus Framework, fully-encrypted database, and memory-optimized table (MOT) management.

HUAWEI

Objectives

- Upon completion of this course, you will be able to:
  - Get familiar with NUMA architecture optimization and resource pooling.
  - Learn about openGauss's AI capabilities, DCF, and MOTs.
  - Understand the application of Ustore in the openGauss database.
  - Use openGauss's fully-encrypted database feature.

HUAWEI

Contents

1. **NUMA Architecture Optimization and Resource Pooling**
2. AI Capabilities
3. Fusion Storage Engine
4. High Availability Based on the Paxos Protocol
5. Fully-encrypted Database
6. MOT Management

HUAWEI

NUMA Architecture Optimization

- The NUMA architecture optimization mainly focuses on the architecture features of the Kunpeng processor and the ARMv8 instruction set. It involves system optimizations at multiple levels, including the OS, software architecture, lock concurrency, logging, atomic operations, and cache access. These optimizations considerably enhance the processing performance of the openGauss database on the Kunpeng platform.

HUAWEI

Feature Description (1)

- openGauss uses the load/store-exclusive (LSE) instruction sets to perform efficient atomic operations on Kunpeng chips, which are based on the Armv8.1 architecture. This significantly improves CPU usage, multi-thread synchronization performance, and Xlog write performance.
- Leveraging the larger L3 cache line provided by Kunpeng chips, openGauss optimizes hotspot data access. This enhancement improves the cache hit ratio, reduces the overhead of cache consistency maintenance, and significantly boosts overall data access performance.

HUAWEI

Feature Description (2)

- openGauss performs a series of targeted optimizations tailored to the NUMA architecture characteristics of Kunpeng multi-core processors. It minimizes the latency associated with inter-core memory access while fully leveraging the many-core processing power of Kunpeng processors. Key technologies, including batch insertion of redo logs, NUMA distribution of hotspot data, and Clog partitioning, significantly enhance the processing performance of the TP system.
- Atomic operations are performed by leveraging the LSE instruction sets. This also improves the multi-thread synchronization performance.

HUAWEI

Arm-based Architecture Optimization (1)

- Global PGPROC array optimization
  - openGauss optimizes its internal data structure. For instance, the PGPROC array is globally optimized so that each client connection's service process (represented by the PGPROC structure) aligns with the NUMA node of the CPU core handling the connection request. This reduces cross-NUMA memory access.
- NUMA distribution of hotspot data
  - In line with the principle of "local data access," openGauss optimizes the distribution of hotspot data across NUMA nodes to ensure that frequently accessed data is stored in or near the local memory of the CPU core processing the data. This approach reduces remote memory access latency and enhances data access speed.

HUAWEI

Arm-based Architecture Optimization (2)

- Clog partitioning
  - Partitioning transaction commit logs (Clogs) enable independent write operations on different NUMA nodes, thereby dispersing I/O workloads and preventing multiple CPU cores from contending for resources on the same NUMA node.
- Batch insertion of redo logs
  - Batch inserting redo logs for transactions reduces the frequency of data transmissions across NUMA boundaries and enhances the parallel processing capabilities of Kunpeng processors.
- Usage of LSE instruction sets
  - Leveraging the LSE instruction sets provided by Kunpeng chips based on the Armv8.1 architecture, openGauss achieves efficient atomic operations, enhancing CPU utilization. This particularly boosts synchronization performance between threads and enhances Xlog write performance.

HUAWEI

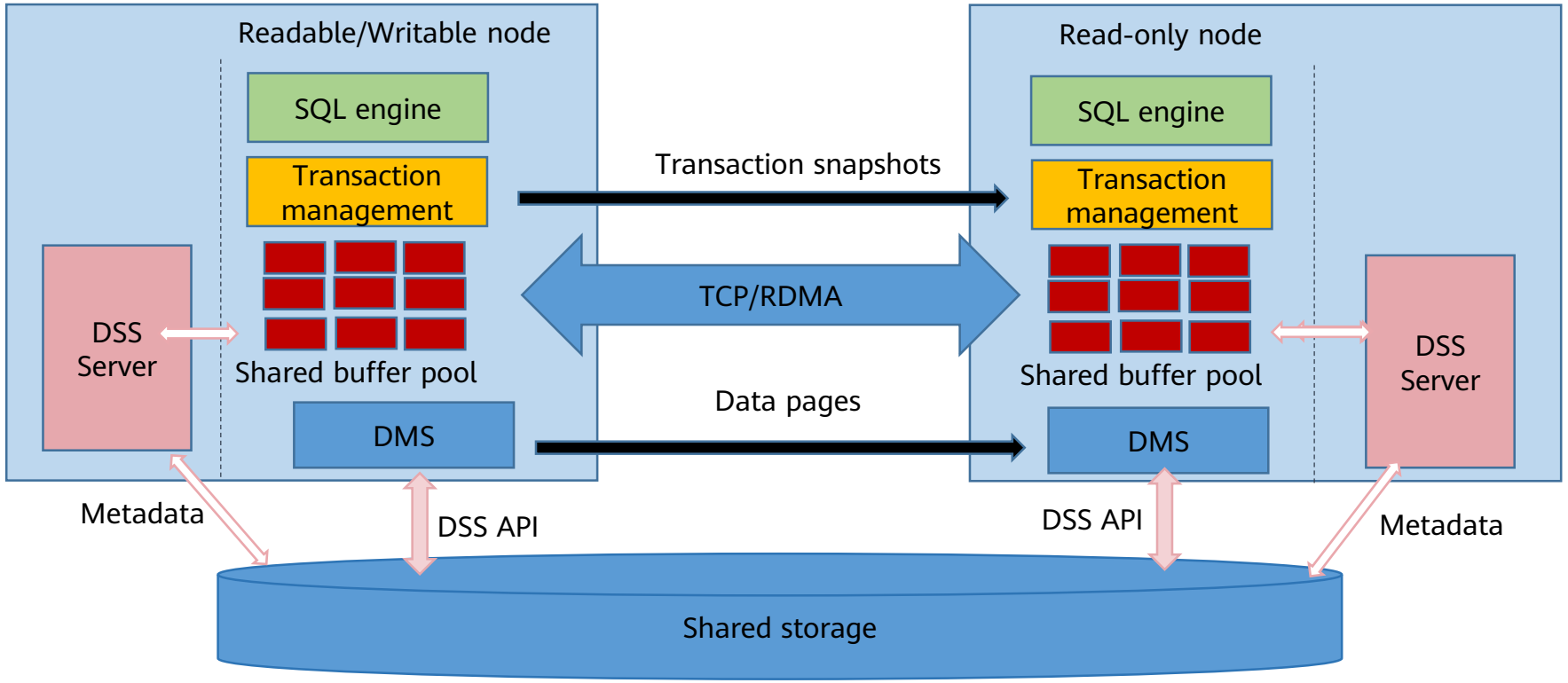
Arm-based Architecture Optimization (3)

- Cache optimization
  - Kunpeng chips feature large L3 caches. By leveraging this feature, openGauss optimizes the distribution and access of hotspot data in the cache, improves the cache hit ratio, and reduces the overhead of cache consistency maintenance, further improving the overall data access performance.
- In short, Huawei openGauss optimizes Arm-based Kunpeng processors at the NUMA layer to maximize the advantages of Kunpeng multi-core processors, eliminate performance bottlenecks caused by uneven memory access, and enhance database processing performance in large-scale concurrent scenarios.

HUAWEI

Resource Pooling (1)

- Resource pooling is a novel HA deployment mode that allows primary and standby nodes to share the same storage. This approach effectively addresses the challenge of doubled storage capacity in traditional HA setups, significantly reducing both storage requirements and costs. Additionally, it enables the standby node to support real-time, consistent reads.



Resource Pooling (2)

- Distributed Storage Service (DSS)
  - DSS components include DSS API and DSS Server. DSS Server is an independent process that directly manages raw devices of disk arrays and provides capabilities similar to distributed file systems externally. DSS API is a dynamic library integrated into the database. DSS components share memory and a client API dynamic library, enabling the database to create, delete, expand, shrink, read, and write files.
- Distributed Memory Service (DMS)
  - DMS is a dynamic library integrated into the database that transmits page data over TCP/RDMA networks, coordinating memory between primary and standby nodes. This memory pooling ensures real-time, consistent reads on the standby node.
- Page exchange between primary and standby nodes is accelerated over RDMA, depending on the CX5 NIC and OCK RDMA dynamic library.

HUAWEI

Resource Pooling (3)

- Storage pooling uses the DSS components to enable the primary and standby nodes to share the same storage. Compared to traditional database setups, resource pooling classifies directories into three types: directories exclusively used by each instance, directories exclusively used but shared among instances, and directories shared by all instances. The directories to be shared must be stored on the disk array, and the directories that are not shared must be stored on the local disk. Additionally, to set up a database on the standby node, you only need to create directories specific to the standby node instead of creating directories shared by all instances. Resource pooling-related GUC parameters are added. You can change the storage mode from page mode to segment-page mode.
- Memory pooling uses the DMS component to implement real-time page exchange between primary and standby nodes and ensure real-time, consistent data reads on the standby node. For instance, after a transaction is committed on the primary node, the transaction can be read on the standby node immediately. There is no delayed read (the transaction isolation level is read committed).
- Memory pooling utilizes OCK RDMA to reduce the latency of page exchanges between the DMS primary and standby nodes. Compared to that over TCP, the latency of consistent reads on the standby node over OCK RDMA is reduced by at least 20%.

HUAWEI

Feature Description

- Compared to the traditional HA deployment mode, primary and standby nodes sharing one copy of data significantly reduces the storage capacity.
- Log replication is not required between the primary and standby nodes. Instead, page exchange is implemented between the primary and standby nodes to support real-time, consistent reads on the standby node.
- By default, the primary and standby nodes exchange pages in real time over the TCP network. To reduce the page exchange latency, you can use the OCK RDMA dynamic library to boost the real-time consistent read performance of the standby node.

HUAWEI

Performance Optimization (1)

- Resource pooling optimizes the visibility logic on the standby node. The commit sequence number (CSN) that corresponds to the transaction XID obtained from the primary node is cached locally to reduce network overhead and message interaction.
  - In the original logic, the standby node determines the visibility of a tuple as follows: if the transaction associated with the tuple's XID is not marked as committed, the standby node retrieves the real-time CSN for the XID from the primary node to assess visibility. When a large number of pages are queried on the standby node, the extended query process and frequent interactions can impact the execution efficiency between the primary and standby nodes. This feature stores transaction commit status information in the local L2 cache of the standby node. When assessing the tuple visibility, the standby node first checks the local cache for the relevant information. If the information is not available locally, the node retrieves it from the primary node and subsequently updates the local cache.

HUAWEI

Performance Optimization (2)

- Resource pooling optimizes how the primary node retrieves the oldest Xmin from the cluster by locally caching this value. This allows the standby node to access snapshots in real time. Additionally, the standby node periodically sends its oldest local Xmin to the primary node, reducing broadcast overhead.
  - In the original logic, each time the primary node generates a new snapshot, it triggers a broadcast message to retrieve the oldest Xmin from all standby nodes in the cluster. This process updates the cluster's oldest Xmin for use in vacuuming and heap tuple pruning operations. When the standby node receives a snapshot from the primary node, the Xmin of that snapshot is recorded in the primary node's memory. Additionally, the standby node periodically sends its oldest local Xmin to the primary node via a background thread. The primary node also periodically clears invalid Xmin information through a background thread.

HUAWEI

Performance Optimization (3)

- Resource pooling optimizes the process standby nodes use to obtain snapshots. The primary node broadcasts the latest snapshot to the standby nodes, which then access this snapshot from their local host. This approach reduces network overhead and minimizes message interactions.
  - In the original logic, each standby node must obtain the latest snapshot from the primary node in real time for every read operation. This frequent interaction between the primary and standby nodes can become burdensome when the standby node is heavily engaged in read services.

HUAWEI

Transparent Write Forwarding on the Standby Node Within a Transaction

- This feature enhances transaction write forwarding in traditional primary/standby mode based on the resource pooling architecture. In the traditional architecture, each transaction committed on the standby node is forwarded to the primary node. This feature enables the standby node to execute read requests among transactions locally and forward write requests to the primary node for execution.
- In the resource pooling architecture, a cluster supports concurrent data writes across multiple nodes. This enhances the linear performance of the entire cluster, especially when there are minimal concurrent data conflicts between primary and standby nodes.

HUAWEI

Transparent Write Forwarding on the Standby Node Within a Transaction - Restrictions

- In the resource pooling architecture, after the "write on standby" feature is enabled, all write SQL statements among transactions committed on the standby node are forwarded to the primary node for execution. Read statements among the transactions are still executed on the standby node.
- Resource pooling does not support DDL and LOCK statements in transactions on the standby node, and an error is reported.
- This feature also forwards write requests among child transactions, if any. Additionally, it forwards cursor operations (including cursors themselves) to the primary node for execution.

HUAWEI

DSS Enhancements

- DSS supports a thread pool, allowing server CPU and memory resources to be used and scaled on-demand and configured as required.
  - DSS supports on-demand configuration of concurrent database connections. By using a thread pool, DSS manages resource usage, such as memory and CPU usage, thereby improving server memory and CPU utilization. This configuration is static and does not take effect dynamically.
- DSS uses black box logs to locate DSS Server faults, which enhances the DFX capability.
  - During a system crash, process and thread context information related to heaps, stacks, and registers is dumped to the black box logs, helping R&D engineers locate and trace DSS Server faults. The black box log function is enabled by default and can be disabled by setting GUC parameters. The setting takes effect after the node service restarts.
- DSS supports the NoF/NoF+ API.

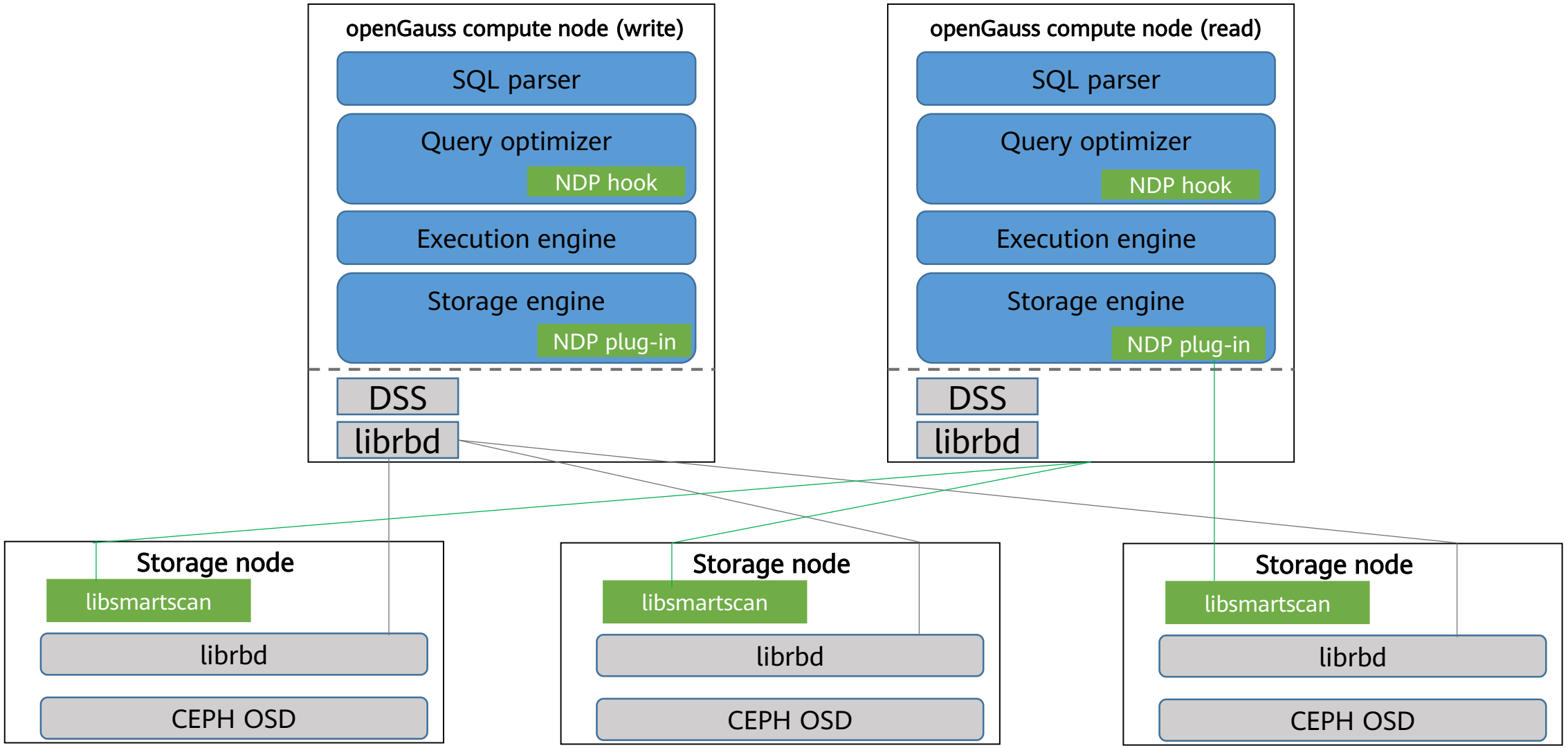
HUAWEI

Resource Pooling Operator Offloading

- Shared storage offers excellent elasticity and reliability, but significantly impacts performance compared to local host storage. This performance degradation is due to delays from network I/O and distributed storage, particularly when many queries overwhelm the buffer pools. In such scenarios, a large volume of data must be transferred from storage nodes to compute nodes, with only a small fraction of data being relevant. This results in substantial network I/O waste and degraded performance. By offloading operators, data filtering is executed on storage nodes to remove unnecessary data, reducing network communications and improving end-to-end performance.

HUAWEI

Resource Pooling Operator Offloading - Solution



HUAWEI

Resource Pooling Operator Offloading - Feature Description

- The openGauss resource pooling feature enables offloading operators, such as Agg, SeqScan, Filter, and Projection, to storage nodes. The filtered, projected, or aggregated results are then transmitted to compute nodes for further processing. This approach, known as near-data processing (NDP), allows compute nodes to perform fewer calculations and generate final results using upper-layer operators. The operator offloading process does not interrupt services. The feature reduces data I/Os transmitted over the network and improves query performance by utilizing the computing power of storage nodes. The solution of this feature consists of two parts:
  - Computing: database kernel
  - Storage: libsmartscan process

HUAWEI

Computing: Database Kernel

- Operator offloading is implemented as a plug-in in the database kernel, which means that it has no impact on the original kernel process if operator offloading is not required.
- Operator offloading is only performed in scenarios where it is beneficial or feasible. In cases where offloading is not suitable, the operators follow the existing kernel process.
- The context information for operator offloading, including snapshot, Clog, CSN log, expression, and table schema, is sent to the libsmartscan process on the storage side.
- Subsequently, DSS retrieves information about the storage units (stripes) corresponding to the file location, constructs an NDP request, and sends the request to the appropriate storage node for processing. NDP processing is applied only to pages or blocks not present in the buffer pool (i.e., those that are being read or written from the current node). If the data is already in the buffer pool, the original kernel process is used instead.
- Finally, compute nodes retrieve the NDP results from storage nodes and output the results to upper-later operators for further calculation or aggregate the NDP results.

HUAWEI

Storage: libsmartscan Process

- Offers the in-storage computing (ISC) function.
- Local data is accessed according to storage semantics through storage APIs.
- During the parsing process, data visibility is assessed. If the visibility cannot be determined or if the page/block for the data version (when using Ustore) is not recognized, the data is processed by compute nodes following the original kernel process.
- Data whose visibility is confirmed is filtered, aggregated, or projected and then sent to compute nodes for further processing.
- Operator offloading does not impact the original execution plan or operator information. The storage engine API hook implements the NDP logic, shielding the execution engine from detecting the offloading of operators.

HUAWEI

Resource Pooling Operator Offloading - Limitations

- This feature depends on the resource pooling feature.
- Currently, the plug-in can be loaded only by setting **shared_preload_libraries**.
- TOAST tables are not supported.
- Ustore is not supported.
- The **synchronize_seqscans** parameter is disabled.

HUAWEI

On-demand Replay for Ultimate RTO

- With on-demand replay for achieving the ultimate RTO, only essential content is rebuilt for disaster recovery. During this process, services are immediately available without requiring full replay, thereby minimizing the RTO. Once services are available, logs are replayed in the background as needed to ensure that users can access the most recent data, matching the state that existed before the fault occurred.
- In the resource pooling architecture, the standby node does not replay logs of the primary node. If the primary node is faulty, the standby node restores data of the primary node from the last checkpoint, which takes a long time. The database is unavailable during this process, significantly affecting system availability. After the on-demand replay feature is enabled, the database becomes immediately available following a primary node fault, achieving the ultimate RTO and enhancing overall database availability.

HUAWEI

On-demand Replay for the Shortest RTO – Feature Description

- In the resource pooling architecture, this feature enables the primary database node to quickly recover from a restart.
- In the resource pooling architecture, this feature accelerates the failover of standby nodes, reducing the RTO.
- This feature is an evolution of the existing ultimate RTO feature.
- Once the ultimate RTO feature is enabled, multi-level pipelines are established to enhance the concurrency and speed up Xlog replay. The standby node utilizes multi-version page reading. A replayer thread maintains a log chain for each page, while a reader thread retrieves the page corresponding to a specified LSN that indicates the location of the WAL. When a query conflicts with the replay process, it times out and is canceled, reporting the error message "canceling statement due to conflict with recovery." In this case, retry based on the error code.

HUAWEI

On-demand Replay for Ultimate RTO – Limitations

- The ultimate RTO feature focuses only on whether the standby node meets the ultimate RTO requirements. The ultimate RTO feature has no inherent flow control and uses the **recovery_time_target** parameter for flow control instead.
- This feature does not support the read operation on the standby node. If you query the standby node, a core dump may occur on the standby node.
- This feature applies only to the resource pooling architecture.
- In the failover scenario where the standby node is promoted to primary, the original primary node cannot be added back to the cluster during on-demand replay.
- Additionally, autovacuum is not allowed during on-demand replay.

HUAWEI

Resource Pooling Functions (1)

- ss_buffer_ctrl()
  - Description: Displays the information recorded in buf_ctrl corresponding to the buffer.
  - Return type: record

  Example 1: Query the buf_ctrl information of a specific buffer.

```
openGauss=# select * from ss_buffer_ctrl() where bufferid = 20;
bufferid | is_remote_dirty | lock_mode | is_edp | force_request | need_flush | buf_id | state | pblk_relno | pblk_blkno | pblk_lsn |
seg_fileno | seg_blockno
----------+-----------------+-----------+--------+---------------+------------+--------+-------+------------+------------+----------+---------
---+-------------
20 | 0 | 1 | 0 | 0 | 0 | 19 | 32 | 0 | 4294967295 | 0 | 2 | 4375
(1 row)
```

Resource Pooling Functions (2)

- ss_txnstatus_cache_stat()
  - Description: Returns statistics about transaction information cache and retrieval. Note that the related count statistical items are not entirely accurate and are meant for performance optimization analysis purposes rather than providing precise statistics..
  - Example: Query statistics about transaction information cache.

```
openGauss=# select * from ss_txnstatus_cache_stat();
vcache_gets | hcache_gets | nio_gets | avg_hcache_gettime_us | avg_nio_gettime_us | cache_hit_rate | hcache_eviction |
avg_eviction_refcnt
-------------+-------------+----------+-----------------------+--------------------+----------------+-----------------+--------------------
263809 | 782159 | 275012 | 1.73883698838727 | 756.186410047562 | .79181213947221 | 0 | 0
(1 row)
```


HUAWEI

Resource Pooling Functions (3)

- query_node_reform_info()
  - Description: Reforms the statistics of a cluster.
  - Example 1: Add a standby node to the cluster.

```
openGauss=# select * from query_node_reform_info();
 reform_node_id |  reform_type  |    reform_start_time  |  reform_end_time  | is_reform_success | redo_start_time | redo_end_time |
xlog_total_bytes | hashmap_construct_time | action
----------+-----------+----------+-----------+---------+------------+------------+----------+---------+---------
            0 | Normal reform | 2023-09-21 16:37:46.414  | 2023-09-21 16:37:47.817  | t                 | -               | -             |
-1 | -                      | stable
            1 | -             | -                       | -                       | t                 | -               | -             |              -1 | -                      |
join in
(2 rows)
```


HUAWEI

Resource Pooling Functions (4)

- Example 2: Remove a standby node from the cluster.

```
openGauss=# select * from query_node_reform_info();
 reform_node_id |  reform_type  |     reform_start_time      |      reform_end_time       | is_reform_success | redo_start_time |
redo_end_time | xlog_total_bytes | hashmap_construct_time |  action
----------------+---------------+----------------------------+----------------------------+-------------------+-----------------+---------------+-------
----------+----------------------+---------
              0 | Normal reform | 2023-09-21 16:37:06.520  | 2023-09-21 16:37:06.568  | t                 | -               | -             |
-1 | -                    | stable
              1 | -             | -                          | -                          | t                 | -               | -             |              -1 | -                    |
kick off
(2 rows)
```


HUAWEI

Resource Pooling Functions (5)

- Example 3: Trigger a cluster failover upon a primary node fault.

```
openGauss=# select * from query_node_reform_info();
 reform_node_id | reform_type |    reform_start_time     |     reform_end_time      | is_reform_success |     redo_start_time      |
redo_end_time       | xlog_total_bytes |  hashmap_construct_time  |  action
----------------+-------------+--------------------------+--------------------------+-------------------+--------------------------+--------------
----------+------------------+--------------------------+----------
              0 | -           | -                        | -                        | t                 | -                        | -                        |               -1 | -
| kick off
              1 | Failover    | 2023-09-21 16:56:45.893  | 2023-09-21 16:56:50.702  | t                 | 2023-09-21 16:56:50.282  | 2023-09-
21 16:56:50.385  |              288 | 2023-09-21 16:56:50.385  | stable
(2 rows)
```


HUAWEI

Resource Pooling Functions (6)

- Example 4: The cluster automatically performs a switchover.

```
openGauss=# select * from query_node_reform_info();
 reform_node_id | reform_type |     reform_start_time      |      reform_end_time       | is_reform_success | redo_start_time |
redo_end_time | xlog_total_bytes | hashmap_construct_time | action
----------------+-------------+----------------------------+----------------------------+-------------------+-----------------+--------------+---------
---------+------------------------+--------
              0 | Switchover  | 2023-09-21 16:59:25.646  | 2023-09-21 16:59:29.182  | t                 | -               | -            |              -1
| -                      | stable
              1 | -           | -                        | -                        | t                 | -               | -            |              -1 | -                      |
stable
(2 rows)
```


HUAWEI

Resource Pooling Functions (7)

- query_page_distribution_info(relname TEXT, forkNum INT4, blockNum INT4)
  - Description: Queries the distribution of pages in the cluster based on **relname**, **forkNum**, and **blockNum**.
  - Example: Insert a row of data into a table and immediately query the page distribution information.

```
openGauss=# create table tb(id int);
CREATE TABLE
openGauss=# insert into tb values(0);select * from query_page_distribution_info('tb', 0, 0);
INSERT 0 1
 instance_id | is_master | is_owner | is_copy |    lock_mode     |  mem_lsn   |  disk_lsn  | is_dirty
-------------+-----------+----------+---------+------------------+------------+------------+----------
           0 | t         | t        | f       | Exclusive lock | 1975555208 | 1975555112 | t
(1 row)
```


HUAWEI

Resource Pooling Functions (8)

- dss_io_stat(duration INT4)
  - Description: Collects statistics on the I/O read/write speed and times within the interval (in seconds) specified by the **duration** parameter.
  - Restriction: The value of **duration** must be less than 60.
  - Example: Write data to a table on the primary node and query DSS I/O statistics within 2 seconds.

```
openGauss=# select * from dss_io_stat(2);
 read_kilobyte_per_sec | write_kilobyte_per_sec | io_times
-----------------------+------------------------+----------
                   404 |                  25664 |     3158
(1 row)
```

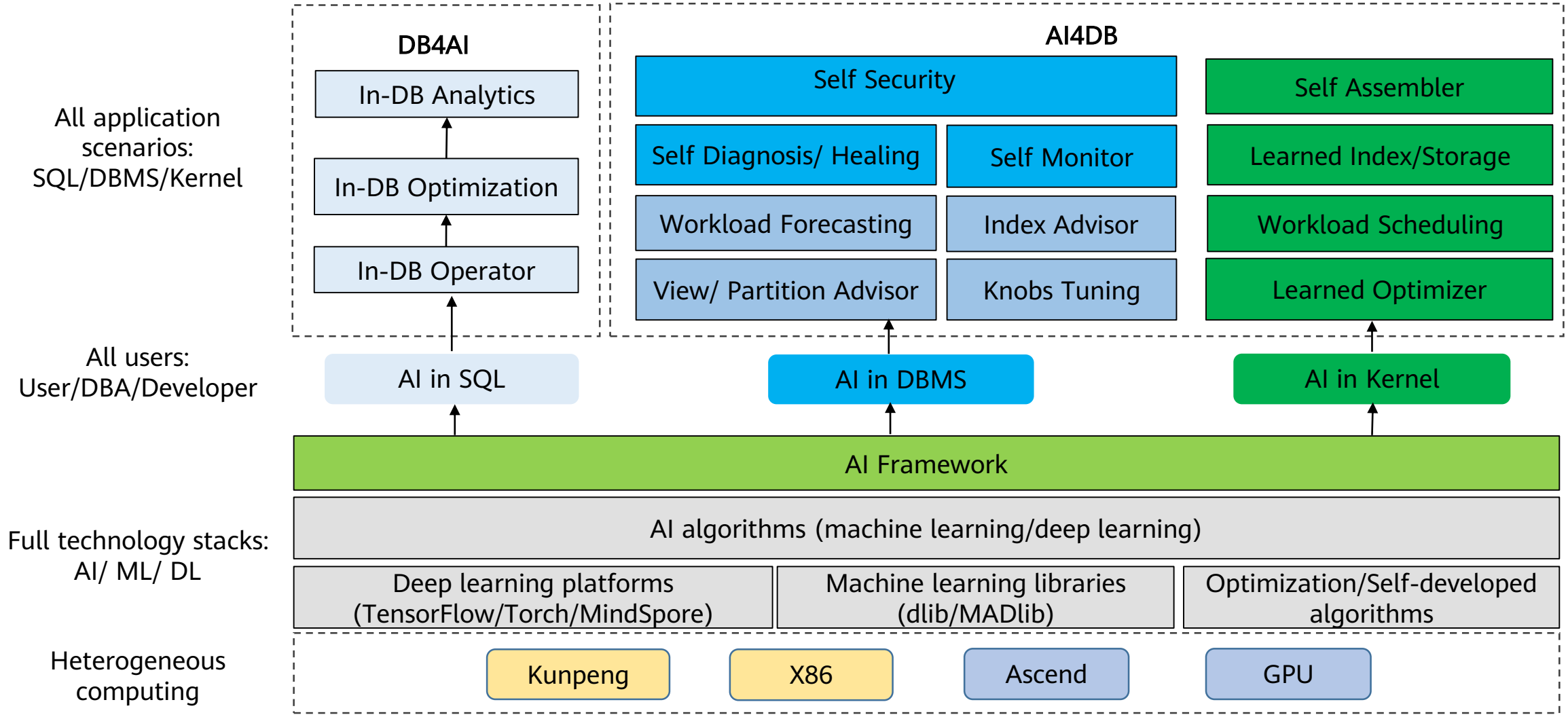

HUAWEI

Contents

1. NUMA Architecture Optimization and Resource Pooling
2. **AI Capabilities**
3. Fusion Storage Engine
4. High Availability Based on the Paxos Protocol
5. Fully-encrypted Database
6. MOT Management

HUAWEI

openGauss AI Overview



HUAWEI

AI4DB: Autonomous Database O&M

- AI4DB primarily focuses on autonomous operations, maintenance, and management of databases, significantly reducing the workload of database O&M personnel. DBMind offers a service-oriented AI4DB framework for monitoring, which includes real-time AI toolkits and out-of-the-box AI O&M functions, such as index recommendation. For monitoring, AI4DB relies mainly on the open-source Prometheus platform. DBMind provides an exporter that generates monitoring data, enabling seamless integration with Prometheus.

HUAWEI

Database Metric Collection, Forecasting, and Exception Detection (1)

- This feature is an AI tool integrated into openGauss and is designed to collect and forecast database metrics, as well as monitor and diagnose exceptions. It is a component of the DBMind suite. Currently, this feature is compatible with the Prometheus platform to collect database system metrics. Prometheus exporters are provided to collect and process database monitoring metrics. By analyzing the time series data of metrics, you can forecast the future workload trend, diagnose problems, and detect exceptions.
- Prometheus is a popular open-source monitoring system in the industry. It is also a time series database. The collector of Prometheus is called an exporter, which is used to collect metrics of monitored modules. To interconnect with the Prometheus platform, DBMind provides two types of exporters: openGauss-exporter for collecting database metrics and reprocessing-exporter for reprocessing the collected metrics.

HUAWEI

Database Metric Collection, Forecasting, and Exception Detection (2)

- **Advantages:**
  - This significantly simplifies the work of O&M personnel, eliminates the need for more personnel, and reduces costs for the company.
  - Users can proactively identify potential issues through metrics collection, monitoring, and forecasting to prevent database incidents that could lead to significant losses.
- **Prerequisites:**
  - The database is functioning normally, and the data directory has been configured in the environment variables.
  - The Python version must be 3.6 or later.
  - The Prometheus monitoring platform has been configured, and the Prometheus service has been started to enable the collection of monitoring data.

HUAWEI

Root Cause Analysis for Slow SQL Statements

- Slow SQL statements continue to be a significant challenge in data operations and management. Diagnosing the root causes of these slow statements can be particularly challenging. This tool leverages database characteristics and the diagnostic experience of DBAs from the live network to provide root cause analysis for slow SQL statements. It identifies multiple potential causes based on their likelihood and provides targeted solutions.
- Environment setup
  - The database is functioning normally.
  - The entire metric collection system (including openGauss exporter, reprocessing exporter, and node exporter) is running normally.
- Command information

```
gs_dbmind component slow_query_diagnosis –help
```


HUAWEI

Root Cause Analysis for Slow SQL Statements - Usage Guidelines

- Enable only the slow SQL diagnosis function. The top 3 possible root causes will be output.

```
gs_dbmind service start -c confpath --only-run slow_query_diagnosis
```

- Diagnose slow SQL statements interactively.

```
gs_dbmind component slow_query_diagnosis show -c confpath --query SQL --start-time timestamps0 --end-time timestamps1
```

- Clean up historical forecast results.

```
gs_dbmind component slow_query_diagnosis clean -c confpath --retention-days DAYS
```

- Stop the service.

```
gs_dbmind service stop -c confpath
```


HUAWEI

Root Cause Analysis for Slow SQL Statements - Troubleshooting

- If users attempt to perform interactive diagnosis on slow SQL statements that have not been executed, the diagnosis results will not be output.
- If the exporter for metric collection is not running, the slow SQL diagnosis feature will be unable to function properly.
- After reconfiguring parameters in the configuration file, you need to restart the service process for the changes to take effect.
- When performing interactive diagnosis on slow SQL statements, the tool relies on the PRC and data collection service to gather the necessary data. Therefore, these services must be running before you can perform interactive diagnosis.
- For interactive diagnosis, the tool checks the SQL statements entered into a database. If the input is invalid, the diagnosis will not be performed.
- During slow SQL diagnosis, SMALL_SHARED_BUFFER will collect column information of related tables. Therefore, ensure that the opengauss_exporter connection user has permission on the schema to which the tables belong.

HUAWEI

Index-advisor: Index Recommendation

- An index in a database functions just like a catalog in a library; it points to the data value stored in a specific column of a table, which accelerates the query process for that table.
- Index-advisor recommends indexes in three forms: single-query indexes, virtual indexes, and workload-level indexes.

HUAWEI

Single-Query Index Recommendation

- The single-query index recommendation feature allows users to perform operations directly within the database. This feature generates index recommendations based on the semantic information of a query statement and the database's statistical data, providing recommended indexes for individual queries entered by the user.
- This feature involves the following API.

Function	Parameter	Description
gs_index_advise	An SQL statement string	Recommends an index for a single query statement.

HUAWEI

Single-Query Index Recommendation Example (1)

- Calling gs_index_advise generates a recommended index for a specific query. The recommended index result consists of two parts: table name and column name.
  - Example 1:

```
postgres=> select * from gs_index_advise('SELECT c_discount from bmsql_customer where c_w_id = 10');

table | column
---------------+----------
bmsql_customer | (c_w_id)
(1 row)
```

  - Based on the preceding output, the **c_w_id** column will be used as an index of the **bmsql_customer** table. To create the index, run the following command:

```
CREATE INDEX idx on bmsql_customer(c_w_id);
```


HUAWEI

Single-Query Index Recommendation Example (2)

- Example 2:

```
postgres=# select * from gs_index_advise('select name, age, sex from t1 where age >= 18 and age < 35 and sex = ''f''; ');

 table | column
-------+------------
t1 | (age, sex)
(1 row)
```

- Based on the preceding output, a composite index **(age, sex)** will be created on the **t1** table. To create the index, run the following command:

```
CREATE INDEX idx1 on t1(age, sex);
```


HUAWEI

Virtual Index

- The virtual index feature enables users to perform operations directly within the database, eliminating the need to create a physical overhead and reducing related overhead costs. It simulates the presence of a physical index to assess its potential impact on query performance. Users can evaluate the effects of the virtual index on specified queries using the optimizer, thus avoiding the time and space costs associated with creating a physical index.
- To enable this feature, set the GUC parameter **enable_hypo_index**. The default value is **off**.
- This feature involves the following APIs.

Function	Parameter	Description
hypopg_create_index	Character string of the statement for creating an index	Creates a virtual index.
hypopg_display_index	None	Displays information about all created virtual indexes.
hypopg_drop_index	OID of an index	Deletes a specified virtual index.
hypopg_reset_index	None	Clears all virtual indexes.
hypopg_estimate_size	OID of an index	Estimates the space required for creating a specified index.

HUAWEI

Virtual Index Example (1)

- Call the hypopg_create_index function to create a virtual index. For example:

```
postgres=> select * from hypopg_create_index('create index on bmsql_customer(c_w_id)');
indexrelid | indexname
 ------------+-----------------------------------
329726 | <329726>btree_bmsql_customer_c_w_id
(1 row)
```

- Enable the GUC parameter **enable_hypo_index**. This parameter controls whether the database optimizer considers the created virtual index when executing the **EXPLAIN** statement. By executing **EXPLAIN** on a specific query statement, you can evaluate whether the index can improve the execution efficiency of the query statement based on the execution plan provided by the optimizer. For example:

```
postgres=> set enable_hypo_index = on;
SET
```

- Before enabling the GUC parameter, run the combined statement of the keyword **EXPLAIN** and the query statement.

```
postgres=> explain SELECT c_discount from bmsql_customer where c_w_id = 10;
                                 QUERY PLAN
-----------------------------------------------------------------------
Seq Scan on bmsql_customer (cost=0.00..52963.06 rows=31224 width=4)
Filter: (c_w_id = 10)
(2 rows)
```


HUAWEI

Virtual Index Example (2)

- After enabling the GUC parameter, run the combined statement of the keyword **EXPLAIN** and the query statement.

```
postgres=> explain SELECT c_discount from bmsql_customer where c_w_id = 10;
QUERY PLAN
------------------------------------------------------------------------------------------------------------
[Bypass]
Index Scan using <329726>btree_bmsql_customer_c_w_id on bmsql_customer (cost=0.00..39678.69 rows=31224 width=4)
Index Cond: (c_w_id = 10)
(3 rows)
```

- If you compare the two execution plans, you may find that the index reduces the execution cost of the specified query statement. In this scenario, you can consider creating a physical index.

Workload-Level Index Recommendation

- To generate workload-level index recommendations, you can run a script outside the database. This feature uses a set of DML statements as workload input to generate a batch of indexes designed to enhance overall workload execution performance.
- Prerequisites:
  - The database is functioning normally, and the client connection is established successfully.
  - The gsql tool has been installed by the current user, and the tool path has been configured in the *PATH* environment variable.
  - An environment running Python 3.6 or later is available.
- Procedure:
  - Prepare a file where each line contains a separate DML statement. Each statement represents a workload input for analysis. To obtain historical service statements, you can check the offline logs of the database.
  - Run the Python script **index_advisor_workload.py**.

```
python index_advisor_workload.py [p PORT] [d DATABASE] [f FILE] [--h HOST] [-U USERNAME] [-W PASSWORD] [--max_index_num MAX_INDEX_NUM] [--multi_iter_mode]
```


HUAWEI

Workload-Level Index Recommendation Example

- Run the **index_advisor_workload.py** script to save multiple DML statements to **tpcc_log.txt**.

```
python index_advisor_workload.py 6001 postgres tpcc_log.txt --max_index_num 10 --multi_iter_mode
```

- The recommendation result is a set of indexes, which are displayed on the screen as multiple CREATE INDEX statements.

```
create index ind0 on bmsql_stock(s_i_id,s_w_id);
create index ind1 on bmsql_customer(c_w_id,c_id,c_d_id);
create index ind2 on bmsql_order_line(ol_w_id,ol_o_id,ol_d_id);
create index ind3 on bmsql_item(i_id);
create index ind4 on bmsql_oorder(o_w_id,o_id,o_d_id);
create index ind5 on bmsql_new_order(no_w_id,no_d_id,no_o_id);
create index ind6 on bmsql_customer(c_w_id,c_d_id,c_last,c_first);
create index ind7 on bmsql_new_order(no_w_id);
create index ind8 on bmsql_oorder(o_w_id,o_c_id,o_d_id);
create index ind9 on bmsql_district(d_w_id);
```

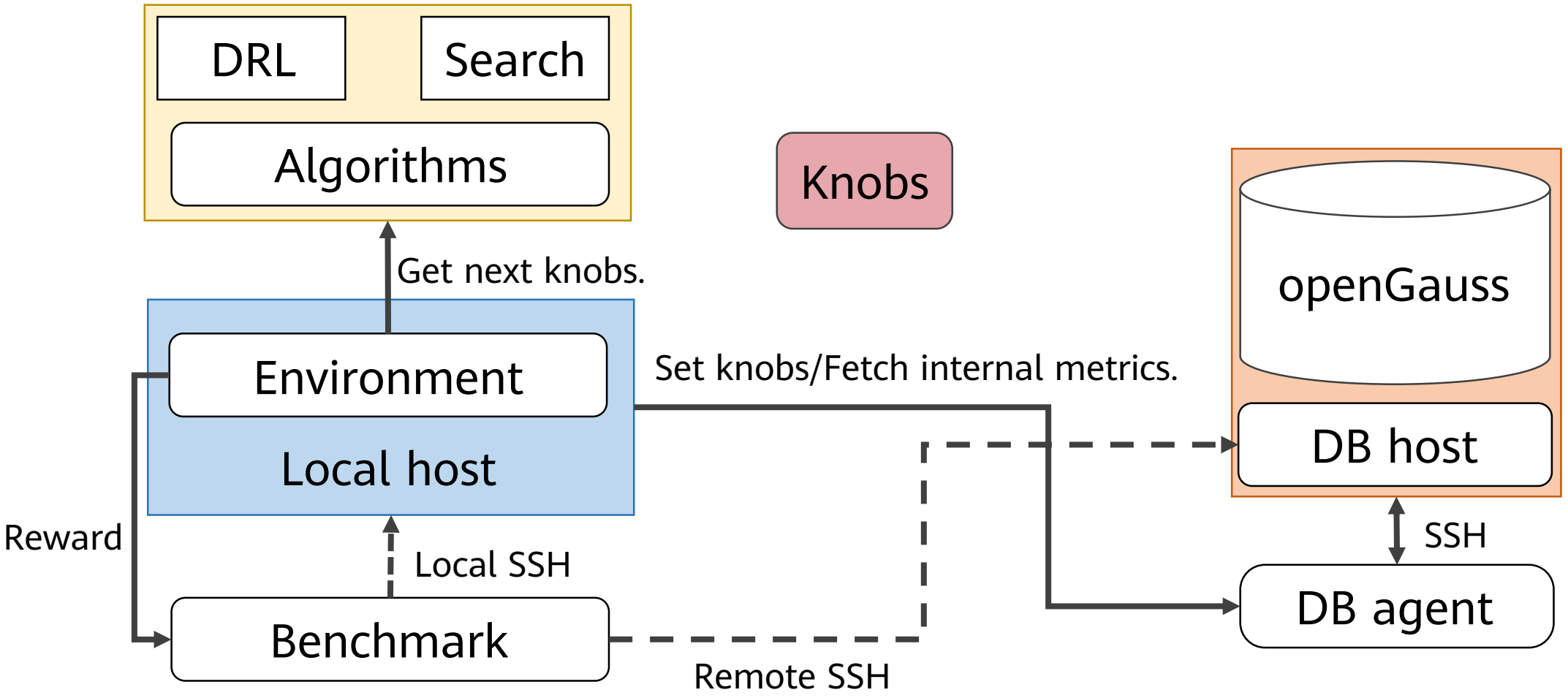

HUAWEI

Automatic Parameter Tuning and Diagnosis

- X-Tuner is a parameter-tuning tool integrated into the database. It uses AI technologies like deep reinforcement learning and a global search algorithm to obtain the optimal database parameter settings without manual intervention. This function does not need to be deployed within the database environment and can be installed and operated independently.
- Before starting the tuning program, ensure that the test environment is functioning normally: the benchmark test script should run successfully, and the database should be accessible. When using the tuning program, you need to provide the username and login password for the database and its instance to control the execution of benchmark performance tests.

HUAWEI

X-Tuner Structure



HUAWEI

X-Tuner Modes

- X-Tuner runs in three modes:
  - **recommend**: logs in to the database using specified account credentials, identifies inappropriate parameter settings and potential risks in the current database, collects characteristics of running workloads, and generates recommended knobs based on the collected information. **In this mode, the database does not need to be restarted. In other modes, the database may need to be repeatedly restarted.**
  - **train**: dynamically adjusts parameters and executes benchmark testing based on the benchmark information provided by users. The reinforcement learning model is trained through iterations to prepare for model tuning.
  - **tune**: Database parameters are further tuned based on optimization algorithms. Currently, two types of algorithms are supported: deep reinforcement learning and global search. Deep reinforcement learning requires X-Tuner to start from the **train** mode, while global search does not. You can directly perform search optimization.

HUAWEI

Installing and Running X-Tuner (1)

- There are two ways to run X-Tuner. One is to run the X-Tuner directly from the source code. The other is to install X-Tuner on the system using Python setup tools, and then run the **gs_xtuner** command to call the X-Tuner tool.
- Method 1: Run it from the source code.
  - Switch to the **xtuner** source code directory. To run it from the openGauss community, use the following source code path: **openGauss-server/src/gausskernel/dbmind/tools/xtuner**. To run it from the installed database system, use the following source code path: *$GAUSSHOME*/**bin/dbmind/xtuner**.
  - You can view the **requirements.txt** file in the current directory. Use pip to install the dependencies from **requirements.txt**.

    ```
    pip install -r requirements.txt
    ```

  - After the installation is complete, configure the environmental variable **PYTHONPATH**, and then run **main.py**. To view the help information, run the following command:

    ```
    cd tuner # Switch to the directory where the main.py entry file is located.
    export PYTHONPATH='..' # Add the upper-level directory to the search path.
    python main.py --help # View help information. The methods for using other functions are similar.
    ```


HUAWEI

Installing and Running X-Tuner (2)

- Method 2: Install X-Tuner.
  - You can use the **setup.py** file to install X-Tuner and then run the **gs_xtuner** command to run the program. Switch to the X-Tuner root directory: **openGauss-server/src/gausskernel/dbmind/tools/xtuner**. If a database system has been installed, the path is *$GAUSSHOME*/**bin/dbmind/xtuner**.
  - Install the program by using Python setup tools.

```
python setup.py install
```

  - If the **bin** directory of Python has been added to the *PATH* environment variable, the **gs_xtuner** command can be called directly from anywhere.
  - To view help information, run the following command:

```
gs_xtuner --help
```


HUAWEI

Slow SQL Statement Discovery

- SQLdiag is a tool for predicting the execution duration of SQL statements in openGauss. Existing prediction technologies are mainly based on evaluations of the execution plans, which are applicable only in OLAP scenarios where the execution plans can be obtained. However, they are not very useful for quick, simple OLTP or HTAP queries. SQLdiag, in contrast, examines the database SQL statement history and analyzes the performance of historical statements to predict the performance of new, unknown services.

HUAWEI

Slow SQL Statement Discovery - Advantages

- Execution plans of SQL statements are not necessary, thus there is no impact on database performance.
- SQLdiag is widely applicable, unlike many other industry algorithms that are limited to specific types of workloads, such as OLTP or OLAP.
- The tool is easy to use. You can train your own prediction model by performing simple operations.
- A typical application of this tool is to perform a perspective analysis on a batch of SQL statements before bringing them online to identify risks in advance.

HUAWEI

Slow SQL Statement Discovery - Prerequisites

- Training datasets are prepared in the following format, and each sample is separated by a newline character:

```
SQL,EXECUTION_TIME
```

- The WDR feature must be enabled if you want to use the provided tool to collect training data. The parameters **track_stmt_stat_level** and **log_min_duration_statement** are involved. For details, see the following sections.
- To ensure accurate prediction, the logs of historical statements provided by users must be as comprehensive and representative as possible.

HUAWEI

Slow SQL Statement Discovery - Procedure

- Provide logs for model training.
- Perform training and prediction.
  - Template-based training and prediction:

```
gs_dbmind component sqldiag [train, predict] -f FILE --model template --model-path template_model_path
```

  - DNN-based training and prediction:

```
gs_dbmind component sqldiag [train, predict] -f FILE --model dnn --model-path dnn_model_path
```


HUAWEI

Native DB4AI Engine

- openGauss offers native support for the DB4AI capability. This is made possible by introducing native AI operators, which streamline the operation process and make full use of the optimization and execution capabilities of the database optimizer and executor. This simplifies the model training and prediction process, leading to higher performance, and allows developers to concentrate on model tuning and data analysis in a shorter time frame. This, in turn, eliminates the need for fragmented technology stacks and redundant code implementation.

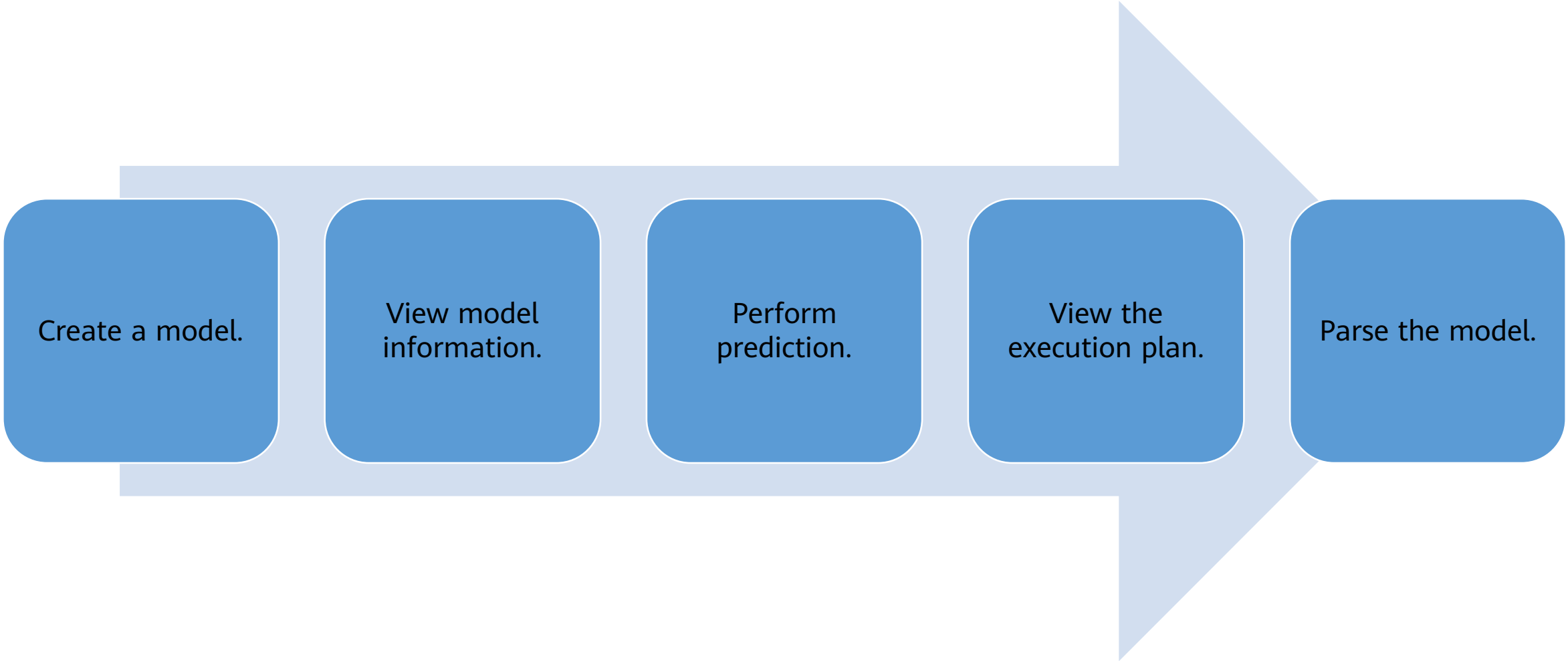
HUAWEI

Native DB4AI Engine - Syntax and Keywords

Name	Description	
Syntax	CREATE MODEL	Creates a model, trains it, and saves the model.
	PREDICT BY	Uses an existing model for prediction.
	DROP MODEL	Deletes a model.
Keyword	TARGET	Target column name of a training or prediction task.
	FEATURES	Data feature column name of a training or prediction task.
	MODEL	Model name of a training task.

HUAWEI

Native DB4AI Engine - Training Model



Creating a Model

- CREATE MODEL
- You can run the **CREATE MODEL** statement to create a model and train it. This SQL statement uses the public iris dataset for model training.
- Multiclass is used as an example to describe how to train a model. Specify **sepal_length**, **sepal_width**, **petal_length**, and **petal_width** as feature columns in the **tb_iris** training set, and use the multiclass algorithm to create and save the **iris_classification_model** model.

```
openGauss=# CREATE MODEL iris_classification_model USING xgboost_regression_logistic FEATURES sepal_length,
sepal_width,petal_length,petal_width TARGET target_type < 2 FROM tb_iris_1 WITH nthread=4, max_depth=8;
MODEL CREATED. PROCESSED 1
```


HUAWEI

Viewing Model Information (1)

- After the training is complete, the model is stored in the gs_model_warehouse system catalog. You can view information about the model and training process in the gs_model_warehouse system catalog.
- The model details are stored in the system catalog in binary mode. You can call the gs_explain_model function to view the model details. The statement is as follows:

```
openGauss=# select * from gs_explain_model("iris_classification_model");
 DB4AI MODEL
-------------------------------------------------------------
 Name: iris_classification_model
 Algorithm: xgboost_regression_logistic
 Query: CREATE MODEL iris_classification_model
 USING xgboost_regression_logistic
 FEATURES sepal_length, sepal_width,petal_length,petal_width
 TARGET target_type < 2
 FROM tb_iris_1
 WITH nthread=4, max_depth=8;
```

Viewing Model Information (2)

```
Return type: Float64
Pre-processing time: 0.000000
Execution time: 0.001443
Processed tuples: 78
Discarded tuples: 0
n_iter: 10
batch_size: 10000
max_depth: 8
min_child_weight: 1
gamma: 0.0000000000
eta: 0.3000000000
nthread: 4
verbosity: 1
seed: 0
booster: gbtree
tree_method: auto
eval_metric: rmse
rmse: 0.2648450136
model size: 4613
```


HUAWEI

Performing Prediction

- Use the existing model to perform prediction.
- Use the SELECT and PREDICT BY keywords to complete the prediction task based on the existing model.
- Query syntax: SELECT... PREDICT BY... (FEATURES...)... FROM...;

```
openGauss=# SELECT id, PREDICT BY iris_classification (FEATURES sepal_length,sepal_width,petal_length,petal_width) as
"PREDICT" FROM tb_iris limit 3;
id  | PREDICT
-----+---------
  84 |       2
  85 |       0
  86 |       0
(3 rows)
```

Viewing the Execution Plan

- View the execution plan.
- You can run the **EXPLAIN** statement to analyze the execution plan in the model training (**CREATE MODEL**) or prediction process (**PREDICT BY**). The keyword EXPLAIN can be followed by a CREATE MODEL or PREDICT BY clause or an optional parameter. The supported parameters are as follows:

```
openGauss=# Explain CREATE MODEL patient_logisitic_regression USING logistic_regression FEATURES second_attack,
treatment TARGET trait_anxiety > 50 FROM patients WITH batch_size=10, learning_rate = 0.05;
                              QUERY PLAN
---------------------------------------------------------------------
 Train Model - logistic_regression  (cost=0.00..0.00 rows=0 width=0)
   ->  Materialize  (cost=0.00..41.08 rows=1776 width=12)
         ->  Seq Scan on patients  (cost=0.00..32.20 rows=1776 width=12)
(3 rows)
```

Parsing the Model

- Parse the model.
- Scenario: If the model name is not be found in the system catalog, the database reports "ERROR."

```
openGauss=# select gs_explain_model("ecoli_svmc");
ERROR:  column "ecoli_svmc" does not exist
```


HUAWEI

Exception Scenarios

- Training phase
  - Scenario 1: When the value of a hyperparameter exceeds its permitted range, the model training fails, and an error message is returned.
  - Scenario 2: The model cannot be saved because the model name already exists.
  - Scenario 3: The value of the **FEATURE** or **TARGETS** column is *****.
  - Scenario 4: The TARGET keyword is either used for unsupervised learning or incorrectly applied for supervised learning in a context where it does not fit.
  - Scenario 5: For a classification task, the **TARGET** column contains only a single class.
  - Scenario 6: The dataset used for model training is empty, as DB4AI filters out columns that contain null values during training.

HUAWEI

DB4AI-Snapshots for Data Version Management (1)

- Concept
  - DB4AI-Snapshots is used by the DB4AI module to manage dataset versions. With the DB4AI-Snapshots component, developers can easily and quickly perform data preprocessing operations such as feature filtering and type conversion. Additionally, developers can perform version control on training datasets like Git. After a data table snapshot is created, it can be used as a view. However, once the data table snapshot is released, it is fixed as static data. To modify the content of this data table, you need to create a data table snapshot with a different version number.

HUAWEI

DB4AI-Snapshots for Data Version Management (2)

- Lifecycle
  - DB4AI-Snapshots can be in the published, archived, or purged state. In the published state, the DB4AI-Snapshots feature becomes available for use. In the archived state, DB4AI-Snapshots is not available for new model training. You can use old data to validate new models. DB4AI-Snapshots in the purged state will not be retrievable within the database system.
  - The DB4AI-Snapshots feature is designed to provide a unified training set, enabling team members to collaboratively train a machine learning model. Therefore, the DB4AI-Snapshots feature is not suitable for scenarios where user data cannot be converted, such as private users, or when separation of duties is enforced (**enableSeparationOfDuty** is set to **ON**).

HUAWEI

DB4AI-Snapshots for Data Version Management (3)

- You can run the **CREATE SNAPSHOT** statement to create a data table snapshot. The created snapshot is in the published state by default. A table snapshot can be created in either MSS or CSS mode, which can be configured using the GUC parameter **db4ai_snapshot_mode**. You can run the **ARCHIVE SNAPSHOT** statement to mark a data table snapshot as archived and run the **PUBLISH SNAPSHOT** statement to mark it as published again. The state of a data table snapshot is marked to facilitate collaboration among data scientists.
- If a table snapshot is no longer required, you can run the **PURGE SNAPSHOT** statement to permanently delete it and recycle the storage space.

HUAWEI

DB4AI-Snapshots for Data Version Management (4)

- Usage guide
  - Create a table and insert table data.
    - If a data table already exists in the database, create a data table snapshot based on the existing data table. To facilitate subsequent demonstration, create a data table named **t1** and insert test data into the table. Query the table content.
  - Use DB4AI-Snapshots.
    - Create a snapshot by running **CREATE SNAPSHOT... AS**.
    - Example 1: In this example, the default version separator is an at sign (@), and the default subversion separator is a period (.). You can set the separator by setting the **db4ai_snapshot_version_delimiter** and **db4ai_snapshot_version_separator** parameters.

HUAWEI

DB4AI-Snapshots for Data Version Management (5)

```
create snapshot s1@1.0 comment is 'first version' as select * from t1;
schema |  name
--------+--------
 public | s1@1.0
(1 row)
```

- The command output indicates that a snapshot has been created for data table **s1**, and the snapshot version is **1.0**. A created data table snapshot can be queried in the same way as a common view but cannot be updated using the INSERT INTO statement.

DB4AI-Snapshots for Data Version Management (6)

- For example, you can use any of the following statements to query the content of data table snapshot **s1** of version 1.0:

```
SELECT * FROM s1@1.0;
SELECT * FROM public.s1@1.0;
SELECT * FROM public . s1 @ 1.0;
id |   name
----+----------
  1 | zhangsan
  2 | lisi
  3 | wangwu
  4 | lisa
  5 | jack
(5 rows)
```

DB4AI-Snapshots for Data Version Management (7)

- You can run the following SQL statement to modify the content of the **t1** data table:

```
UPDATE t1 SET name = 'tom' where id = 4;
insert into t1 values (6, 'john');
insert into t1 values (7, 'tim');
```

- When the content of data table **t1** is retrieved again, although the content of data table **t1** has changed, the query result of data table snapshot **s1@1.0** will not change. Because data in data table **t1** has changed, to use the content of the current data table as version 2.0, you can create snapshot **s1@2.0** by using the following SQL statement:

```
create snapshot s1@2.0 as select * from t1;
```


HUAWEI

DB4AI-Snapshots for Data Version Management (8)

- Delete the data table snapshot **SNAPSHOT**.

```
purge snapshot s1@3.0;
schema |  name
--------+--------
 public | s1@3.0
(1 row)
```


HUAWEI

DB4AI-Snapshots for Data Version Management (9)

- Sample from snapshots of a data table.
  - Example: Use the sampling rate 0.5 to extract data from snapshot **s1**.

```
sample snapshot s1@2.0 stratify by name as nick at ratio .5;
schema |     name
--------+------------
 public | s1nick@2.0
(1 row)
```

  - You can use this feature to create a training set and a test set. For example:

```
SAMPLE SNAPSHOT s1@2.0  STRATIFY BY name AS _test AT RATIO .2, AS _train AT RATIO .8 COMMENT IS 'training';
schema |      name
--------+----------------
 public | s1_test@2.0
 public | s1_train@2.0
(2 rows)
```

DB4AI-Snapshots for Data Version Management (10)

- Publish a data table snapshot.
  - Run the following SQL statement to mark the data table snapshot **s1@2.0** as published:

```
publish snapshot s1@2.0;
schema |  name
--------+--------
 public | s1@2.0
(1 row)
```


HUAWEI

DB4AI-Snapshots for Data Version Management (11)

- Archive a data table snapshot.
  - Run the following statement to mark a data table snapshot as archived:

```
archive snapshot s1@2.0;
schema |  name
--------+--------
 public | s1@2.0
(1 row)
```

  - You can query the db4ai-snapshots view to display the status of the current data table snapshot and other information.

```
select * from db4ai.snapshot
```

DB4AI-Snapshots for Data Version Management (12)

- GUC parameters
  - db4ai_snapshot_mode:
  - There are two snapshot modes: MSS (materialized mode, used to store data entities) and CSS (computing mode, used to store incremental information). A snapshot mode can be switched between the MSS and CSS modes. The default snapshot mode is MSS.

```
db4ai_snapshot_version_delimiter:
```

  - Specifies the version delimiter of data table snapshots. The at sign (@) is the default version delimiter of data table snapshots.

```
db4ai_snapshot_version_separator
```

  - Specifies the subversion separator of data table snapshots. The period (.) is the default subversion separator of data table snapshots.

HUAWEI

ABO

- Intelligent cardinality estimation
  - Intelligent cardinality estimation leverages lightweight algorithms within databases to model the distribution of multi-column data. This enables accurate estimation of the cardinality of multi-column equality queries. In scenarios where data skew occurs, and columns are closely related, more accurate estimation results can be obtained to provide accurate cost reference for the optimizer, improving plan generation accuracy and database query execution efficiency.
- Adaptive plan selection
  - This feature improves query performance by initiating plan selection based on the selectivity of the base table's conditions. It offers multi-plan cache management and adaptive selection for queries involving partial indexes and offsets. In typical scenarios, this can result in several times improvement in query throughput. Adaptive plan selection is useful when a general cache plan is used for plan execution. Range linear expansion is used for cache plan exploration, and range coverage matching is used for plan selection. This approach addresses performance issues associated with traditional single cache plans that cannot adjust to varying query conditions and prevents the need for frequent calls to the query optimizer.

HUAWEI

Contents

1. NUMA Architecture Optimization and Resource Pooling
2. AI Capabilities
3. **Fusion Storage Engine**
4. High Availability Based on the Paxos Protocol
5. Fully-encrypted Database
6. MOT Management

HUAWEI

Fusion Storage Engine

- The fusion engine architecture supports a pluggable storage framework, and the in-place update storage engine Ustore is used. By versioning indexes, transaction information is enriched. Lockless Xlog updates significantly enhance the Xlog write performance. Parallel page replay improves the efficiency of replay on standby. Additionally, the enterprise-grade flashback feature provides a stable environment for users to query data.

HUAWEI

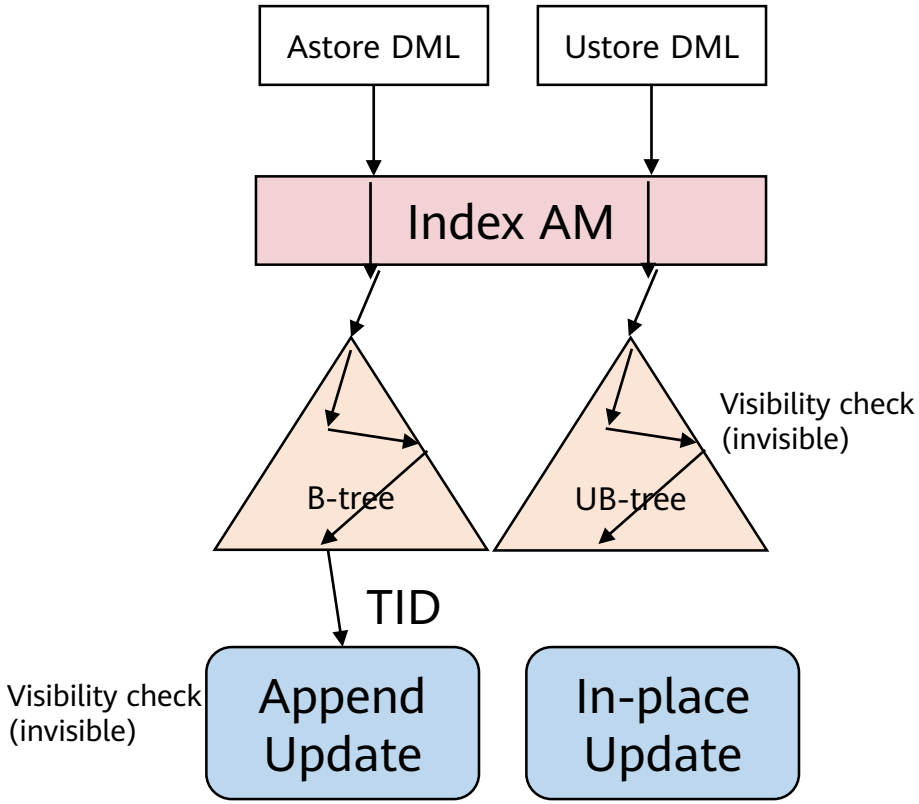
In-place Update Storage Engine

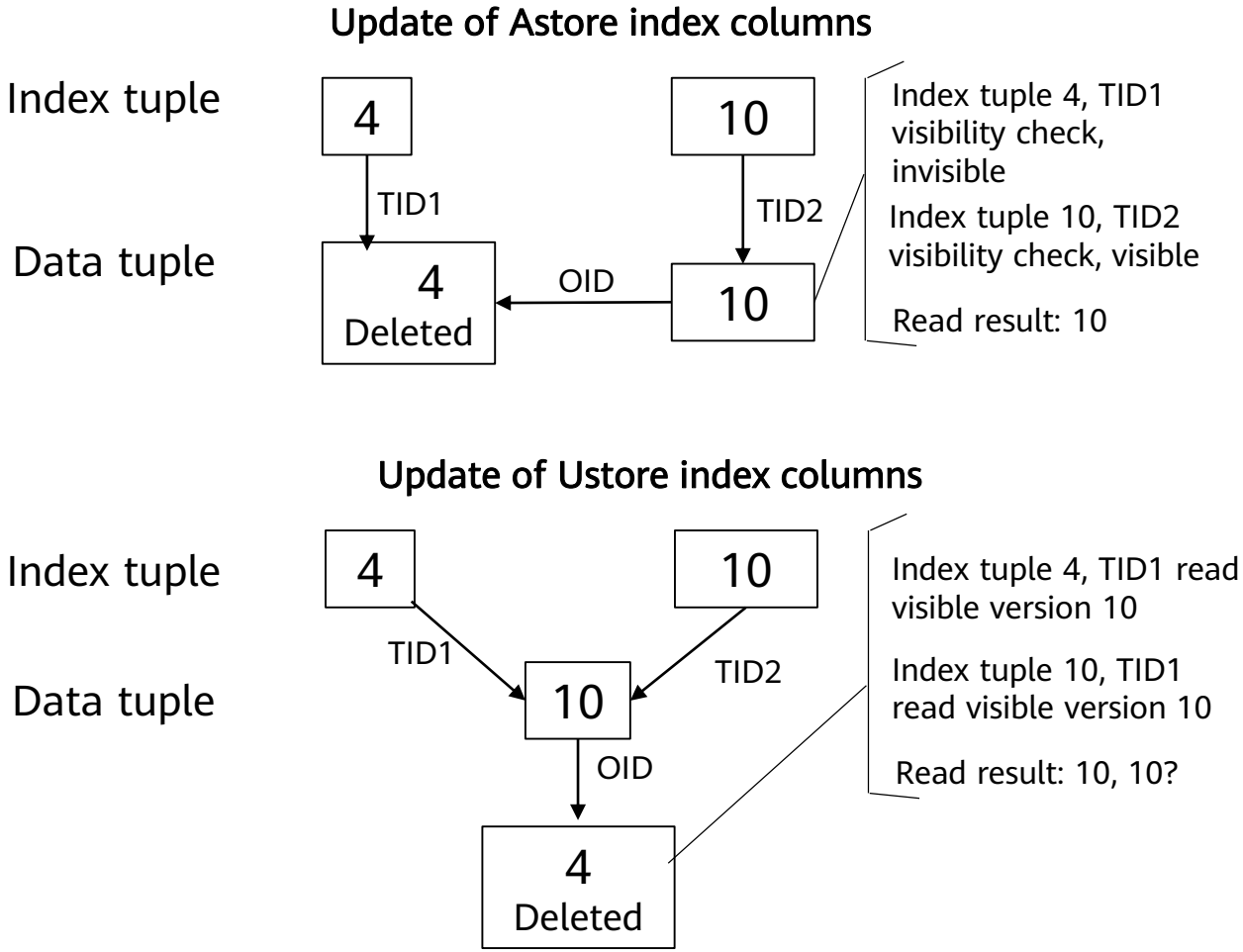
- The openGauss Kernel now features the in-place update storage engine Ustore, in addition to the existing row storage engine that updates data in append mode. The row storage engine delivers good performance for services such as addition, deletion, and heap-only tuple (HOT) updates (that is, updates on the same page). However, it is not efficient in recycling space in non-HOT updates across pages. To address this issue, Ustore is introduced.
- Ustore uses undo segments to perfectly solve the problems of space bloat and large tuples during append updates.

HUAWEI

Index Versioning

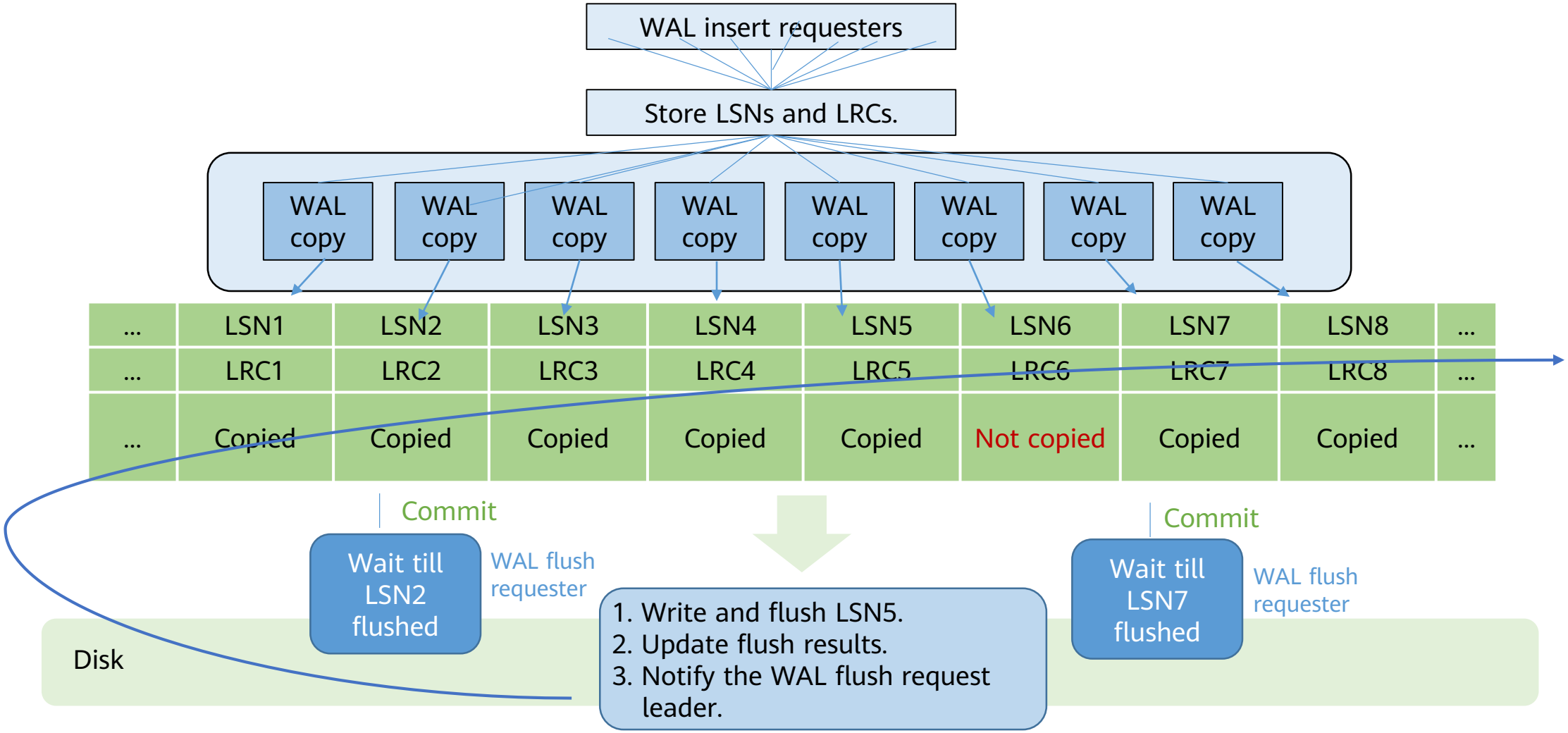
- Comparison between UB-tree and B-tree indexes regarding search and update





HUAWEI

Lockless Xlog Update



HUAWEI

Parallel Page Replay

- Prefixes and suffixes are used to reduce the write times of WAL updates. Since Ustore usually replays a range of WAL pages in a DML operation, replayer threads are further classified to facilitate targeted log replay. In addition, Ustore data pages are distributed by block number during the replay process. Benefit: Ustore optimizes the write logic of in-place WAL updates and improves the degree of replay parallelism for DML operations.

HUAWEI

Enterprise-Grade Flashback (1)

- Flashback is a part of the database recovery technology. It enables DBAs to selectively and efficiently cancel the impact of a committed transaction and restore data incorrectly modified due to manual operations. Before the flashback technology was introduced, a committed database modification could be retrieved only by means of backup and restoration or PITR. The restoration took several minutes or hours. With the flashback technology, you can restore committed data in a database to its previous state in just seconds, regardless of the database size.
- This feature supports the following flashback modes:
  - Flashback query: enables you to query a snapshot of a table at a certain time point. This feature can be used to view and logically rebuild data that was accidentally deleted or modified. The flashback query feature is based on the MVCC mechanism. You can query and retrieve an earlier version of data.

Enterprise-Grade Flashback (2)

- Flashback table: enables you to restore a table to a specific time point. When only one table or a group of tables is logically damaged (not the entire database), this feature can be used to quickly restore the table data. Using the MVCC mechanism, the flashback table feature removes incremental data added after a specified time point and restores data that was deleted at or before that time point.
- Flashback DROP: enables you to restore accidentally deleted tables and their auxiliary structures, including indexes and table constraints, from the recycle bin. Flashback DROP is based on the recycle bin mechanism. You can restore dropped physical table files from the recycle bin.
- Flashback TRUNCATE: enables you to restore accidentally truncated tables, including their physical data and indexes, from the recycle bin. Flashback TRUNCATE is based on the recycle bin mechanism. You can restore truncated physical table files from the recycle bin.

HUAWEI

Contents

1. NUMA Architecture Optimization and Resource Pooling
2. AI Capabilities
3. Fusion Storage Engine
4. **High Availability Based on the Paxos Protocol**
5. Fully-encrypted Database
6. MOT Management

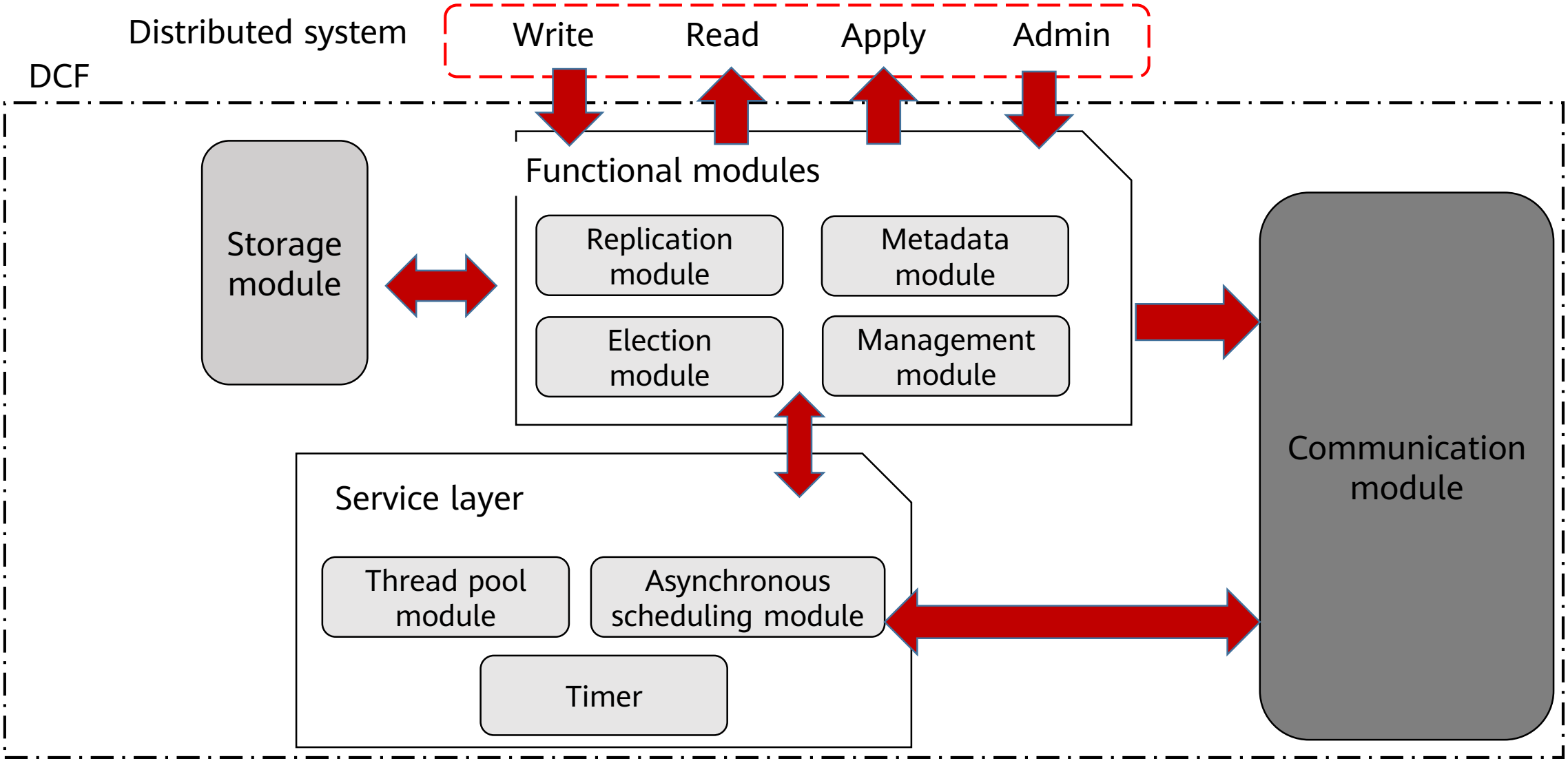
HUAWEI

High Availability Based on the Paxos Protocol

- Description
  - With DCF enabled, database nodes can support Paxos-based replication and quorum capabilities. Based on Paxos's leader election and log replication capabilities, data compression, and flow control are achieved among database nodes during the replication process, preventing high bandwidth usage. Multiple types of nodes are supported based on the Paxos protocol, and the node roles can be adjusted. You can query the status of the current database instance.
- Prerequisites
  - To use this feature, ensure that DCF has been enabled during installation and deployment.

HUAWEI

Architecture



HUAWEI

Feature Description (1)

- During log replication, DCF compresses logs before transmission to reduce network bandwidth usage.
- DCF supports passive nodes, which do not participate in leader election but only synchronize and replay logs. When this type of node is overloaded during log synchronization, flow control is performed.
- DCF supports logger nodes, which participate in leader election and voting. However, the logger nodes only replicate DCF logs. Xlogs are not replicated, and redo is not performed.

HUAWEI

Feature Description (2)

- DCF supports online switchover between follower and passive nodes. That is, a node can switch over between the follower and passive roles without interrupting services.
- DCF supports forcible startup of the minority of nodes. If the majority of nodes in a database instance are faulty, a leader node is elected from a minority of standby nodes, and other normal standby nodes replicate logs from the new primary node.
- DCF supports leader selection. If the original primary node is faulty, a new primary node is automatically selected from the normal standby nodes on the premise that data consistency is ensured.

HUAWEI

Contents

1. NUMA Architecture Optimization and Resource Pooling
2. AI Capabilities
3. Fusion Storage Engine
4. High Availability Based on the Paxos Protocol
5. **Fully-encrypted Database**
6. MOT Management

HUAWEI

Fully-encrypted Database

- Overview
- Using gsql to Operate a Fully-encrypted Database with Software and Hardware Integrated
- Using the JDBC Driver to Operate a Fully-encrypted Database with Software and Hardware Integrated

Fully-encrypted Database with Software and Hardware Integrated

- A fully encrypted database aims to protect privacy throughout the data lifecycle. Data is always encrypted during transmission, computing, and storage regardless of the service scenario or environment. After the data owner encrypts data on a client and sends the encrypted data to the server, attackers cannot obtain valuable information even if they steal user data by exploiting system vulnerabilities. This ensures data privacy is protected.
- Currently, the fully encrypted database supports two connection modes: gsql and JDBC.

HUAWEI

Fully Encrypted Database with Software and Hardware Integrated - Feature Description (1)

- From a user perspective, the encrypted equality query feature comprises three parts that rely on the newly added keytool and the enhanced gsql client in openGauss.
- This feature provides the client-side key management function. Users can use keytool to generate, destroy, and update CMKs, and import and export keys. With the import and export functions of keytool, CMKs can be transmitted between different clients. In addition, keytool implements key management on a single client. By configuring management files, you can store and update keys.
- In addition, this feature provides the key creation and encrypted table creation functions. The SQL syntax CREATE CLIENT MASTER KEY and CREATE COLUMN ENCRYPTION KEY are added to record and manage CMK and CEK metadata in the database. The CMK and CEK information is recorded in the new system catalog. The CREATE TABLE syntax is extended to specify a column encryption key and encryption algorithm for each sensitive information column in a table, facilitating subsequent ciphertext data storage.

HUAWEI

Fully Encrypted Database with Software and Hardware Integrated - Feature Description (2)

- This feature provides users with the encrypted equality query capability, which is the core of the feature. Although users are unaware of the ciphertext query, the query of sensitive data is restricted by the specifications of the current encrypted equality query.
- This feature is used to store and manage data based on sensitive data protection requirements and implement query tasks based on ciphertext data.

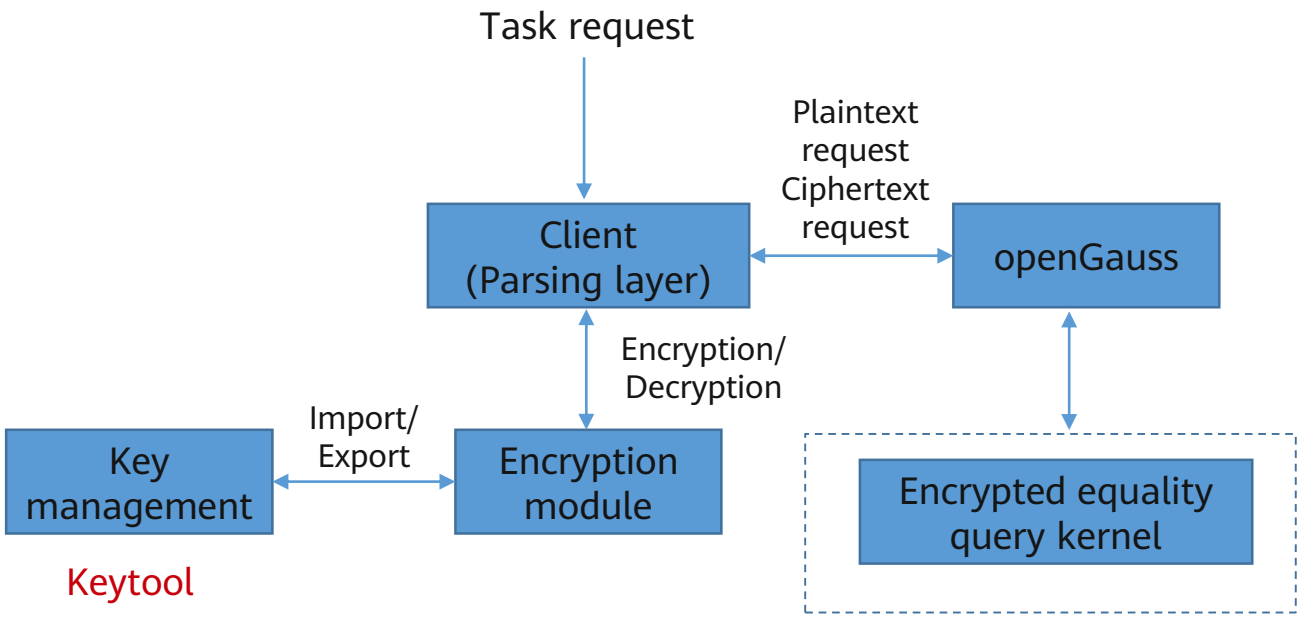
HUAWEI

Fully-encrypted Database with Software and Hardware Integrated - Limitations

- Different data types cannot be implicitly converted.
- The set operation between ciphertexts of different data types is not supported.
- Range partition cannot be created from encrypted columns.
- Encrypted tables cannot be copied to files.
- The encrypted equality query is supported. Other encrypted queries, such as sorting, range query, and fuzzy query, are not supported.
- Multiple SQL statements cannot be executed at the same time. This limitation does not apply to scenarios where the INSERT INTO statement is executed in multiple batches.
- The encrypted database does not encrypt empty strings whose length is zero.

HUAWEI

Fully-encrypted Equality Query Process



- A three-layer key management mechanism is used, which comprises the following keys: root key (RK), column master key (CMK), and column encryption key (CEK).
- The client encrypts and decrypts data, and the server processes the encrypted data.
- Columns that do not need to be encrypted are still processed in plaintext.

Using gsql to Operate a Fully Encrypted Database with Software and Hardware Integrated (1)

- Log in as the **omm** user to the host where the CN is located.
  - Enable the fully encrypted database feature and connect to the encrypted database.

```
gsql -p PORT postgres -r -C
```

- Create a CMK and a CEK.
  - Create a CMK on the client.

```
openGauss=# CREATE CLIENT MASTER KEY ImgCMK1 WITH (KEY_STORE = localkms, KEY_PATH = "key_path_value1",
ALGORITHM = RSA_2048);
openGauss=# CREATE CLIENT MASTER KEY ImgCMK WITH (KEY_STORE = localkms, KEY_PATH = "key_path_value2",
ALGORITHM = RSA_2048);
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK1 WITH VALUES (CLIENT_MASTER_KEY = ImgCMK1, ALGORITHM
= AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
openGauss=# CREATE COLUMN ENCRYPTION KEY ImgCEK WITH VALUES (CLIENT_MASTER_KEY = ImgCMK, ALGORITHM  =
AEAD_AES_256_CBC_HMAC_SHA256);
CREATE COLUMN ENCRYPTION KEY
```


HUAWEI

Using gsql to Operate a Fully Encrypted Database with Software and Hardware Integrated (2)

- Query the system catalog that stores the key information. The output is similar to the following:

```
openGauss=# SELECT * FROM gs_client_global_keys;
 global_key_name | key_namespace | key_owner | key_acl |        create_date
-----------------+---------------+-----------+---------+---------------------------
 imgcmk1         |          2200 |        10 |         | 2021-04-21 11:04:00.656617
 imgcmk          |          2200 |        10 |         | 2021-04-21 11:04:05.389746
(2 rows)
```

```
openGauss=# SELECT column_key_name,column_key_distributed_id ,global_key_id,key_owner  FROM gs_column_keys;
 column_key_name | column_key_distributed_id | global_key_id | key_owner
-----------------+---------------------------+---------------+-----------
 imgcek1         |                 760411027 |         16392 |        10
 imgcek          |                3618369306 |         16398 |        10
(2 rows)
```


HUAWEI

Using gsql to Operate a Fully Encrypted Database with Software and Hardware Integrated (3)

- Create an encrypted table.

```
openGauss=# CREATE TABLE creditcard_info (id_number    int, name         text encrypted with (column_encryption_key =
ImgCEK, encryption_type = DETERMINISTIC),
credit_card  varchar(19) encrypted with (column_encryption_key = ImgCEK1, encryption_type = DETERMINISTIC));
NOTICE:  The 'DISTRIBUTE BY' clause is not specified. Using 'id_number' as the distribution column by default.
HINT:  Please use 'DISTRIBUTE BY' clause to specify suitable data distribution column.
CREATE TABLE
```

- Insert data into an encrypted table and perform an equality query.

```
openGauss=# INSERT INTO creditcard_info VALUES (1,'joe','6217986500001288393');
INSERT 0 1
openGauss=# INSERT INTO creditcard_info VALUES (2, 'joy','6219985678349800033');
INSERT 0 1
openGauss=# select * from creditcard_info where name = 'joe';
 id_number | name |     credit_card
-----------+------+--------------------
         1 | joe  | 6217986500001288393
(1 row)
```


HUAWEI

Using gsql to Operate a Fully Encrypted Database with Software and Hardware Integrated (4)

```
openGauss=# select id_number,name from creditcard_info;
 id_number |                                                
```

Using the JDBC Driver to Operate a Fully Encrypted Database with Software and Hardware Integrated

- To enable JDBC to perform operations on a fully encrypted database, set **enable_ce** to **1**.
- When JDBC is used to operate an encrypted database, each database connection is tied to a single thread. Avoid changing database threads, as this may lead to conflicts.
- When JDBC is used to operate an encrypted database, the encryption configuration varies between connections. To ensure that each connection's encryption configuration is valid, the client should call the IsValid method. In this case, it is important to set the **refreshClientEncryption** parameter to **1** (the default value). If only a single client is interacting with the encrypted database, you can set the **refreshClientEncryption** parameter to **0**.

HUAWEI

Contents

1. NUMA Architecture Optimization and Resource Pooling
2. AI Capabilities
3. Fusion Storage Engine
4. High Availability Based on the Paxos Protocol
5. Fully-encrypted Database
6. **MOT Management**

HUAWEI

MOT and Its Benefits

- MOT features high performance (low query and transaction latency), high scalability (high throughput and concurrency), and, in some cases, even cost savings (high resource utilization).
  - **Low latency**: quick query and transaction response.
  - **High throughput**: peak hours and continuous high concurrency.
  - **High resource utilization**: hardware resource maximization.
  - With MOT, the throughput of applications can increase by 2.5 to 4 times. For instance, when a TPC-C benchmark test (including interactive transactions and log synchronization) is run on Huawei TaiShan servers equipped with Arm/Kunpeng processors and Dell x86 servers equipped with Intel Xeon processors, the throughput gain provided by MOT increases by 2.5 times on 2-socket servers, by 3.7 times on 4-socket servers, and reaches 4.8 million tpmC on 4-socket, 256-core Arm servers.

HUAWEI

Key Technologies (1)

- Memory-optimized structure: To achieve high concurrency, high throughput, and predictable low latency, all data and indexes are stored in the memory without being cached in page buffers. Additionally, the lock-holding time is minimized. The data structure and algorithms are memory-optimized.
- Lock-free transaction management: While ensuring strict consistency and data integrity, MOT uses optimistic policies to achieve high concurrency and high throughput. During a transaction, MOT does not lock any version of the data row being updated, significantly reducing resource contention in certain large memory systems. Optimistic concurrency control (OCC) in transactions is implemented without locks. All data modification is performed in the part of memory dedicated to private transactions (also called private transactional memory). This means that during a transaction, related data is updated in the private transactional memory, thereby implementing lock-free read and write. In addition, a lock is held briefly only during the commit phase.

HUAWEI

Key Technologies (2)

- Lock-free index: MOT stores tables and indexes in memory, which requires a highly efficient index structure and algorithms. MOT's indexes are based on the state-of-the-art Masstree data structure, which is a fast and scalable key-value (KV) index storage mechanism for many-core systems and implemented using the B+ tree Trie index structure. This approach achieves excellent performance on many-core servers in the case of high-concurrency workloads. In addition, MOT uses advanced technologies to optimize performance, such as lock optimization, cache awareness, and memory prefetch.
- NUMA-aware memory management: It is handled directly by the memory controller, eliminating the need for additional interconnects like Intel QPI. The intelligent memory control module of MOT pre-allocates memory pools for various memory objects to improve performance, reduce locks, and ensure stability. Transactional memory is always allocated to NUMA local nodes. After transactions end, memory is released to the pool. In addition, system memory allocation (OS malloc) is minimally used in transactions to prevent unnecessary locks.

HUAWEI

Key Technologies (3)

- High durability: As an in-memory storage engine, MOT optimizes numerous algorithms to enhance persistence design, achieving a good trade-off between durability and targeted speed and throughput. These optimizations include:
  - Parallel logging, applicable to all disk-based tables in openGauss
  - Log buffering and lock-free transaction preparation for each transaction
  - Incremental logging. Only changes are logged.
  - NUMA-aware group commit, in addition to synchronous and asynchronous logging
  - State-of-the-art checkpointing asynchronously using logical consistency (CALC), minimizing memory and computing overheads

HUAWEI

Key Technologies (4)

- Seamless integration of MOT and openGauss
  - MOT is a high-performance memory-optimized storage engine integrated into the openGauss database installation package. MOT, along with disk-based storage engines, is included in openGauss to support a range of application scenarios. They share auxiliary database services, such as WAL, replication, checkpointing, and HA. Users benefit from unified deployment, configuration, and access to disk-based tables and MOTs. Users can flexibly select a cost-effective storage engine based on their specific requirements. For instance, performance-sensitive data that could cause bottlenecks is stored in memory.

HUAWEI

MOT Application Scenarios (1)

- High throughput: MOT is especially applicable to high throughput scenarios, as it allows massive transactions with low latency for each single transaction. Examples of such applications include real-time decision-making systems, payment systems, financial instrument transactions, sports lotteries, mobile games, advertisement placement, and the like.
- Caching elimination: Cloud computing and mobile applications often experience periodic or peak workloads. In addition, more than 80% of the workloads of many applications are read workloads with frequently repeated queries. Typically, a caching layer is deployed for applications to meet the individual requirements of peak workloads, reduce response latency, and provide the best user experience. Such an additional layer increases the complexity and time of development as well as operational costs. MOT is a good alternative solution, which simplifies the application architecture, shortens the development cycle, and reduces the CAPEX and OPEX through consistent high-performance data storage.

HUAWEI

MOT Application Scenarios (2)

- Performance acceleration: Tables with high contention can benefit from MOT, even if the tables are based on disks. Due to lower latency, reduced contention and locking, and increased server throughput, the conversion of such tables results in a significant performance improvement.
- Large-scale stream data extraction: MOT can meet requirements for extracting large-scale stream data in the cloud (for mobility, M2M, and IoT), transactional processing (TP), analytical processing (AP), and machine learning (ML). MOT is particularly fast at continuously extracting large amounts of data from many different sources at the same time.
- Reduced TCO: 30% to 90% TCO can be saved by improving resource utilization and eliminating the caching layer.

HUAWEI

MOT Usage (1)

- Grant permissions to a user.
  - The following describes how to grant a database user the permission to access the MOT storage engine. This operation is performed only once for each database user typically during initial configuration.
  - To enable a user to create and access MOTs (through DDL, DML, and SELECT operations), execute the following statement only once:

```
GRANT USAGE ON FOREIGN SERVER mot_server TO <user>;
```

  - All keywords are case-insensitive.

HUAWEI

MOT Usage (2)

- Create or delete an MOT.
  - Except for the differences in the statements for creating and deleting MOTs compared to disk-based tables in openGauss, the syntax for all other SELECT, DML, and DDL commands is identical for both MOTs and disk-based tables.
  - Create an MOT.

```
create FOREIGN table test(x int) [server mot_server];
```

  - Drop the MOT named **test**.

```
drop FOREIGN table test;
```


HUAWEI

MOT Usage (3)

- Create an index for an MOT.
  - Standard statements for creating and deleting indexes are supported in openGauss. For example:

```
create index text_index1 on test(x) ;
```

  - Create an **ORDER** table for TPC-C benchmark testing and create an index.

```
create FOREIGN table bmsql_oorder (
o_w_id integer not null,
o_d_id integer not null,
o_id integer not null,
o_c_id integer not null,
o_carrier_id integer,
o_ol_cnt integer,
o_all_local integer,
o_entry_d timestamp,
primary key (o_w_id, o_d_id, o_id)
);
create index bmsql_oorder_index1 on bmsql_oorder(o_w_id, o_d_id, o_c_id, o_id) ;
```


HUAWEI

Quiz

1. Can a disk table be directly converted to an MOT? ( )

   A. True

   B. False

HUAWEI

Summary

- This chapter provides an overview of the advanced features of the openGauss database. These include NUMA architecture optimization and resource pooling, AI capabilities, fusion storage engine (introduction of the in-place update storage engine), Paxos-based high availability implemented using the Distributed Consensus Framework, fully-encrypted database, and memory-optimized table (MOT) management. By thoroughly studying this chapter, you will have an in-depth understanding of the advanced features of the openGauss database.

HUAWEI

Thank you.

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI

Chapter 8 Troubleshooting



HUAWEI

Foreword

This chapter outlines various troubleshooting methods for openGauss, covering techniques for diagnosing issues that may arise with the operating system, network, disk, and database. Additionally, it includes several troubleshooting cases that address problems such as memory stalling, disk full errors, and memory shortages that may occur during TPC-C runtime.

HUAWEI

Objectives

- After completing this course, you will be able to:
  - Master common troubleshooting methods, including techniques for diagnosing database faults.
  - Gain knowledge of typical troubleshooting cases, such as a database becoming read-only when the disk usage reaches its upper limit.

HUAWEI

Contents

1. **Essential Troubleshooting Methods**
2. Typical Troubleshooting Cases

HUAWEI

OS Faults

- Symptom: When you query the database status, if all instances on a node are abnormal, an OS fault occurs.
- Troubleshooting method:
  - Log in to the node using SSH or other remote login tools. If the login fails, run the **ping** command to check the network status.
    - If the ping operation does not receive a response, the server may be down, restarting, or experiencing a network issue. If the system has crashed due to an OS kernel panic, the restart process can take up to 20 minutes. Attempt to connect to the host every 5 minutes. If a successful connection is not established within 20 minutes, the server may be down, or there is a network problem. In this case, contact the system administrator for onsite assistance.
    - If the ping operation succeeds but SSH login fails, or commands cannot be executed, the server may not respond to external connections due to insufficient system resources (for example, CPU or I/O resources are overloaded). In this case, try again. If the fault persists for 5 minutes, contact the system administrator for onsite assistance.
  - If you experience slow response times after successfully logging in, check the system's running status. This includes gathering system information, checking the system version, hardware configuration, parameter settings, and the number of logged-in users.

OS Faults - Checking the System Running Status (1)

- Run the following command to check the current online users:

```
[root@openGauss36 ~]# who
root     pts/0        2023-11-07 16:32 (10.70.xxx.xx8)
wyc      pts/1        2023-11-10 09:54 (10.70.xxx.xx8)
root     pts/2        2023-10-10 14:20 (10.70.xxx.xx8)
root     pts/4        2023-10-09 10:14 (10.70.22x.xx3)
root     pts/5        2023-10-09 10:14 (10.70.21x.xx3)
root     pts/7        2023-10-31 17:03 (10.70.xxx.xx8)
root     pts/9        2023-10-20 10:03 (10.70.xxx.xxx)
```

- Run the **cat /etc/openEuler-release** and **uname –a** commands to check the system version and kernel information.

```
[root@openGauss36 ~]# cat /etc/openEuler-release
openEuler release 20.03 (LTS)
[root@openGauss36 ~]# uname -a
Linux openGauss36 4.19.90-2003.4.0.0036.oe1.aarch64 #1 SMP Mon Mar 23 19:06:43 UTC 2020 aarch64 aarch64 aarch64
GNU/Linux
[root@openGauss36 ~]#
```


HUAWEI

OS Faults - Checking the System Running Status (2)

- Run the **cat /proc/cpuinfo** command to check the CPU information.

```
[root@openGauss36 ~]# cat /proc/cpuinfo
   processor          : 0
   BogoMIPS           : 200.00
   Features: fp asimd evtstrm aes pmull sha1 sha2 crc32 atomics fphp asimdhp cpuid asimdrdm jscvt fcma dcpop asimddp
asimdfhm
   CPU implementer    : 0x48
   CPU architecture: 8
   CPU variant        : 0x1
   CPU part           : 0xd01
   CPU revision       : 0
```

- Run the **cat /proc/meminfo** command to check the system memory information.

```
[root@openGauss36 ~]# cat /proc/meminfo
   MemTotal:       534622272 kB
   MemFree:        253322816 kB
   MemAvailable:   369537344 kB
   Buffers:          2429504 kB
   Cached:         253063168 kB
   SwapCached:            0 kB
```

```
   Active:         88570624 kB
   Inactive:       171801920 kB
   Active(anon):    4914880 kB
   Inactive(anon): 67011456 kB
   Active(file):   83655744 kB
   Inactive(file): 104790464 kB
   ......
```


HUAWEI

OS Faults - Other Essential Commands

- **top -H**:
  - Check the CPU usage to determine whether a specific process's CPU usage is high. If it is high, run the **gdb** or **gstack** command to print the process's stack trace and check whether the process is in an infinite loop.
- **iostat -x 1 3**:
  - Check the I/O usage to determine whether the current disk's I/O is saturated. Check the ongoing jobs to determine whether to handle the jobs with high I/O usage.
- **vmstat 1 3**:
  - Check the memory usage in the current system. Run the **top** command to obtain the processes that consume a large amount of memory and behave unexpectedly.
- **sysctl -a**:
  - Run the **cat /etc/sysctl.conf** command as the **root** user to obtain system parameter information.

HUAWEI

Network Faults

- When the database runs normally, the network layer is transparent to upper-layer users. However, during the long-term operation of a database, network exceptions or errors may occur. Typical exceptions caused by network faults are as follows:
  - Network errors reported due to database startup failure.
  - Abnormal status. For example, all instances on a host are in the **UnKnown** state, or all services are switched over to standby instances.
  - Network connection failure.
  - Network disconnection reported during database SQL query.
  - Process response failures during database connection or query execution. When a network fault occurs in a database, locate and analyze the fault by using network-related Linux command line tools (such as **ping**, **ifconfig**, **netstat**, and **lsof**) and process stack-related Linux command line tools (such as **gdb** and **gstack**) based on database log information. This section lists the typical network faults and describes how to analyze and locate them.

HUAWEI

Network Faults - Network Errors Reported Due to Database Startup Failure (1)

- Symptom 1:
  - If the log file contains the following error information, the port may be being listened to by another process:

```
LOG: could not bind socket at the 10 time, is another postmaster already running on port 26000?
```

- Troubleshooting method:
  - Check the process that is listening on the port. Replace the port number in the following command with the actual one:

```
[root@openGauss36 ~]# netstat -anop | grep 15970
tcp        0      0 127.0.0.1:15970         0.0.0.0:*               LISTEN      3920251/gaussdb      off (0.00/0/0)
tcp6       0      0 ::1:15970               :::*                    LISTEN      3920251/gaussdb      off (0.00/0/0)
unix  2      [ ACC ]     STREAM     LISTENING     197399441 3920251/gaussdb      /tmp/.s.PGSQL.15970
unix  3      [ ]         STREAM     CONNECTED     197461142 3920251/gaussdb      /tmp/.s.PGSQL.15970
```

Based on the query result, forcibly stop the process that is occupying the port or change the database's listening port.

HUAWEI

Network Faults - Network Errors Reported Due to Database Startup Failure (2)

- Symptom 2:
  - When the **gs_om -t status --detail** command is executed to query the database status, the command output shows that the connection between the primary and standby nodes is not established.
- Troubleshooting method:
  - In openEuler OS, run the **systemctl status firewalld.service** command to check whether the firewall is enabled on this node. If it is enabled, run the **systemctl stop firewalld.service** command to disable it.

```
[root@openGauss36 mnt]# systemctl status firewalld.service
firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; disabled; vendor preset: enabled)
   Active: inactive (dead)
     Docs: man:firewalld(1)
```

The command varies depending on the OS. You can run the corresponding command to view and modify the configuration.

HUAWEI

Network Faults - Abnormal Database Status

- Symptom:
  - All instances on a node are in the **UnKnown** state, all services are switched over to standby instances, or many error messages, including "Connection reset by peer" and "Connection timed out," are displayed.
- Troubleshooting method:
  - If you cannot connect to the faulty server through SSH, run the **ping** command on other servers to send data packages to it. If the ping operation succeeds, the connection fails because resources, such as memory, CPUs, and disks, on the faulty server are used up.
  - If you can connect to the faulty server through SSH, run the **/sbin/ifconfig eth***?* command every other second (replace the question mark (?) with the NIC number). Check the value change of **dropped** and **errors**. If they increase rapidly, the NIC or NIC driver may be faulty.

```
[root@openGauss36 ~]# ifconfig enp125s0f0
enp125s0f0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.90.56.36  netmask 255.255.255.0  broadcast 10.90.56.255
        inet6 fe80::7be7:8038:f3dc:f916  prefixlen 64  scopeid 0x20<link>
        ether 44:67:47:7d:e6:84  txqueuelen 1000  (Ethernet)
        RX packets 129344246  bytes 228050833914 (212.3 GiB)
        RX errors 0  dropped 647228  overruns 0  frame 0
        TX packets 96689431  bytes 97279775245 (90.5 GiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Network Faults - Network Connection Failure (1)

- Symptom 1: A node fails to connect to other nodes, and the "Connection refused" error is reported in the log.
- Troubleshooting method:
  - Check whether the port is incorrectly configured. Incorrect configuration may cause the connection port to be different from the listening port of the peer end. Check whether the port number that is recorded in the **postgresql.conf** configuration file of the faulty node is the same as the listening port number of the peer end.
  - Run the **netstat -anp** command to check whether the listening port of the peer end is normal or whether the peer process exists.
- Symptom 2: When SQL query operations are performed on the database, the connection descriptor fails to be obtained. The following error information is displayed:

```
WARNING:  29483313: incomplete message from client:4905,9
WARNING:  29483313: failed to receive connDefs at the time:1.
ERROR:  29483313: failed to get pooled connections
```

Locate the error in the log and loop up to find more error details. The following lists typical errors generated by incorrect primary and standby information.

```
FATAL:  dn_6001_6002: can not accept connection in pending mode.
FATAL:  dn_6001_6002: the database system is starting up
FATAL:  dn_6009_6010: can not accept connection in standby mode.
```

Network Faults - Network Connection Failure (2)

- Troubleshooting method:
  - Run the **gs_om -t status --detail** command to query the status and check whether a switchover occurs. If a switchover occurs, reset the instance status.

```
[   Cluster State   ]

cluster_state   : Normal
redistributing  : No
current_az      : AZ_ALL

[  Datanode State   ]

   node   node_ip         port      instance                       state
-----------------------------------------------------------------------------------------
1  primary 192.168.56.26   26000      6001 /opt/huawei/install/data/dn   P Primary Normal
2  standby 192.168.56.27   26000      6002 /opt/huawei/install/data/dn   S Standby Normal
```

  - In addition, check whether a core dump or restart occurs on the node that fails to be connected. In the OM log, check whether a restart occurs.

Network Faults - Network Disconnection Error Reported During SQL Execution

- Symptom: The query fails, and the following error information is displayed:

```
ERROR:  dn_6065_6066: Failed to read response from Datanodes. Detail: Connection reset by peer. Local: dn_6065_6066
Remote: dn_6023_6024
ERROR: Failed to read response from Datanodes Detail: Remote close socket unexpectedly
ERROR: dn_6155_6156: dn_6151_6152: Failed to read vector response from Datanodes
```

If the connection fails, the following error information may be displayed:

```
ERROR:  Distribute Query unable to connect 10.145.120.79:14600 [Detail:stream connect connect() fail: Connection timed
out
ERROR: Distribute Query unable to connect 10.144.192.214:12600 [Detail:receive accept response fail: Connection timed out
```

- Troubleshooting method:
  - Use gs_check to check whether the network configuration meets the requirements.
  - Check whether a process core dump, restart, or switchover occurs.
  - If problems still exist, contact technical support.

HUAWEI

Disk Faults

- Typical disk issues include insufficient disk space, bad sectors on the disk, and the disk not being mounted.
- These disk issues vary in terms of their consequences:
  - File system corruption. In this case, the database management system automatically and periodically checks disks, identifies the disk as a fault, and stops the instance. When you check the database status, the instance status is abnormal.
  - No damage to the file system. For example, if the disk space is insufficient and the database management system does not detect this, the service process exits abnormally when accessing the faulty disk. Such faults can manifest as a database startup failure, checksum verification failure, page read/write failure, or page validation error.
- The disk faults can manifest as follows:
  - The instance status is always "Unknown" (if the disk fault corrupts the file system).
  - The service process exits abnormally when accessing the faulty disk (if the disk fault does not corrupt the file system).

HUAWEI

Disk Faults - File System Corruption

The instance status is always "Unknown" (if the disk fault corrupts the file system).

- Troubleshooting method:
  - Check the logs. If the logs contain information similar to "data path disc writable test failed," the file system is corrupted.
  - Possibly, the disk is not mounted. Run the **ls –l** command to check whether the permission on the directory corresponding to the disk is abnormal.
  - Or, the disk has bad sectors. In this case, the OS rejects read and write operations to protect the file system. You can use the bad sector check tool. For example, run the **badblocks** command to check whether bad sectors exist.

```
[root@openeuler123 mnt]# badblocks /dev/sdb1 -s -v
Checking blocks 0 to 2147482623
Checking for bad blocks (read-only test): done
Pass completed, 0 bad blocks found. (0/0/0 errors)
```


HUAWEI

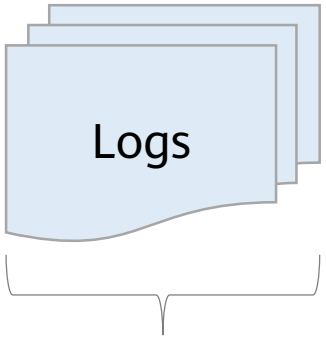
Disk Faults - No Damage to the File System

- The service process exits abnormally when accessing the faulty disk (if the disk fault does not corrupt the file system). Troubleshooting method:
  - Check the logs. The log contains read and write errors, such as "No space left on device" and "invalid page header n block 122838 of relation base/16385/152715". Run the **df -h** command to check the disk space. If the disk usage is 100% as shown below, the read and write errors are caused by insufficient disk space:

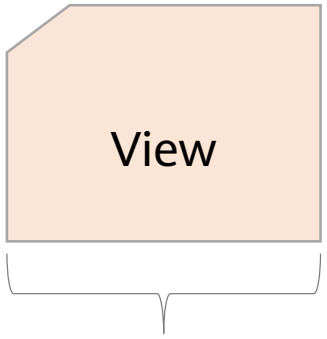
```
[root@openeuler123 mnt]# df -h
Filesystem                 Size     Used    Avail   Use%   Mounted on
devtmpfs                   255G     0       255G    0%         /dev
tmpfs                      255G     35M     255G    1%         /dev/shm
tmpfs                      255G     57M     255G    1%         /run
tmpfs                      255G     0       255G    0%         /sys/fs/cgroup
/dev/mapper/openeuler-root  196G   8.8G   178G    5%   /
tmpfs                      255G     1.0M    255G    1%         /tmp
/dev/sda2                  9.8G     144M    9.2G    2%         /boot
/dev/sda1                   10G     5.8M    10G     1%         /boot/efi
/dev/mapper/openeuler-home  1.5T    69G     1.4T    5%     /home
tmpfs                      51G      0   51G    0%             /run/user/0
tmpfs                      51G      0   51G    0%             /run/user/1004
/dev/sdb1                  2.0T   169G   1.9T    9%            /data
```

Database Faults

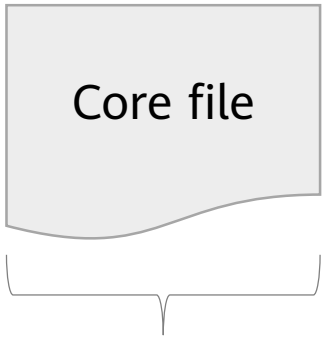
- Three methods for locating database faults:



Database logs record server operations (starting, running, and stopping). Database users can view logs to quickly locate faults, identify root causes, and rectify the faults.



pg_stat_activity
pg_thread_wait_status
pg_locks



An abnormal exit of a database process triggers a core dump. A core file helps locate faults and determine fault causes. If a core dump occurs during the process running, collect the core file immediately to further analyze and locate the fault.

Contents

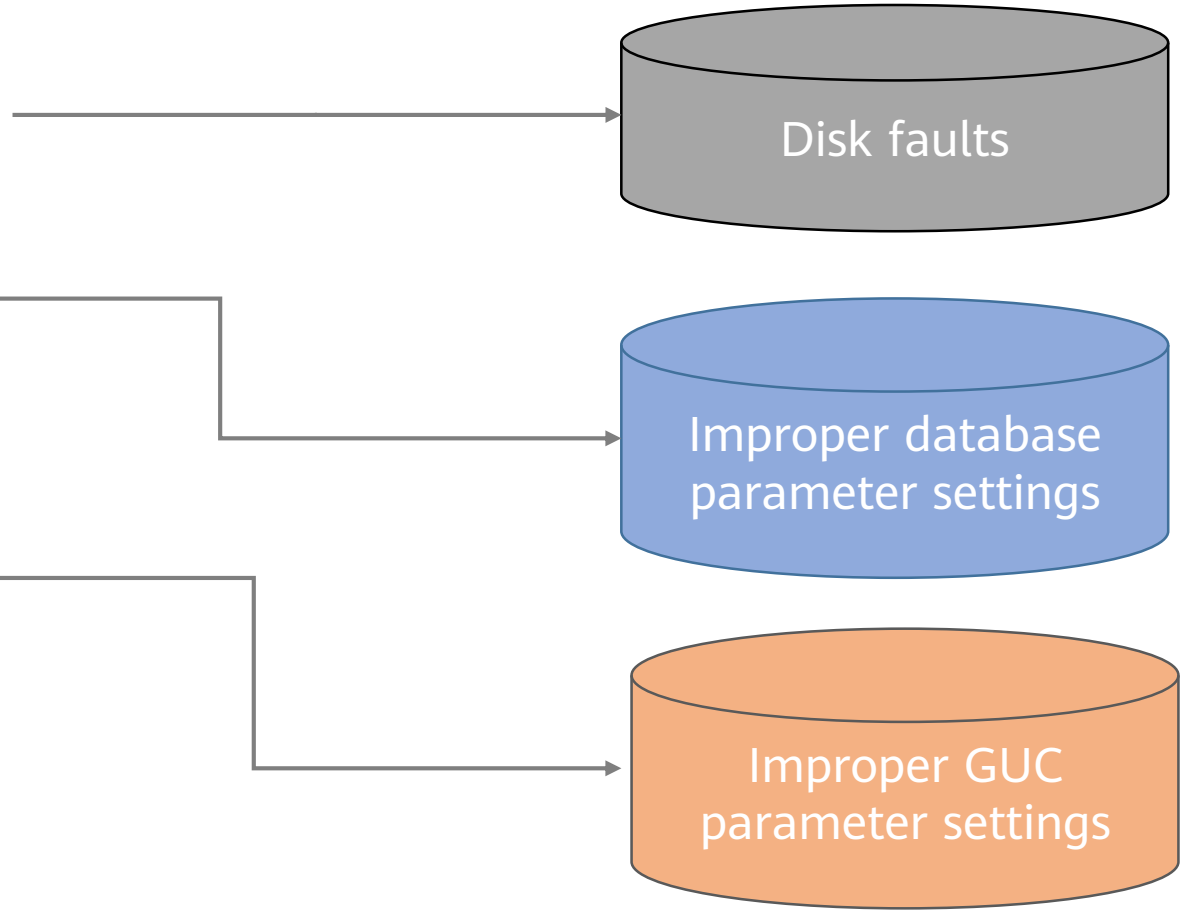
1. Essential Troubleshooting Methods
2. **Typical Troubleshooting Cases**

HUAWEI

Core Dump

Typically, a core dump may occur in the following scenarios:

- Core dump occurs due to disk full errors.
- Core dump occurs due to improper setting of the GUC parameter **log_directory**.
- Core dump occurs when **RemoveIPC** is enabled.



Core Dump Due to Disk Space Errors

- Symptom:
  - At TPC-C runtime, the disk is full, the database process (gaussdb) is down, and log content similar to the following is generated:

```
-rw-------. 1 omm dbgrp    1611 Mar 28 09:57 core-gaussdb-14734-2024_03_28_09_57_39-bbox.lz4
-rw-------. 1 omm dbgrp    2543 Mar 27 23:33 core-gaussdb-63834-2024_03_27_23_33_24-bbox.lz4
```

- Possible cause:
  - When the disk is full, Xlogs cannot be written, and the program logs a "panic" error and exits.
- Troubleshooting method:
  - Externally monitor the disk usage and periodically clean up the disk.

HUAWEI

Core Dump Due to Incorrect Setting of the GUC Parameter **log_directory**

- Symptom:
  - After the database process is started, a core dump occurs, and no log is recorded.
- Possible cause:
  - The directory specified by the GUC parameter **log_directory** cannot be read, or you do not have permission to access this directory. As a result, the verification fails during the database startup, and the program logs a "panic" error and exits.
- Troubleshooting method:
  - Set the GUC parameter **log_directory** to a valid path.

HUAWEI

Core Dump Occurs When RemoveIPC Is Enabled

- Symptom:
  - The **RemoveIPC** parameter in the OS configuration is set to **yes**. The database breaks down during running, and the following log information is displayed:

```
FATAL: semctl(1463124609, 3, SETVAL, 0) failed: Invalid argument
```

- Possible cause:
  - If **RemoveIPC** is set to **yes**, the OS releases the IPC resources (shared memory and semaphore) when the corresponding user exits, causing the openGauss database to break down.
- Troubleshooting method:
  - Re-set **RemoveIPC** to **no**.

HUAWEI

Memory Stalling or Disk Full at TPC-C Runtime

- Symptom:
  - At TPC-C runtime, the disk is full, and the memory stalls. After the fault is rectified, the TPC-C runs automatically.
- Possible cause:
  - When the disk where the performance log (**gs_profile**) is located is full, the database enters an infinite waiting state because data can no longer be written. As a result, the TPC-C stops. After the disk fault is rectified, performance logs can be properly written, and the TPC-C runs automatically.
- Troubleshooting method:
  - Externally monitor the disk usage and periodically clean up the disk.

HUAWEI

Standby Node in the Need Repair (WAL) State

- Symptom:
  - The "Need Repair (WAL)" fault occurs on a standby node of openGauss.
- Possible cause:
  - The primary and standby instances are disconnected due to network faults or disk full errors. As a result, logs are not synchronized between the primary and standby instances, and the database fails to start.
- Troubleshooting method:
  - Run the **gs_ctl build -D** command to rebuild the faulty node.

HUAWEI

Memory Shortage

- Symptom:
  - The client reports the error message "memory usage has reached the max_dynamic_memory," or the logs contain this error message.
- Possible cause:
  - The possible cause is that the value of the GUC parameter **max_process_memory** is too small. This parameter limits the maximum memory that can be used by an openGauss instance.
- Solution:
  - Use gs_guc to change the value of **max_process_memory**. You need to restart the instance for the change to take effect.

HUAWEI

Memory Shortage - Troubleshooting Method (1)

- Troubleshooting method:
  - If the database memory is insufficient, the port connection may fail. In this case, connect to the next port (that is, the port number of the faulty node plus 1).
  - After the login is successful, query the total memory usage.

```
openGauss=# SELECT * FROM gs_shared_memory_detail;
 contextname                        | level |              parent                | totalsize | freesize | usedsize
------------------------------------+-------+------------------------------------+-----------+----------+----------
 ProcessMemory                      |     0 |                                    |    516096 |   176192 |   339904
 WaitCountGlobalContext             |     1 | ProcessMemory                      |     30800 |     1760 |    29040
 sql count lookup hash              |     2 | WaitCountGlobalContext             |      8192 |     7984 |      208
 SQL used space on datanode         |     1 | ProcessMemory                      |    122880 |    52704 |    70176
 StreamNodeGlobalContext            |     1 | ProcessMemory                      |         0 |        0 |        0
 stream connect sync hash           |     2 | StreamNodeGlobalContext            |      8192 |     2640 |     5552
 StreamGlobalContext                |     1 | ProcessMemory                      |         0 |        0 |        0
 stream info lookup hash            |     2 | StreamGlobalContext                |      8192 |     2768 |     5424
 DynamicFuncCtrolMgr                |     1 | ProcessMemory                      |         0 |        0 |        0
 dynamic stmt track control hash table |     2 | DynamicFuncCtrolMgr                |      8192 |     2640 |     5552
 PercentileContext                  |     1 | ProcessMemory                      |   2400088 |        0 |  2400088
 CaptureViewContext                 |     1 | ProcessMemory                      |         0 |        0 |        0
```


HUAWEI

Memory Shortage - Troubleshooting Method (2)

- Analyze the memory usage obtained in the previous step.
  - If the value of **dynamic_used_shrctx** is excessively large, query the gs_shared_memory_detail view.

```
select contextname, sum(totalsize)/1024/1024 sum, sum(freesize)/1024/1024, count(*) count from
gs_shared_memory_detail group by contextname order by sum desc limit 10;
```

According to the memory context information, memory leakage may occur in the context with the largest memory usage.

```
 contextname                 |        sum                 |       ?column?                | count
-----------------------------+----------------------------+-------------------------------+-------------------
 AshContext                  | 63.3320922851562500  |  .00772094726562500000  |     1
 IncreCheckPointContext      | 52.2579498291015625  |  .00373840332031250000  |     1
 GlobalAuditMemory           | 16.0081176757812500  |  .00769042968750000000  |     1
 CBBTopMemoryContext         | 12.4787292480468750  |  .05073547363281250000  |     1
 StorageTopMemoryContext |  8.4577789306640625  |  .00376892089843750000   |     1
 DoubleWriteContext          |  6.5549163818359375  |  .02331542968750000000   |     1
 GPRC_Bucket_Context         |  3.0000000000000000  |     1.5546875000000000     |   128
 PercentileContext           |  2.2889022827148438   | 0.00000000000000000000 |     1
 WalContext                  |  2.0079040527343750  |  .00605773925781250000   |     1
 Workload manager memory context |  1.7892150878906250 |  .00379943847656250000 |     1
```


HUAWEI

Memory Shortage - Troubleshooting Method (3)

- If the value of **dynamic_used_memory** is large and the value of **dynamic_used_shrctx** is small, query the gs_session_memory_detail view.

```
select contextname, sum(totalsize)/1024/1024 sum, sum(freesize)/1024/1024, count(*) count from
gs_session_memory_detail group by contextname order by sum desc limit 10;
```

- Check whether the context with the largest memory usage is a common context based on the memory context information. If it is not, memory leakage may occur in the context.

```
 contextname                    |        sum                 |       ?column?              | count
--------------------------------+----------------------------+-----------------------------+-------------------------------
 gs_signal                      | 64.5457000732421875 |    3.3509979248046875   |     1
 SessionCacheMemoryContext  | 32.4762420654296875 |    6.6469192504882813   |    28
 StorageTopMemoryContext    | 23.2120819091796875 | .28353881835937500000 |    28
 ThreadTopMemoryContext     |  8.2134704589843750 | .17477416992187500000   |    28
 Wal Sender                 |  8.0079574584960938 | .00474548339843750000   |     1
 Undo Recycler              |  8.0078887939453125 | .00767517089843750000   |     1
 DefaultTopMemoryContext    |  7.4426803588867188 | .95897674560546875000   |    28
 CBBTopMemoryContext        |  5.9329833984375000 | .64689636230468750000   |    28
 Timezones                  |  2.2297973632812500 | .07391357421875000000   |    28
 PLpgSQL function cache     |  1.9800033569335938 | .00263977050781250000   |     1
```


HUAWEI

Memory Shortage - Troubleshooting Method (4)

- Query the detailed memory allocation information of the context that may have memory leakage.
  - gs_get_shared_memctx_detail(text) returns the memory allocation details of the specified memory context, including the file, line number, and size of each memory allocation (the size of the same line in the same file is accumulated). The memory context can only be queried through the pg_shared_memory_detail view. The input parameter is the memory context name (that is, the **contextname** column in the result returned by the pg_shared_memory_detail view). Only users with the SYSADMIN or MONADMIN permission can execute this function. The query result contains the following three fields.

Parameter	Type	Description
file	text	Name of the file to which the memory is allocated.
line	int8	Line number of the code in the file to which the memory is allocated.
size	int8	Size of the allocated memory. The value is accumulated if the memory is allocated multiple times to the same line in the same file.

HUAWEI

Memory Shortage - Troubleshooting Method (5)

- gs_get_session_memctx_detail(text) returns the memory allocation details of the specified memory context, including the file, line number, and size of each memory allocation (the size of the same line in the same file is accumulated). This function is valid only in thread pool mode. The memory context can only be queried through the pv_session_memory_context view. The input parameter is the memory context name (that is, the **contextname** column in the result returned by the pv_session_memory_context view). Only users with the SYSADMIN or MONADMIN permission can execute this function. The query result contains the following three fields.

Name	Type	Description
file	text	Name of the file to which the memory is allocated.
line	int8	Line number of the code in the file to which the memory is allocated.
size	int8	Size of the allocated memory. The value is accumulated if the memory is allocated multiple times to the same line in the same file.

HUAWEI

Memory Shortage - Troubleshooting Method (6)

- Most context-related memory leakage faults can be resolved using the preceding method. However, if the value of **other_used_memory** is high, use the tool provided by Jemalloc to collect malloc call and memory allocation relationship information for analysis. The procedure is as follows:
  - In the debug version, run the following command to set the environment variables:

```
export MALLOC_CONF=prof:true,prof_final:false,prof_gdump:true,lg_prof_sample:20
```

    In the command, **20** indicates that a heap file is generated every 2^20 bytes (1 MB). This value can be modified. However, after the value is increased, the number of heap files decreases, but some memory allocation information is lost.

  - Restart the cluster. In a cluster environment, kill the om_monitor process and manually restart it.
    - Check whether a large number of heap files are generated in the data directory. If so, the configuration is successful. If not, repeat the preceding steps until heap files are generated.

Memory Shortage - Troubleshooting Method (7)

- Run related services for a period of time and obtain the earliest and latest heap files.
- Run the **jeprof** command to process heap files and generate PDF files. You can obtain jeprof from **binarylibs/*${platForm}*/jemalloc/debug/bin**. To use jeprof, you need to install graphviz by running **yum install graphviz**.
- To generate a PDF file, run the following command:
  - Full: jeprof –show_bytes –pdf gaussdb *.heap > out.pdf
  - Incremental: jeprof –pdf gaussdb –base=start.heap end.heap > out.pdf
- Analyze the generated PDF files.

```
GenericMemoryAllocator
        AllocSetAlloc
              3579.8(89.2%)
            of 4015.1(100%)
```

The output shows the total size of memory allocated and its proportion to the total memory allocated in the system.

HUAWEI

Primary Node Hangs During Demotion in Switchover

- Symptom:
  - In a cluster deployed with one primary and multiple standby nodes, if system resources are insufficient and a switchover occurs, the primary node hangs when it is demoted to standby.
- Possible cause:
  - If system resources are insufficient, third-party management threads cannot be created. As a result, worker threads cannot exit, and the primary node hangs when it is demoted to standby.
- Troubleshooting method:
  - Run the following command to stop the process of the primary node so that a standby node can be promoted to primary (do not use this method if the scenario is not applicable):

```
kill -9 PID
```


HUAWEI

Disk Space Usage Reaches Its Upper Limit, and the Database Becomes Read-Only (1)

- Symptom:
  - The following error is reported when a non-read-only SQL statement is executed:

    ```
    ERROR: cannot execute %s in a read-only transaction.
    ```

  - The following error is reported when certain non-read-only SQL statements (such as insert, update, create table as, create index, alter table, and copy from) are executed:

    ```
    canceling statement due to default_transaction_read_only is on.
    ```

- Possible cause:
  - After the disk space usage reaches the upper limit, the database enters the read-only mode. In this mode, only read-only statements can be executed.
- Troubleshooting method:
  - Use either of the following methods to connect to the database in maintenance mode:

HUAWEI

Disk Space Usage Reaches Its Upper Limit, and the Database Becomes Read-Only (2)

- Method 1:

```
gsql -d postgres -p 8000 -r -m
```

- Method 2:

```
gsql -d postgres -p 8000 -r
```

After the connection is successful, run the following command.

```
set xc_maintenance_mode=on;
```

- Run the **DROP** or **TRUNCATE** statement to delete user tables that are no longer used until the disk space usage falls below the upper limit. Deleting user tables is a temporary solution for the insufficient disk space issue. To permanently solve the problem, it is necessary to expand the disk space.
- Disable the read-only mode of the database as the **omm** user.

```
gs_guc reload -D /gaussdb/data/dbnode -c "default_transaction_read_only=off"
```


HUAWEI

Analyzing the Running Status of Query Statements (1)

- Symptom:
  - Some query statements are executed for an excessively long time in the system. In this scenario, you can analyze the running status of the query statements.
- Troubleshooting method:
  - Log in to the host as the **omm** user and connect to the database. In the following command, **postgres** is the name of the database to be connected, and **8000** is the port number:

```
gsql -d postgres -p 8000
```

  - Set the parameter **track_activities** to **on**.

```
SET track_activities = on;
```

The database system collects the running information about active queries only if the parameter is set to **on**.

HUAWEI

Analyzing the Running Status of Query Statements (2)

- View the running query statements. The pg_stat_activity view is used as an example.

```
SELECT datname, usename, state, query FROM pg_stat_activity;
datname  | usename | state  | query
----------+---------+--------+-------
postgres | omm     | idle   |
postgres | omm     | active |
```

  - If the **state** column is **idle**, the connection is idle and users need to enter a command. To identify only active query statements, run the following command:

```
SELECT datname, usename, state, query FROM pg_stat_activity WHERE state != 'idle';
```

- Analyze whether a query statement is in the active or blocked state. To identify blocked query statements, run the following command:

```
SELECT datname, usename, state, query FROM pg_stat_activity WHERE waiting = true;
```

  - The command output lists the blocked query statements that are waiting for lock resources to be released by other sessions.

HUAWEI

Forcibly Terminating a Session

- Symptom:
  - In some cases, administrators must forcibly terminate abnormal sessions to keep the system healthy.
- Troubleshooting method:
  - Find the thread ID of the faulty session from the current active session view.

```
SELECT datid, pid, state, query FROM pg_stat_activity;
```

A command output similar to the following is displayed, where the pid value indicates the thread ID of the session.

```
datid |       pid       | state  | query
-------+-----------------+--------+------
13205 | 139834762094352 | active |
13205 | 139834759993104 | idle   |
```

  - Terminate the session using its thread ID.

```
SELECT pg_terminate_backend(139834762094352);
```


HUAWEI

Low Query Efficiency (1)

- Symptom:
  - A query that used to take a few milliseconds to complete now requires several seconds, and a query that used to take several seconds now requires up to half an hour.
- Troubleshooting method:
  - Run the **analyze** command to analyze the database.
    - Run the **analyze** command to update statistics such as data sizes and attributes in all tables. You are advised to perform the operation when the service load is light. If the query efficiency is improved or restored after the command execution, it indicates that the autovacuum process is not functioning normally and further analysis is required to identify the root cause.
  - Check whether the query statement returns unnecessary information.
    - If an application requires only a part of data information but the query statement returns all information, add a LIMIT clause to the query statement to restrict the number of returned records.

HUAWEI

Low Query Efficiency (2)

- Check whether the query statement has a slow response even when it is solely executed.
  - Run the query statement when there are no requests or only a few query requests in the database and monitor the query efficiency. If the efficiency is high, the issue is likely caused by a heavily loaded host in the database system or an inefficient execution plan.
- Check the same query statement repeatedly to check the query efficiency.
  - One major cause of low query efficiency is that the required information is not cached in the memory or is replaced by other query requests due to insufficient memory resources. This can be verified by running the same query statement repeatedly. If the query efficiency increases gradually, the issue is likely caused by this reason.

HUAWEI

Error Occurs Upon Altering a Table Partition (1)

- Symptom:
  - When **ALTER TABLE PARTITION** is performed, the following error message is displayed:
- Possible cause:
  - If the ALTER TABLE PARTITION statement contains both the DROP PARTITION and ADD PARTITION operations, openGauss always performs the DROP PARTITION operation first and then performs the ADD PARTITION operation regardless of their sequence. However, performing DROP PARTITION and then performing ADD PARTITION causes a partition gap (unallocated space). As a result, an error is reported.
- Troubleshooting method:
  - To prevent partition gaps, set **END** in DROP PARTITION to the value of **START** in ADD PARTITION.

Error Occurs Upon Altering a Table Partition(2)

The following is an example:

```
-- Create a partitioned table partitiontest.
openGauss=#  CREATE TABLE partitiontest
(
c_int integer,
c_time TIMESTAMP WITHOUT TIME ZONE
)
PARTITION BY range (c_int)
(
partition p1 start(100)end(108),
partition p2 start(108)end(120)
);
-- An error is reported when the following statements are used:
openGauss=#  ALTER TABLE partitiontest ADD PARTITION p3 start(120)end(130), DROP PARTITION p2;
ERROR:  start value of partition "p3" NOT EQUAL up-boundary of last partition.
openGauss=#  ALTER TABLE partitiontest DROP PARTITION p2,ADD PARTITION p3 start(120)end(130);
ERROR:  start value of partition "p3" NOT EQUAL up-boundary of last partition.
-- Change them as follows:
openGauss=#  ALTER TABLE partitiontest ADD PARTITION p3 start(108)end(130), DROP PARTITION p2;
openGauss=#  ALTER TABLE partitiontest DROP PARTITION p2,ADD PARTITION p3 start(108)end(130);
```


HUAWEI

Table Size Does not Change After VACUUM FULL Is Executed on the Table (1)

- Symptom:
  - A user runs the **VACUUM FULL** command to clear a table, but the table size does not change.
- Possible cause:
  - No data has been deleted from the table specified by **table_name**. Therefore, **VACUUM FULL** *table_name* has nothing to delete. As a result, the table size does not change.
  - Concurrent transactions exist during the execution of **VACUUM FULL** *table_name*, causing the recently deleted data to be skipped. As a result, the table size does not change.

HUAWEI

Table Size Does not Change After VACUUM FULL Is Executed on the Table (2)

- Troubleshooting method (for the second possible cause):
  - Wait until all concurrent transactions are complete, and run the **VACUUM FULL** *table_name* command again.
  - If the table size still does not change, ensure that no service operations are performed on the table, and then execute the following SQL statement to query the active transaction list status:

```
select txid_current();
```

  - The command output returns the current XID. Then, run the following command to check the active transaction list:

```
select txid_current_snapshot();
```

  - If any XID in the active transaction list is smaller than the current transaction XID, stop the database and start it. Run **VACUUM FULL** to clear the table again.

HUAWEI

Quiz

1. (Multiple-answer question) Before and after the **VACUUM FULL** command is executed, the size (32 MB) of table A does not change. Which of the following are possible causes? ( )

   A. No data has been deleted from table A and **VACUUM FULL** has nothing to delete. As a result, the table size does not change.

   B. Concurrent transactions exist during the execution of **VACUUM FULL**, causing the recently deleted data to be skipped. As a result, the table size does not change.

   C. After **VACUUM FULL** is executed, the possible cause is that the database system is abnormal and the command is not correctly executed.

   D. Table A may have a backup, and logs may be redone after the VACUUM FULL operation is performed.

HUAWEI

Summary

This chapter outlines various troubleshooting methods for openGauss, covering techniques for diagnosing issues that may arise with the operating system, network, disk, and database. Additionally, it includes several troubleshooting cases that address problems such as memory stalling, disk full errors, and memory shortages that may occur during TPC-C runtime.

HUAWEI

More Information

- openGauss Community
  - https://opengauss.org/en/
- Huawei Talent
  - https://e.huawei.com/en/talent/cert/#/careerCert
- Related books:
  - openGauss Database Core Technologies

Recommendations

- Huawei Career Certification
  - HCIA-openGauss
  - HCIP-openGauss

HUAWEI

Thank you.

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © 2024 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI